

Trabajo Fin de Grado
Grado en Ingeniería de las Tecnologías de
Telecomunicación

Caracterización de Sensores de Imagen CMOS

Autor: Manuel Lorente Almán

Tutor: Vicente Baena Lecuyer

Dep. Ingeniería Electrónica
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla

Sevilla, 2018



Trabajo Fin de Grado
Grado en Ingeniería de las Tecnologías de Telecomunicación

Caracterización de Sensores de Imagen CMOS

Autor:
Manuel Lorente Almán

Tutor:
Vicente Baena Lecuyer
Profesor titular

Dep. de Ingeniería Electrónica
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla
Sevilla, 2018

Trabajo Fin de Grado: *Caracterización de Sensores de Imagen CMOS*

Autor: Manuel Lorente Almán

Tutor: Vicente Baena Lecuyer

El tribunal nombrado para juzgar el Proyecto arriba indicado, compuesto por los siguientes miembros:

Presidente:

Vocales:

Secretario:

Acuerdan otorgarle la calificación de:

Sevilla, 2018

El Secretario del Tribunal

A mi familia

A mis amigos

A mis compañeros de AnaFocus

A mis compañeros de remo

A mis maestros

El diseño de sensores de imagen CMOS al igual que el de gran cantidad de sistemas electrónicos comprenden dos procesos generales: diseño y caracterización. Éstos son diseñados con una serie de características las cuales son importantes medir con precisión. Los CIS (CMOS Image Sensors) son dispositivos electrónicos que convierten la información en forma de luz en información digital que puede ser reproducida en formato de imagen.

Eficiencia cuántica, la cual define la conversión fotón-electrón y su recolección. Capacidad de llenado, la cual si es alta indica que un gran número de electrones pueden ser almacenados en el píxel. Ganancia de conversión, que define la capacidad del sensor de convertir estos electrones almacenados en tensión. Ruidos, que establecen el rango dinámico del sensor definiendo el piso de ruido, es decir su rango de percepción de la luz, actualmente existen sensores con un piso de ruido del orden de 1 e⁻. Función de transferencia de modulación, el cual es un parámetro crucial en la caracterización espectral de un sensor pues define la resolución espacial y la relación de contraste en un sensor permitiendo detectar acoplos o diafonías en el mismo. Todas estas y más son las principales características a medir en un sensor de imagen para verificar que tanto el diseño como la configuración es robusta para su posterior implementación en una aplicación real como podría ser el detectar granos de arroz defectuosos en una cadena de producción a una muy alta velocidad, captar una imagen en 3D de un escenario con el fin de detectar obstáculos o detección de bordes para identificar una matrícula en una autopista de un coche a gran velocidad.

A lo largo del trabajo se irán introduciendo el resto de características y explicando sus principios físicos así como los procedimientos de medida, cálculos e instrumentación de laboratorio necesaria para asegurar que el sensor está en su punto óptimo de operación.

La principal aportación de este trabajo ha sido el desarrollo de una librería Python que comprenda tanto la simulación de características de sensores de imagen CMOS, como el análisis y cálculo de parámetros característicos típicos a partir de imágenes reales como la generación automática de reportes que de manera clara y concisa permita sacar conclusiones sobre el punto de operación de un sensor, posibles desviaciones en el comportamiento e incluso comparar diferentes sensores desde un punto de vista cualitativo.

Gracias al desarrollo de esta librería mencionada anteriormente, se ha logrado migrar a nivel empresarial en Teledyne-AnaFocus de un entorno para la caracterización de sensores CMOS basado en Matlab a un entorno Python con un significativo ahorro económico en término de licencias asociado y un aumento en cuanto a la capacidad de desarrollo de software para la caracterización debido a la gran cantidad de recursos libres en la red relativos a Python. Por otra parte, el desarrollo de esta librería ha permitido alinear métodos de trabajo en Teledyne-e2v y Teledyne-AnaFocus (comprada por Teledyne-e2v en 2015) cuyas sinergias hasta la actualidad eran casi inexistentes debido al uso de distinto software en ambas organizaciones.

Resumen	9
Índice	11
Índice de Tablas y Figuras	15
Glosario y abreviaturas	21
1 Introducción	25
1.1 <i>Motivación al trabajo y objetivos</i>	25
1.2 <i>Organización del trabajo</i>	25
2 Generalidades sobre Sensores de Imagen CMOS	27
2.1 <i>Introducción</i>	27
2.1.1 Principios básicos	27
2.1.2 Funcionamiento	27
2.2 <i>Fundamentos de los Sensores de Imagen CMOS</i>	29
2.2.1 CCD vs CMOS	29
2.2.2 Ejemplos de layout de un CIS	31
2.2.3 Principales características de los CIS	31
2.3 <i>Arquitectura del píxel</i>	32
2.3.1 Fotodiodo	32
2.3.2 Píxel 3T	33
2.3.3 Píxel 4T	33
2.3.4 Píxel 5T	34
2.4 <i>Operación interna del píxel</i>	35
2.4.1 Técnica de doble muestreo correlado	35
2.4.2 Métodos de exposición	38
2.4.3 Interfaces de salida más comunes	39
2.4.4 EMVA ₁₂₈₈	40
3 Caracterización de Sensores de Imagen CMOS	41
3.1 <i>Introducción</i>	41
3.2 <i>Laboratorio típico de trabajo</i>	41
3.3 <i>Fuente de led</i>	43
3.3.1 Estabilidad	43
3.3.2 Respuesta espectral y uniformidad espacial	44
3.4 <i>Flujo típico de caracterización</i>	45
4 Medidas funcionales	47
4.1 <i>Inspección visual</i>	47
4.1.1 Package y wire bonding	47
4.2 <i>Medidas eléctricas</i>	47
4.2.1 Alimentaciones	47
4.2.2 Consumo	48
4.3 <i>Interfaces de programación y comunicación del sensor</i>	49
4.3.1 Generación del reloj interno del sistema	49
4.3.2 SPI	50

4.3.3	Interfaz de comunicación	50
4.3.4	Reset externo	53
4.3.5	Señal de exposición	54
4.4	<i>Bloques de señal mixta</i>	55
4.4.1	Power on Reset	55
4.4.2	Referencias internas analógicas	56
4.5	<i>Canal de lectura</i>	56
4.6	<i>Funcionalidades del sensor</i>	57
4.6.1	Correcciones on-chip	57
4.6.2	Offset analógico	58
4.6.3	Ganancia analógica	58
4.6.4	Tipos de trigger	59
5	Medidas electro-ópticas	61
5.1	<i>Corriente oscura</i>	61
5.1.1	Fundamento teórico	61
5.1.2	Método de medida	62
5.1.3	Condiciones de la medida	62
5.1.4	Procedimiento de medida y cálculos	63
5.2	<i>Foto respuesta</i>	64
5.2.1	Fundamento teórico	64
5.2.2	Método de medida	68
5.2.3	Condiciones de la medida	68
5.2.4	Procedimiento de la medida y cálculos	68
5.3	<i>Medidas espectrales</i>	72
5.3.1	Fundamento teórico	72
5.3.2	Método de medida	73
5.3.3	Condiciones de la medida	74
5.3.4	Procedimiento de la medida y cálculos	74
5.4	<i>Función de Transferencia de Modulación (MTF)</i>	77
5.4.1	Fundamento teórico	77
5.4.2	Método del Borde Inclinado	80
5.4.3	Método de medida	81
5.4.4	Condiciones de la medida	82
5.4.5	Procedimiento de la medida y cálculos	83
5.5	<i>Ruidos y No Uniformidades</i>	87
5.5.1	Parámetros a medir y fundamento teórico	87
5.5.2	Método de medida	93
5.5.3	Condiciones de la medida	93
5.5.4	Procedimiento de medida y cálculos	93
5.6	<i>Otros artefactos</i>	97
5.6.1	Blooming	97
5.6.2	Image Lag	98
6	Simulaciones y resultados	101
6.1	<i>Lenguaje y entorno</i>	101
6.2	<i>Librería en Python</i>	102
6.3	<i>Gráficas y resultados</i>	104
6.4	<i>Generación automática de reporte</i>	108
7	Conclusiones	111
7.1	<i>Conclusiones</i>	111
7.2	<i>Líneas futuras de trabajo</i>	111
	Anexo: Código Python	113
	Referencias	187

ÍNDICE DE TABLAS Y FIGURAS

Figura 2-1 Fotoconversión	27
Figura 2-2 Proceso de adquisición de imagen en un CIS	27
Figura 2-3 Recombinación por electrón-hueco	28
Figura 2-4 Esquema de conversión fotoeléctrica	28
Figura 2-5 Esquemático simplificado de un píxel	29
Figura 2-6 Diagrama de potencial de un píxel	29
Figura 2-7 Funcionamiento sensor CCD	30
Figura 2-8 Funcionamiento sensor CMOS	30
Tabla 2-1 CCD vs CMOS	30
Figura 2-9 Layout CIS de área	31
Figura 2-10 Layout CIS lineal	31
Figura 2-11 Estructura básica fotodiodo pinned	32
Figura 2-12 Diagrama de polarización dentro de un píxel	33
Figura 2-13 Píxel 3T	33
Figura 2-14 Píxel 4T	34
Figura 2-15 Píxel 5T	34
Figura 2-16 Píxel 5T desde punto de vista físico	35
Figura 2-17 Cronograma básico píxel 5T	35
Figura 2-18 Cronograma técnica CDS en un píxel	36
Figura 2-18.2 Fase de lectura del piso de ruido	36
Figura 2-18.1 Fase de Reset del PPD	36
Figura 2-18.4 Fase de lectura de señal y ruido	36
Figura 2-18.3 Fase de transferencia de carga a la FD	36
Figura 2-19 Máquina de estados de control del píxel	37
Figura 2-20 Canal de imagen de un CIS	38
Figura 2-22 Rolling Shutter	38
Figura 2-21 Imagen captada en Rolling Shutter	38
Figura 2-24 Global Shutter	39
Figura 2-23 Imagen captada en Global Shutter	39
Tabla 2-2 Interfaces de salida más comunes	39
Figura 3-1 Sensores de Imagen CMOS de área y línea	41
Figura 3-2 Banco típico de caracterización	41
Tabla 3-1 Instrumentación típica de un banco de caracterización	41
Figura 3-3 Fuente de luz para caracterización	43

Figura 3-4 Estabilización de irradiancia en un led	43
Figura 3-5 Estabilización térmica de un led	44
Figura 3-6 Respuesta espectral de la fuente de LEDs	44
Figura 3-7 Uniformidad fuente de LEDs	45
Figura 3-8 Respuesta angular fuente de LEDs	45
Tabla 3-2 Plan típico de caracterización	46
Figura 4-1 Pinout LCC de un CIS	47
Figura 4-2 Wire-bonding al microscopio de un CIS	47
Tabla 4-1 Árbol de alimentaciones típico de un CIS	48
Figura 4-3 Diagrama para test de comunicaciones en un CIS	49
Figura 4-4 Esquema para la generación del reloj en un CIS	49
Tabla 4-2 Comandos SPI típicos en un CIS	50
Figura 4-5 Patrón de test para testar comunicaciones	50
Figura 4-7 Diagrama de ojo salida LVDS	53
Figura 4-8 Medida de Skew en un puerto LVDS	53
Figura 4-9 Esquema generación Reset externo	54
Figura 4-10 Diagrama para test de bloques analógicos	55
Figura 4-11 Punto de test para visualización del Power-On-Reset	55
Figura 4-12 Punto de test para visualización de las señales GATIO	56
Figura 4-13 Medida ruido en el canal de lectura del píxel	57
Figura 4-14 Esquema correcciones en un CIS	57
Figura 4-15 Ejemplo correcciones FPN	58
Figura 4-16 Medida responsividad de un CIS frente a ganancia analógica	58
Figura 4-17 Cronograma trigger single-edge	59
Figura 4-18 Cronograma trigger PWC	59
Figura 5-1 Efecto túnel	62
Figura 5-2 Set-up medida corriente oscura	63
Figura 5-3 Corriente oscura asociada al fotodiodo	63
Figura 5-4 Corriente oscura asociada a la floating diffusion	64
Figura 5-5 Típica curva de responsividad de un CIS	64
Figura 5-6 Esquema simplificado de un píxel 3T	65
Figura 5-7 Típica curva de fototransferencia de carga	66
Figura 5-8 Principales características de un píxel	66
Figura 5-9 Medida de linealidad en un CIS	67
Figura 5-10 Ejemplo de relación señal a ruido y ruido temporal de un CIS	67
Figura 5-11 Set-up para la medida de fotorespuesta de un CIS	68
Figura 5-12 Cálculos directos en la curva de fotorespuesta	69
Figura 5-13 Relaciones entre las medidas espectrales de un CIS	72
Figura 5-14 Efecto fotoeléctrico en el CIS	72

Figura 5-15 Efecto rizado o “Etaloning” en un CIS	73
Figura 5-16 Set-up para medidas espectrales	74
Figura 5-17 Banco óptico	74
Figura 5-18 Ejemplo medidas eficiencia para un mismo CIS	77
Figura 5-19 Curva conceptual MTF	78
Figura 5-20 Comparativa resolución espacial en imágenes reales	78
Figura 5-21 Interferencias a nivel de píxel	79
Figura 5-22 Tipos de interferencias entre píxeles	79
Figura 5-23 Concepto teórico medida MTF	80
Figura 5-24 Concepto real medida MTF	81
Figura 5-25 MTF horizontal y vertical	81
Figura 5-26 Medida MTF paso a paso	81
Figura 5-27 Curva MTF al detalle	82
Figura 5-28 Set-up medida MTF	82
Figura 5-29 Cálculo ESF	84
Figura 5-30 Curva SFR para un píxel	84
Figura 5-31 Ejemplo curva ESF para un CIS	85
Figura 5-32 Ejemplo LSF para un CIS frente a la ideal	85
Figura 5-33 V-MTF y H-MTF	86
Figura 5-34 H-MTF	86
Figura 5-35 V-MTF	86
Figura 5-36 Ruidos en un CIS	87
Figura 5-37 Ruidos en un píxel	87
Figura 5-38 Ruido temporal en una imagen	88
Figura 5-39 Cálculo del ruido temporal en una imagen	88
Figura 5-40 Esquema ruido de lectura de un píxel	89
Figura 5-41 Esquema ruido de reset en un píxel	89
Figura 5-42 Ruido reset en un píxel	90
Figura 5-43 Imágenes con ruido de patrón fijo o espacial y tras aplicar correcciones	91
Figura 5-44 DSNU y PRNU en la curva de ruido espacial	91
Figura 5-51 Información de los píxeles aportada por un histograma	92
Figura 5-52 Análisis de ruido en el histograma de una imagen	92
Figura 5-45 Diferentes ruidos en una misma imagen real	93
Figura 5-46 Set-up para medida de ruidos	93
Figura 5-47 Cálculo ruido espacial en un sensor lineal	94
Figura 5-48 Cálculo nivel de señal y el ruido en una imagen	94
Figura 5-49 Curvas de componentes espacial y temporal del ruido de patrón fijo	95
Figura 5-50 Distribución ruido PRNU	96
Figura 5-53 Fuentes de ruido DSNU	97

Figura 5-54 Conjunto de ruidos y artefactos en un CIS	97
Figura 5-55 Imágenes con y sin efecto Blooming	98
Figura 5-56 Medida de Blooming	98
Figura 5-57 Efecto Lag	99
Figura 6-1 Entorno Python de trabajo	101
Figura 6-2 IDE Spyder	102
Figura 6-3 Librería creada para caracterización y simulación	103
Tabla 6-1 Descripción funciones y scripts creados	103
Tabla 6-2 Sensores simulados para la comparativa	104
Tabla 6-3 Comparativa performance electro-óptica para los dos casos en estudio	105
Figura 6-4 Ejemplo uso MikTex	108

GLOSARIO Y ABREVIATURAS

Término	Unidades	Descripción
CIS	-	Sensor de Imagen CMOS
A		Area del píxel
c	m/s	Velocidad de la luz
DSNU	DN	No uniformidad en oscuridad
DR	dB, bits	Rango dinámico
E	W/m ² , Photons/píxel	Irradiancia
H	Js	Constante de Planck
K	DN/e ⁻	Ganancia del sistema total
PRNU	%	No uniformidad en la foto-respuesta
SNR	dB, bits	Relación señal a ruido
t_{exp}	s	Tiempo de exposición
QE	%	Eficiencia cuántica
λ	nm	Longitud de onda de la luz
μ	-	Valor medio
σ	-	Desviación estándar temporal
σ²	-	Varianza temporal
DS	e ⁻	Corriente oscura
p	-	Número de fotones
q	-	Ruido de cuantización
3T / 4T / 5T	-	Píxel con arquitectura de 3, 4, 5 transistores
CCD	-	Charge Coupled Device
CTE	-	Eficiencia en la transferencia de carga
CTF	-	Función de transferencia de contraste
VFPN	%	Ruido de patrón fijo vertical
HFPN	%	Ruido de patrón fijo horizontal
MTF	-	Función de transferencia de modulación
LSF	-	Función de extensión de línea
ESF	-	Función de extensión de borde

1 INTRODUCCIÓN

1.1 Motivación al trabajo y objetivos

Tener procedimientos de medida precisos y estándar es un concepto clave a la hora de tener un exitoso diseño de sensores de imagen. La medida de cualquier sistema opto-electrónico es un arte en sí mismo y debe ser preciso y acertado. Con el crecimiento de la tecnología en circuitos integrados, gran cantidad de técnicas avanzadas de diseño se han ido incorporando en los diseños de CIS, esto convierte en todo un reto determinar correctamente los parámetros característicos de cada sensor por lo que es aún más necesario la integración y estandarización de procedimientos y más si cabe en grandes multinacionales en las que se trabaja desde diferentes puntos del mundo sobre una misma tecnología.

La principal motivación de este trabajo fue una comprensión profunda de los CIS desde el proceso de fabricación hasta el test y caracterización final y el poder crear un nuevo conjunto de librerías para la caracterización de sensores basado en Python mostrando su potencial y adaptabilidad en detrimento de MatLab, LabView y otros softwares de carácter privado empleados en la actualidad.

Este trabajo tiene dos objetivos claros, por una parte pretende ser un documento de referencia, el cual me hubiese gustado encontrarme a mi llegada a Teledyne-AnaFocus allá por 2015 para poder en poco tiempo ponerme al día tanto con la tecnología como con lo que es en sí el test y caracterización de un circuito integrado como el principio de una nueva metodología de desarrollo de caracterización de sensores de imagen basado en un lenguaje de propósito general y gratuito como es Python con el objetivo de migrar desde Matlab en las oficinas de Sevilla.

Gracias a los 3 años que llevo en Teledyne-AnaFocus y el más de medio año que estuve en Teledyne-e2v en Grenoble (Francia) he podido recopilar diferentes visiones y métodos de trabajo de distintos grupos sobre un producto común lo que hace que este documento tenga además un tercer propósito, de integración y potenciación de sinergias entre ambas empresas, al haber sido adquiridas por una matriz mayor recientemente como es la americana Teledyne en 2017.

Este proyecto presenta el estado actual de los sensores de imagen CMOS así como sirve de guía tanto para poder testar funcionalmente un sensor a nivel de circuitería interna integrada como para caracterizarlo teniendo en cuenta la variabilidad y los agentes externos que influyen en el diseño de sensores antes de su paso a producción masiva o entrega al cliente final. Por último este proyecto incluye una librería software de procesamiento de imágenes en Python para este propósito.

1.2 Organización del trabajo

Este trabajo consta de 7 capítulos. Éste primero de ellos proporciona una introducción del proyecto e ilustra los objetivos, motivaciones y estructura del mismo.

El capítulo 2 incluye el estado del arte de sensores de imagen CMOS así como su arquitectura interna y principios de funcionamiento e interfaces de comunicaciones con el exterior, fundamentos ópticos a tener en cuenta y una introducción a estándares de medida como la EMVA1288.

El capítulo 3 desarrolla la importancia de la instrumentación de laboratorio necesaria a la hora de caracterizar sensores CMOS y testarlos funcionalmente así como los distintos instrumentos y estaciones de trabajo. Este capítulo es seguido por el capítulo 4 el cual explica todas las pruebas funcionales a realizar idealmente, eléctricos, de consumo, visuales, de integridad de señal, así como las principales funcionalidades internas del sensor tanto a nivel de circuito integrado como CIS en general, aunque esto último variará en cada diseño. Por

otra parte el capítulo 5 incluye todas las medidas y procedimientos para una caracterización óptica completa, escenarios, datos a extraer y procedimientos de medida. Llegando al final, el capítulo 6 contiene los resultados en dos casos concretos de dos sensores con características diferentes concebidos para una misma aplicación de manera que se pueda comparar ventajas y desventajas de ambos diseños, todo ello apoyándose en simulaciones y procesamiento de los datos a través de Python.

Finalmente el último capítulo incluye las conclusiones del trabajo y líneas maestras a desarrollar en un futuro en este campo de la ingeniería de test de circuitos integrados en general y sensores de imagen CMOS en particular. Así como funcionalidades a añadir en la librería desarrollada en Python.

Además tras el último capítulo se incluyen apéndices con información adicional que he utilizado durante la elaboración del trabajo como el código Python para simular el comportamiento de un CIS y su posterior medida de performance óptica, cómo generar reportes automáticos a partir del análisis de los datos de caracterización.

2 GENERALIDADES SOBRE SENSORES DE IMAGEN CMOS

Este capítulo tiene como objetivo dar una breve introducción a los sensores de imagen CMOS (CIS), su funcionamiento, arquitectura interna, modos de captura, interfaces de comunicaciones para la recuperación de datos y una pequeña introducción a estándares oficiales de caracterización.

2.1 Introducción

2.1.1 Principios básicos

Los CIS son dispositivos semiconductores empleados en la fabricación de cámaras digitales. Detectan la información en forma de luz u otra radiación electromagnética y crea una imagen que representa dicha información. De manera simplificada, esta conversión consiste en un circuito integrado que capta la información y la convierte en una tensión equivalente que más tarde es traducida a un dato digital.

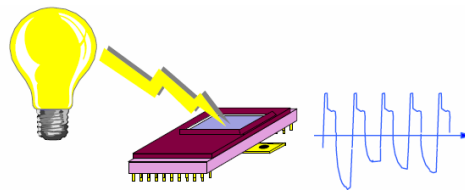


Figura 2-1 Fotoconversión

Existen dos clases de sensores en función de su aplicación y diseño; lineales y matriciales o de área. Los sensores lineales se utilizan para aplicaciones de escaneo a muy alta velocidad de objetos en movimiento y los de área para el resto de aplicaciones como podría ser tecnologías 3D como el Time of Flight o LIDAR. Además, las aplicaciones de los sensores de imagen pueden variar también en función de su espectro de visión, es decir la longitud de onda que capta con mayor sensibilidad, en nuestro caso vamos a centrarnos en CIS que en principio trabajen dentro del espectro visible.

2.1.2 Funcionamiento

Veamos cómo funciona la cadena de adquisición de una imagen en una cámara que integre un CIS:

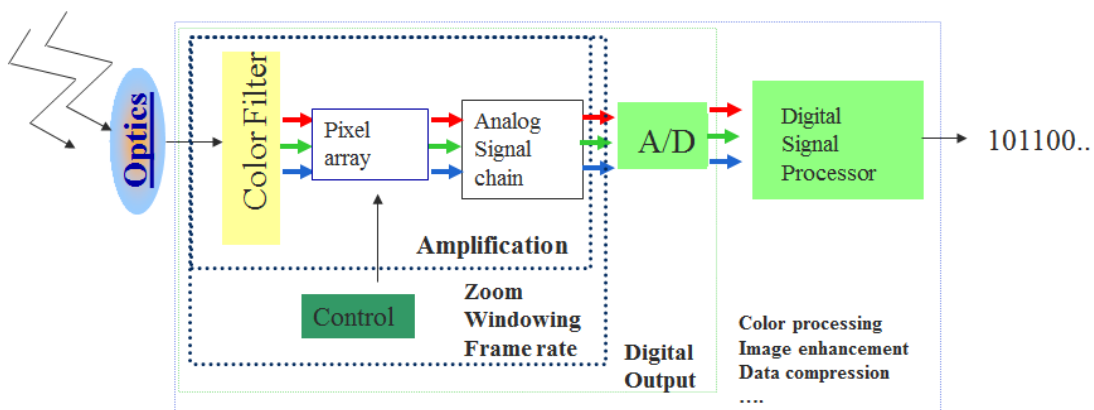


Figura 2-2 Proceso de adquisición de imagen en un CIS

La primera interfaz física que se encuentra en una cámara siempre es la óptica, la cual direcciona la luz hacia el CIS haciéndolo pasar por estructuras dedicadas a la aplicación del sensor como pudiesen ser filtros de colores para tener una cámara cromática o microlentes para obtener la mayor cantidad de información de la imagen. Tras este paso se llega al sensor donde se produce todo el proceso de adquisición de fotones y posterior conversión a señal digital por un convertidor analógico-digital. Además se le puede dotar al sensor de inteligencia como podría ser capacidad de amplificar la señal, seleccionar regiones de interés, balance de blanco y cualquier algoritmo de procesamiento. Aunque lo usual para cualquier fabricante es añadir estos algoritmos a nivel de cámara fuera del chip embebido en una FPGA o dispositivo microprocesador.

El principio físico de adquisición de una imagen es tan básico como el principio fotoeléctrico el cual consiste en la emisión de electrones por un material, en este caso el silicio, al incidir sobre él una radiación electromagnética como puede ser la luz. Al excitar el silicio con la luz los electrones se les dota de energía de tal manera que si están cerca del nivel de Fermi y se les proporciona la energía suficiente para superarlo, se desprende del material y puede ser almacenado.

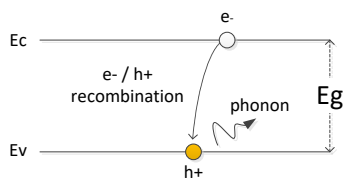


Figura 2-3 Recombinación por electrón-hueco

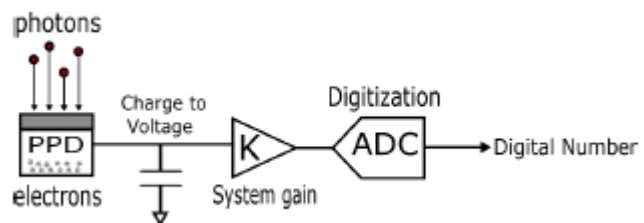
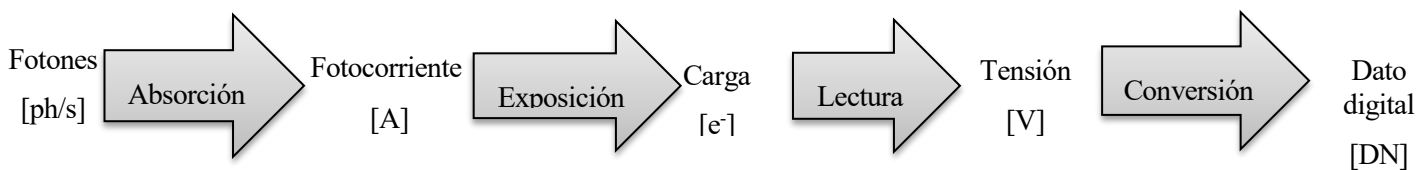


Figura 2-4 Esquema de conversión fotoeléctrica

Veamos la descripción básica de esta conversión fotoeléctrica y cómo se aprovecha para crear una información digital en forma de imagen:



1. Conversión foto-eléctrica
El fotodiodo capta los fotones incidentes y almacena los electrones durante un tiempo de integración, también llamado tiempo de exposición.
2. Conversión carga-tensión
Se hace sobre una capacidad, que puede ser la parásita del fotodiodo en una estructura 3T o sobre una separada llamada Floating Diffusion (FD) en un píxel 4T o 5T.
3. Amplificación
Gracias a un transistor tras la capacidad, llamado seguidor de corriente o *source followr (SF)*.
4. Lectura
Un transistor de selección permite conectar línea por línea cada píxel a la columna y poder leer los datos.
5. Conversión analógico-digital
La señal analógica recolectada es seguida por un convertidor analógico-digital (ADC) el cual determina la resolución del sensor.

Por ejemplo, este sería el esquemático más básico de un píxel conceptualmente y su comportamiento frente a la luz y un tiempo de exposición variable.

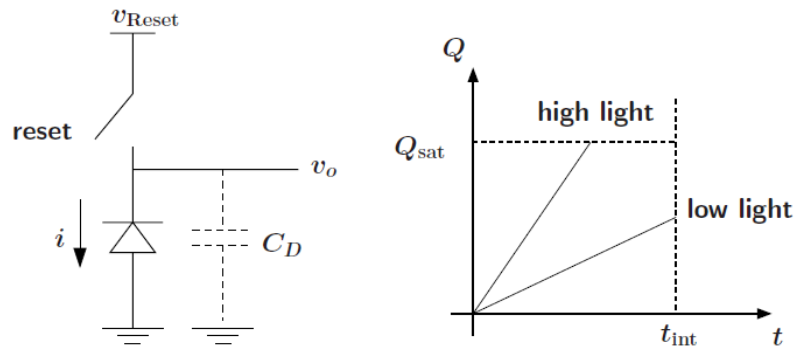


Figura 2-5 Esquemático simplificado de un píxel

A la hora de adquirir una imagen acumulando carga en el fotodiodo hay dos eventos esenciales y que vale la pena explicar:

- Fase de reset
La fase de reset provoca una limpieza del píxel estableciendo su potencial al potencial fijado por el transistor de reset, generalmente una tensión digital limpia de ruido y problemas de integridad.
- Fase de integración
Durante la fase de integración o tiempo de exposición, el nivel de carga continua en el fotodiodo varía en función del nivel de luz captado por el píxel.

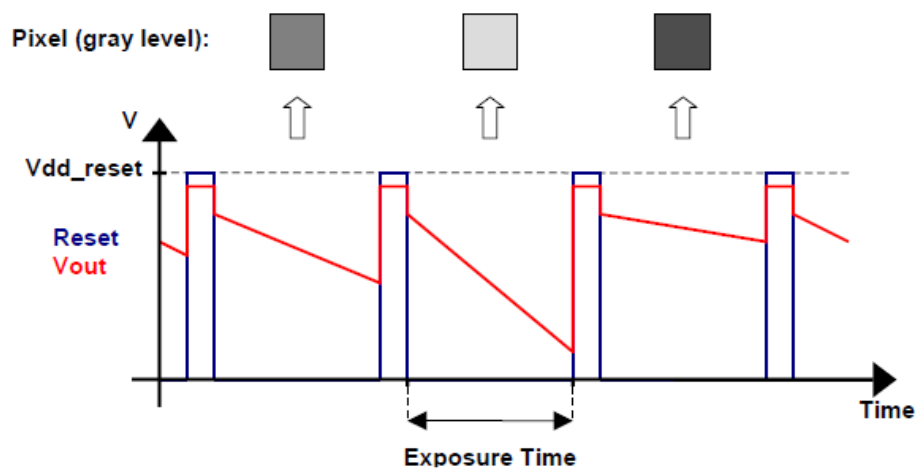


Figura 2-6 Diagrama de potencial de un píxel

2.2 Fundamentos de los Sensores de Imagen CMOS

2.2.1 CCD vs CMOS

En la actualidad, los sensores de imagen CMOS han superado a los CCDs en la mayoría de campos. Un sensor CMOS es una tecnología integrada que ofrece un amplio rango de funcionalidades, rápida lectura, bajo consumo, bajo coste y mejores parámetros característicos.

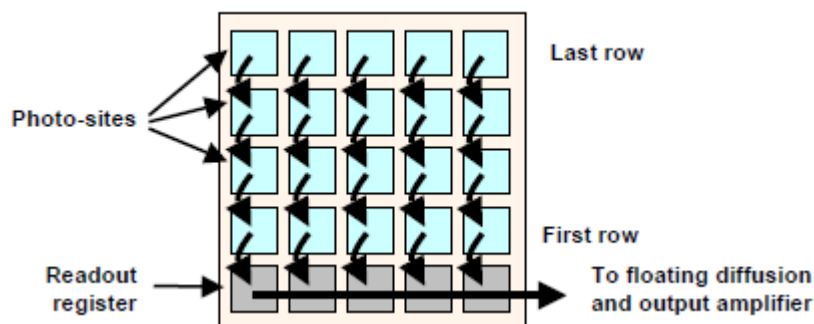


Figura 2-7 Funcionamiento sensor CCD

Aunque los CCDs ofrecen una imagen excelente, su proceso de fabricación está dedicado a elementos fotosensibles en vez de transistores y por lo tanto esta es una dificultad importante a la hora de implementar el avance de la tecnología en los transistores usando la tecnología CCD. Sin embargo, el mismo rendimiento puede ser conseguido usando sensores CMOS, es incluso posible implementar todos los bloques funcionales embebidos en el sensor, por ejemplo, control del sensor on-chip, técnicas de doble muestreo correlado, circuitería para la supresión de ruidos de patrón fijo, etc.

El estado del arte en el diseño de sensores de imagen CMOS a día de hoy consiste en sensores de píxel activo, es decir, en un circuito integrado compuesto por una matriz de píxeles y unidades de procesamiento de señal donde además se produce una amplificación de señal dentro del píxel, de ahí la nomenclatura de activo. Idealmente un sensor de imagen CMOS debería poseer; una eficiencia cuántica del 100%, es decir, que por cada fotón incidente se genere un par electrón-hueco, alta velocidad de captura, bajo ruido, respuesta lineal, sin diafonías o acoplos y sin artefactos en las imágenes, pero veremos que esto no siempre es así.

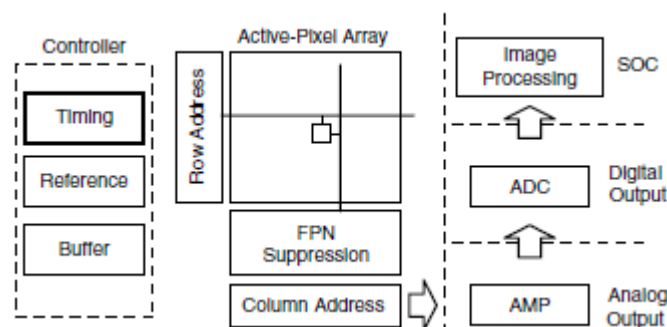


Figura 2-8 Funcionamiento sensor CMOS

Veamos a continuación las principales diferencias entre sensores CMOS y CCD.

Tabla 2-1 CCD vs CMOS

	CCD	CMOS
Ventajas	Excelentes prestaciones: resolución, ruidos, uniformidad, sensibilidad Fill Factor o eficiencia en torno al 90% Corriente oscura reducida (5 pA a 0.5 nA/cm²) Tecnología muy madura	Tecnología CMOS estandarizada Consumo moderado Región de interés seleccionable Permite integrar conversión A/D en el chip Alta velocidad Alta escala de integración de circuitos de procesamiento de señal Coste razonable del Silicio (para volúmenes importantes)
Inconvenientes	Tensiones de alimentación elevadas (~15V)	Eficiencia del sensor en torno a un 60%

	Alto consumo Ruido por transferencia de carga	Ruido debido al mismatch entre transistores. Ruido kTC Corriente oscura elevada ($1\text{nA}/\text{cm}^2$ standard, $50\text{pA}/\text{cm}^2$)
--	--	--

2.2.2 Ejemplos de layout de un CIS

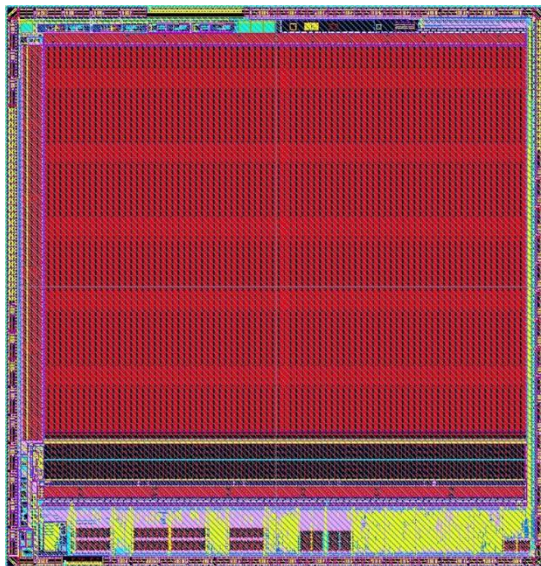


Figura 2-9 Layout CIS de área

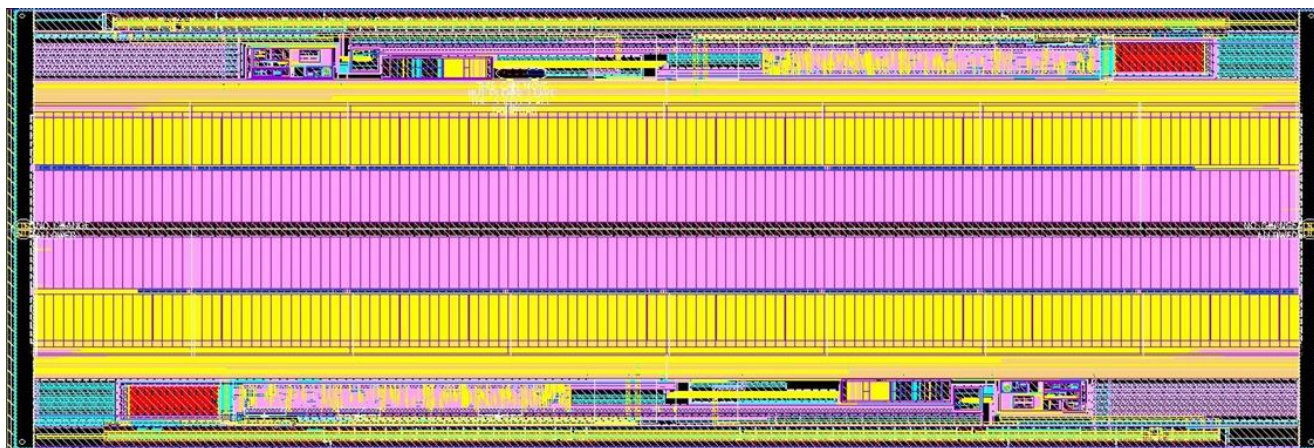


Figura 2-10 Layout CIS lineal

2.2.3 Principales características de los CIS

Las principales características de los sensores de imagen y sobre las que pivotarán el resto de capítulos que siguen son las siguientes:

- Rango dinámico; o rango de niveles de iluminación que puede ser representado en una imagen, en dB.
- Sensibilidad; que es la mínima cantidad de exposición detectable por el sensor, en Lux·s. Es el primer parámetro que te permite directamente comparar dos sensores de imagen en condiciones de baja iluminación.
- Relación señal a ruido.
- Ruidos que provocan artefactos en las imágenes o deterioran su calidad.
- Función transferencia de modulación; permite medir la resolución espacial de un sensor.

2.3 Arquitectura del píxel

2.3.1 Fotodiodo

El fotodiodo es un dispositivo semiconductor que convierte la luz en corriente. El funcionamiento del fotodiodo se basa en el efecto fotoeléctrico que establece que cuando un fotón con suficiente energía incide en un fotodiodo este crea un par electrón-hueco. A partir de esta transformación se crea un campo eléctrico en el fotodiodo que polariza el diodo de manera que conduzca corriente. Un fotodiodo enclavado (pinned) es una variación en la estructura del fotodetector con una gran región de deplexión la cual permite bajo ruido, baja corriente oscura y una eficiencia elevada.

El fotodiodo pinned (PPD) tiene regiones p⁺/n/p con un implante superficial de p⁺ en una capa de tipo n sobre un sustrato epitaxial de tipo p. El enclavamiento se refiere a cuando el nivel de Fermi se fija a un cierto voltaje. Un PPD es diseñado para tener una región de recolecta suficientemente grande para que se vacíe al resetearlo. Cuando el PPD se vacía, este se aísla del circuito de lectura y se drenará toda la carga fuera de región de recolecta, este proceso se llama transferencia de carga. Una de las mayores virtudes es que este diodo puede tener un completo estado de vaciado, lo cual permite aplicar la técnicas de eliminación de ruido como el doble muestreo correlado (CDS). Por otra parte, la capa fijada p⁺ disminuye la corriente oscura previniendo a la interfaz de ser agotada y absorbiendo todos los portadores que se generan en la superficie aislándolos de la región de deplexión. Cuando se diseña la región de deplexión de un PPD para que se vacíe a un cierto voltaje, se está enclavando el PPD a ese voltaje.

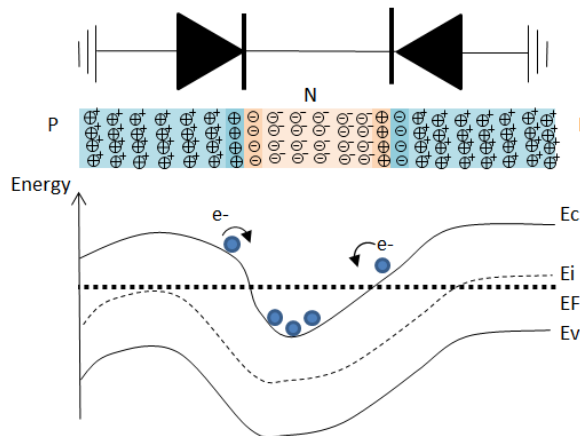


Figura 2-11 Estructura básica fotodiodo pinned

La tensión V_{pin} establece el punto de potencial a partir del cual se acumulan cargas en el fotodiodo pinned. Si este valor es demasiado bajo, la saturación del píxel (Q_{SAT}) es igualmente baja debido a que el fotodiodo no puede acumular mucha carga.

$$[1] \quad Q_{SAT} = C_{PhD} * \Delta V = C_{PhD} * (V_{pin} - V_{TRA})$$

Donde,

C_{PhD} : capacidad parásita del fotodiodo que permite almacenar electrones.

V_{TRA} : tensión a partir de la cual los electrones empiezan a transferirse desde el PPD.

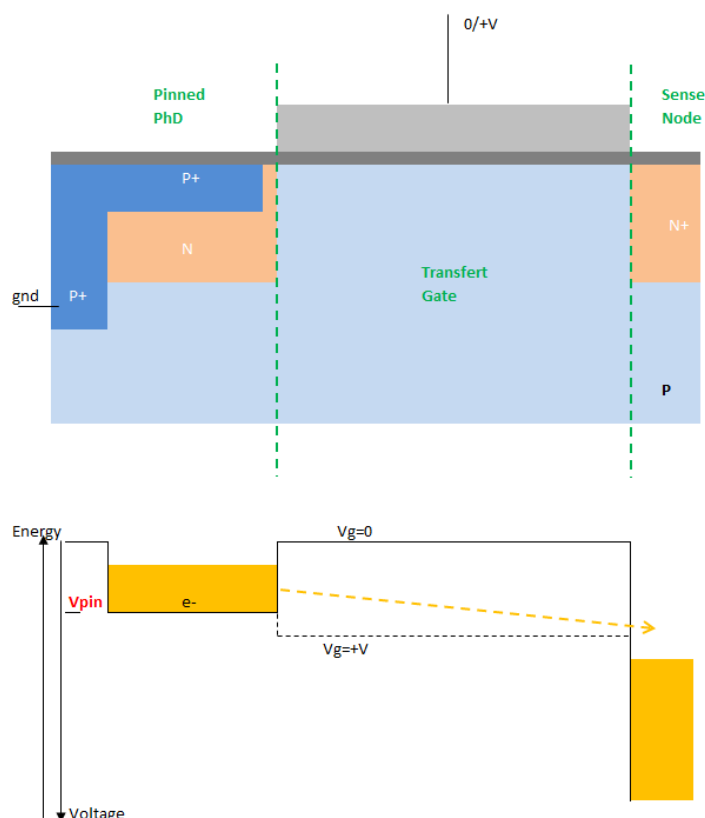


Figura 2-12 Diagrama de polarización dentro de un píxel

2.3.2 Píxel 3T

Un CIS actual utiliza una arquitectura de píxel activo 4T para el sensado de luz y su lectura. Antiguamente, los CIS solían tener únicamente un fotodiodo en el píxel encargado de la recolección de carga y más tarde llegó el píxel pasivo (PPS) el cual incluía un fotodiodo y un selector de fila para posteriormente amplificar cada columna de píxeles. La principal ventaja de los PPS era la de ser un píxel de pequeño tamaño pero a su vez la lectura de columna era lenta y con bastante ruido. El píxel activo (APS) consiste en un fotodiodo, un selector de fila y una amplificación a nivel de píxel lo que genera una lectura más rápida y bajo ruido.

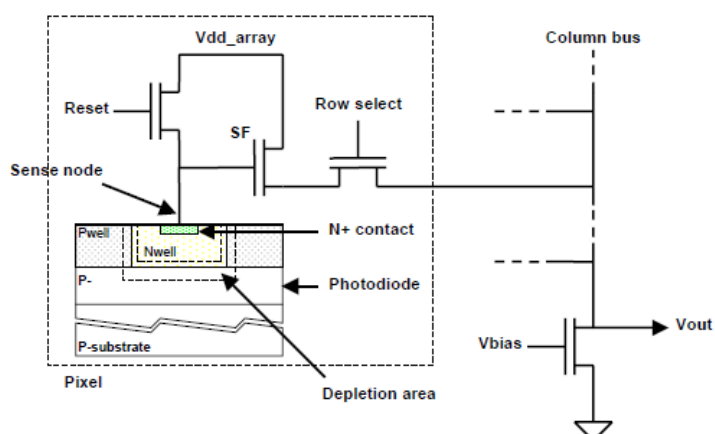


Figura 2-13 Píxel 3T

2.3.3 Píxel 4T

Aunque existe la única diferencia de un transistor entre la arquitectura 3T y 4T del píxel, hay una diferencia significativa en prestaciones del píxel. La siguiente figura muestra el diseño y el principio de funcionamiento de un píxel 4T. El esquema de un píxel 4T consiste básicamente en un fotodiodo p+/n/p, un transistor de

transferencia de carga (TG) desde el PPD hasta un nodo de difusión flotante (FD) de capacidad C_{FD} para almacenar carga del fotodiodo. Además, posee un transistor de reset para resetear el nodo FD tras cada lectura, un seguidor de corriente que actúa como buffer, y de amplificador que aísla el nodo de sensado el canal de lectura además de un transistor de selección de fila.

La principal diferencia entre el píxel 3T y el 4T es el transistor de transferencia TG, el cual separa el PPD de la FD. Por lo tanto, un píxel 4T permite almacenar la señal lo que nos posibilita el poder hacer concurrencia entre integración de señal y lectura del dato anterior; es decir, leer la señal de un frame anterior mientras se está exponiendo el siguiente frame. Lo cual proporciona más velocidad de lectura y mayor SNR. Además, a diferencia del píxel 3T, el 4T permite implementar la técnica CDS, la cual es útil a la hora de eliminar el ruido de reset, esta técnica se discutirá más adelante. La otra ventaja de la TG es que previene de diafonías e interferencias con los píxeles vecinos mitigando efectos como el blooming (exceso de carga en los píxeles vecinos cuando saturan que hace que se contaminen sus píxeles adyacentes).

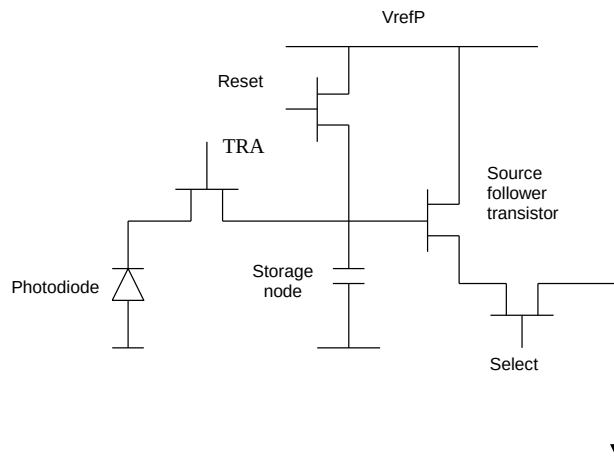


Figura 2-14 Píxel 4T

2.3.4 Píxel 5T

El píxel 5T hereda todas las características del 4T pero añade un transistor de reset global el cual permite un control de blooming para evitarlo y liberar la señal de reset a la hora de limpiar el fotodiodo de manera que pueda ser utilizada para otros menesteres como pudiese ser crear etapas intermedias de reset de manera que se amplíe el rango útil del píxel, lo que viene a ser incrementar el rango dinámico del mismo.

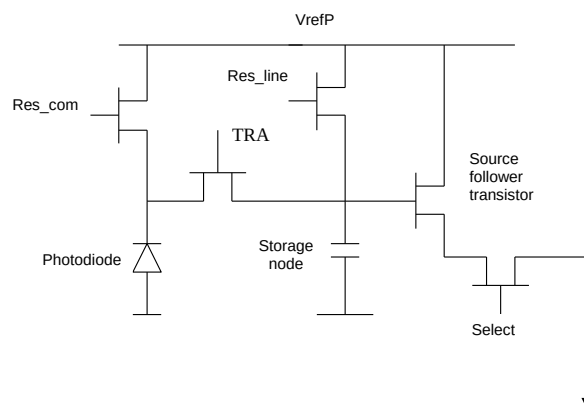


Figura 2-15 Píxel 5T

2.4 Operación interna del píxel

La operación y cronograma de un píxel APS 5T se muestra en la próxima figura. La figura muestra cómo el potencial de todos los componentes del píxel 5T va cambiando durante la operación del píxel y el cronograma de las distintas señales que intervienen en la estimulación del píxel para obtener la señal.

El primer paso comienza con un periodo de integración de la señal en el que el fotodiodo genera carga libremente acorde a los fotones incidentes y al mismo tiempo el nodo FD es reseteado para que la nueva lectura no esté influenciada por ninguna carga de la lectura anterior, por lo cual se establece un nivel de reset que fijará el piso de ruido del píxel. Ahora la TG se activa de manera que toda la carga acumulada en el fotodiodo se transfiere a la FD y más tarde al buffer donde esta carga se convierte en una señal de tensión la cual es muestreada gracias a un circuito de muestro (*Sample and Hold*) activado por el transistor de selección.

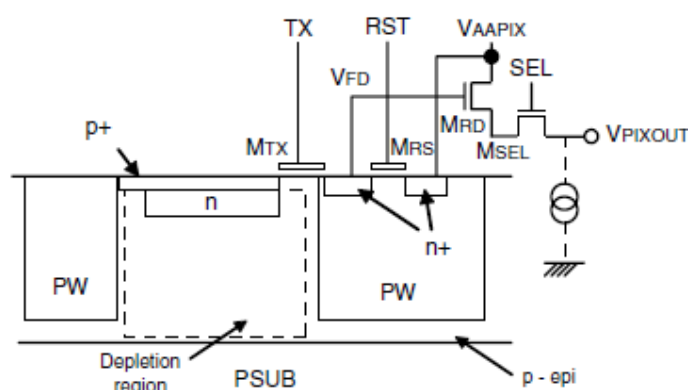


Figura 2-16 Pixel 5T desde punto de vista físico

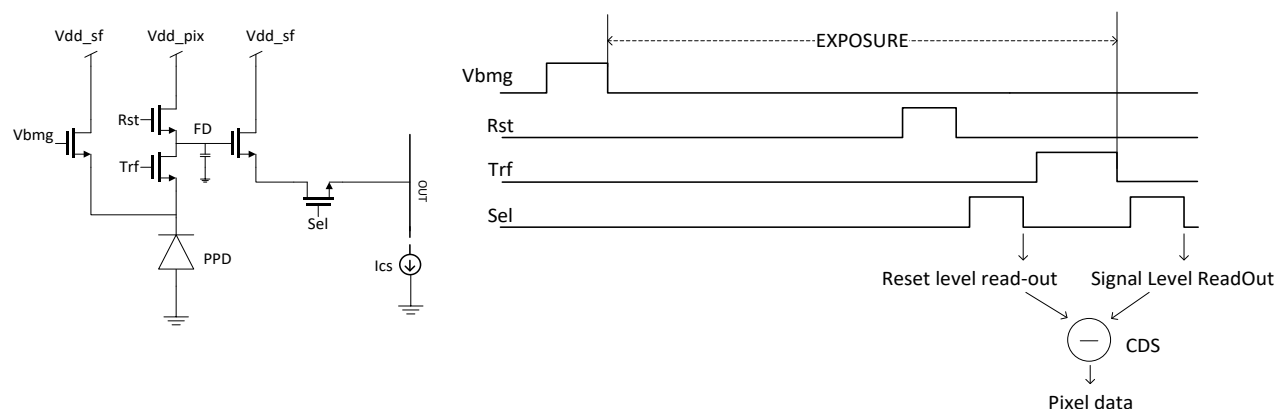


Figura 2-17 Cronograma básico píxel 5T

2.4.1 Técnica de doble muestreo correlado

Hay ciertas técnicas usadas durante la lectura de la señal para mejorar las prestaciones del sensor. En la técnica del doble muestreo correlado (CDS), el píxel es leído dos veces, una para el nivel de señal de reset y otra tras la transferencia de carga, y la diferencia entre ambas señales es el valor de señal verdadero. Esta técnica tiene algunas ventajas, no solo elimina el ruido de reset (ruido kTC) y el offset de señal que exista en el píxel, sino que además suprime el ruido Flicker $1/f$ (el cual sólo es correlado para bajas frecuencias).

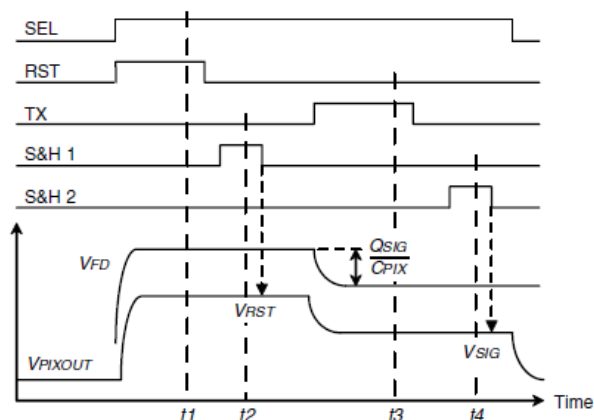


Figura 2-18 Cronograma técnica CDS en un píxel

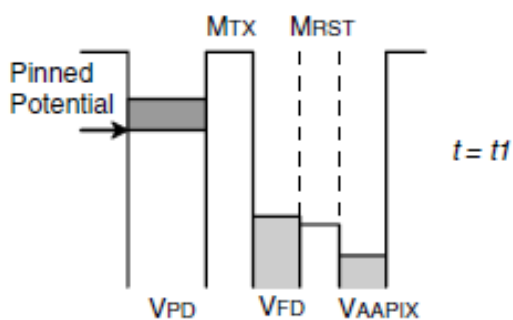


Figura 2-18.1 Fase de Reset del PPD

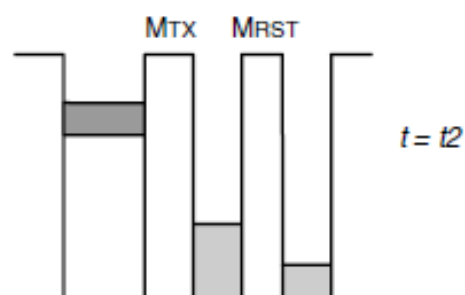


Figura 2-18.2 Fase de lectura del piso de ruido

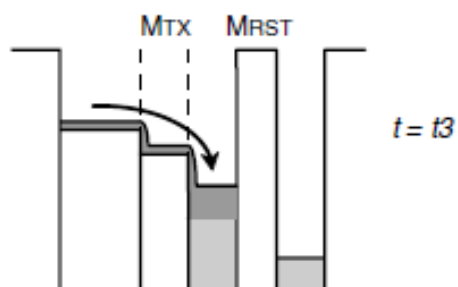


Figura 2-18.3 Fase de transferencia de carga a la FD

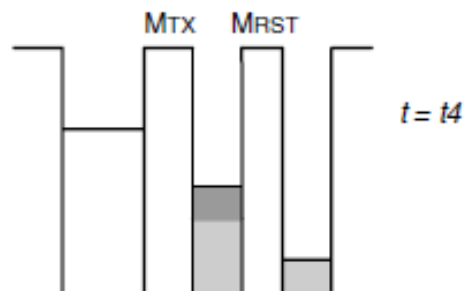


Figura 2-18.4 Fase de lectura de señal y ruido

A continuación vamos a explicar la forma de onda general de las señales que controlan el píxel, las cuales son generadas por máquinas de estados finitos internas al sensor y permiten una gran flexibilidad en la configuración de un sensor para adaptarse a multitud de aplicaciones.

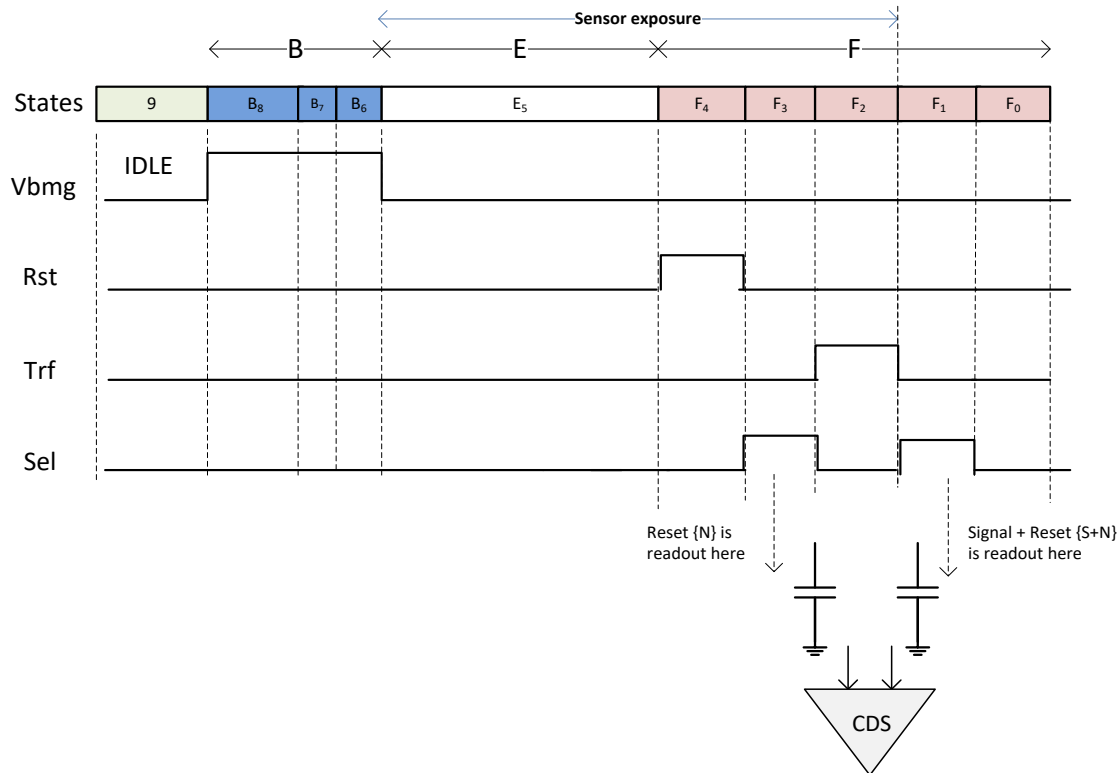


Figura 2-19 Máquina de estados de control del píxel

- Idle
Estado de espera del píxel, todas las señales tienen una señal de espera cuando no se está en operación.
- Estados B
Requisito para preparar la exposición, pues suele servir de reset global a todos los píxeles.
- Estado E
Exposición efectiva o tiempo que el sensor estará captando fotones.
- Estados F
Indica el final de la exposición y de lectura. Por ejemplo: activar la señal de RST para resetear la FD y leer el ruido de RST seguido de una transferencia de carga hacia la FD la cual requiere activar la señal TRF para transferir la carga desde el fotodiodo al nodo FD de lectura.

La duración de estos estados determinará el periodo de línea y este a su vez en función de la resolución del sensor, la velocidad de operación del mismo.

Otra técnica de muestreo es el muestreo correlado múltiple o falso CDS, en el cual se repite la operación del verdadero CDS durante diferentes instantes de tiempos y se suman, y la diferencia entre la media de los dos niveles se toma como valor de señal.

En el siguiente esquema se simplifica el canal de imagen de un sensor desde el píxel hasta la serialización de los datos habiendo convertido el dato analógico previamente y su posterior salida por su correspondiente interfaz de salida como podría ser la tecnología LVDS.

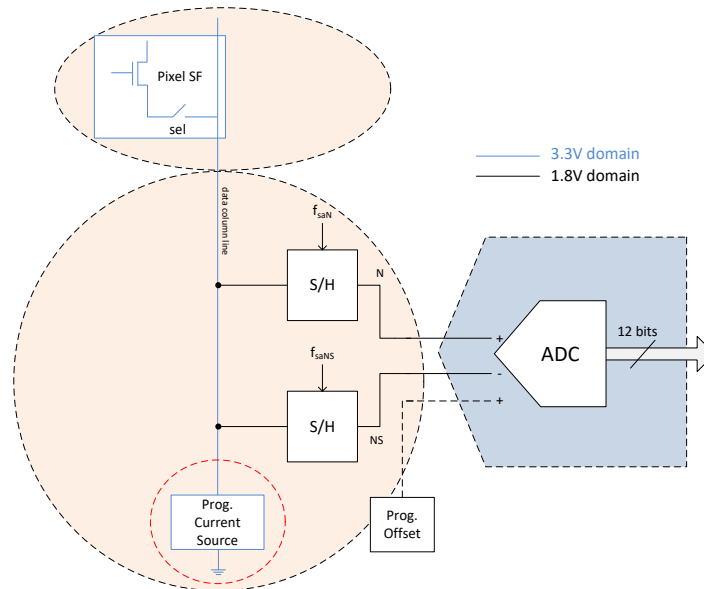


Figura 2-20 Canal de imagen de un CIS

2.4.2 Métodos de exposición

La elección de un modo de funcionamiento u otro a la hora de capturar la imagen dependerá directamente de la escena a captar y de la velocidad de los objetos en la misma. Por ejemplo, en cámaras de alta velocidad o de entornos industriales, el tiempo de integración deberá ser elegido para que la imagen del objeto no se desplace por más de un píxel durante el tiempo de exposición. Los CIS pueden operar en dos modos de captura llamados Global shutter y Rolling Shutter.

➤ Rolling Shutter

En este modo los píxeles son direccionados fila a fila, es decir, píxeles de una misma fila son expuestos/reseteados simultáneamente, luego los de la siguiente línea y así sucesivamente. Los píxeles de diferentes filas del array son expuestos y leídos fila a fila por lo que el tiempo total de un frame es el tiempo desde que comienza la exposición de la primera fila hasta la lectura de la última.

Como principal característica de este modo, tenemos que el tiempo de integración de cada línea no se produce en el mismo instante de tiempo.

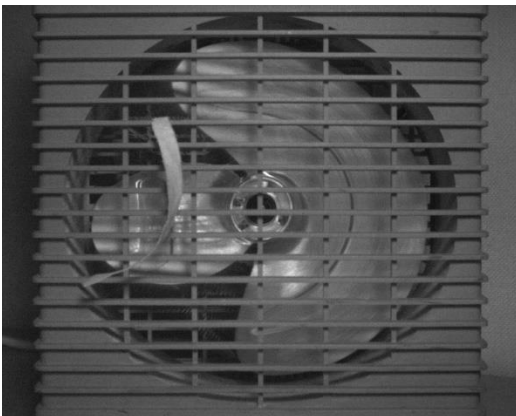


Figura 2-21 Imagen captada en Rolling Shutter

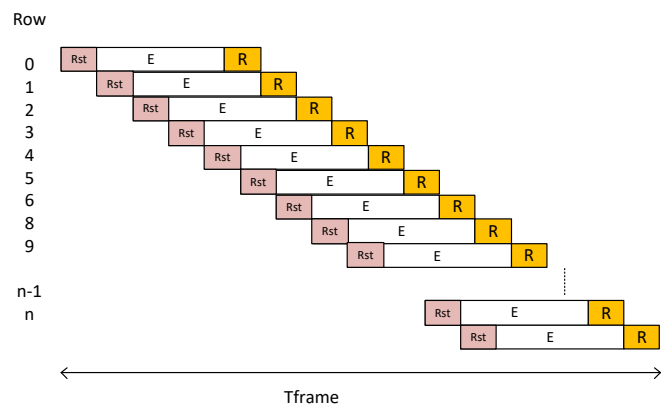


Figura 2-22 Rolling Shutter

Para la estructura de un píxel Rolling shutter basta con un píxel con tres transistores, además dota de

una buena relación señal a ruido. Aunque su principal inconveniente es la distorsión de la imagen cuando la velocidad de movimiento del objeto es grande y el periodo de integración es demasiado corto comparado con el tiempo de lectura.

➤ Global Shutter

En el modo Global shutter, todos los píxeles del sensor son expuestos y reseteados al mismo tiempo y la carga de cada fila es almacenada para posteriormente ser leída fila a fila.

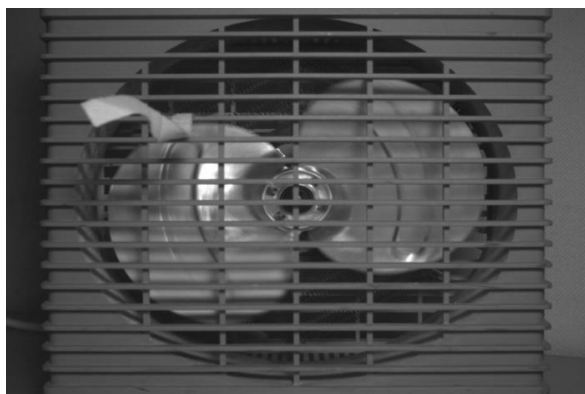


Figura 2-23 Imagen captada en Global Shutter

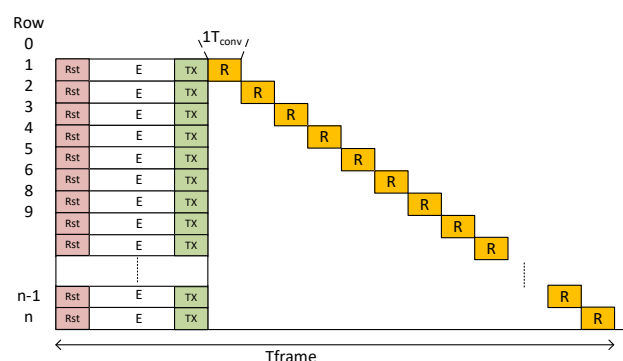


Figura 2-24 Global Shutter

Para el diseño de un píxel global shutter, habría que añadir un área de memoria a cada píxel a través de un transistor adicional el cual idealmente debería ser no sensible a la luz. En el caso de que este transistor no fuese totalmente insensible a la luz puede provocar efectos de lag, debido a una mala limpieza. Este modo de sensado únicamente es implementable en arquitecturas 4T o 5T, en las que se puede almacenar la carga en una capacidad independiente de la del PPD.

2.4.3 Interfaces de salida más comunes

Tabla 2-2 Interfaces de salida más comunes

	Descripción	Ventajas	Inconvenientes
LVDS	Bus paralelo de par de señales en el que la información se envía codificada en diferencia de voltaje, lo que significa que la señal es transmitida por duplicado por el emisor pero con diferente voltaje, una señal transmitida con signo positivo, mientras que la otra es transmitida con signo negativo.	Robustez Extendido en el mercado. Gran cantidad de frame grabbers incluyen esta interfaz. Bajo consumo	Limitación de la velocidad de transmisión a unos 80MHz Limitación de longitud del cable a unos 60m No existen conectores estándar Precio elevado de los frame grabber.
Camera Link	Es un protocolo de comunicación serie para transferencia de datos a alta velocidad. Serializa los datos y el sincronismo con hasta cuatro salidas de 8/10/12 bits.	Limitación de velocidad de hasta de 85MHz para la señal de reloj del píxel. Control serie RS232 incluido. Cables y conectores estándares.	Precio elevado de los frame grabber.

USB	Bus serie estándar industrial que define los cables, conectores y protocolos usados en un bus para conectar, comunicar y proveer de alimentación eléctrica entre computadoras, periféricos y dispositivos electrónicos	Protocolo ampliamente conocido. La alimentación puede ser proporcionada por el bus. No precisa frame grabber.	Limitación de velocidad a estándares 8Mb/s (USB 1.2), 80Mb/s (USB 2.0) y USB 3.0. Longitud del cable limitada a 5m a no ser que se utilice un repetidor. Ancho de banda compartido por todos los dispositivos que comparten el bus.
Gigabit Ethernet	Bus serie (fibra óptica o cable). Es una ampliación del estándar Ethernet (concretamente la versión 802.3ab y 802.3z del IEEE) que consigue una capacidad de transmisión de 1 gigabit por segundo	Uso de cable Ethernet. No existe limitación en la longitud del cable. No precisa frame grabber.	Limitación de velocidad a 1Gb/s (actualmente bajo desarrollo 10Gb/s). No siempre se respeta el estándar.

2.4.4 EMVA₁₂₈₈

La EMVA₁₂₈₈ es un estándar de medida desarrollado por la Asociación Europea de Visión Industrial (European Machine Vision Association) cuyo propósito es definir métodos de medida y caracterización de sensores de imagen y cámaras utilizados en aplicaciones en entornos industriales.

Para este propósito son necesarios varios requisitos:

- Una fuente de luz homogénea para las medidas de sensibilidad, linealidad y no uniformidades.
- Una cámara climática para las medidas de corriente oscura dependiente de temperatura.
- Banco óptico para las medidas espectrales y eficiencia cuántica en el espectro de luz visible (410nm – 1050nm) así como para las medidas de eficiencia parásita a la luz.
- Un mecanismo de luz pulsada sincronizada con la adquisición de imágenes para la medida de lag.
- Píxeles tapados en el área sensora con el propósito de medida de blooming.
- Plataforma rotatoria y banco óptico para medir la respuesta angular.
- Banco necesario para la medida de la resolución efectiva o MTF.

3 CARACTERIZACIÓN DE SENSORES DE IMAGEN CMOS

Este capítulo desarrolla la importancia de la instrumentación de laboratorio necesaria a la hora de caracterizar sensores de imagen CMOS y testarlos funcionalmente así como los distintos instrumentos y características de los mismos que influyen en la medida.

3.1 Introducción

Si echamos un vistazo en perspectiva a la caracterización de CIS encontraremos los siguientes objetivos:

- Búsqueda del punto de operación.
- Medidas electroópticas estándar: EMVA1288, señal oscura, ruidos temporal y de patrón fijo.
- Medidas electroópticas específicas: MTF, QE, PSRR, eficiencia del shutter, lag, blooming, etc.
- Validación funcional: secuencias internas, bloques digitales, consumo, salida LVDS, etc.

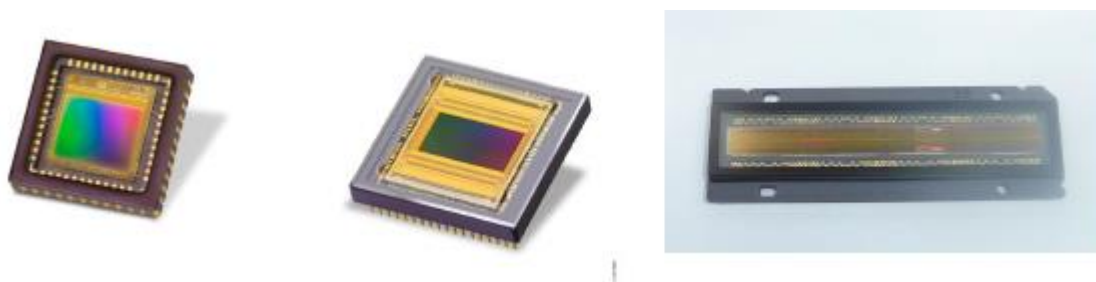


Figura 3-1 Sensores de Imagen CMOS de área y línea

3.2 Laboratorio típico de trabajo

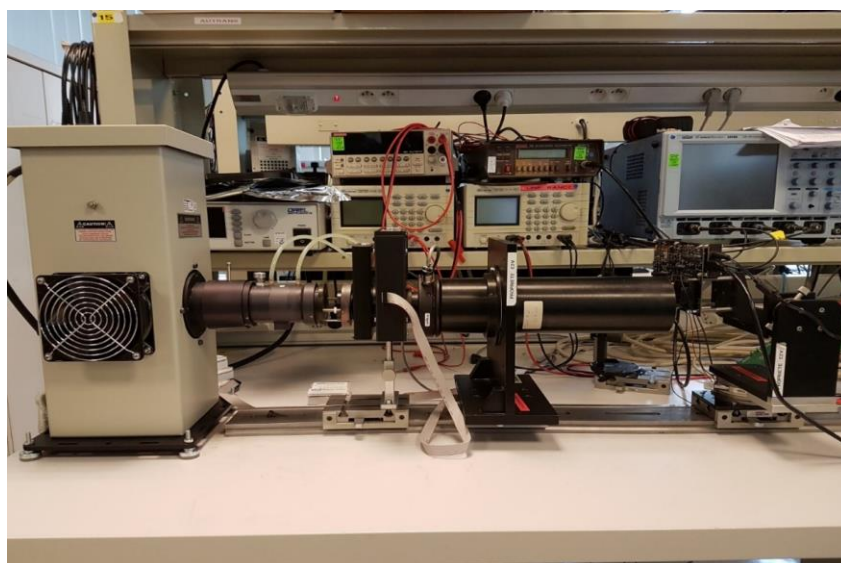


Figura 3-2 Banco típico de caracterización

Tabla 3-1 Instrumentación típica de un banco de caracterización

Descripción	Fabricante	Cantidad	€/ud	TOTAL €
PC	DELL	1	2000	2,000
Frame grabber NI Cam-Linkfull / NI PCIe-1433	NATIONAL INSTRUMENTS	1	2600	2,600
Cable, Power over Camera Link (PoCL), MDR to MDR, 5M	NATIONAL INSTRUMENTS	2	180	360
IRIS motorizado	NEWPORT	1	3400	3,400
SHUTTER motorizado	NEWPORT	1	2000	2,000
Controlador de shutter, RS232, Requiere pulso TTL externo	NEWPORT	1	2100	2,100
Cable GPIB 6M	RADISOSPARES	6	110	660
Múltiples filtros, 2.0 in. Filters, 1.5 Inch Series Flange	NEWPORT	1	130	130
Fuente de luz QTH, 1000 W QTH, Research Lamp Housing, F/1	NEWPORT	1	7350	7,350
Líquido refrigerante fuente de luz, 2 Inch Series	NEWPORT	1	950	950
"R&S HMC8043-G" Fuente alimentación, Programmable, 3 salidas 0 V, 32 V, 0 A, 3 A	FARNELL	2	1050	2,100
Multímetro digital Keysight Technologies 34401A, 3A ac 750V ac, RS	RADISOSPARES	1	1550	1,550
KEITHLEY 6485/E Multímetro digital, corriente DC, 20 mA	FARNELL	1	2600	2,600
Difusor de luz 50.8mm 25°	NEWPORT	1	600	600
Etapa de rotación con ajuste micrométrico, Ø46 mm, ±2° Fine Adjustment	NEWPORT	1	690	690
Montura óptica, 101.6 mm, 12.7 mm Dia. Molded Composite, M4, M6	NEWPORT	1	10	10
Estructura mecánica del tunel óptico	e2v	1	5000	5,000
			TOTAL	34,100

3.3 Fuente de led

Los diodos emisores de luz (LEDs) ofrecen un gran número de ventajas frente a las fuentes de luz convencionales, como pueden ser un bajo consumo de potencia, mejor pureza espectral, gran esperanza de vida y bajo coste. Con el rápido desarrollo de la industria LED durante las pasadas décadas, los LEDs se han popularizado en un creciente número de aplicaciones y son considerados clave para reemplazar las fuentes de luces convencionales.

Una matriz de luces LED suele ser utilizada para la caracterización de sensores de imagen CMOS, por lo que es esencial conocer sus características. Las características clave de las fuentes de luz LEDs son la uniformidad espacial, la estabilidad térmica y temporal y su respuesta espectral, la cual ayuda a la hora de caracterizar los sensores de imagen.

Es un factor clave poseer una fuente de luz estable y uniforme ya que afecta directamente a la medida de las prestaciones de los sensores de imagen CMOS.

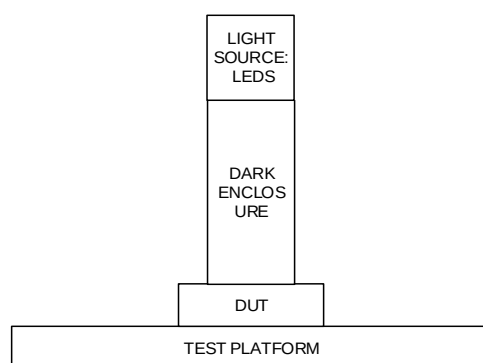


Figura 3-3 Fuente de luz para caracterización

3.3.1 Estabilidad

Las fuentes de luz idealmente emiten una potencia óptica constante con una intensidad constante también. Esto no debería variar dependiendo de la temperatura o el tiempo, por lo que una fuente de luz debe tener una buena estabilidad térmica y temporal. Los LEDs precisan de cierto tiempo para ser térmicamente estables, por lo que es importante conocer este tiempo. Veamos cómo medir esta estabilidad:

1. Medida de fotocorriente gracias a un fotómetro convencional el cual puede transformar la luz incidente en una fotocorriente equivalente (A) la cual puede ser convertida a su vez en intensidad de luz (W/cm^2) usando un factor de conversión diferente para cada longitud de onda y el cual se puede encontrar en la hoja de datos de la fuente de luz.
2. Por ejemplo en las siguientes gráficas se puede observar cómo la luz de LED tarda unos 10' en proporcionar una intensidad constante con una variación de $1\text{nW}/\text{cm}^2$ y ser térmicamente estable.

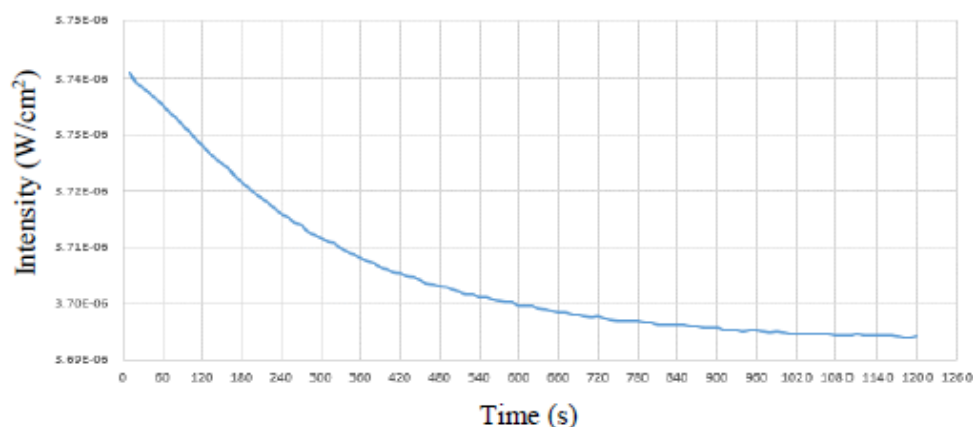


Figura 3-4 Estabilización de irradiancia en un led

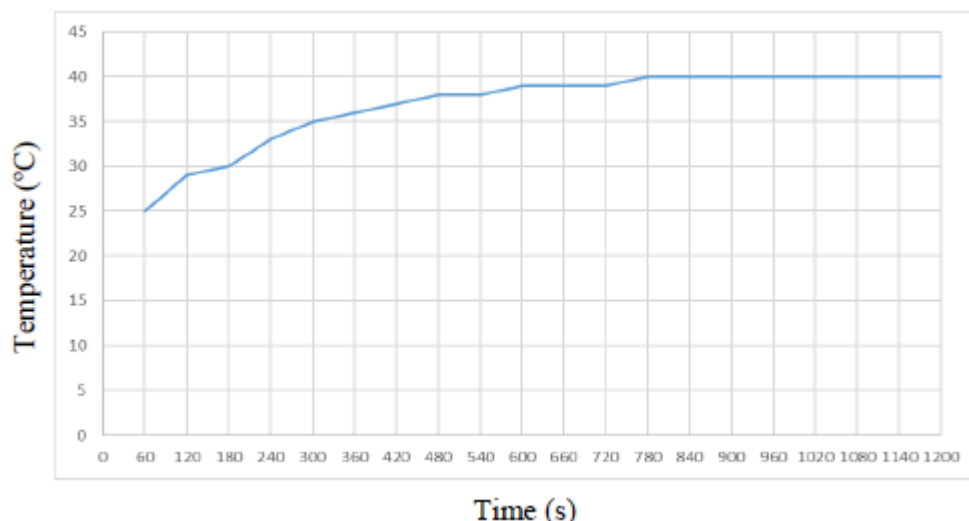


Figura 3-5 Estabilización térmica de un led

3.3.2 Respuesta espectral y uniformidad espacial

La respuesta espectral se define mediante la respuesta espectral de los LEDs. Generalmente, los proveedores de fuentes de iluminación LED proporcionan este tipo de información. Para la medida de la eficiencia cuántica es crucial conocer la respuesta espectral de la fuente de luz midiendo la fotocorriente barriendo la longitud de onda de un mon cromador con un paso de al menos 1nm para las tres longitudes de onda correspondiente a los tres colores primarios.

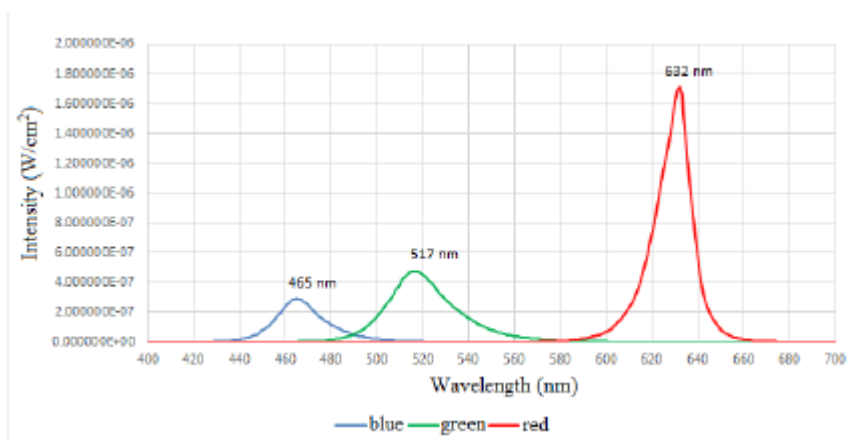


Figura 3-6 Respuesta espectral de la fuente de LEDs

La uniformidad espacial es una de las características más importantes de cualquier fuente de luz. Una fuente de luz LED debería ser espacialmente uniforme, es decir, poseer un mismo nivel de intensidad en el espacio a igual distancia de la fuente.

La medida de esta uniformidad es de carácter relativa, es decir normalizando respecto a la máxima intensidad medida por un fotodiodo calibrado. Lo ideal es medir a diferentes distancias como pueden ser 10cm, 20cm y 30cm entre la fuente de luz y el fotodiodo para poder caracterizar correctamente esta uniformidad.

Los resultados de procedimiento recién mencionado son las siguientes tres figuras. El mapa de calor en 3D muestra la intensidad de luz normalizada de la fuente de luz con el máximo en el centro y la mínima intensidad de luz en los bordes. Las gráficas en 2D muestra en escala de grises la intensidad de luz recibida por cualquier sensor y como se puede ver en un área de 2x2cm² la variación de uniformidad de nuestra fuente convencional es de menos de un 1%.

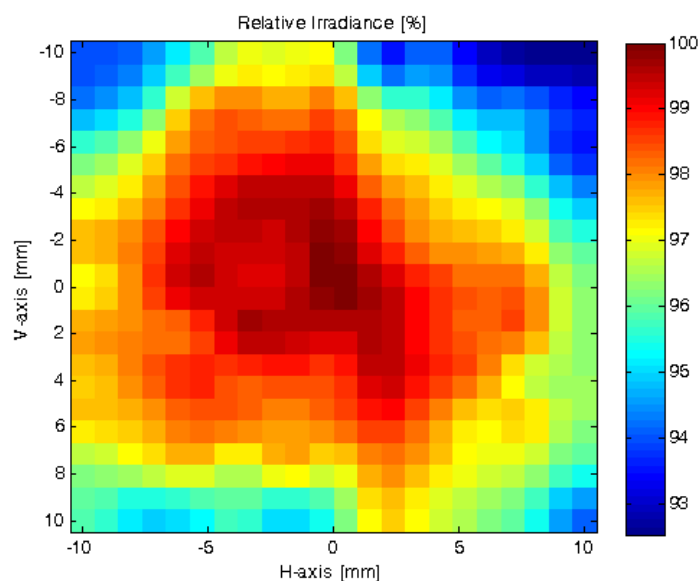


Figura 3-7 Uniformidad fuente de LEDs

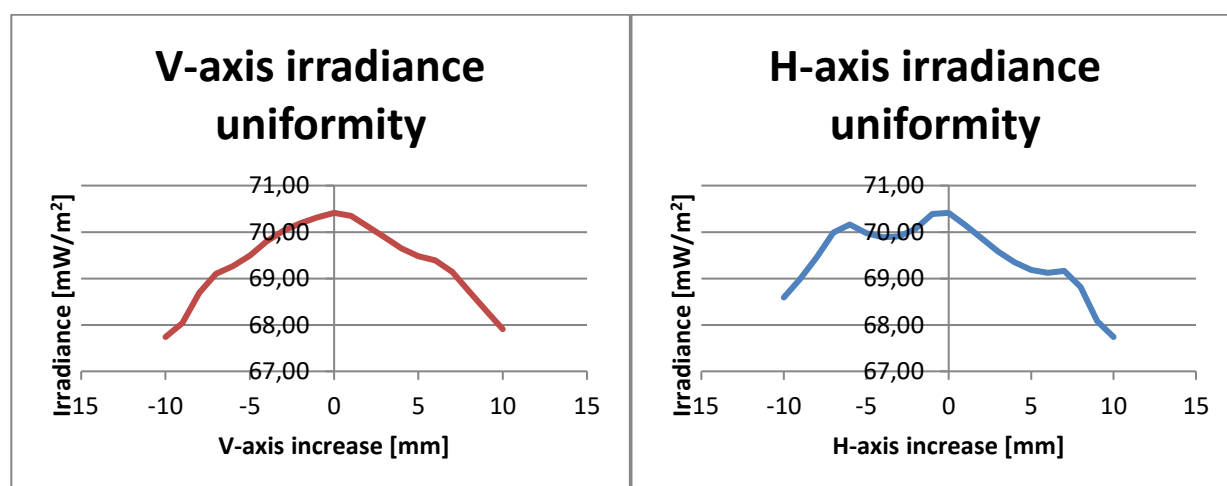


Figura 3-8 Respuesta angular fuente de LEDs

3.4 Flujo típico de caracterización

Una buena planificación de la caracterización es vital para generar datos fiables en tiempo y coste. La planificación del proceso debe cubrir aspectos como los objetivos de las medidas, información necesaria adicional a calcular, selección de los métodos apropiados o el detallado del reporte final. La idea es generar un plan de trabajo orientado en la precisión la robustez y la rapidez de las medidas y reproducibilidad.

Normalmente las medidas son realizadas con diferentes muestras de diferentes lotes de obleas de silicio, no más de 3-4 ya que entraríamos en el terreno de la cualificación del producto y ello se escapa del alcance de la caracterización.

La mayor parte de las medidas que se realizan son evoluciones de las propuestas por la EMVA1288 para adaptarlas a las necesidades tanto de los clientes, como de la aplicación, como al software y hardware dedicado que se diseña para cada producto. No obstante a la hora de presentar un reporte de caracterización siempre se acuerda con el cliente el método de medida que se va a emplear siempre pudiendo incluir medidas EMVA1288 que garanticen la fiabilidad de las performances medidas con el sensor siguiendo otros métodos.

Los procedimientos de medida deben incluir toda la información básica para poder reproducir de nuevo la medida, como la configuración del sensor, el set-up de medida, las condiciones y los objetivos de la medida así

como posibles factores a tener en cuenta o posibles fuentes de error que se puedan ir encontrando a la hora de reproducir la medida.

Veamos un ejemplo general de plan de caracterización:

Tabla 3-2 Plan típico de caracterización

	Test	Tarea	Descripción
Tests funcionales	Secuencia de inicio	Bonding	Comprobar que cada pad del anillo de pads está correctamente conectado según el diseño referencia del chip
		Power up	Comprobar integridad de las alimentaciones y las tierras
		Consumo de corriente	Medir consume en espera y en funcionamiento del sensor y comparar con valores de simulación
	Comunicaciones y programabilidad	Funcionalidad SPI	Lectura ID del chip
			Lectura STATUS del chip
			Correcta lectura/escritura de registros internos del sensor
			Lanzar instrucción START/STOP de operación del sensor y comprobar su aplicación
		LVDS	Comprobar con un patrón conocido
		Reset externo	Escribir registros internos con valores conocidos no por defecto
			Aplicar un reset externo al senso
			Comprobar que los registros contienen los valores por defecto
	Bloques de señal mixta	Power on Reset	Chequear reset al inicio del sistema
		Referencias internas	Comprobar referencias internas de tensión de los bloques digitales y analógicos
	Circuitaría de lectura	Funcionalidad	Leer fila de test con y sin iluminación
Medidas de caracterización	Performance óptica		DR, SnR, DSNU, PRNU, VFPN/HFPN in dark, Temp Noise, NEE, CG, Dark Current, Response curves, QExFF
	Consumos	Medida corrientes	Medida de consumo para los modos más depurados y demandados

4 MEDIDAS FUNCIONALES

Este capítulo detalla todas las pruebas funcionales a realizar idealmente a la llegada del primer silicio de un CIS o tras un cambio en el diseño del chip, eléctricas, de consumo, visuales, de integridad de señal, así como las principales funcionalidades internas del sensor a nivel de circuito integrado como CIS en general, aunque esto último variará en cada diseño.

4.1 Inspección visual

4.1.1 Package y wire bonding

Comprobar que cada pad del anillo de pads está correctamente conectado según el diseño referencia del chip así como el package en buen estado.

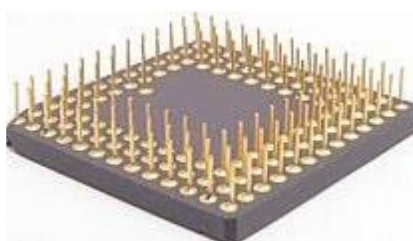


Figura 4-1 Pinout LCC de un CIS

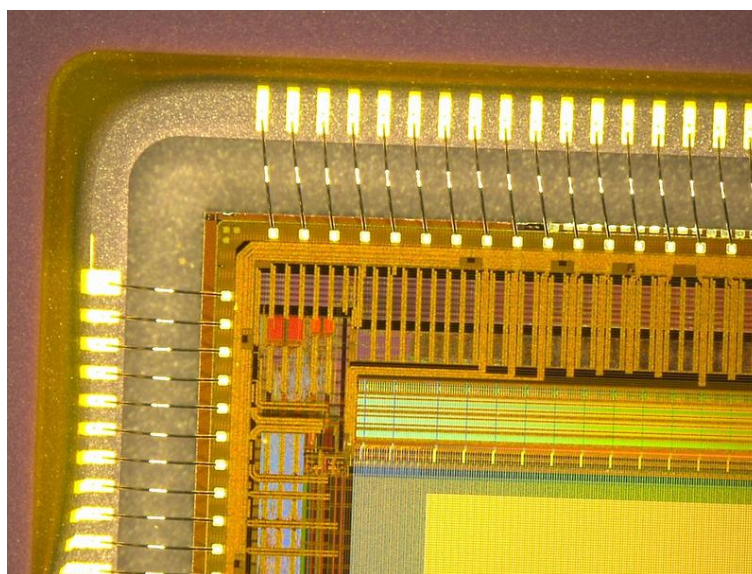


Figura 4-2 Wire-bonding al microscopio de un CIS

4.2 Medidas eléctricas

4.2.1 Alimentaciones

En esta medida se comprueba la impedancia entre diferentes pines de alimentación del chip lo que permite rápidamente detectar posibles problemas de metalización provocados por nodos de potencia mal dimensionados.

4.2.2 Consumo

Este paso tiene como objetivo la medida de la demanda de corriente del chip tras encenderlo para verificar que la distribución de las alimentaciones es la esperada.

Tabla 4-1 Árbol de alimentaciones típico de un CIS

Alimentación	Descripción
ANA_SUP	Alimentación analógica limpia. Común para todos los bloques analógicos, normalmente ~3.3V.
PIX_SUP	Alimentación analógica limpia. Común para drenador de RST todos los píxeles, normalmente ~3.3V.
MIX_SUP	Alimentación mixta analógica y digital. Común para todos los canales de lectura en la interfaz entre dominio analógico y digital y puerta de los transistores del píxel, normalmente ~3.3V.
ANA_SUP_1V8	Alimentación analógica limpia a 1.8V. Común para todos los bloques analógicos.
DIG_SUP	Alimentación digital limpia. Común para todos los bloques digitales, normalmente ~3.3V.
LVDS_SUP	Alimentación digital para los LVDS. Normalmente ~3.3V.
PLL_ASUP	Alimentación analógica limpia para los PLL.
PLL_DSUP	Alimentación digital limpia para los PLL.

4.3 Interfaces de programación y comunicación del sensor

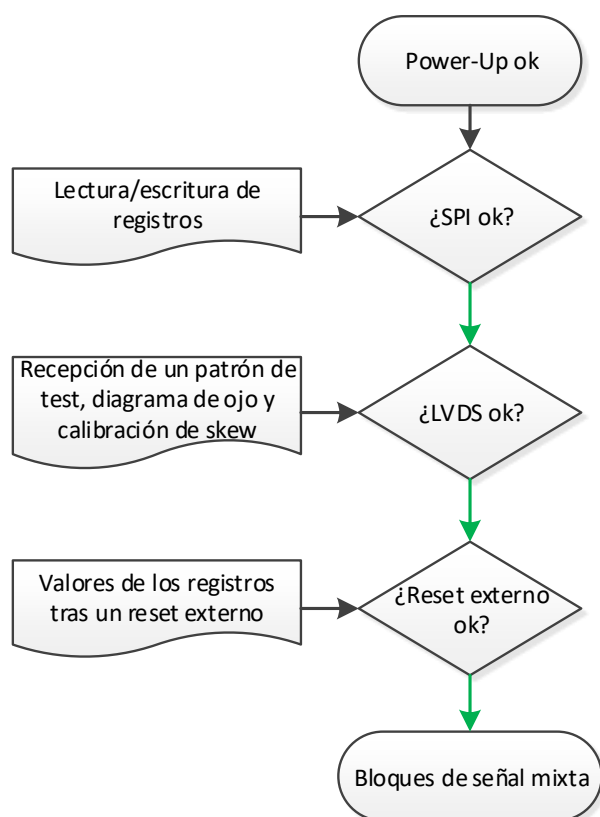


Figura 4-3 Diagrama para test de comunicaciones en un CIS

4.3.1 Generación del reloj interno del sistema

Los CIS suelen recibir todas sus referencias de reloj de una misma fuente, un cristal generalmente con una frecuencia nominal de 9.8304MHz y que gracias a un PLL embebido se permite obtener diferentes frecuencias de sistema para diferentes bloques dentro del sensor como podría ser la rampa para la conversión AD o la salida LVDS.

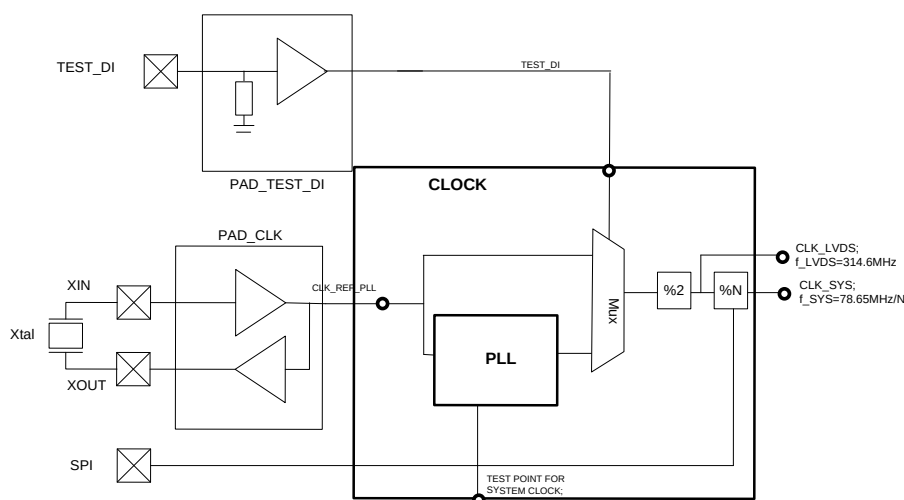


Figura 4-4 Esquema para la generación del reloj en un CIS

4.3.2 SPI

Con esta etapa se trata de comprobar que la interfaz SPI para comunicar con el sensor y programar sus registros internos funciona correctamente.

Consiste en mandar comandos como los siguientes y verificar con un osciloscopio el comportamiento del sensor según qué comando para testar la comunicación.

Tabla 4-2 Comandos SPI típicos en un CIS

Instrucción	Descripción
UPDATE REG	Update registers
UPDATE ROI	Changes RoI
UPDATE ALL	Update all registers and RoI
START	Start sensing
STOPHARD	Stop sensing immediately
READ	Read configuration register
WRITTE	Write configuration register

4.3.3 Interfaz de comunicación

4.3.3.1 Generación reloj LVDS y sincronización puertos LVDS

Es importante testar la verificación de la integridad de señal en la salida LVDS, para ello se utiliza un patrón programado en un registro de la FPGA que durante el tiempo de idle o espera es el que aparece en la salida LVDS con el propósito de detectar errores en el canal de comunicación.

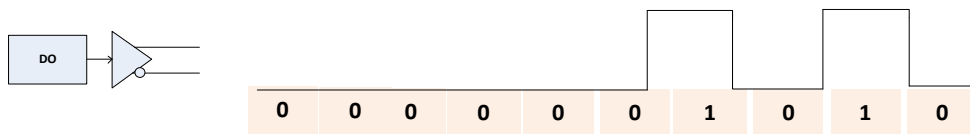


Figura 4-5 Patrón de test para testar comunicaciones

4.3.3.2 Test de datos LVDS

Para este test se realizará un diagrama de ojo con la señal LVDS, este diagrama es una representación de la transición entre 1s y 0s y la capacidad del receptor de detectar si se recibió un 1 o un 0 correctamente.

Las medidas a realizar son las siguientes:

➤ Nivel de uno/cero

En un patrón para testar comunicaciones, es el valor medio de tensión de un uno/cero lógico. Este valor es calculado como la media de la distribución del histograma hasta un 20% del mismo.

$$\mu_{ones\ level}[mV]$$

$$\mu_{zeros\ level}[mV]$$

➤ Apertura del ojo

Es la apertura horizontal del diagrama de ojo y se puede calcular directamente mediante un osciloscopio midiendo con los cursores verticales a partir tensión de referencia de $\pm 50mV$ desde el centro del ojo. La apertura del ojo será la intersección de los cursores verticales evitando los efectos de jitter.

$$eye_{aperture}[ps]$$

➤ Amplitud del ojo

Es la diferencia entre los niveles de unos y ceros.

$$eye_{amplitude}[mV] = \mu_{ones\ level} - \mu_{zeros\ level}$$

➤ Tiempo de subida y bajada

Es el tiempo que tarda el dato en seguir la pendiente de subida/bajada del ojo entre el 20% y el 80%.

$$t_{rise}[ps] = \mu_{80\%} - \mu_{20\%}$$

$$t_{fall}[ps] = \mu_{20\%} - \mu_{80\%}$$

➤ Desequilibrio de subida y bajada

El tiempo de desequilibrio entre el flanco de subida y el flanco de bajada se calcula con la siguiente ecuación:

$$RF_{bal}[\%] = \frac{t_{rise}}{t_{fall}} * 100$$

➤ Valor eficaz de jitter

El jitter es la incertidumbre temporal en la temporización ideal de un evento síncrono. Para calcularlo, se captura la variación temporal del cruce de los flancos de subida y bajada en el ojo. Esta desviación estándar es el valor eficaz de jitter.

$$jS_{RMS}[ps]$$

➤ Relación señal a ruido

Define como la relación entre el nivel de señal y el piso de ruido teniendo en cuenta la distorsión. Para calcular la SNR se analiza el diagrama de ojo con la siguiente ecuación:

$$SNR = \frac{\mu_{ones\ level} - \mu_{zeros\ level}}{\sigma_{ones\ level} - \sigma_{zeros\ level}}$$

➤ Tasa de error de bit

La tasa de error de bit (BER) es el número de bits erróneamente recibidos por unidad de tiempo a lo largo de un canal de comunicación debido a ruidos, interferencias, distorsión o errores de sincronización.

Para su cálculo se considera que un error es detectar cero cuando se transmite un uno y viceversa. Es sencillo de entender con el siguiente histograma:

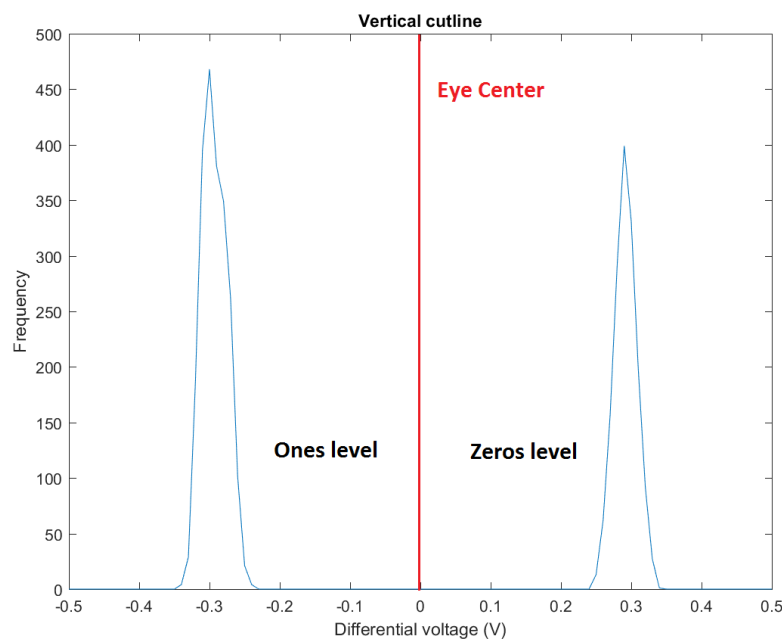


Figura 4-6 Histograma característico equivalente al cálculo de la BER

➤ Capacitancia máxima en la carga

Si se asume que el flanco de subida/bajada sigue una dinámica exponencial, se puede modelar el tiempo como una red RC y conociendo la impedancia resistiva del canal, se puede calcular la impedancia capacitiva.

$$C_{load}[pF]$$

➤ Amplitud diferencial de señal

Es la amplitud del ojo medida entre los dos niveles lógicos $VO_+[mV]$ y $VO_-[mV]$.

➤ Desequilibrio de la amplitud de señal

Es la relación entre las dos amplitudes y debe estar por debajo de un 10%.

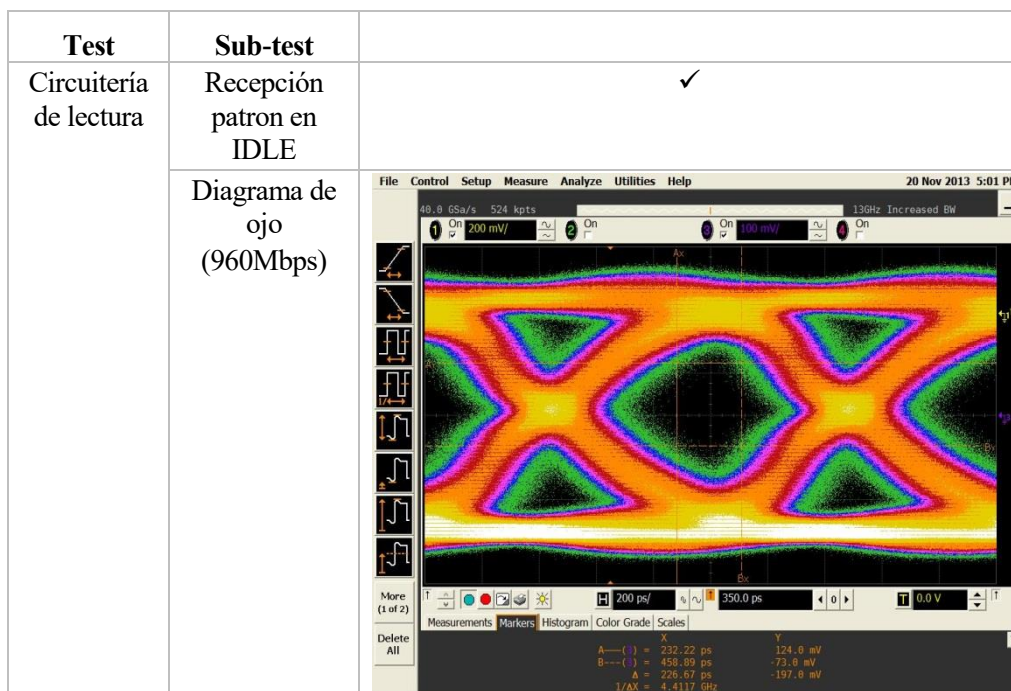
$$SAI[\%] = \frac{VO_+}{VO_-}$$

➤ Distorsión del Duty Cycle

Es la medida de la desviación del Duty Cycle normalmente causada por desviaciones en el ancho del pulso del dato. Se mide a través de la diferencia temporal entre el 50% de los flancos de subida y bajada del ojo.

$$DCD[\%] = \frac{\text{time difference between rising and falling edges at 50\%}}{\text{bit period}} \times 100$$

Un ejemplo de test de datos LVDS sería el siguiente:



4.3.3.3 Calibración de Skew

En orden de garantizar una correcta recepción y composición del código de sincronismo y de datos, el skew respecto al reloj de recuperación y cada puerto LVDS debe ser medido y cada puerto retrasado para equalizar este skew.

➤ Skew entre canales

Es la diferencia de tiempo que existe entre los canales comparando con la señal de reloj como referencia. Se mide respecto al 50% de nivel del reloj y el canal en el flanco más próximo.

$$T_{CCS}[ps] = |t_{clk} - t_{sgn}|$$

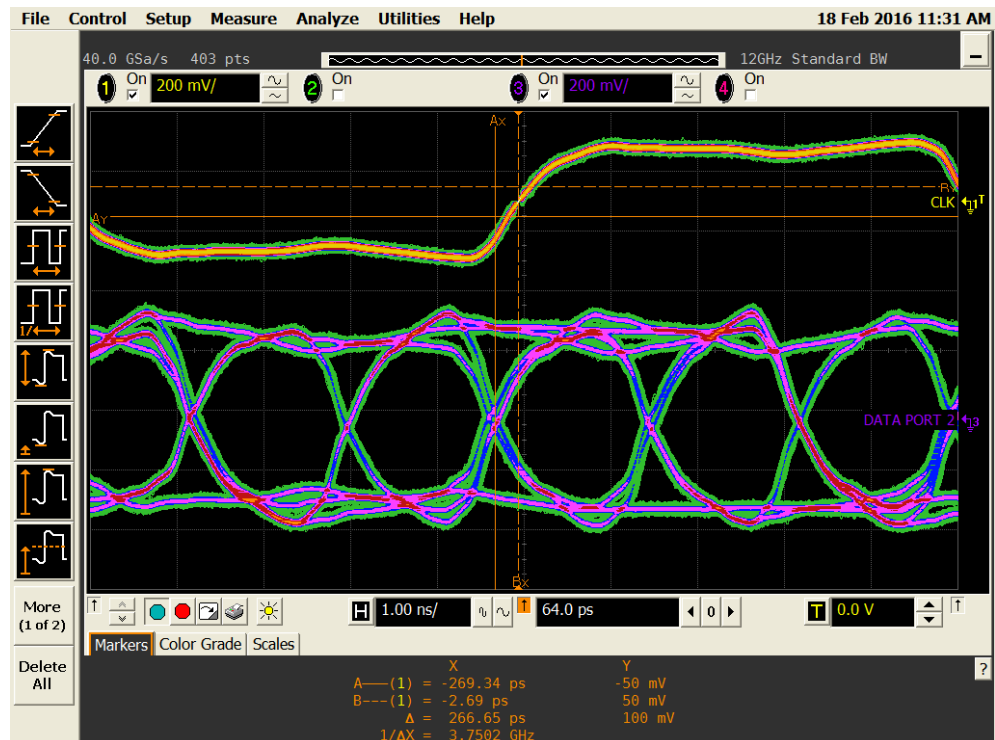


Figura 4-7 Diagrama de ojo salida LVDS

➤ Skew diferencial

Es la diferencia temporal que existe entre las diferentes salidas de un mismo puerto LVDS. Se mide igual que el skew entre canales pero entre dos salidas por ejemplo; p y n.

$$T_{skew\ dif}[ps] = |t_p - t_n|$$

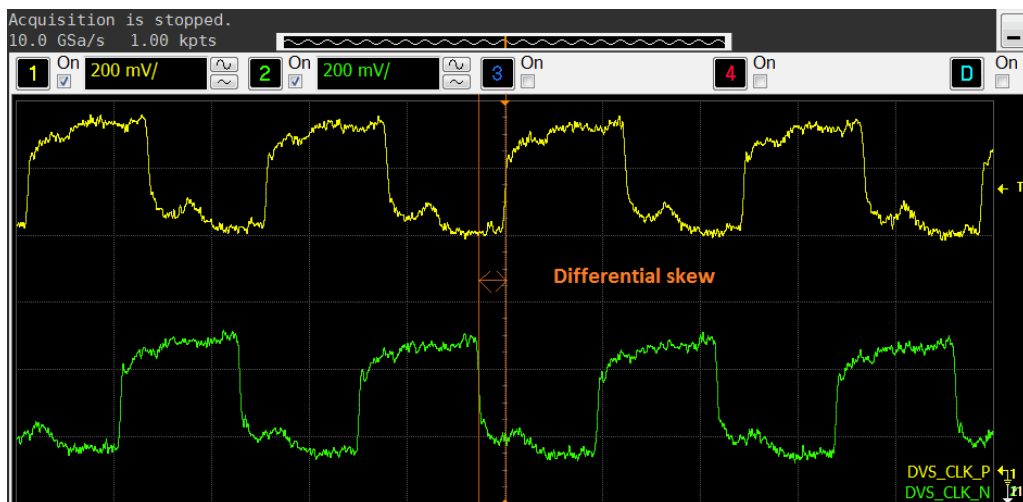


Figura 4-8 Medida de Skew en un puerto LVDS

4.3.4 Reset externo

Haciendo uso del pin de RESTN como entrada desde una FPGA. Inicialmente el sistema fue programado con la configuración básica y se escribieron varios registros vía SPI. Tras ello se aplica el reset externo y se comprueba con una lectura el valor anterior programado.

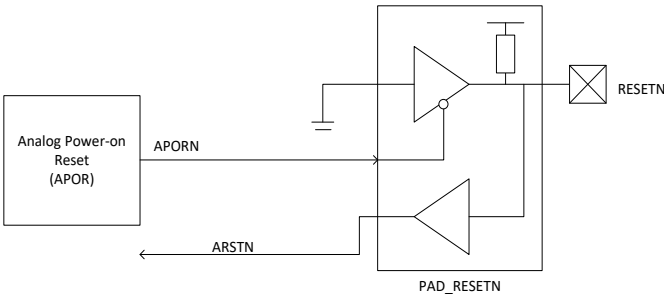
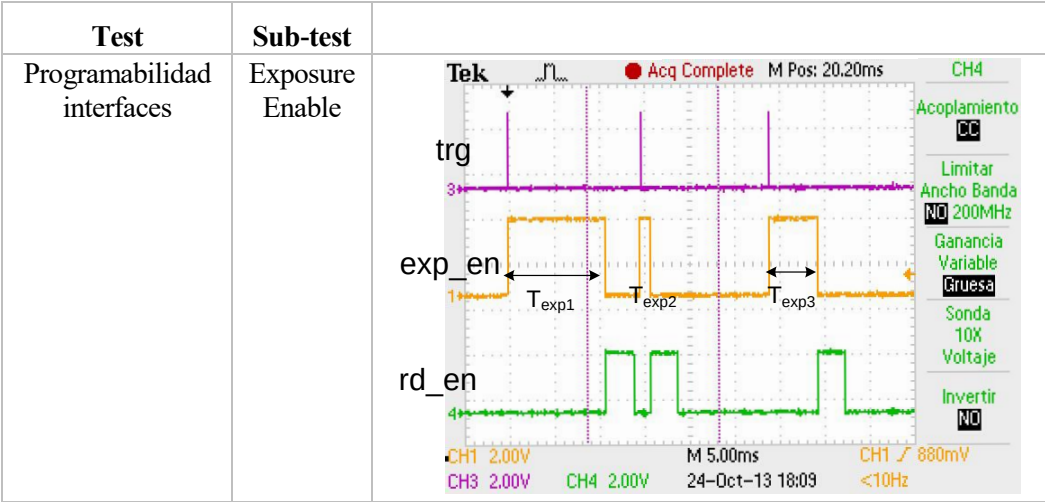


Figura 4-9 Esquema generación Reset externo

4.3.5 Señal de exposición

En los diseños se suele añadir un pin de test para la señal exposición que indica cuando el sensor está captando luz con objetivo de sincronizar con otros eventos externos al sensor.

Suponiendo que nuestro sensor funciona con un trigger “single-edge” o lo que viene a ser lo mismo que por cada flanco de subida de una señal llamada trigger a la entrada del sensor se suceden una serie de procesos en cadena desde la exposición (activando la señal de *exposure_en*) hasta la lectura del dato (*readout_enable*).



4.4 Bloques de señal mixta

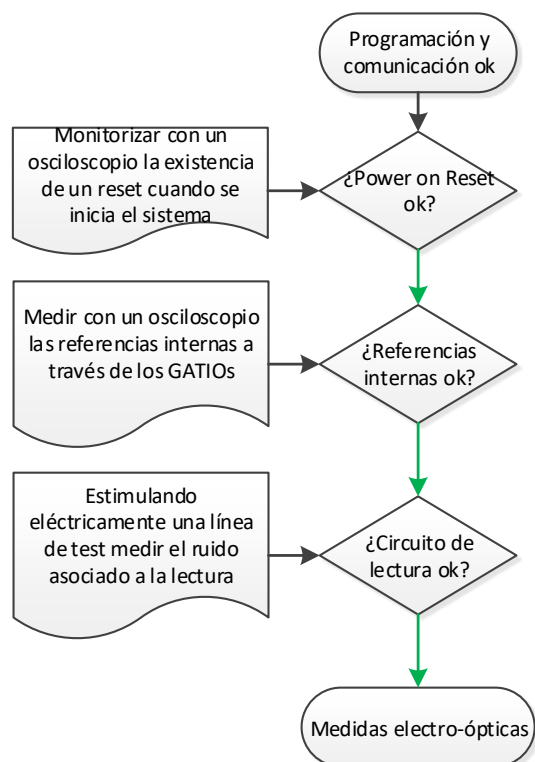


Figura 4-10 Diagrama para test de bloques analógicos

4.4.1 Power on Reset

Esta operación es esencial pues durante la inicialización del sistema se activa un bloque de auto-reset observable desde un pin del chip.

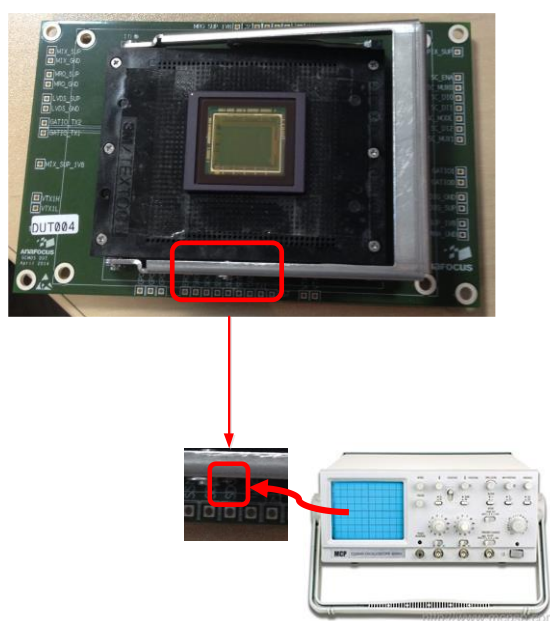


Figura 4-11 Punto de test para visualización del Power-On-Reset

4.4.2 Referencias internas analógicas

Este bloque genera las tensiones internas de referencia para todo el chip, todas, tanto las fijas como las variables las cuales hay que verificar tanto su integridad como su programación según se desee. Se miden a través de pines GATIO (General Analog Test Input/Output) con la ayuda de un osciloscopio.

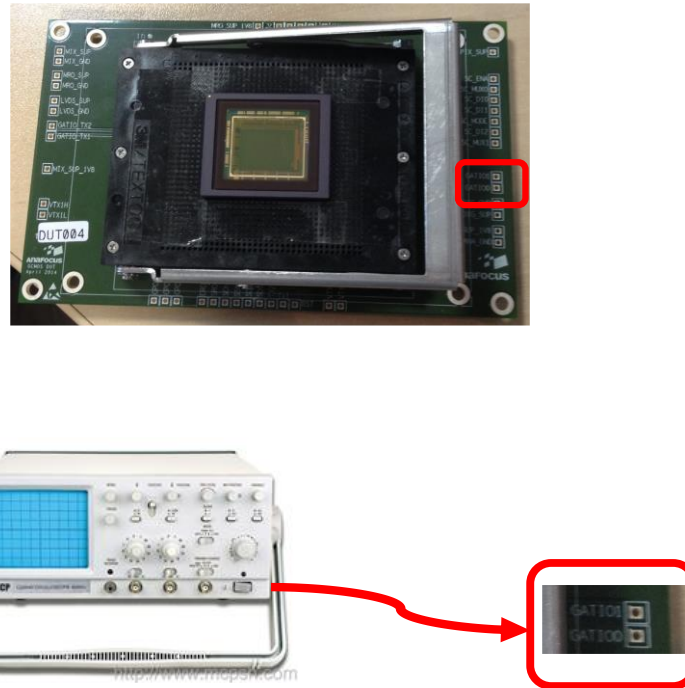


Figura 4-12 Punto de test para visualización de las señales GATIO

4.5 Canal de lectura

En esta sección se introduce la funcionalidad del circuito de lectura y su ruido asociado son medidos usando referencias internas programables de una fila de test del sensor, esta fila es igual que cualquiera del array activo pero con la ausencia del fotodiodo de manera que se fija un valor de tensión y a la salida se debería leer ese mismo valor más el ruido que añade el canal. El procedimiento es sencillo:

1. Se capturan dos imágenes consecutivas fijando la fila de test a una alimentación limpia de ruido de las referencias internas del sensor. La diferencia entre estas dos señales se computa con el objetivo de eliminar el ruido de patrón fijo o FPN. El ruido temporal restante y su desviación estándar se computan tal que así:

$$temporal_noise = Im_1 - Im_2$$

$$\sigma_{TEMP} = \frac{std(temporal_noise)}{\sqrt{2}}$$

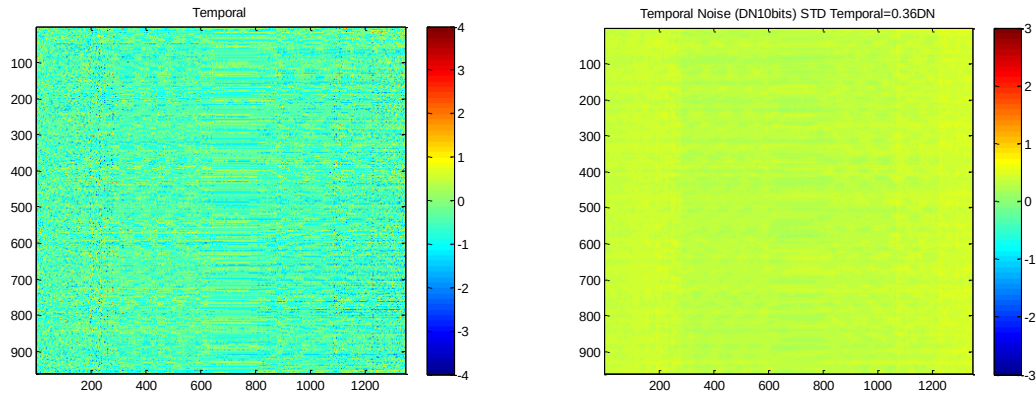


Figura 4-13 Medida ruido en el canal de lectura del píxel

A partir de estos datos se pueden calcular los ruidos temporales de lectura por fila y columna. El ruido medida en DN_{rms} debe ser convertido en $Volt_{rms}$ teniendo en cuenta la resolución de la conversión, la del sensor y su tensión de escalado. Por ejemplo para un caso genérico de un canal con un convertidor de 14bits:

$$Vnoise_rms = \frac{\sigma_{TEMP}(DN_{14bit})}{2^{10}} \cdot V_{FS}$$

4.6 Funcionalidades del sensor

4.6.1 Correcciones on-chip

Es posible incluir inteligencia en algunos sensores en forma de algoritmos de procesamiento de imagen con el objetivo de eliminar ruidos y proporcionar una imagen más limpia. Tenemos dos tipos de calibraciones/correcciones posibles embebidas en el propio chip:

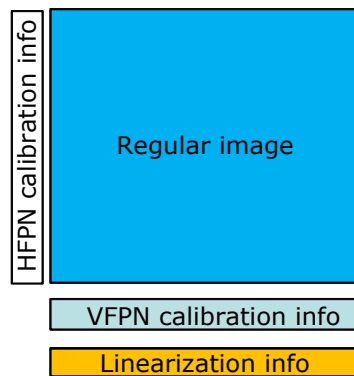


Figura 4-14 Esquema correcciones en un CIS

- Ruido de fila (Temporal Row Noise, TRN) y ruido de patrón fijo horizontal (HFPN).
Se promedian N píxeles oscuros (están tapados con una metalización para este propósito), dispuestos en una parte fija del array (normalmente a la izquierda). Una vez promediado, el resultado por fila se sustrae a los píxeles de la misma fila. Este procedimiento permite habilitar un control del nivel de negro u oscuridad frente a cambios en las condiciones externas.
- Corrección de ruido de patrón fijo vertical (VFPN).
Se puede activar esta corrección igual que la anterior si se dispone de píxeles tapados en la parte superior del array, en caso contrario se puede hacer uso de la fila de test capturando un número elevado de imágenes para posteriormente aplicarle un promedio y calcular los coeficientes VFPN. Los cuales se sustraen por columna de todos los píxeles en todas las imágenes.

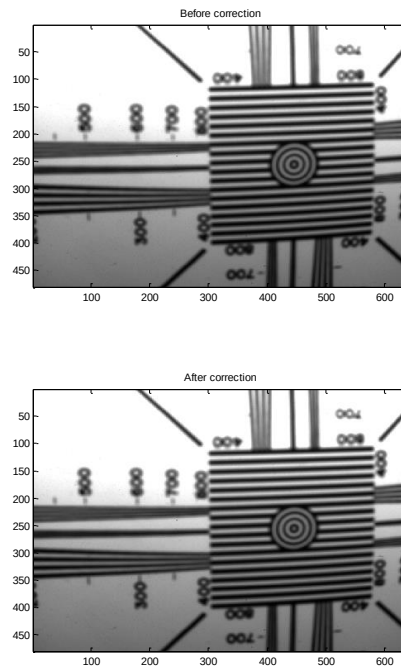


Figura 4-15 Ejemplo correcciones FPN

4.6.2 Offset analógico

Este offset es el retraso entre la rampa analógica y digital del CAD las cuales comienzan a contar a la vez idealmente y cuando se cruzan es cuando convierten el dato. Controlando este retraso se permite añadir un offset a la conversión para evitar que en ausencia de luz haya algún píxel a cero por lo que define el nivel de negro para garantizar un correcto funcionamiento en los algoritmos de calibración.

4.6.3 Ganancia analógica

Es equivalente a modificar el rango del convertidor A/D de manera que modificando la pendiente de la rampa de conversión se puede modificar la sensibilidad del sensor. Veamos un ejemplo para diferentes niveles de ganancia e iluminación fijada.

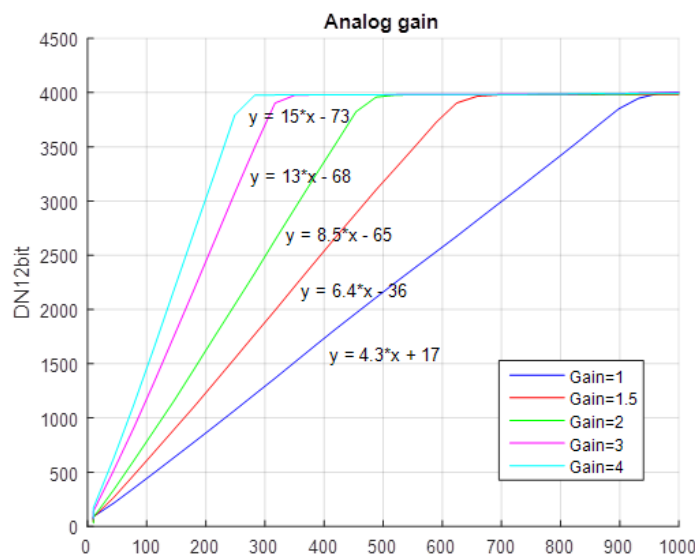


Figura 4-16 Medida responsividad de un CIS frente a ganancia analógica

4.6.4 Tipos de trigger

El trigger es la señal externa al sensor que le indica cómo debe operar, idealmente se contemplan dos tipos de trigger:

➤ **Single-Edged**

En este modo de trigger el pulso activa una secuencia de operaciones, es decir, cada vez que se detecta un flanco de subida en el trigger se activa la maquinaria de operación interna del píxel. Un ejemplo:

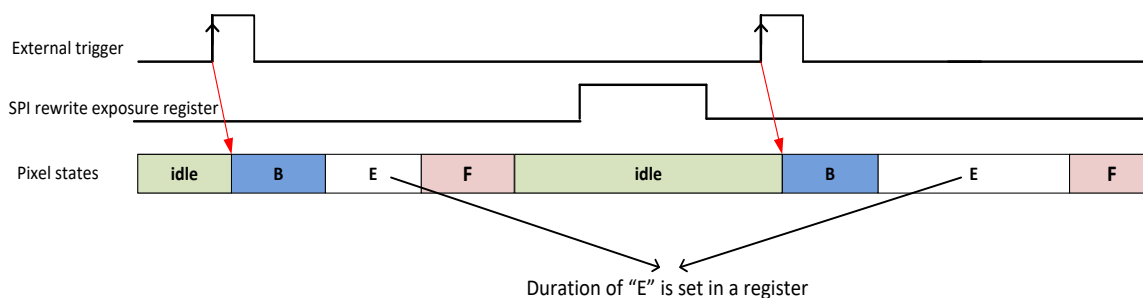


Figura 4-17 Cronograma trigger single-edge

➤ **PWC (Pulse Width Control)**

En este modo se produce un control por ancho de pulso pues el flanco de subida del trigger lanza el evento de reset global del sensor o estados B y el pulso de bajada lanza el proceso de lectura o estados F.

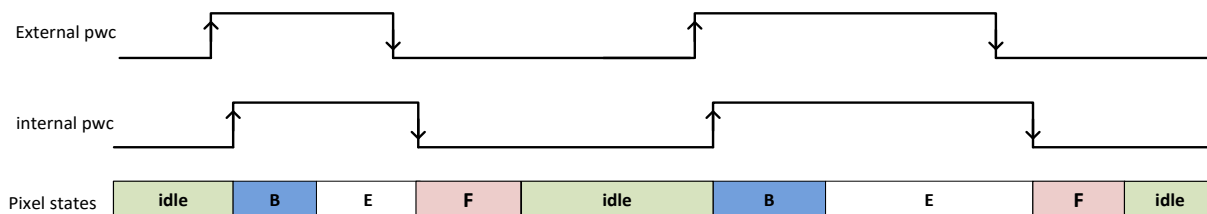


Figura 4-18 Cronograma trigger PWC

5 MEDIDAS ELECTRO-ÓPTICAS

Uno de los propósitos de este trabajo era recoger y unificar procedimientos de medida para los parámetros característicos de los sensores de imagen CMOS, para este propósito es esencial comprender estos parámetros y también su relación con el diseño del píxel. Aunque algunos de estos parámetros están principalmente limitados por la circuitería de lectura, la mayoría lo están por el diseño del píxel, como pueden ser la eficiencia cuántica o la linealidad de la foto-respuesta. Esta sección explicará detalladamente estos parámetros entre otros de vital importancia a la hora de diseñar un sensor de imagen.

5.1 Corriente oscura

5.1.1 Fundamento teórico

La corriente oscura es una pequeña fuga de corriente cuando no hay presencia de fotones en un CIS y se debe principalmente a la aleatoria generación de pares electrón-hueco en el PPD. La corriente oscura no es únicamente culpa del PPD, también la FD o la TG contribuyen a la misma. La corriente oscura por sí misma no supone ningún problema para el sensor pues puede ser calibrada, pero es la variación temporal de la misma la que introduce ruido en el sistema. En sensores de muy alta velocidad la corriente oscura es prácticamente despreciable pues para un corto periodo de integración es el ruido de lectura el que domina, sin embargo para sensores que trabajen con tiempos de integración elevados (del orden de 10ms), es posible que suponga un problema.

Esta corriente es un ruido que depende de la tecnología de fabricación y del diseño del CIS. Hay diferentes componentes de la corriente oscura que son resultados de diferentes mecanismos físicos, los principales vamos a discutirlos a continuación.

5.1.1.1 Generación térmica

La generación térmica y la recombinación son fenómenos comunes en cualquier dispositivo opto-eléctrico. En ausencia de luz, debido a impurezas y defectos del cristal, algunos electrones tienen un nivel de energía superior al nivel de Fermi lo que produce una transición indirecta del electrón a una banda de conducción generando corriente oscura.

5.1.1.2 Efecto túnel

Es un fenómeno común que ocurre en uniones p-n altamente dopadas con una fina capa de depleción como es el PPD. Durante el efecto túnel los portadores penetran en la banda en vez de superar la barrera. Este efecto ocurre en los PPD bajo un alto campo eléctrico donde los portadores pueden atravesar desde el sustrato de tipo p hasta la región n y contribuir a la corriente oscura del píxel. A mayor dopado del silicio mayor corriente oscura se tendrá. Uno de los principales inconvenientes a la hora de reducir la corriente oscura producida por el efecto túnel es que se debe reducir el campo eléctrico, y esto afecta directamente a las prestaciones del píxel.

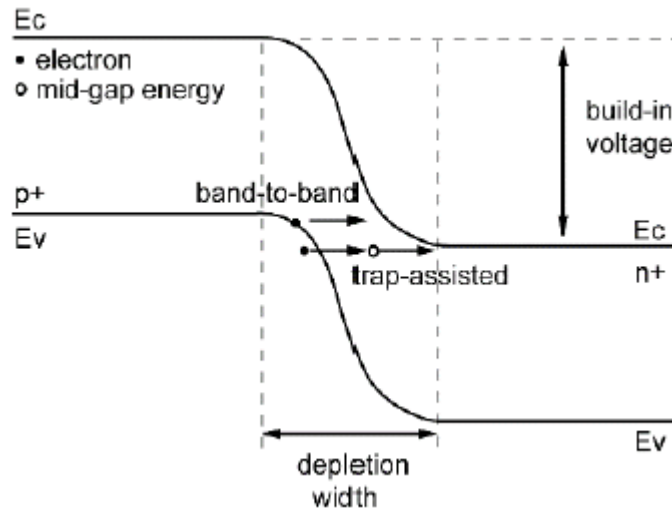


Figura 5-1 Efecto túnel

5.1.1.3 Corriente de difusión

Es la corriente resultante de la diferencia de concentraciones entre dos regiones en una unión p-n. Cuando hay una región con un mayor potencial que otra se genera un movimiento neto de portadores desde la región con mayor concentración a la región con menor y este movimiento neto genera una corriente de arrastre, llamada corriente de difusión.

5.1.1.4 Proceso de fabricación

La técnica STI (Shallow Trench Isolation) es ampliamente utilizada en los circuitos integrados como técnica de aislamiento. Suele utilizarse para separar distintos dispositivos semiconductores en la mayoría de tecnologías MOS y bipolares. Los pasos claves de esta técnica, consiste en rasar un patrón en el silicio para colocar material dieléctrico para rellenar las zanjas creadas y posteriormente eliminar el exceso utilizando un proceso químico-mecánico. La principal ventaja de esta técnica es que previene fugas de corriente eléctricas entre dispositivos adyacentes lo cual tiene un impacto directo en la disipación de potencia en el circuito integrado y en la corriente oscura.

Muchos sensores de imagen usan esta técnica de STI para prevenir el crosstalk entre píxeles debido a la difusión de portadores generados en el sustrato. La solución que se suele tomar es extender el pozo de tipo p+ para cubrir todo el STI y el fotodiodo como se muestra en la siguiente figura. Se ha descubierto recientemente que la corriente oscura total depende fuertemente de la distancia entre la zona en la que se aplica el STI y el fotodiodo. Para disminuir esta corriente se puede incrementar la distancia entre el fotodiodo y el STI a costa de perder algo de factor de llenado (FF).

5.1.2 Método de medida

La corriente oscura del fotodiodo depende directamente de la temperatura y del tiempo de integración pues mide la variación estadística de la generación/recombinación en cada píxel. Por lo que habrá que medir en condiciones de oscuridad y elevados tiempos de integración.

5.1.3 Condiciones de la medida

El set-up típico de medida será el de la siguiente figura y la lista de equipos la siguiente:

1. DUT
2. Cámara link y frame grabber o ethernet dependiendo de la interfaz de salida del sensor.
3. Fuente de alimentación (CI-0033 TTi_QL355TP_Power_supply).
4. Condiciones de oscuridad.

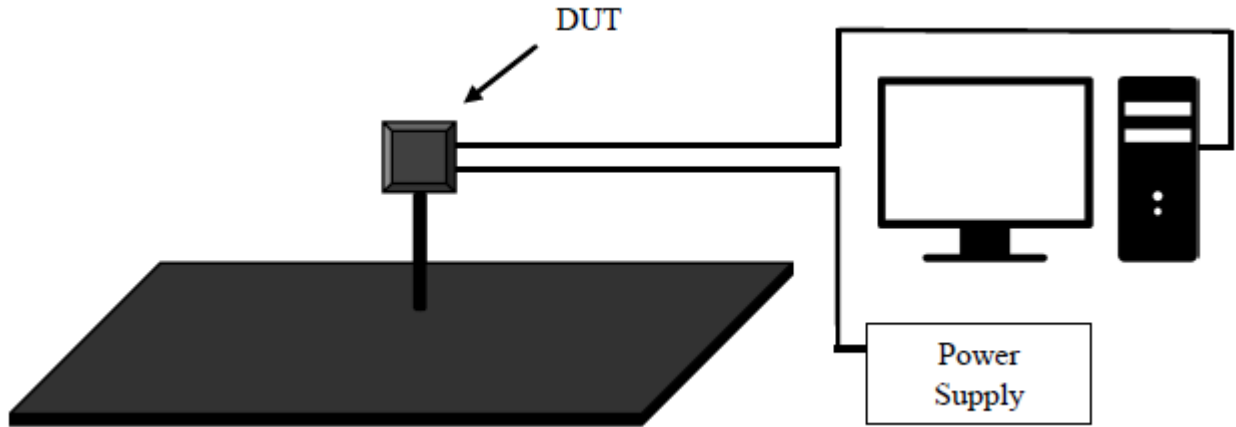


Figura 5-2 Set-up medida corriente oscura

5.1.4 Procedimiento de medida y cálculos

La idea es capturar dos imágenes, una de ellas con un corto periodo de integración y otra con un gran periodo de integración y tomando la diferencia entre ambas calcular la desviación estándar con el fin de computar la corriente oscura:

1. Colocar el set-up anteriormente detallado.
2. Tomar capturas dummy (al menos 20 o 30) para que el sistema sea estable.
3. Tomar una imagen 1 con un corto periodo de integración (t_{int1}) o lo que es lo mismo a máxima velocidad.
4. Tomar una imagen 2 con un largo periodo de integración (t_{int2}) al menos 100 o 1000 veces mayor que t_{int1} para que la corriente oscura sea significativa.
5. Capturar una tercera imagen dummy con algo más de tiempo de integración que t_{int2} con el fin de comprobar que las imágenes 1 y 2 se encuentran en una región lineal.
6. Cálculo de la corriente oscura tanto en e-/s como en tensión eficaz.

$$DS \left[\frac{e^-}{s} \right] = \frac{\mu_{image2} - \mu_{image1}}{t_{int2} - t_{int1}}$$

$$Dark\ current\ [V_{rms}] = \frac{\sigma_{image2} - \sigma_{image1}}{t_{int2} - t_{int1}}$$

Unas curvas típicas tras esta medida serían las siguientes, tanto para el PPD como para la FD:

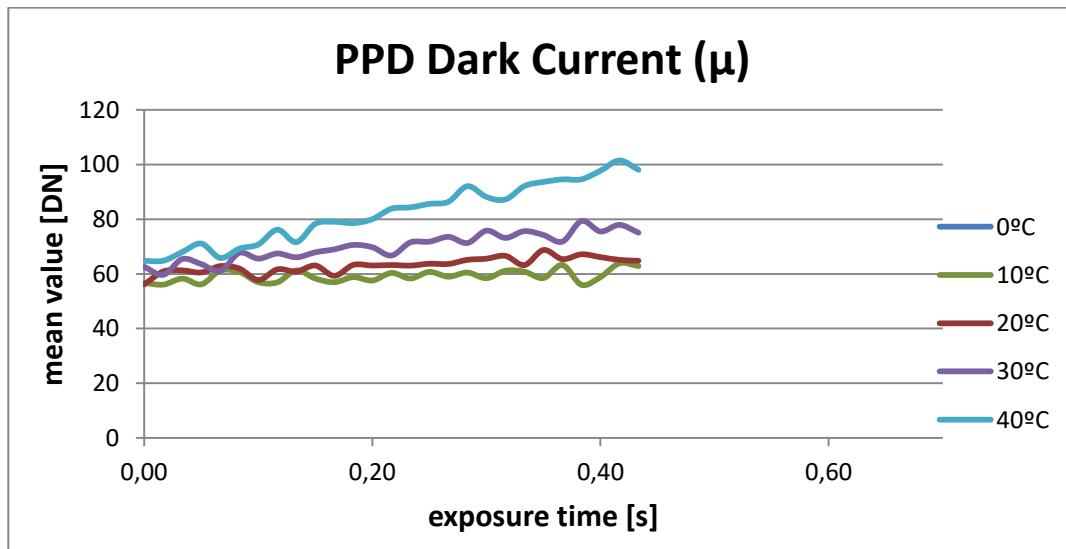


Figura 5-3 Corriente oscura asociada al fotodiodo

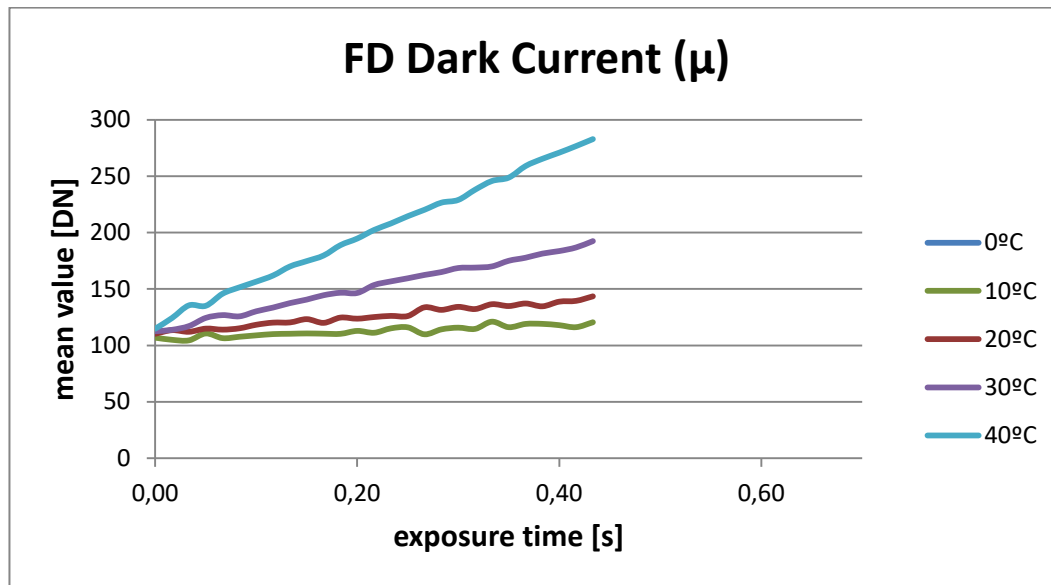


Figura 5-4 Corriente oscura asociada a la floating diffusion

5.2 Foto respuesta

5.2.1 Fundamento teórico

En esta sección vamos a explicar todos los parámetros medibles a raíz de analizar las imágenes generadas tras un barrido de luz uniforme y homogénea con un tiempo de exposición constante, o bien, barriendo el tiempo de exposición con el nivel de iluminación fijado.

5.2.1.1 Responsividad

La curva de foto-respuesta o responsividad es la medida de la habilidad del sensor de convertir potencia óptica incidente (número de fotones incidentes en el sensor dado un cierto tiempo de integración) en señal eléctrica (ganancia del sistema multiplicada por el número de electrones generados), y depende de la longitud de onda.

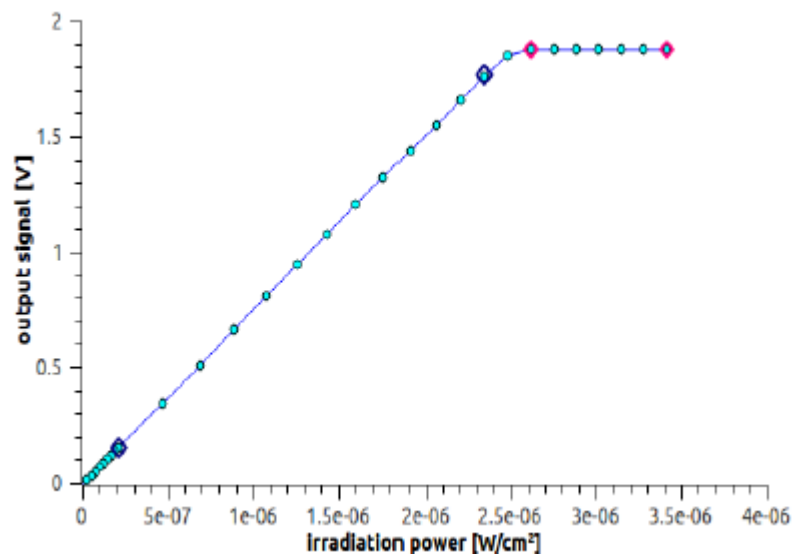


Figura 5-5 Típica curva de responsividad de un CIS

5.2.1.2 Capacidad de llenado (FWC)

La capacidad de llenado o Full-Well Capacity (FWC) define la capacidad del píxel de almacenar/generar carga de manera que cuando el estado del píxel alcanza el FWC se dice que el sensor satura. Normalmente el PPD posee una amplia región de deplexión donde todos los portadores libres son recolectados, esta cantidad de carga que puede ser recolectada define el FWC. De otra manera se define el FWC como la capacidad de la FD de almacenar carga y para un buen diseño del píxel, la cantidad de carga almacenable en ambos nodos debe ser la misma. Acorde al diseño, hay un límite para el número de portadores libres que pueden ser generados en un PPD y este sería la cantidad de carga almacenable en la FD.

La fórmula para el FWC puede ser deducida considerando que el fotodiodo integra la carga como se muestra en la siguiente figura donde V_{rst} es la tensión de reset del fotodiodo, I_{ph} es la fotocorriente, C_{PD} es la capacidad parásita del fotodiodo y V_{PD} es la tensión en el fotodiodo.

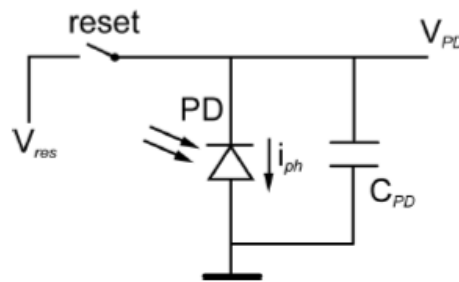


Figura 5-6 Esquema simplificado de un píxel 3T

Para el anterior fotodiodo su capacidad de llenado se calcula como:

$$FWC[e^-] = \frac{1}{q} \int_{V_{rst}}^{V_{PD(min)}} C_{PD} \cdot V_{PD} dV$$

Donde q es la carga del electrón y $V_{PD(min)}$ es la barrera de potencial del fotodiodo o V_{PIN} . Normalmente el FWC se expresa en electrones.

5.2.1.3 Saturación (V_{sat})

Se dice que un sensor satura cuando alcanza su valor de FWC para una potencia óptica incidente y la señal de salida no depende más de la cantidad de luz. Puede ser gráficamente computada de la curva de responsividad como el punto a partir del cual la salida del sensor es constante.

El número de electrones que hace que un píxel sature es la capacidad de saturación, la que depende fuertemente de la ganancia del píxel pero que no debe confundirse con el FWC. Pues para que un sensor sature o bien sus píxeles saturan (llenándose de electrones el PPD o la FD), o el ADC satura, o se aplica una ganancia digital a la señal que provoca que sature.

5.2.1.4 Curva de foto-transferencia (PTC)

La PTC (Photo Transfer Curve) da una buena visión de muchas de las características del sensor como la ganancia de conversión (CG) y si el valor de QE está disponible, información sobre no linealidades, tensión de saturación y FWC. Normalmente la PTC es evaluada para una longitud de onda correspondiente a un pico de respuesta.

La ganancia de conversión (CG) define qué cantidad de voltaje (V_{signal}) corresponde a un cierto número de electrones generados por el píxel (μ_e). La CG es uno de los parámetros más importantes de los CIS. Este parámetro influye tanto en la linealidad y uniformidad de la respuesta del píxel como sobre la sensibilidad a la luz y ruido del píxel.

Un método fácilmente deducible y ampliamente utilizado para el cálculo de la CG es el basado en la curva de foto transferencia de carga o Photo Transfer Curve (PTC). Este método está basado en leyes físicas básicas y emplea el hecho de que el nivel de ruido shot del píxel (PSN) es proporcional a la raíz cuadrática del nivel de señal. Por lo tanto la CG puede ser determinada como la pendiente de la PTC, es decir la pendiente de la curva entre la varianza del ruido y el nivel medio de señal barriendo la luz o el tiempo de exposición.

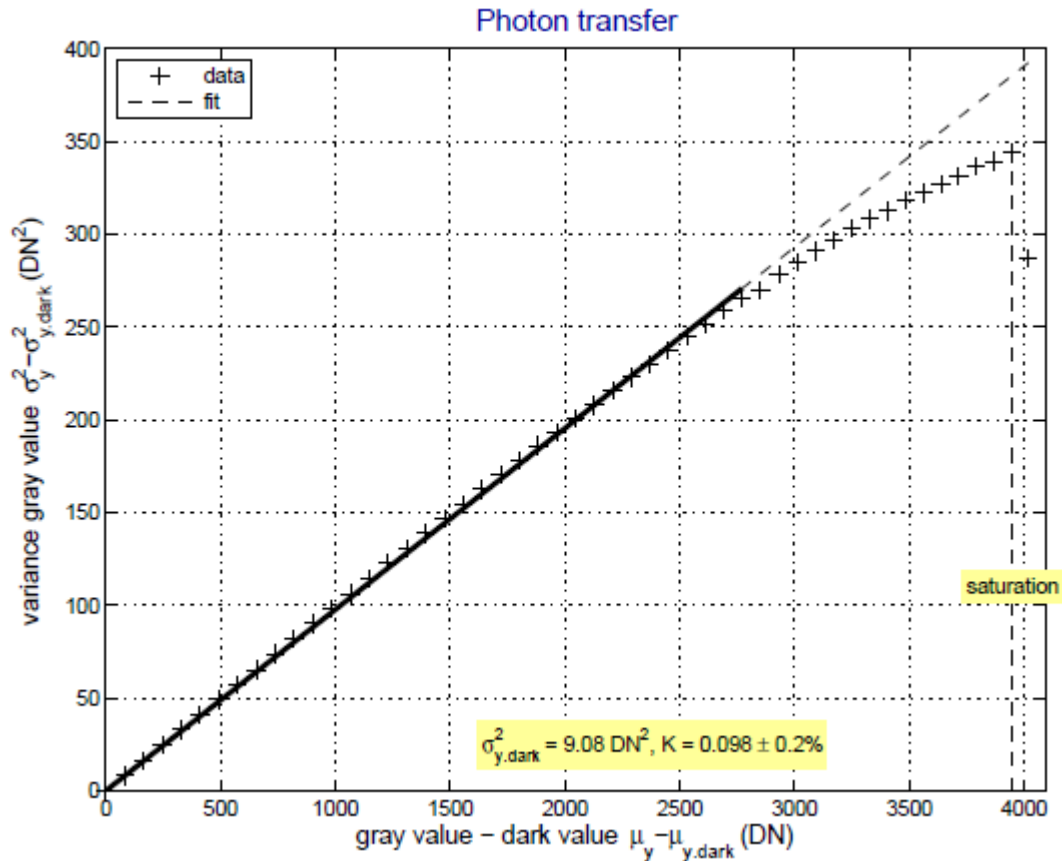


Figura 5-7 Típica curva de fototransferencia de carga

5.2.1.5 Rango dinámico

Es el ratio entre las el valor de señal de saturación y el piso de ruido, y gráficamente se puede modelar como la siguiente curva.

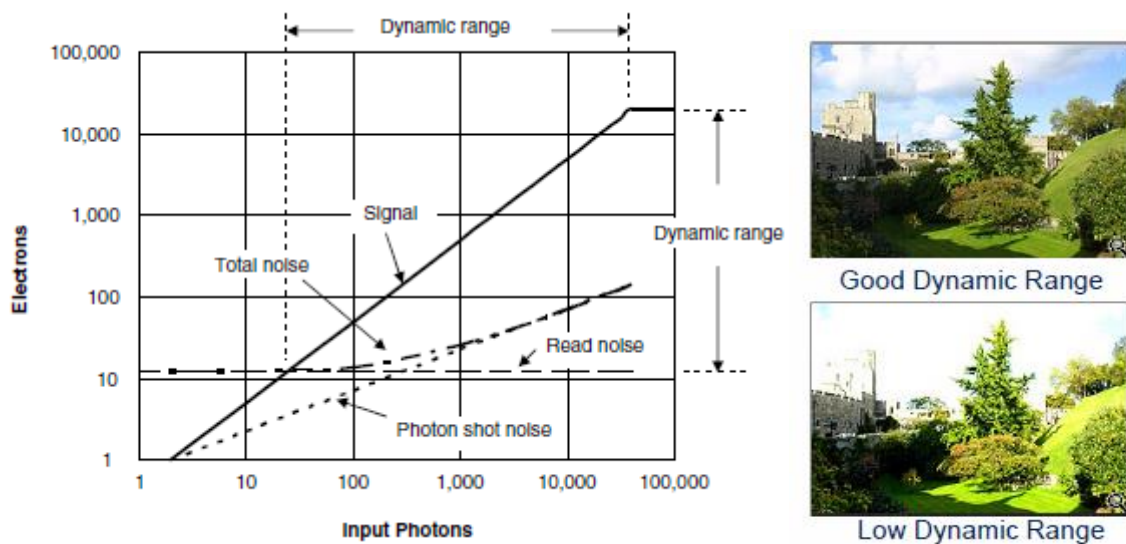


Figura 5-8 Principales características de un píxel

5.2.1.6 Linealidad

Idealmente un CIS debería tener un comportamiento lineal, es decir, que responda linealmente a la luz

incidente, pero debido a desviaciones que provienen tanto del píxel como de la cadena de procesamiento de señal el sensor se desvía de una perfecta respuesta lineal.

La mayor fuente de no linealidad en un CIS suele venir del seguidor de corriente del píxel 4T, al utilizarse como amplificador de trans-impedancia, su ganancia depende de la resistencia de la fuente la cual induce a no linealidad. Otros transistores son únicamente utilizados como llaves por lo cual no contribuyen a no linealidades en el sensor. Por otra parte otras fuentes de no linealidades son:

- Lag en el PPD por una ineficiente limpieza.
- No linealidades del PD o la FD por culpa del *mismatch* de transistores debido al proceso de fabricación.
- No linealidades debidas a procesamiento analógico y multiplexaciones de señal.

Normalmente el rango lineal del píxel suele estar limitado por el rango lineal del ADC a su salida.

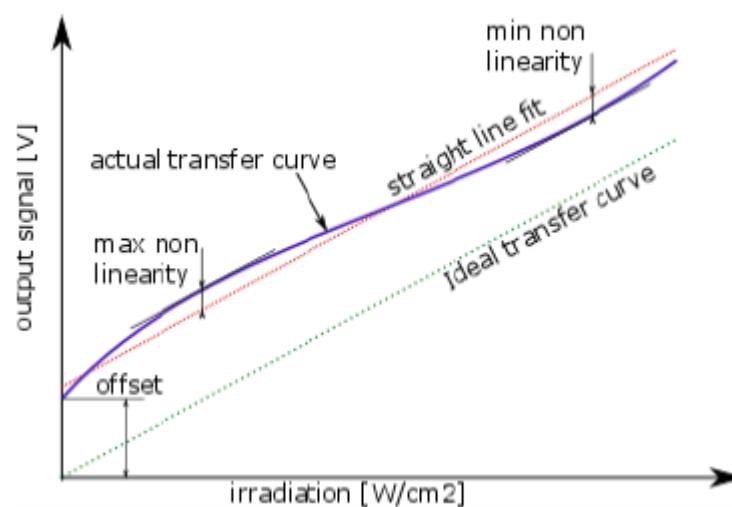


Figura 5-9 Medida de linealidad en un CIS

5.2.1.7 Relación señal a ruido

La relación señal a ruido es una medida del ratio entre el nivel de señal de salida y el ruido en decibelios, es una buena medida comparativa entre sensores de imagen en término de prestaciones

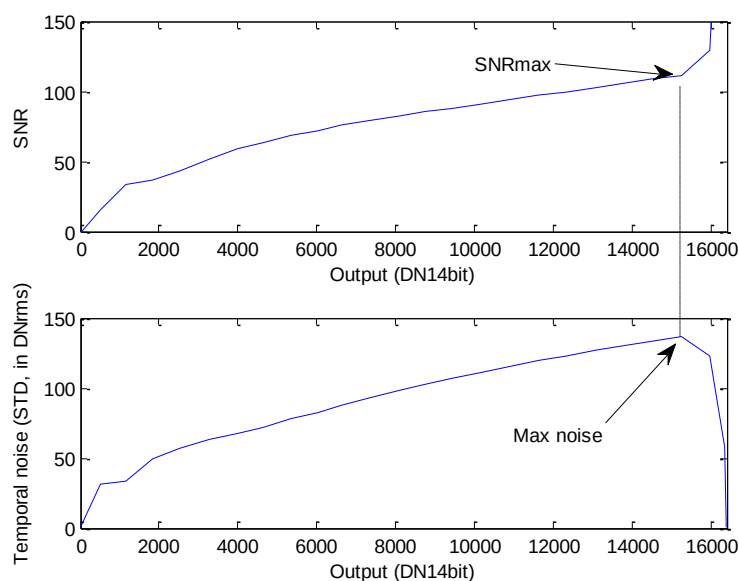


Figura 5-10 Ejemplo de relación señal a ruido y ruido temporal de un CIS

5.2.2 Método de medida

Para determinar la foto respuesta y calcular entre otros, algunos de los parámetros anteriormente descritos, se necesitan tanto la potencia óptica incidente, la cual puede ser calculada con ayuda de un foto-detector calibrado para una longitud de onda en concreto y la señal eléctrica de salida de la DUT, la cual se determina midiendo la señal de salida para un tiempo de integración fijo y diferentes niveles de luz desde oscuridad a saturación en el sensor.

5.2.3 Condiciones de la medida

El set-up de la medida es el de la siguiente figura. Idealmente este set-up debe establecerse en un banco óptico, con posibilidad de oscuridad total y ausencia de interferencias externas.

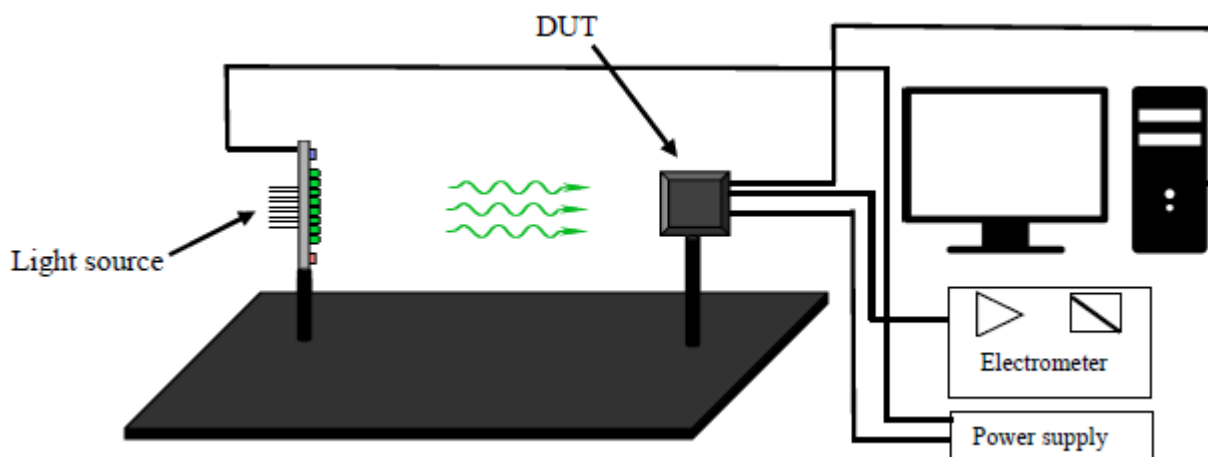


Figura 5-11 Set-up para la medida de fotorespuesta de un CIS

Los equipos utilizados en la medida son los siguientes:

1. DUT.
2. Un detector de referencia.
3. Raíles para el desplazamiento de la DUT.
4. Electrómetro.
5. Cámara link, frame grabber o ethernet dependiendo de la interfaz de salida del sensor.
6. Fuente de alimentación.
7. Fuente de luz led.

5.2.4 Procedimiento de la medida y cálculos

Primeramente se debe medir la fotocorriente de un detector calibrado para determinar la irradiancia (E) que incide en la DUT, los fabricantes de fotodiodos proporcionan una tabla para dicha conversión. Simultáneamente se puede medir la señal eléctrica de salida de la DUT a la salida del ADC para diferentes niveles de iluminación (de oscuridad hasta saturación) para obtener la curva de foto respuesta. Para calcular esta curva basta con tomar al menos 2 imágenes y calcular la media de todas las imágenes para cada paso en el barrido de luz y obtener de esta manera un nivel medio de señal.

Veamos paso a paso:

1. Utilizar una fuente de luz halógena y dejar que alcance un equilibrio térmico durante al menos 20' tras haberla encendido.
2. Asegurar que el sensor alcanza saturación en las condiciones de cada medida y colocar el sensor en la misma posición en que se ha medido la irradiancia.
3. Registrar las condiciones del test en la plantilla de la medida de forma que pueda reproducirse la medida fácilmente en un futuro.

4. Calibrar y controlar la corriente de la fuente de LED con el fin de barrer la luz desde condiciones de oscuridad a saturación.
5. Para cada paso de iluminación tomar al menos 2 imágenes y calcular la media en [V] a la vez que se mide la irradiancia E [W/cm^2] de la luz usando el detector de referencia.
6. Tomar una imagen en oscuridad para obtener el offset de referencia (μ_{dark}).
7. Trazar la curva del nivel medio de señal ($\mu - \mu_{\text{dark}}$) en [V o DN] frente a la intensidad de luz [W/cm^2] para obtener la curva de responsividad.

Hay una serie de aspectos a tener en cuenta para asegurar la precisión de la medida:

- En el caso de que la fuente de luz no sea lineal se puede hacer un barrido frente al tiempo de integración ya que idealmente los CIS se basan en la propiedad de que $t_{\text{int}} \times I_{\text{light}} = E$ (exposición) sea constante, por lo que para calcular la irradiancia idealmente es indiferente barrer la luz o el tiempo de integración.
- A veces conviene utilizar una región de interés en la que la luz incida uniformemente y esté libre de artefactos para las medidas.

La caracterización de la responsividad proporciona información sobre muchos parámetros útiles como lo son el FWC, la CVF o CG, linealidad o saturación:

➤ Capacidad de llenado

Es importante cómo se defina la capacidad de llenado (FWC), en este caso nos referiremos al nivel de saturación del sensor. Por lo tanto se calculará como el punto de intersección de nuestra recta de mejor ajuste entre en 10% y el 90% de la curva de responsividad como se puede observar en la siguiente gráfica.

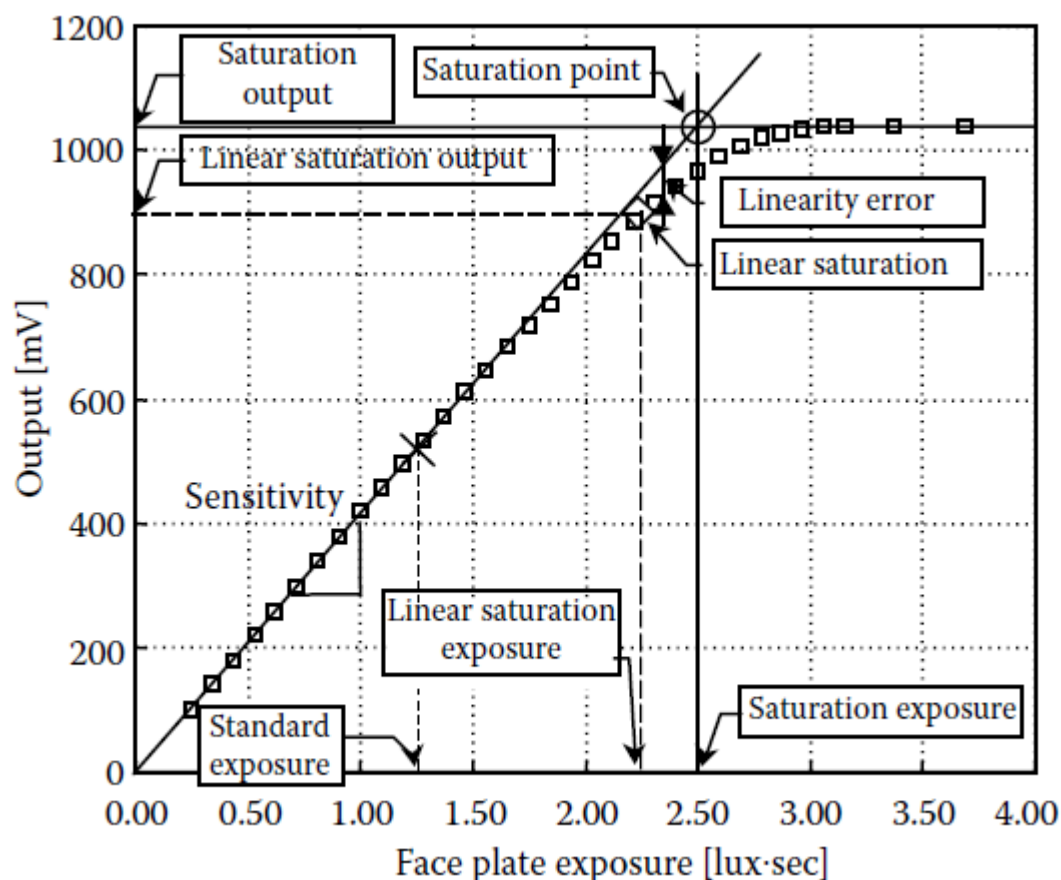


Figura 5-12 Cálculos directos en la curva de fotorespuesta

Otro método de cálculo es en función de la curva de ruido temporal de cada imagen del barrido. En

este caso el FWC corresponderá al nivel de señal de salida al que ya no depende más la salida del sensor del nivel de luz, por lo que el ruido comienza a caer.

➤ Factor de conversión de voltaje o ganancia de conversión.

Para calcular la ganancia de conversión se va a hacer uso del ya comentado método basado en la varianza de la media o método basado en la curva PTC.

Los pasos para este cálculo son sencillos:

1. Para el conjunto de imágenes capturadas, calcular la varianza de señal (ruido temporal) y el nivel medio para cada píxel a lo largo de todo el barrido. Este resultado es el promedio para todos los píxeles de su varianza (σ^2) en $[V^2 \text{ o } DN^2]$ y el nivel medio de señal (μ) en $[v \text{ o } DN]$ para toda la serie y μ_{dark} como offset de referencia.
2. Trazar la curva de varianza entre (σ^2) y ($\mu - \mu_{dark}$) y la pendiente entre el 0% y 70% de esta curva (PTC) es la ganancia de conversión en $[\mu V/e^-]$.

$$CG \left[\frac{\mu V}{e^-} \right] = \frac{V_{signal}}{\mu_e}$$

Donde,

CG: ganancia de conversión

V_{signal} : valor medio de señal del sensor

μ_e : número de electrones foto-generados

3. Calcular la capacidad de saturación y la capacidad de llenado del sensor a partir de los datos del barrido:

$$FWC[e^-] = \frac{\mu_{max}}{CG}$$

$$V_{sat} [V] = \frac{\mu_{sat} - \mu_{dark}}{CG}$$

Donde,

FWC: capacidad de llenado

CG: ganancia de conversión

V_{sat} : tensión de saturación

μ_{sat} : nivel medio imagen saturación

μ_{dark} : nivel medio imagen en oscuridad

Cabe destacar que este método no es del todo preciso y alcanza su mayor precisión si la respuesta del CIS es lineal pues asume que sólo el ruido shot contribuye al ruido total del sensor lo cual no es totalmente cierto.

➤ Rango dinámico, linealidad y relación señal a ruido.

1. Lo primero y directo es calcular el rango dinámico.

$$DR[dB] = 20 \cdot \log \frac{V_{sat}}{\sigma_{dark}}$$

Donde,

V_{sat} : nivel de señal de saturación (sustrayendo la señal en oscuridad)

σ_{dark} : ruido temporal en oscuridad

2. Más tarde podemos calcular la no linealidad a partir del nivel máximo de señal tras calcular su recta de mejor ajuste.

$$LE[\%] = \frac{E_i - \mu_i}{\mu_i} \cdot 100$$

Donde,

LE: error de linealidad

E_i : error de la recta de mejor ajuste [DN]

μ_i : nivel medio de la imagen i [DN]

3. Por último la relación señal a ruido para cada nivel de señal.

$$SNR[dB] = 20 \cdot \log \frac{\mu_i}{\sigma_i}$$

Donde,

μ_i : nivel medio de la imagen i

σ_i : desviación estándar de la imagen i

4. El nivel de negro se calcularía como la media del valor de las imágenes en oscuridad.
5. El Full Scale (FS) se computaría como la media entre el nivel de saturación y el nivel de oscuridad. Sería el rango útil del CIS.

$$FS[DN] = \mu_{sat} - \mu_{dark}$$

Donde,

μ_{sat} : nivel medio imagen saturada

μ_{dark} : nivel medio imagen en oscuridad

5.3 Medidas espectrales

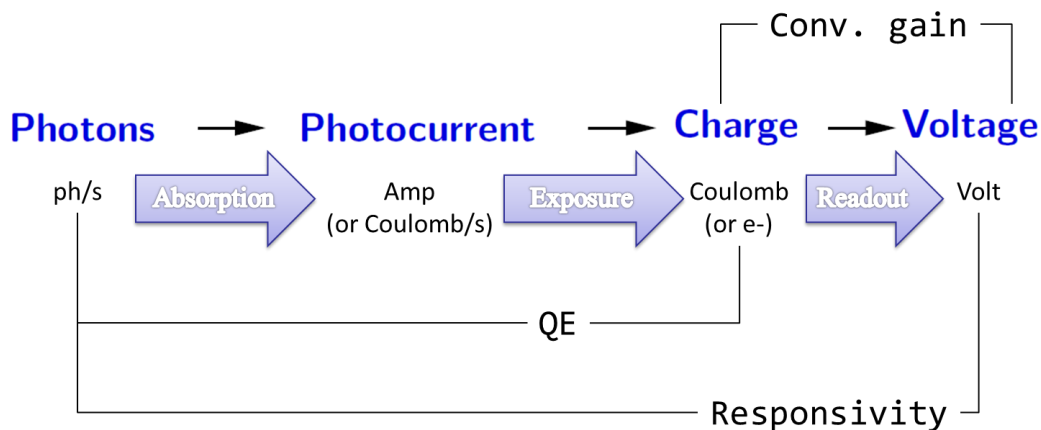


Figura 5-13 Relaciones entre las medidas espectrales de un CIS

5.3.1 Fundamento teórico

5.3.1.1 Eficiencia Cuántica

La eficiencia cuántica o *quantum efficiency* (QE) es uno de las características más importantes de cualquier dispositivo electro-óptico, incluyendo los CIS. La QE es el ratio entre el promedio de número de electrones generados (μ_e) en el píxel que contribuyen a la señal de salida y el promedio del número de fotones incidentes (μ_p) en el píxel durante el tiempo de exposición.

La siguiente figura muestra la generación de un par electrón-hueco en una unión p-n debido a la incidencia de un fotón con una cierta longitud de onda y energía.

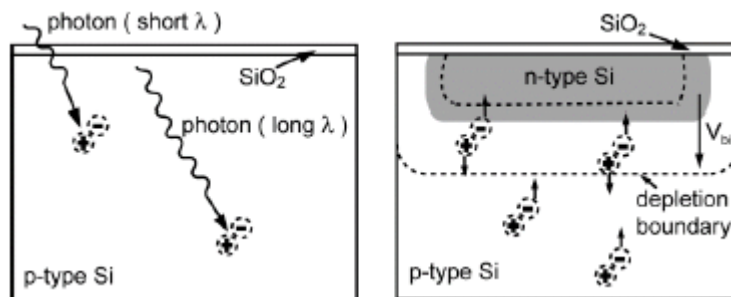


Figura 5-14 Efecto fotoeléctrico en el CIS

Como ya se ha comentado anteriormente, cuando un fotón es absorbido por el PPD, este genera portadores libres (electrón-hueco) que contribuyen a la conductividad del material, este fenómeno se debe al mundialmente conocido como efecto foto-eléctrico.

5.3.1.2 Respuesta Espectral

La sensibilidad espectral o respuesta espectral (SR) determina la cantidad de foto-corriente generada por el CIS en función de los fotones incidentes expresados en unidad de A/W y para diferentes longitudes de onda de la luz.

En los resultados de la medida, la QE y la SR pueden oscilar debido al efecto de franjas o efecto Etaloning mostrado en la siguiente figura.

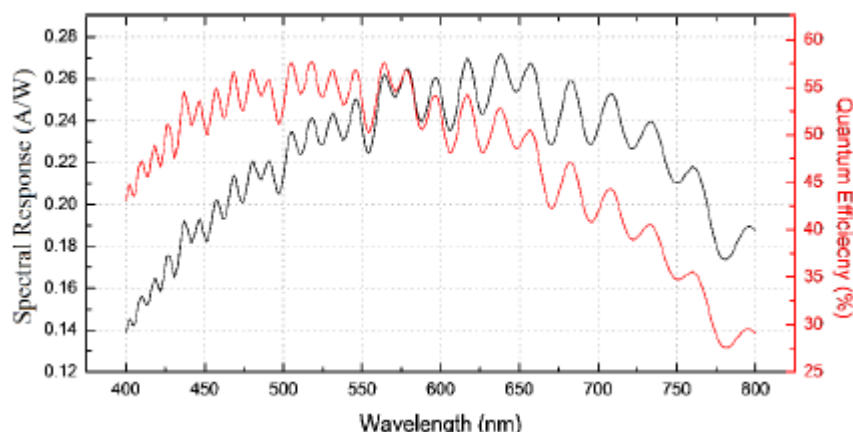


Figura 5-15 Efecto rizado o “Etaloning” en un CIS

Este efecto consiste en que cuando un fotón con cierta longitud de onda choca con el PPD, este atraviesa las diferentes capas de SiO_2 y parte de la capa epitaxial antes de alcanzar la zona de depleción. El efecto Etaloning es el resultado de patrones de interferencias creados por las múltiples reflexiones de los fotones dentro de las capas de metal, las cuales se deben a la diferencia de capacidad de reflexión y transmisión de las mismas.

5.3.1.3 Fill Factor

Normalmente en un CIS se considera la eficiencia cuántica incluyendo el factor de llenado o Fill Factor (FF). La FFQE (*Fill Factor-Quantum Efficiency*) se refiere a todo el área de luz ocupada por el sensor, no únicamente el área sensible a la luz. Normalmente la QE se expresa en porcentajes o tanto por uno.

Idealmente la QE de un CIS debería ser en torno al 100%, un par electrón-hueco generado por cada fotón incidente, aunque en realidad el efecto fotoeléctrico tiene algunas limitaciones asociadas al PPD.

- La primera de ellas es la pérdida de fotones incidentes debido a problemas ópticos como pueden ser la reflexión de la luz en la superficie del sensor o la foto absorción por parte de algunas capas de metales del layout.
- La segunda limitación es la ineficiencia del fotodiodo a la hora de recolectar todos los pares electrón-hueco generados, esta ineficiencia es el resultado de la generación de pares fuera de la región de depleción del PPD. Como la absorción de fotones incidentes depende del coeficiente de absorción del silicio, el cual depende de la longitud de onda, los fotones con una gran longitud de onda penetrarán más y los portadores libres puede que salgan de la zona de depleción de manera que sea difícil recolectarlos disminuyendo la QE.

5.3.1.4 Sensibilidad

Es la mínima exposición que permite ser detectada por el sensor en $\text{Lux}\cdot\text{s}$. Esta es la principal característica que permite comparar sensores en bajas condiciones de iluminación. Está asociada a la sensibilidad parásita a la luz del sensor, que indica la relación entre la respuesta del PD y de la FD a la luz dependiente del tiempo de exposición, es decir si se capta luz durante el proceso de lectura, es especialmente importante en el modo de captura Global Shutter debido a que se exponen muchas filas de píxeles a la vez antes de leer y el dato permanece en cada fila una cantidad de tiempo diferente hasta ser leído.

5.3.2 Método de medida

La eficiencia cuántica es un parámetro que se suele medir de manera relativa en tanto por ciento o tanto por uno y que junto a la respuesta espectral dependen directamente de la longitud de onda de la luz y de la cantidad de fotones incidentes. Conociendo la QE se puede deducir fácilmente la SR y viceversa. Normalmente dependiendo del proyecto existen dos situaciones:

1. El diseño del CIS está orientado a un valor de QE y SR determinado.
2. El CIS está diseñado cuidando más concienzudamente otra serie de características.

5.3.3 Condiciones de la medida

Está claro que para la medida es necesario disponer de una fuente de luz monocromática, una intensidad de luz calibrada y un dispositivo para medir la corriente. Idealmente para estos propósitos se utiliza un banco óptico la siguiente lista de equipos:

1. DUT.
2. Un detector de referencia.
3. Fuente de alimentación.
4. Electrómetro.
5. Monocromador.

El set-up del test es el mostrado en la siguiente figura asumiendo la utilización de toda la instrumentación anteriormente mencionada. Destacar el uso de una luz de amplio espectro como una lámpara halógena entra en el monocromador es colimada y difractada a través de una estructura regulable en función de la longitud de onda que se desee obtener para ser proyectada hacia la DUT. Tanto el monocromador como el electrómetro son controlados de forma remota por el ingeniero.

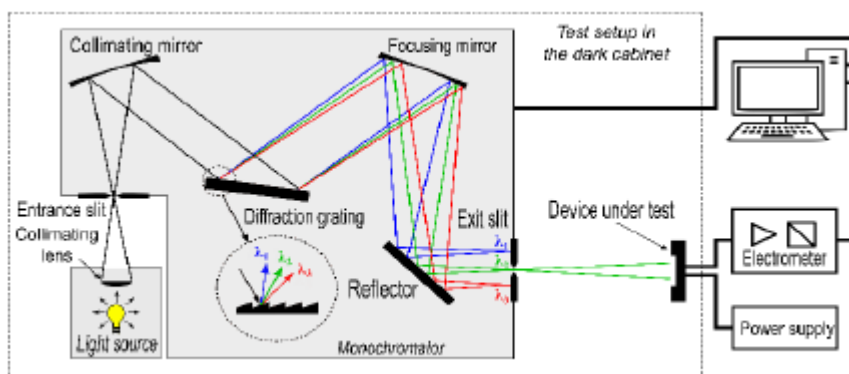


Figura 5-16 Set-up para medidas espectrales



Figura 5-17 Banco óptico

5.3.4 Procedimiento de la medida y cálculos

El principio detrás de la medida de la QE consiste en primeramente medir la fotocorriente con un detector calibrado en función de la longitud de onda para determinar la irradiancia E que incide en la DUT, los fabricantes de fotodiodos proporcionan una tabla para dicha conversión. Con este dato se puede medir sin problemas la respuesta espectral y si el CVF es conocido, se puede calcular para la tensión de salida de la DUT el número de electrones que salen. Luego basta con calcular el ratio entre estos electrones y el número de fotones incidentes para calcular la QE.

De manera más sistemática, paso a paso:

1. Utilizar una fuente de luz halógena y dejar que alcance un equilibrio térmico durante al menos 20' tras haberla encendido.
2. Colocar el detector de referencia bajo las condiciones de iluminación deseadas y medir la corriente para cada longitud de onda de interés. Utilizar el dato de calibración del detector de referencia (en A/W) para calcular la irradiancia E (en W/cm²) de la salida en corriente del detector (en A). Todo ello resulta en el promedio de fotones incidentes (μ_p).
3. Colocar la DUT en la misma posición en la que se colocó el detector de referencia y seleccionar una región de interés (ROI) con el objetivo de obtener un resultado lo más preciso.
4. Ahora con la DUT iluminada con la misma longitud de onda se hace un barrido de tiempo de exposición para cada longitud de onda de interés y una luz fija. Quitándole a cada imagen, una imagen en oscuridad y midiendo la señal en tensión de salida de la DUT y utilizando el nivel medio de señal y el CVF se pueden obtener los electrones generados (μ_e).

Los cálculos asociados a esta medida son los siguientes:

1. Cálculo de la responsividad para determinada longitud de onda.

$$Resp(\lambda) = (\mu_y - \mu_{y,dark}) \cdot \frac{h \cdot c}{\lambda \cdot AE \cdot t_{exp}}$$

Donde,

μ_y : valor medio imagen con iluminación uniforme [DN]

$\mu_{y,dark}$: valor medio imagen en oscuridad [DN]

λ : longitud de onda del fotón incidente [nm]

h: constante de Planck = 6.626×10^{-34} [J·s]

c: velocidad de la luz = 2.99792×10^{10} [cm/s]

A: área del píxel [cm²]

E: intensidad de la luz incidente en el sensor [W/cm²]

t_{exp} : tiempo de exposición de la imagen [s]

Esta respuesta si se calcula para diferentes ángulos de rotación corresponde a la respuesta angular del sensor.

2. Cálculo de la eficiencia del Shutter como medida comparativa entre la sensibilidad del PD y la FD

$$SE(\lambda) = \frac{Resp_{PPD}(\lambda) - DC_{PPD}}{Resp_{FD}(\lambda) - DC_{FD}}$$

Donde,

SE (λ): eficiencia del Shutter

DC_{PPD}: corriente oscura PPD [DN/s]

DC_{FD}: corriente oscura FD [DN/s]

Resp_{PPD} (λ): responsividad PPD a cierta longitud de onda [DN/s]

Resp_{FD} (λ): responsividad FD a cierta longitud de onda [DN/s]

3. Sensibilidad parásita a la luz (PLS) como inversa de la eficiencia del Shutter. Con esta medida se caracteriza la respuesta del PDD frente a la FD.

$$PLS(\lambda) = \frac{1}{SE(\lambda)}$$

4. Cálculo del ruido equivalente en exposición (NEE), definido como la exposición a la cual el nivel de señal alcanza el piso de ruido de lectura, es decir cuando la SNR es 1, este es el umbral a partir del cual el sensor genera un dato real.

$$NEE(\lambda) = \frac{\sigma_y \cdot CG}{Resp(\lambda)}$$

Donde,

NEE(λ): ruido equivalente de exposición [e^-]

σ_y : desviación estándar de la imagen en oscuridad [DN]

CG : ganancia de conversión [DN/uV]

Resp(λ): responsividad PPD a cierta longitud de onda [DN/us]

5. Cálculo de la exposición en saturación (ESAT), definida como el máximo valor de exposición al cual el sensor genera un dato válido sin perder información.

$$ESAT(\lambda) = \frac{\mu_{y.sat} - \mu_{y.dark}}{Resp(\lambda)}$$

Donde,

ESAT(λ): Exposición en saturación [us]

$\mu_{y.sat}$: valor medio imagen saturada [DN]

$\mu_{y, dark}$: valor medio imagen en oscuridad [DN]

Resp(λ): responsividad PPD a cierta longitud de onda [DN/us]

6. Cálculo de la eficiencia cuántica. Existen diferentes fórmulas:

$$QE(\lambda) = \frac{\mu_e}{\mu_p} \quad ; \quad QE[\%] = \frac{SR \cdot h \cdot c}{q \cdot \lambda} \times 100$$

$$SR \left[\frac{A}{W} \right] = \frac{QE \cdot \lambda \cdot q}{h \cdot c} \quad ; \quad SR[A/W] = \frac{I_{ph}}{E \cdot A}$$

Donde,

λ : longitud de onda del fotón incidente [nm]

q: carga de electrón = 1.602×10^{-19} [C]

h: constante de Planck = 6.626×10^{-34} [J·s]

c: velocidad de la luz = 2.99792×10^{10} [cm/s]

A: área del píxel [cm²]

E: intensidad de la luz incidente en el sensor [W/cm²]

μ_e : número de electrones generados en el píxel

$$\mu_e = \frac{V_{out}}{CVF}$$

μ_p : número de fotones incidentes en el píxel de área A [cm^2] durante el tiempo de integración t_{int} [s] para una irradiancia conocida E [W/cm^2],

$$\mu_p = \frac{A \cdot t_{int} \cdot E}{h \cdot c / \lambda}$$

I_{ph} : fotocorriente, que puede ser calculada en función de la carga foto-inducida colectada durante el periodo de integración t_{int} .

$$I_{ph} = \frac{Q_{ph}}{t_{int}} = \frac{q \cdot (V_{out}/CVF)}{t_{int}}$$

Existe una serie de aspectos a tener en cuenta para asegurar la precisión de la medida:

- Hay que tener en cuenta que el nivel de señal de salida del sensor debe ser mayor que la corriente oscura para una medida precisa y correcta.
- Lo ideal es hacer la medida en un habitáculo en total oscuridad y sin óptica.
- La inestabilidad de la fuente de luz puede afectar a la medida.
- Des-alineamientos a la hora de reemplazar el foto-detector por el sensor puede provocar disparidad en los resultados de la medida.

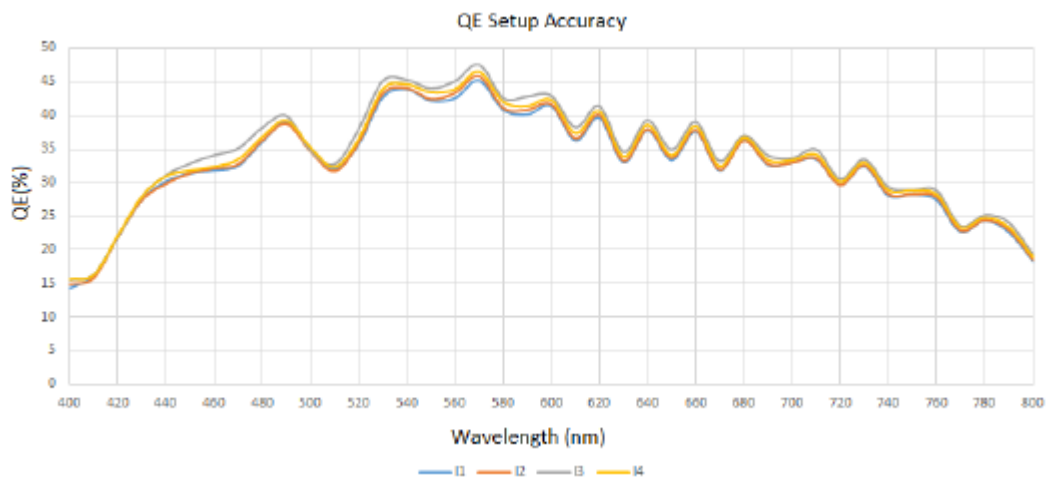


Figura 5-18 Ejemplo medidas eficiencia para un mismo CIS

5.4 Función de Transferencia de Modulaci3n (MTF)

5.4.1 Fundamento te3rico

La funci3n de transferencia de modulaci3n o *Modulation Transfer Function* (MTF) es la habilidad de un sensor de captar contrastes en una frecuencia espacial en particular. Es una medida directa sobre la calidad y la resoluci3n del sensor de imagen. La respuesta ideal de un píxel es un impulso con un ancho equivalente a su 3rea pero debido a interferencias 3pticas y el3ctricas los píxeles suelen compartir informaci3n con sus vecinos. Por lo tanto el resultado de la respuesta de un píxel viene a ser aproximadamente una respuesta Gaussiana m3s que una respuesta impulsiva. Este cross-talk 3ptico y el3ctrico puede ser cuantificado por la MTF la cual es b3sicamente la FFT de la respuesta del píxel.

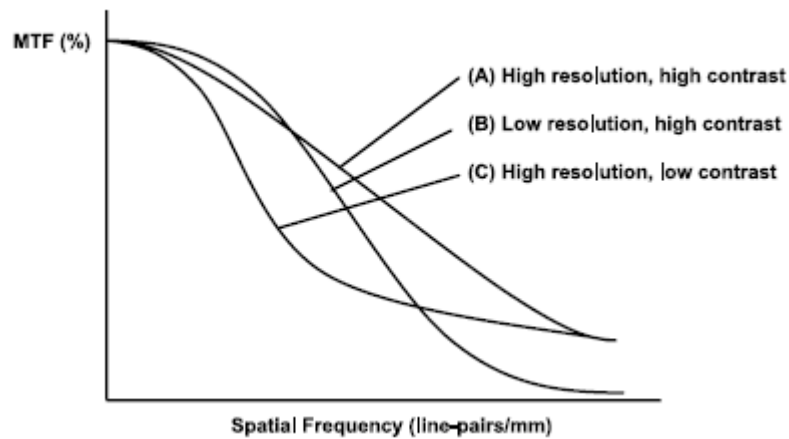


Figura 5-19 Curva conceptual MTF

(a) High resolution, high contrast



(b) Low resolution, high contrast



(c) High resolution, low contrast



Figura 5-20 Comparativa resolución espacial en imágenes reales

Para calcular la respuesta en frecuencia, el píxel puede ser expuesto alternativamente a un patrón de líneas blancas/negras con un grosor de la línea equivalente al área del píxel. La situación especial en la que la frecuencia espacial de línea blanca/negra corresponde exactamente a dos veces el área del píxel corresponde a la frecuencia de Nyquist:

$$f_{Nyq} = \frac{1}{2P} \quad [lp/mm]$$

Siendo P el área del píxel en cuestión.

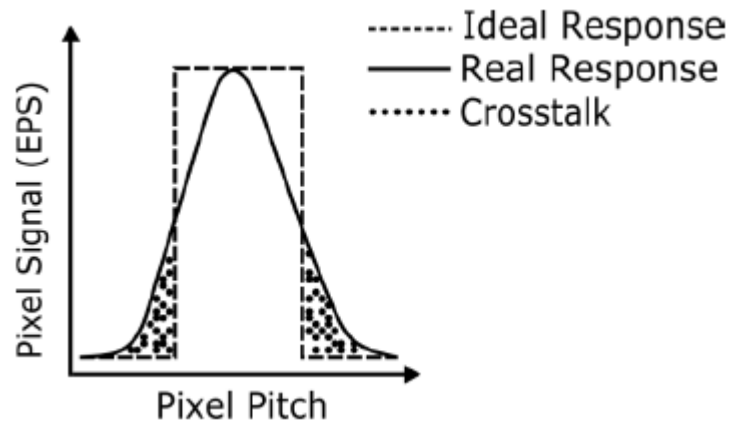


Figura 5-21 Interferencias a nivel de píxel

Estas interferencias pueden poseer diversas fuentes listadas a continuación:

- Difusión lateral de los fotones cuando una luz incidente rebota dentro del chip.
- Interferencias ópticas debidas a fenómenos como difracción, refracción, dispersión, interferencias y reflexión de la luz.
- Interferencias eléctricas resultado de la foto-generación de portadores teniendo la posibilidad de moverse a píxeles vecinos.
- Otras limitaciones físicas como el ángulo de incidencia de la luz.

La MTF a la frecuencia de Nyquist es a menudo empleada como medida de referencia para estas interferencias:

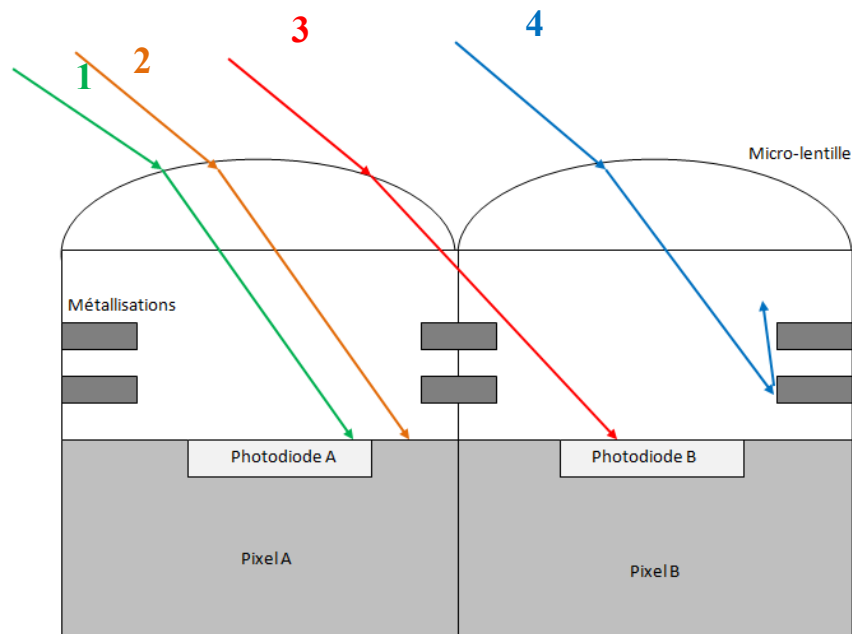


Figura 5-22 Tipos de interferencias entre píxeles

1. La microlente corrige correctamente la trayectoria del fotón el cual es bien captado por el píxel correspondiente.
2. A pesar de la microlente el fotón es perdido ya que no logra ser captado por el fotodiodo.
3. Se produce crosstalk, ya que a pesar de la microlente el fotón llega al fotodiodo incorrecto.
4. El fotón choca contra los metales y se redirecciona a otro lugar incluso fuera del píxel.

5.4.2 Método del Borde Inclinado

Hay diferentes métodos para medir la MTF de un sensor como el método del seno o del borde afilado pero estos métodos sufren como inconvenientes un gran tiempo de computación y de gran cantidad de imágenes capturadas. Sin embargo, el método del borde inclinado o Slanted Edge Method únicamente requiere de una imagen para computar la MTF.

Este método consiste en colocar un borde sobre el sensor ligeramente inclinado respecto a las filas o columnas. Este borde inclinado verticalmente permite obtener la respuesta espacial horizontal en frecuencia (Spatial Frequency Response, SFR) del sensor. En ese caso, la respuesta de cada línea tiene una separación de borde diferente (ESF) debido a diferentes fases y estos datos se pueden utilizar para calcular para calcular la MTF.

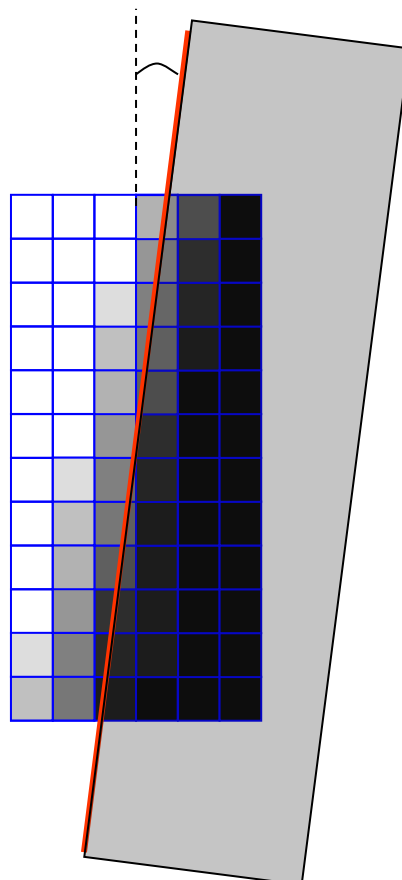


Figura 5-23 Concepto teórico medida MTF

5.4.3 Método de medida

El siguiente procedimiento de medida está basado en el método del borde inclinado y resultará en la función de extensión del borde (ESF), la función de extensión de línea (LSF) y la función de transferencia de modulación (MTF) del sensor.

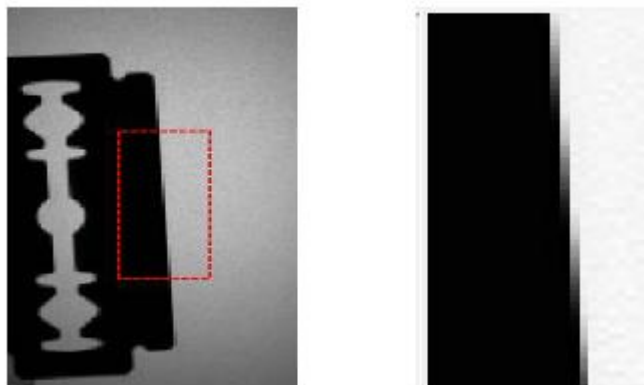


Figura 5-24 Concepto real medida MTF

Primero se deben corregir las imágenes de no uniformidades. Tras ello se debe encontrar el punto de transición (posición del borde), es decir la posición de los píxeles donde el valor del píxel tiene un 50% del pico de intensidad para más tarde determinar la respuesta SFR de cada transición.

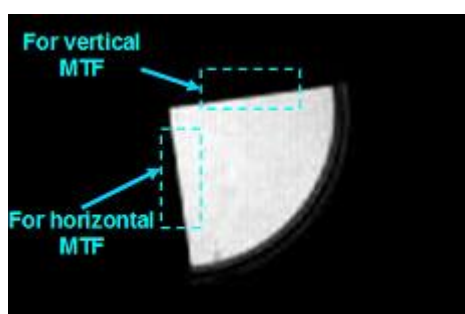


Figura 5-25 MTF horizontal y vertical

Para calcular la MTF del sistema primero hay que determinar la ESF la cual asume que la SFR para todas las líneas perpendiculares al borde inclinado es la misma. La respuesta de una única línea dará una bajo-muestreada ESF, por lo que combinando la respuesta de múltiples líneas con una posición diferente respecto al borde el muestreo de la ESF es mucho mejor.

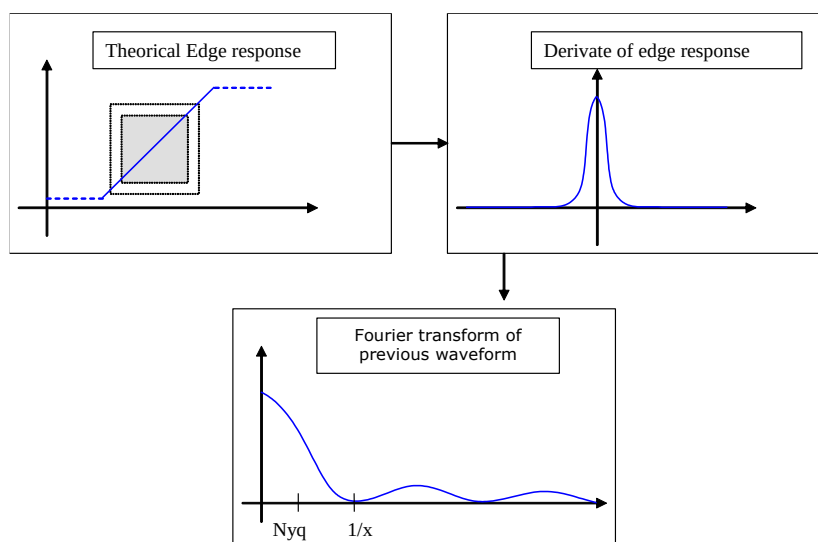


Figura 5-26 Medida MTF paso a paso

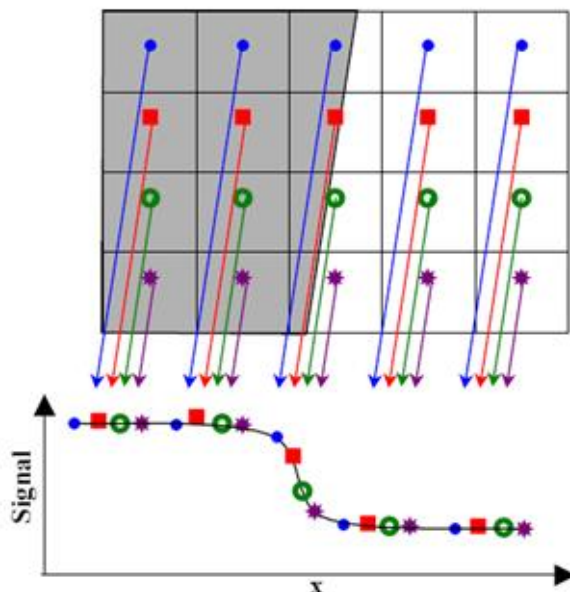


Figura 5-27 Curva MTF al detalle

5.4.4 Condiciones de la medida

El set-up de la medida es el de la siguiente figura. Idealmente este set-up debería establecerse en un banco óptico, con posibilidad de oscuridad total y ausencia de interferencias externas. Para construir este set-up se debe enfocar concienzudamente el centro de la lente con el centro del objeto con borde inclinado, el cual deberá colocarse entre la luz y el sensor.

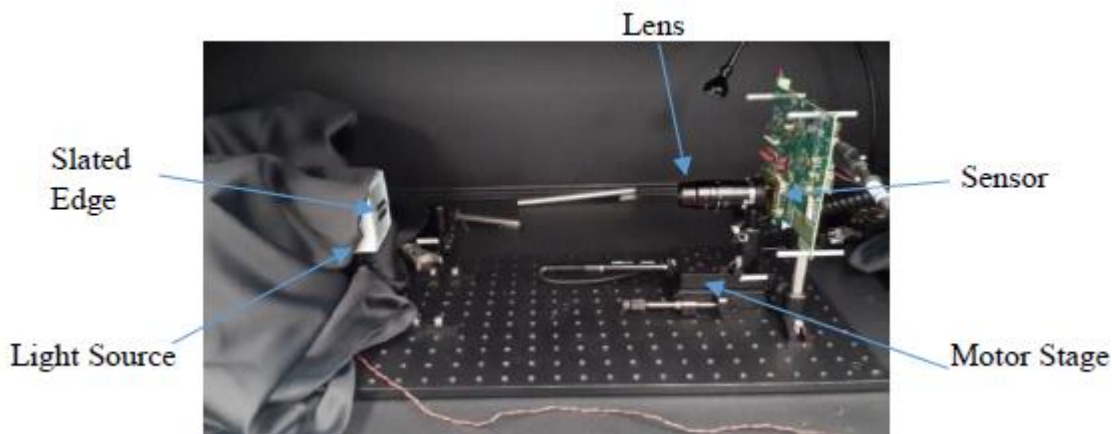


Figura 5-28 Set-up medida MTF

Los equipos utilizados en la medida son los siguientes:

1. DUT
2. Raíles para el desplazamiento de la DUT a la hora de enfocar.
3. Objeto con borde inclinado.
4. Lente apropiada para el sensor.
5. Cámara link, frame grabber o ethernet dependiendo de la interfaz de salida del sensor.
6. Fuente de alimentación.
7. Fuente de luz led.
8. Motor controlable (opcional) para modificar la distancia entre sensor y objeto y enfocar más fácilmente.

5.4.5 Procedimiento de la medida y cálculos

1. Colocar la DUT como se describe en el set-up de medida con una distancia de no más de 30cm entre sensor y fuente de luz.
2. Ajustar la luz a un 50% de saturación y esperar un tiempo hasta que la luz sea estable.
3. Elegir y colocar la óptica adecuada.
4. Capturar tres imágenes bajo las mismas condiciones para tres distancias diferentes con el fin de obtener la imagen mejor enfocada:
 - a. Tomar una imagen en oscuridad como referencia.
 - b. Tomar una imagen con luz como segunda referencia.
 - c. Tomar una imagen con el objeto de borde inclinado entre 1 y 10 grados de ángulo.
5. Corregir las imágenes de no uniformidades en oscuridad y con luz mediante las imágenes de referencia capturadas.

En el siguiente algoritmo de tratamiento de datos cabe reseñar que los cálculos están hechos para un borde vertical, en el caso de tener un borde horizontal basta con intercambiar las columnas por las filas.

Para calcular la MTF de un sensor hay que seguir los siguientes pasos:

1. Corrección de la imagen:
 - a. Tanto la imagen de luz como la imagen de borde son corregidas en offset restándoles la imagen en oscuridad de referencia.
 - b. La corrección obtenida (luz-oscuridad) se utilizará para crear un mapa de ganancia para cada píxel, a este mapa lo llamaremos factor de ganancia.
 - c. Las imágenes capturadas sufren de no uniformidades y offset. Por lo que para medir la MTF con precisión es necesario corregir las imágenes capturadas y aplicar una corrección para eliminar los artefactos.

$$C = \frac{(Edge - Dark) \cdot m}{(Light - Dark)}$$

Donde,

C: imagen corregida

Edge: imagen en crudo a corregir

Dark: imagen oscura de calibración

Light: imagen con luz de calibración

m: Valor medio de (Light-Dark)

Es esencial que el borde inclinado se coloque en el centro del sensor y en el centro de la lente debido a que el contraste y la resolución de una imagen poseen típicamente su valor óptimo en el centro de la imagen. Otro punto importante de la medida es considerar que si se capturan las imágenes con una lente, un mal enfoque reduce la forma y el contraste de la imagen por lo que podría degradar la MTF. Básicamente cualquier tipo de aberración óptica puede degradar la MTF del sensor.

- d. Normalizar la imagen corregida.
2. Calcular la ESF:

Es la suma de la SFR para todas las transiciones. Para determinar las diferentes SFR se debe computar la posición del píxel ($Threshold_{col}$) a partir de la cual el valor del píxel es mayor de 0.5 en un valor de intensidad normalizado como se muestra en la figura y determinar el punto de transición exacto ($col_{intercept}$) para todas las líneas del borde con la siguiente fórmula:

$$col_{intercept} = Threshold_{col} + \frac{(col_0 - 0.5)}{(col_1 - col_0)}$$

Donde,

$col_{intercept}$: punto de transición

$Threshold_{col}$: columna para la que el valor del píxel es mayor que 0.5

Col_0 : Valor del píxel en la posición $Threshold_{col}$ de la línea

Col_1 : Valor del píxel en la posición $(Threshold_{col} - 1)$ de la línea

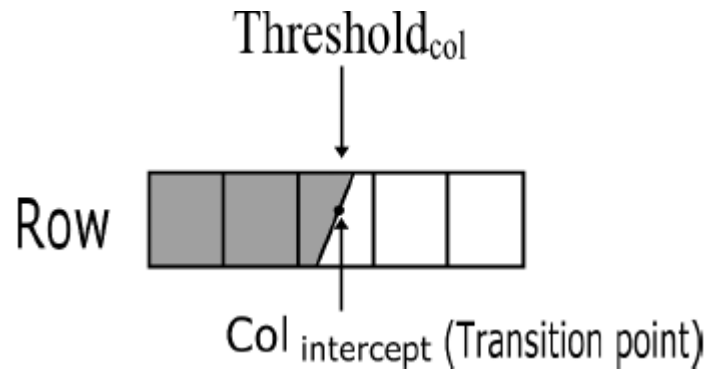


Figura 5-29 Cálculo ESF

Ahora a partir de este punto de transición y para los 10 píxeles colindantes a ambos lados se genera la curva SFR para cada fila como en la siguiente figura.

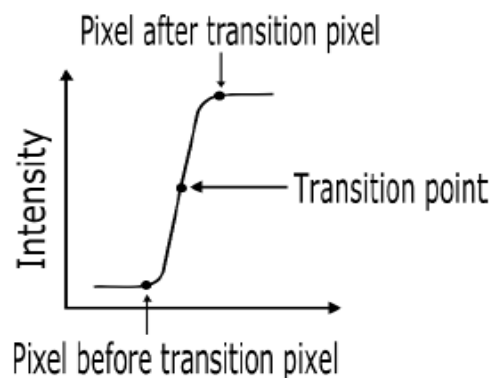


Figura 5-30 Curva SFR para un píxel

Por último resta combinar todas las curvas SFR y dibujar respecto a la posición del píxel la curva ESF. El dato de la ESF está altamente sobre-muestreado por lo que una interpolación lineal de esta curva podría arrojar resultados bastante más fiables.

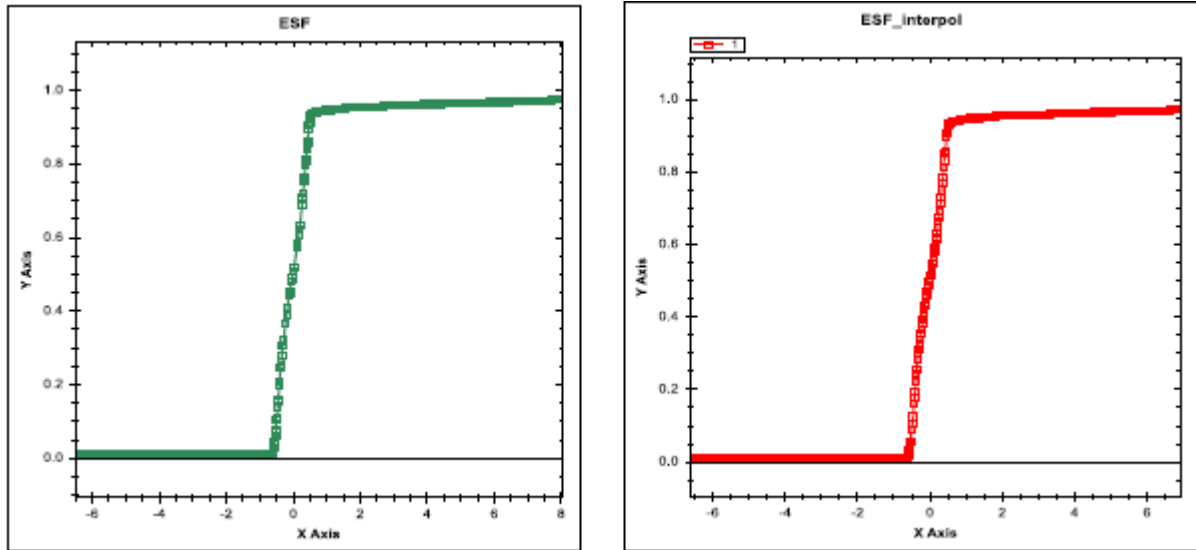


Figura 5-31 Ejemplo curva ESF para un CIS

3. Cálculo de la LSF:

El LSF es la derivada de la ESF por lo que se podría emplear un método derivativo de dos o tres puntos para determinarlo:

$$LSF(x) = \frac{d}{dx} ESF(x)$$

$$LSF_i = \frac{1}{2} \left(\frac{ESF_{i+1} - ESF_i}{col_{i+1} - col_i} - \frac{ESF_i - ESF_{i-1}}{col_i - col_{i-1}} \right)$$

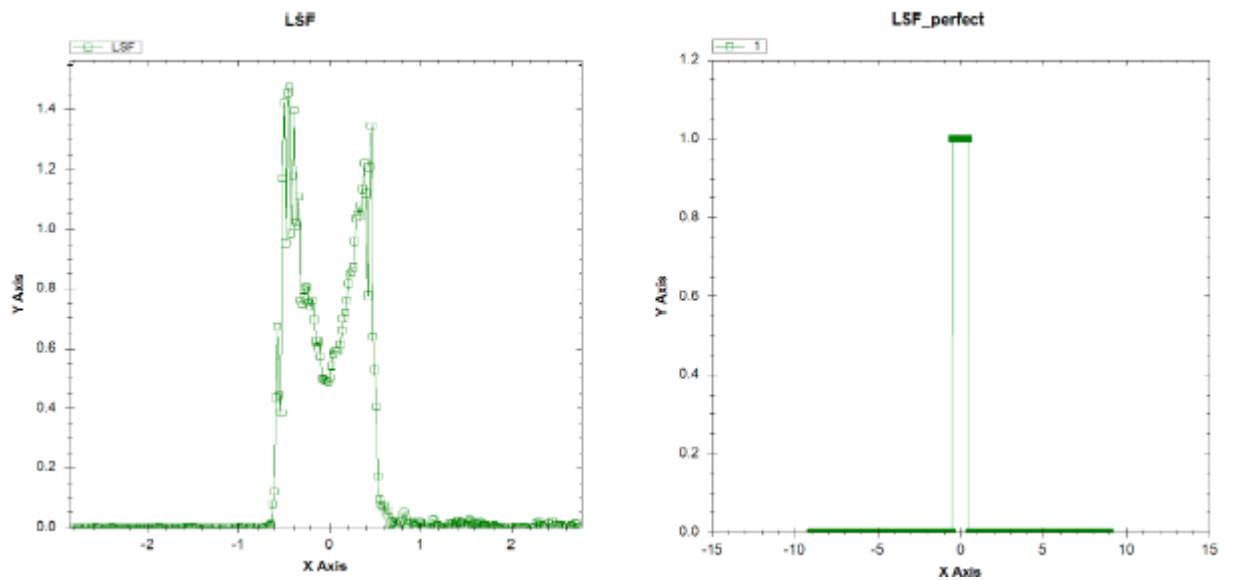


Figura 5-32 Ejemplo LSF para un CIS frente a la ideal

4. Cálculo de la MTF:

A través de la FFT de la LSF y tomando el módulo normalizado, teniendo en cuenta la máxima frecuencia de muestreo realizable o frecuencia de Nyquist como dos veces el área del píxel.

$$MTF = |FFT(LSF)|$$

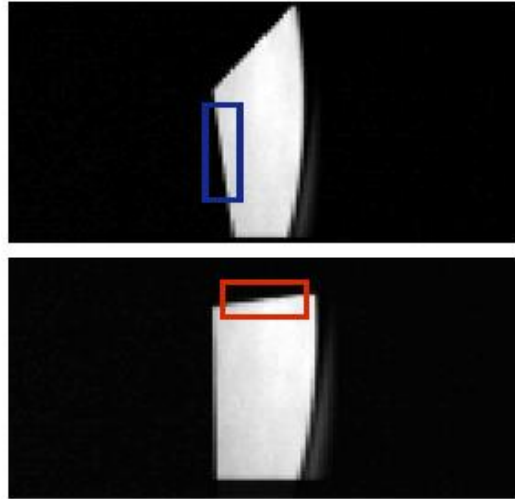


Figura 5-33 V-MTF y H-MTF

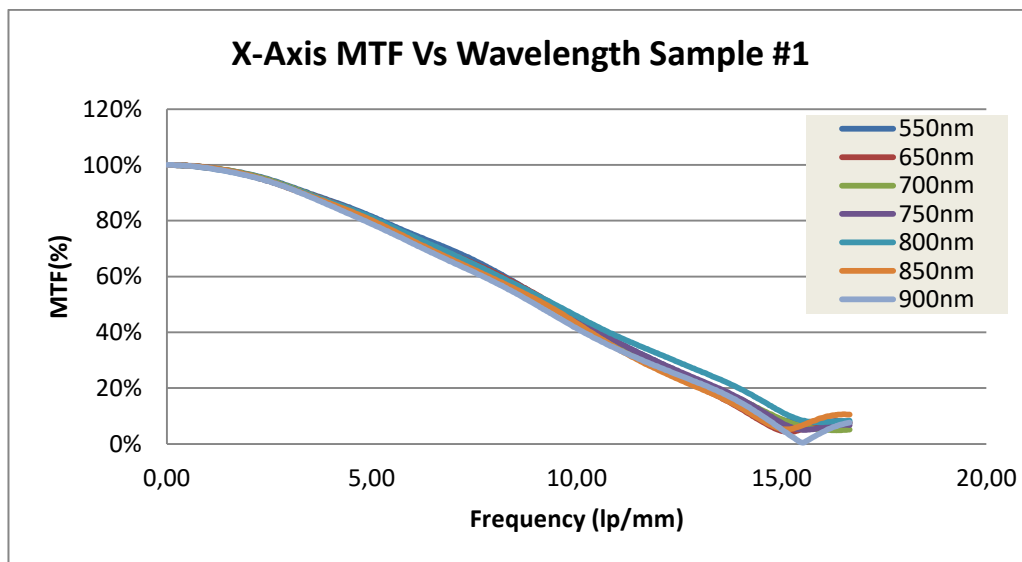


Figura 5-34 H-MTF

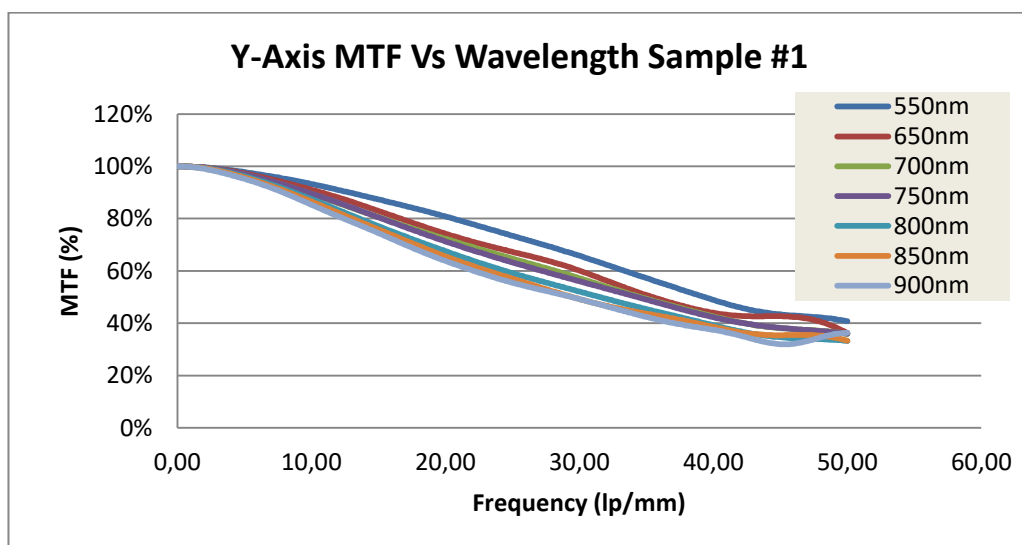


Figura 5-35 V-MTF

5.5 Ruidos y No Uniformidades

Todos los circuitos integrados sufren el ruido y por lo tanto los CIS también. Los CIS convierten la información en forma de luz en una señal eléctrica y durante el procesado de esta información un CIS sufre de dos tipos de ruidos; ruido temporal y ruido espacial. El ruido se define como cualquier fluctuación en el valor de una señal modelable como un proceso aleatorio. Matemáticamente el ruido puede ser estimado computando la varianza de la señal. Por lo tanto en nuestro caso lo computamos como números de electrones. El estado del arte actual de los CIS permite reducir el ruido de los sensores por debajo de $1 e^-_{\text{rms}}$.

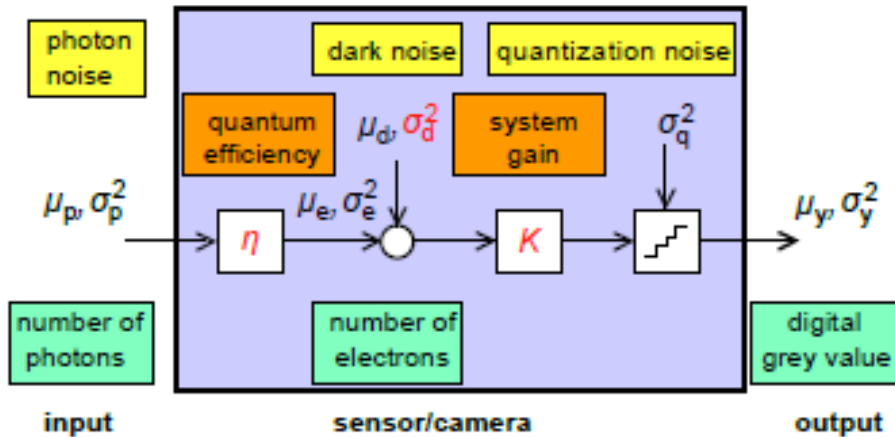


Figura 5-36 Ruidos en un CIS

Antes de medir el ruido es clave conocer de qué parte del sensor proviene cada ruido. En la siguiente figura se dibuja sobre el esquema de un píxel 4T las principales fuentes de ruido. La figura únicamente indica la mayor fuente de ruido: ruido de reset, corriente oscura, ruido flicker ($1/f$), no uniformidad en la foto respuesta (PRNU) y no uniformidad en ausencia de señal (DSNU), además hay otras componentes de ruido que también contribuyen al ruido total del sistema, pero de manera residual.

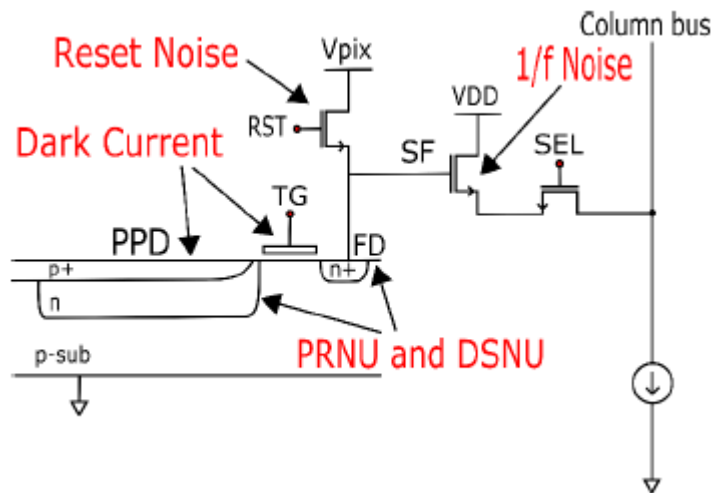


Figura 5-37 Ruidos en un píxel

5.5.1 Parámetros a medir y fundamento teórico

5.5.1.1 Ruido temporal

El ruido temporal es la variación de la respuesta del píxel para una iluminación constante a lo largo de una toma consecutiva de imágenes. Este ruido se puede evaluar calculando la desviación estándar de la respuesta del píxel para cada píxel a lo largo de un buen número de frames tomados con un periodo de tiempo para una iluminación específica y constante. Este ruido no es calibrable pues sólo se puede mitigar con técnicas de diseño orientadas a bajo nivel de ruidos.

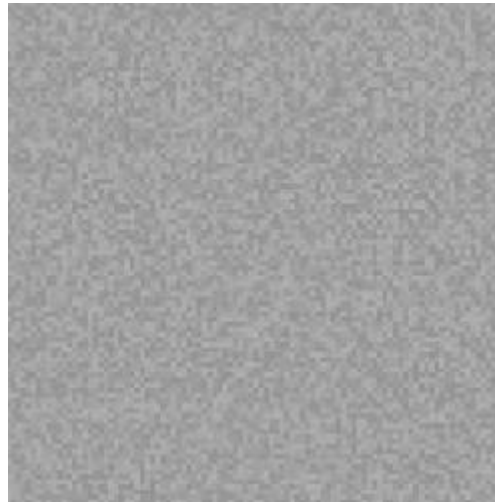


Figura 5-38 Ruido temporal en una imagen

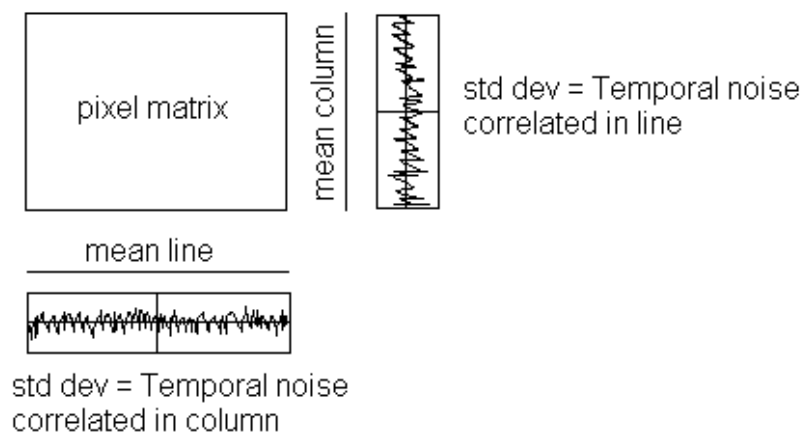


Figura 5-39 Cálculo del ruido temporal en una imagen

El ruido temporal posee los siguientes componentes ordenados de mayor a menor contribución:

1. Ruido shot de los componentes resistivos
2. Ruido de reset o kTC a no ser que se cancele con una técnica CDS (doble muestreo correlado).
3. Ruido Flicker de los transistores, principalmente del SF.
4. Ruido de cuantización del ADC.
5. Interferencias electromagnéticas (EMI).
6. Jitter en el timing de las señales.
7. Contribuciones de otros bloques dentro del chip o fuera.

Para estimar el ruido de un sensor de imagen es clave modelarlo matemáticamente. La variable más utilizada para este modelado es la variable aleatoria Gaussiana. Esta variable es utilizada para describir la distribución de la magnitud de gran cantidad de fuentes de ruido. Si la señal de ruido es una variable aleatoria Gaussiana, el proceso de ruido blanco es un ruido blanco Gaussiano (WGN). Dos de los procesos de ruido más importantes en los circuitos integrados como lo son el ruido térmico y el ruido shot, son modelables como un proceso WGN.

➤ Ruido fotónico shot

El ruido shot es debido a la variación estadística de los pares electrón-hueco generados debido a la aleatoria llegada de fotones incidentes. Esta incidencia de luz se puede modelar como una distribución Poisson, por lo tanto la magnitud de ruido shot (η_{photon}) es igual a la raíz cuadrada del electrón foto generado (μ_e):

$$\eta_{photon} = \sqrt{\mu_e}$$

El ruido shot es uno de los ruidos fundamentales en cualquier sistema fotónico ya que procede de la física de los semiconductores y no del diseño de los sensores de imagen.

➤ Ruido térmico shot

El ruido térmico shot es el resultado de la generación térmica aleatoria de los pares electrón-hueco en oscuridad, lo que quiere decir que depende exponencialmente de la temperatura. Sigue también una distribución estadística de Poisson, por lo que lo podemos computar como

$$\eta_{dc} = \sqrt{\mu_{dark}}$$

Donde μ_{dark} es el número de electrones generados en oscuridad.

➤ Ruido de lectura

El ruido de lectura es inherente en los sensores de imagen debido a la variación temporal del valor del píxel debido al paso del tiempo y está asociado a la electrónica del procesamiento de señal. Es el resultado de varias fuentes de ruido como pueden ser el ruido kTC o el ruido de cuantización.

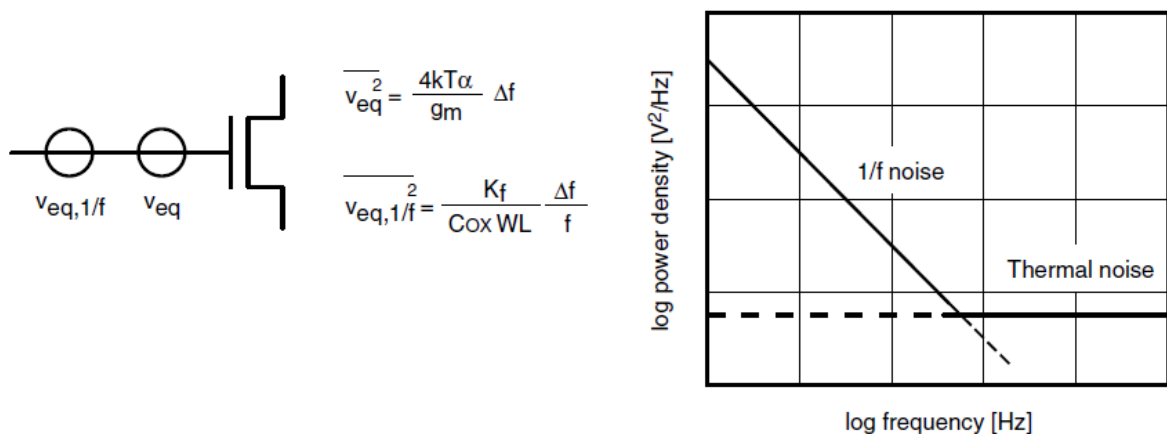


Figura 5-40 Esquema ruido de lectura de un píxel

Se suele medir con un corto periodo de integración para evitar el DSNU y es medido en oscuridad para tener el ruido shot fotónico al mínimo. Además el ruido de lectura se mide en una temperatura estable del sistema y se calcula computando la varianza de cada píxel a lo largo de una serie de imágenes tomadas en oscuridad para el mismo tiempo de integración y tomando la media de la varianza de los píxeles de la imagen.

➤ Ruido reset o kTC

Este es el ruido térmico de la llave de reset sobre el píxel. Este es el ruido muestreado a la FD debido a la redistribución de cargas e incertidumbre de la carga en la capacidad FD cuando el reset pasa de ON a OFF. El transistor de reset y la FD pueden ser modeladas como dos resistencias en serie con una llave como se muestra en la siguiente figura.

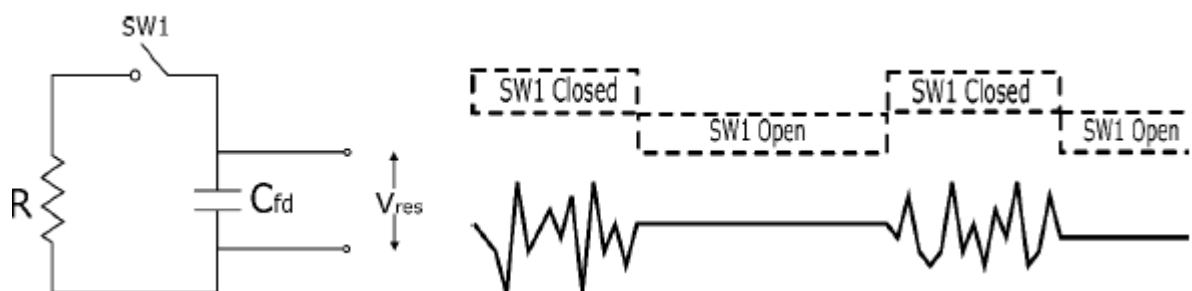


Figura 5-41 Esquema ruido de reset en un píxel

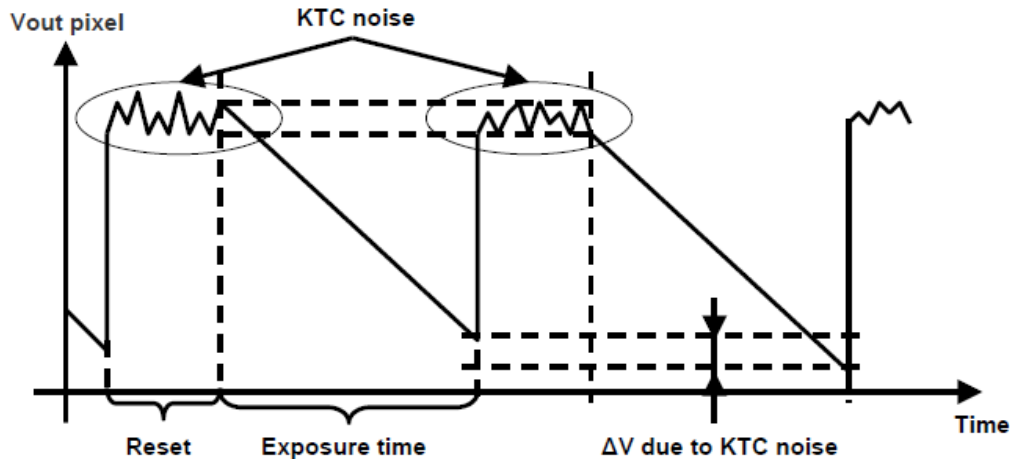


Figura 5-42 Ruido reset en un píxel

En los CIS modernos este ruido se cancela con la técnica de doble muestreo correlado (CDS). Matemáticamente, el ruido de reset:

$$V_{res} = \sqrt{\frac{kT}{C_{FD}}}$$

Donde,

V_{res} : Tensión de ruido reset

k : constante de Boltzmann

T : temperatura absoluta

C_{FD} : Capacidad de la floating diffusion.

➤ **Ruido Flicker.**

El ruido Flicker o ruido $1/f$ es el ruido cuya densidad espectral de potencia es inversamente proporcional a la frecuencia. Ruido Flicker depende de los transistores CMOS, más bien de la tecnología CMOS utilizada y del escalado en el diseño de los transistores, es el ruido predominante del ruido de lectura.

Este ruido es el resultado de la fluctuación aleatoria en la densidad de carga debido a la captura aleatoria y la emisión de portadores en la puerta del transistor y debido a la fluctuación de la movilidad de estos portadores. Estos dos efectos están correlados y son considerados en el modelado.

➤ **Ruido de cuantización**

Equivalente a $1/12 \text{ DN}^2$ y corresponde el ruido teórico correspondiente a digitalizar el dato.

5.5.1.2 Ruido espacial

El ruido espacial es la variación de respuesta del píxel dentro de una imagen que se captura en el mismo instante de tiempo frente a una luz homogénea y uniforme. Hay variaciones estadísticas de offset y ganancia entre los píxeles con una iluminación constante provocando ruidos de patrón fijo (Fixed Pattern Noise FPN) horizontal o vertical según sea por fila o columna respectivamente. Este ruido espacial se puede evaluar calculando los coeficientes de variación en toda la imagen, afortunadamente este es un ruido calibrable.

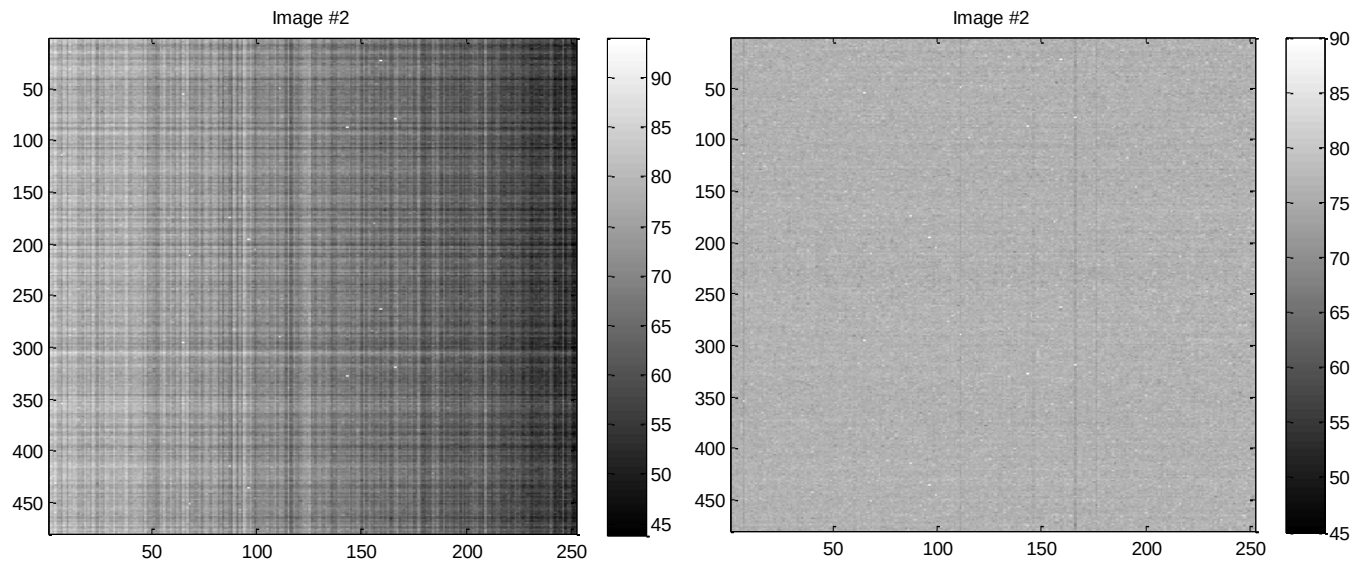


Figura 5-43 Imágenes con ruido de patrón fijo o espacial y tras aplicar correcciones

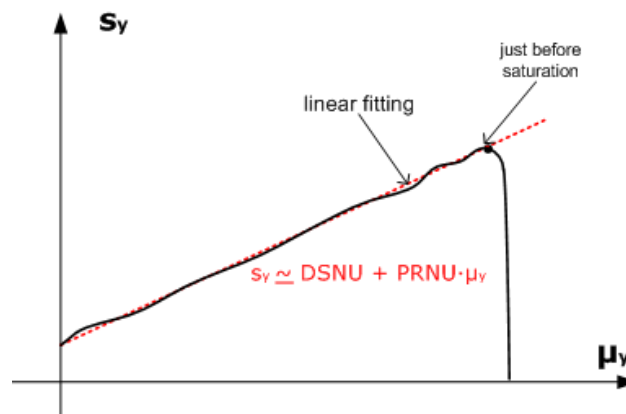


Figura 5-44 DSNU y PRNU en la curva de ruido espacial

➤ DSNU

El ruido de patrón fijo en oscuridad o Dark Signal Non-Uniformity (DSNU), es un ruido en oscuridad con carácter de offset y es el resultado del mismatch a nivel de transistores entre píxeles. Generalmente el FPN producto del mismatch puede ser cancelado con la técnica de CDS durante la lectura de la señal del píxel, pero en este caso el DSNU representa la variación de la corriente oscura de los píxeles en todo el área sensora y este es un valor fijo, de ahí su carácter de offset. Una vez del DSNU se normaliza respecto a la corriente oscura es un valor independiente del tiempo de exposición.

➤ PRNU

Foto respuesta no uniforme o PRNU también llamada ruido de ganancia es el resultado de la variación de la respuesta de cada píxel y es proporcional a la luz. Existen diferentes componentes del PRNU; el primero de ellos es la imperfección de los fotodiodos durante el proceso de fabricación. Esta imperfección resulta en un diferente tamaño de los fotodiodos lo que resulta una variación de las regiones activas de toda el área sensora. Otras contribuciones al PRNU podrían ser variaciones de la capacidad de recolección, variaciones de la ganancia de conversión y variaciones en el tamaño de la capacidad FD.

Es importante comprender que mientras se mide PRNU existe una componente de DSNU y otra de FPN, esta última se puede eliminar con técnicas de lectura CDS pero el DSNU debe ser extraído a la hora de computar el PRNU.

5.5.1.3 Ruido de patrón fijo (FPN)

El cálculo del espectrograma de una imagen permite analizar patrones periódicos espaciales los cuales pueden perturbar el procesamiento de una imagen o la observación humana.

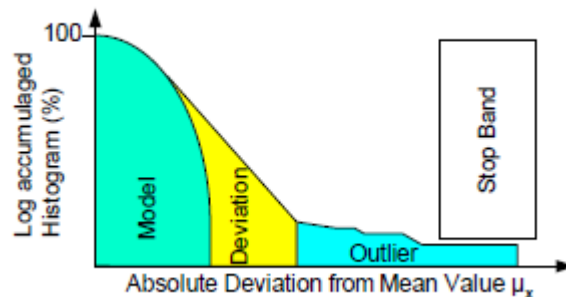


Figura 5-45 Información de los píxeles aportada por un histograma

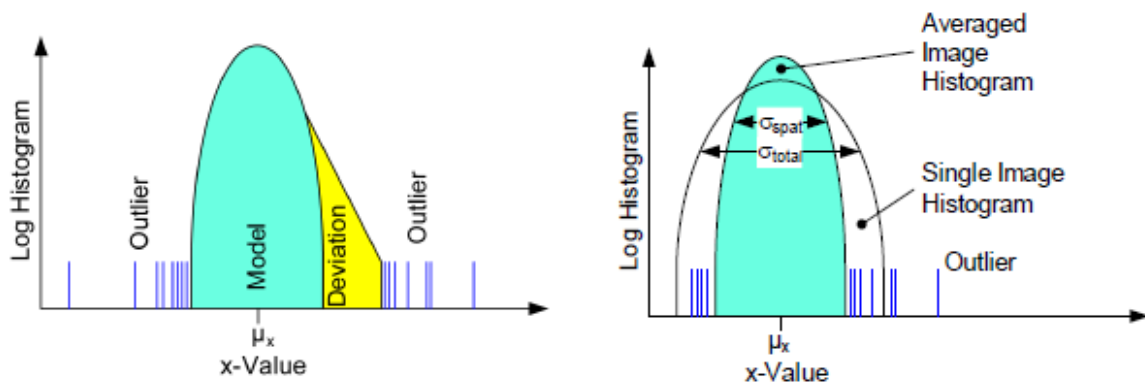


Figura 5-46 Análisis de ruido en el histograma de una imagen

Normalmente se asume que el ruido espacial está normalmente distribuido de manera aleatoria pero esto sólo es así si no es correlación en las fuentes de la variación. Sin embargo, para una adecuada descripción de las no uniformidades, varios efectos deben ser considerados:

- Variaciones graduales.
Imperfecciones en el proceso de fabricación pueden provocar variaciones graduales de baja frecuencia en todo el chip. Este efecto no es fácil de medir pues requiere de una irradiación homogénea, la cual es complicada de conseguir. Afortunadamente este efecto no es muy significativo y el ojo humano no lo percibe aunque a veces hay que corregirlo aplicando calibraciones para aplicaciones que requieren de una respuesta plana y uniforme.
- Variaciones periódicas.
Causadas por interferencias o acoplos en el sensor y a las que el ojo humano es muy sensible. Calculando el espectro de densidad de potencia de fila o columna del sensor, puede ofrecernos en forma de picos a las frecuencias espaciales específicas a las que se produce un acoplo en ciclos/píxel y detectar cualquier bloque analógico o digital que sea fuente del FPN.
- Outliers.
Píxeles aislados o grupos de píxeles desviados de manera significativa de la media frente a una iluminación uniforme.
- Variaciones aleatorias.
Si la no uniformidad espacial es totalmente aleatoria, i.e., no muestra correlación espacial, su PSD está distribuida uniformemente y el espectro se puede considerar un espectro de ruido blanco.

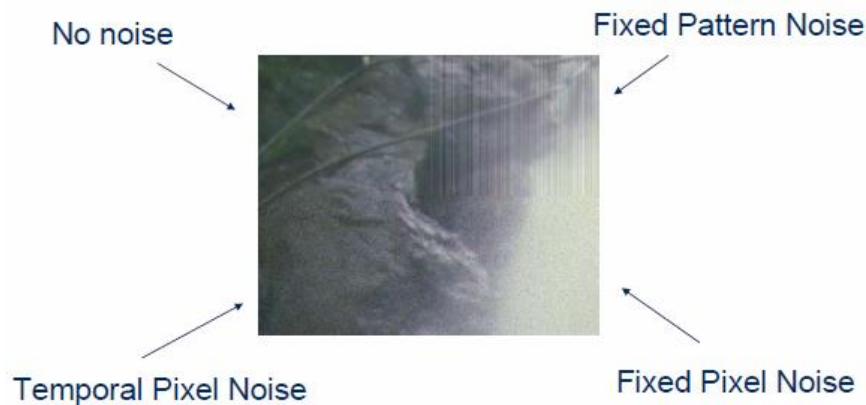


Figura 5-47 Diferentes ruidos en una misma imagen real

5.5.2 Método de medida

En el caso del ruido temporal se medirá con un corto periodo de integración para evitar el DSNU y es medido en oscuridad para tener el ruido shot al mínimo.

En el caso del ruido espacial al ser un tipo de no uniformidad espacial en la imagen se podrá evaluar en una misma imagen viendo el comportamiento del nivel medio píxel a píxel bajo oscuridad e iluminación para computar FPN y PRNU respectivamente.

5.5.3 Condiciones de la medida

El set-up típico de medida será el de la siguiente figura y la lista de equipos la siguiente:

1. DUT
2. Cámara link y frame grabber o ethernet dependiendo de la interfaz de salida del sensor.
3. Fuente de alimentación.
4. Fuente de luz led y posibilidad de condiciones de oscuridad total.

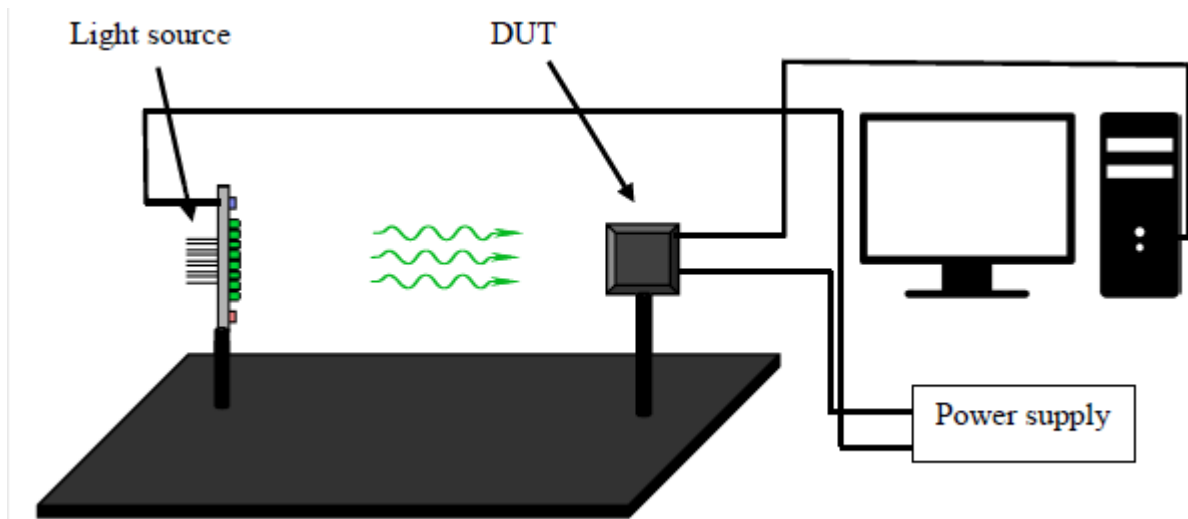


Figura 5-48 Set-up para medida de ruidos

5.5.4 Procedimiento de medida y cálculos

Las medidas las haremos con una temperatura estable del sistema y se calcula computando la varianza de cada píxel a lo largo de una serie de imágenes tomadas en oscuridad, luz controlada y saturación para el mismo tiempo de integración y tomando la media de la varianza de los píxeles de la imagen o bien promediando varias imágenes en las mismas condiciones para eliminar componente del ruido temporal y calcular la componente espacial.

El procedimiento es el siguiente:

1. Colocar el set-up anteriormente detallado y comprobar que se puede alcanzar la saturación en el sensor en las condiciones de la medida.
2. Tomar capturas dummy (al menos 20 o 30) para que el sistema sea estable.
3. Tomar al menos 10 o 20 imágenes para los cálculos de manera que promediando se eliminen errores de offset o de patrón fijo y con un corto periodo de integración para que la influencia del DSNU sea mínima en oscuridad.
4. Encender la fuente de luz y esperar unos 10 minutos hasta que sea térmicamente estable.
5. Capturar al menos 10 o 20 imágenes al 50% del nivel de saturación y en saturación.
6. En el caso del ruido espacial, calcular la media de todas las imágenes en cada una de las tres condiciones de iluminación.

El ruido temporal es la variación aleatoria en los valores medidos a lo largo del tiempo, por lo que para evaluarlo en oscuridad tomaremos la varianza de la señal a lo largo de una serie de capturas para cada píxel:

$$Varianza(\sigma^2) = \frac{\sum_{j=1}^k \frac{\sum_{i=1}^N (P_{ij} - M_j)^2}{N}}{k}$$

$$Ruido_{rms}[V \text{ o } DN] = \sqrt{\sigma}$$

Donde,

P_{ij} : píxel j -ésimo de la imagen i -ésima

M_j : valor medio de los j -ésimos píxeles de N imágenes

N : total de imágenes capturadas con luz uniforme

K : total de píxeles de una imagen

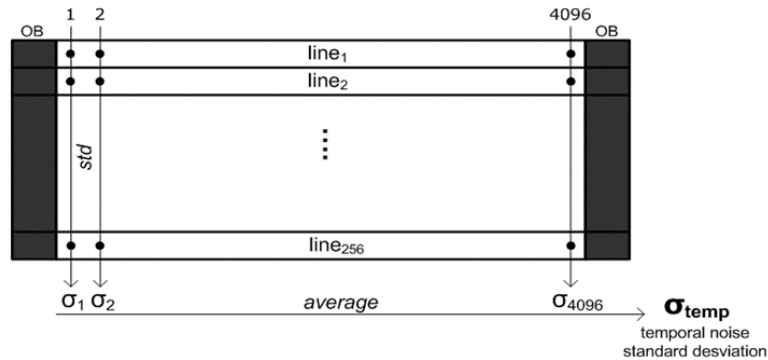


Figura 5-49 Cálculo ruido espacial en un sensor lineal

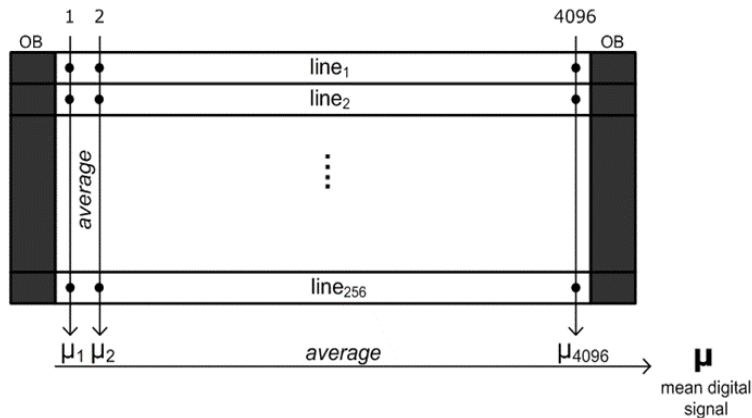


Figura 5-50 Cálculo nivel de señal y el ruido en una imagen

Si se divide este valor por la CG ya calculada se puede obtener el ruido en electrones.

El FPN es la variación estadística del offset en ausencia de luz píxel a píxel que podría ser o bien por columna o bien por fila, VFPN y HFPN respectivamente:

$$FPN[\%] = \frac{\sigma_{dark}}{\mu_{dark}} \times 100$$

Se puede hacer la distinción entre ruido de patrón fijo en oscuridad (constante en el tiempo independiente de la luz) y con iluminación (constante a lo largo del tiempo dependiendo de la cantidad de luz):

$$HFPN_{dark} = \frac{\sigma_{y.row.dark}}{\mu_{y.sat} - \mu_{y.dark}} \times 100$$

$$HFPN_{bright} = \frac{\sqrt{(\sigma_{y.row.50})^2 - (\sigma_{y.row.dark})^2}}{\mu_{y.50} - \mu_{y.dark}} \times 100$$

$$VFPN_{dark} = \frac{\sigma_{y.col.dark}}{\mu_{y.sat} - \mu_{y.dark}} \times 100$$

$$VFPN_{bright} = \frac{\sqrt{(\sigma_{y.col.50})^2 - (\sigma_{y.col.dark})^2}}{\mu_{y.50} - \mu_{y.dark}} \times 100$$

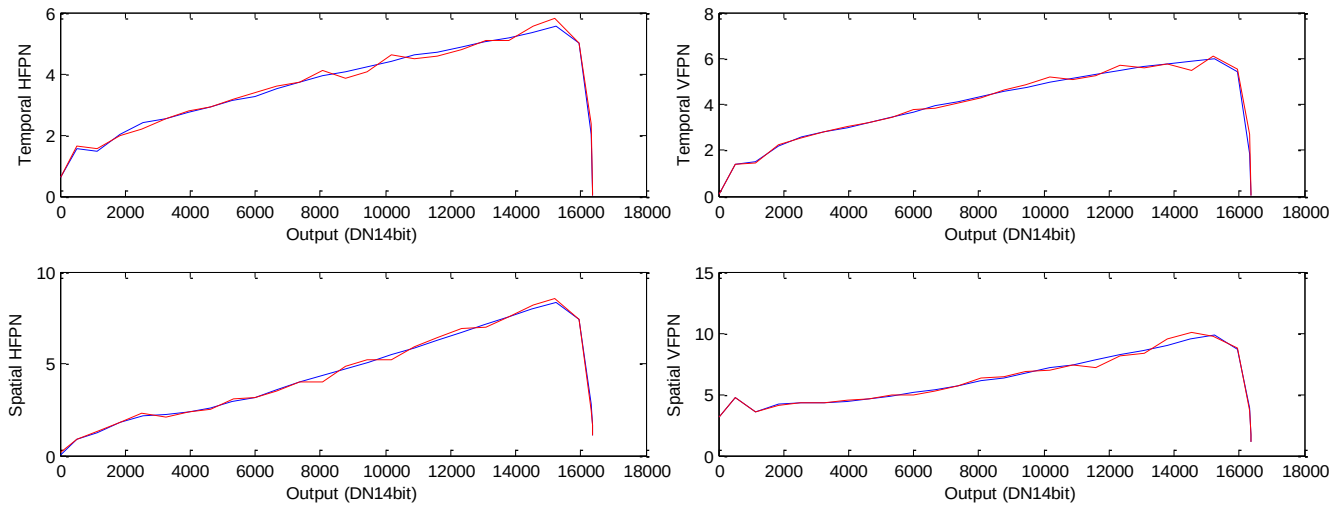


Figura 5-51 Curvas de componentes espacial y temporal del ruido de patrón fijo

El PRNU es el resultado de la variación de la respuesta de cada píxel y es proporcional a la luz:

$$PRNU[\%] = \frac{\sigma_{(light - dark)}}{\mu_{(light - dark)}} \times 100$$

Una manera efectiva de detectar píxeles defectuosos es a través del histograma del PRNU pues la mayoría de ruidos se pueden modelar como una distribución Gaussiana.

$$P = e^{\left(\frac{-(x - \bar{x})^2}{2 \cdot \sigma^2}\right)}$$

La cual si se representa en escala logarítmica corresponde a una parábola negativa, de manera que si determinamos cierto umbral de aceptación podemos cuantificar cuántos píxeles tienen un comportamiento anómalo.

$$\log(P) = -\left(\frac{(x - \bar{x})^2}{2 \cdot \sigma^2}\right) \cdot \log(e)$$

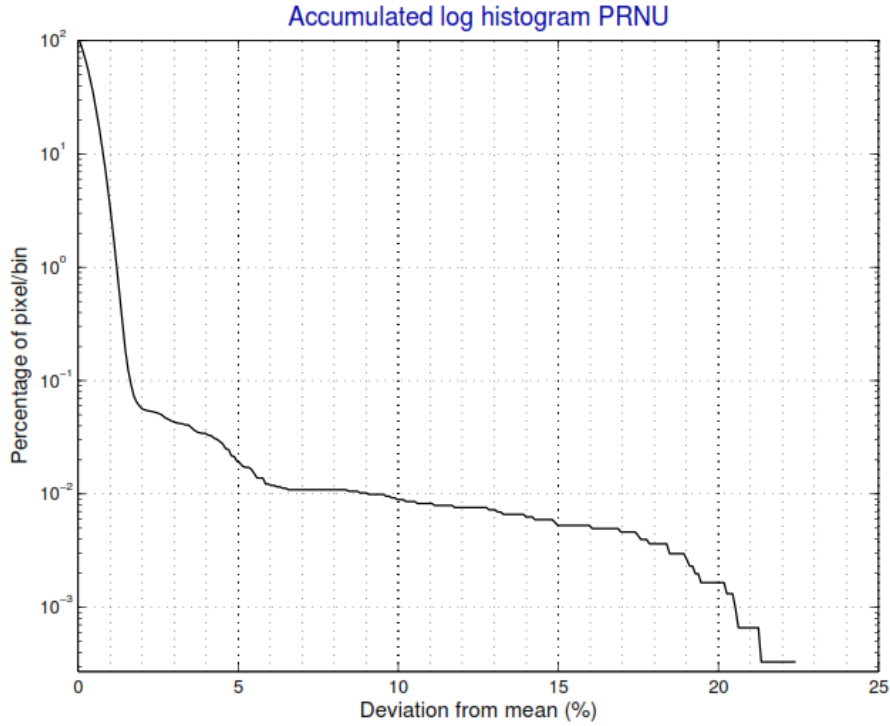


Figura 5-52 Distribución ruido PRNU

Para el cálculo de los espectrogramas horizontal/vertical:

1. Restar el nivel medio de la imagen y .
2. Calcular la FFT de cada fila/columna $y[m]$.

$$\bar{y}[m][\nu] = \frac{1}{\sqrt{N}} \cdot \sum_{n=0}^{N-1} y[m][\nu] \cdot \exp\left(-\frac{2\pi i n \nu}{N}\right) \quad \text{para } 0 \leq \nu \leq N.$$

3. Calcular el espectro promedio de potencia $p[\nu]$ a lo largo de las M filas/columnas.

$$p[\nu] = \frac{1}{M} \cdot \sum_{m=0}^{M-1} \bar{y}[m][\nu] \cdot \bar{y}^*[m][\nu] - \sigma_{y,stack}^2 / L.$$

4. La variación espacial total es el valor medio de este espectro de potencia para filas o columnas. Exceptuando las primeras ocho frecuencias para el PRNU.

$$DSNU: \quad s_y^2 = \frac{1}{N-1} \cdot \sum_{\nu=1}^{N-1} p[\nu], \quad PRNU: \quad s_y^2 = \frac{1}{N-15} \cdot \sum_{\nu=8}^{N-8} p[\nu]$$

5. La parte de ruido blanco de la varianza espacial $s_{y,white}^2$ está dada por la media de esta PSD, siendo fácilmente computable.

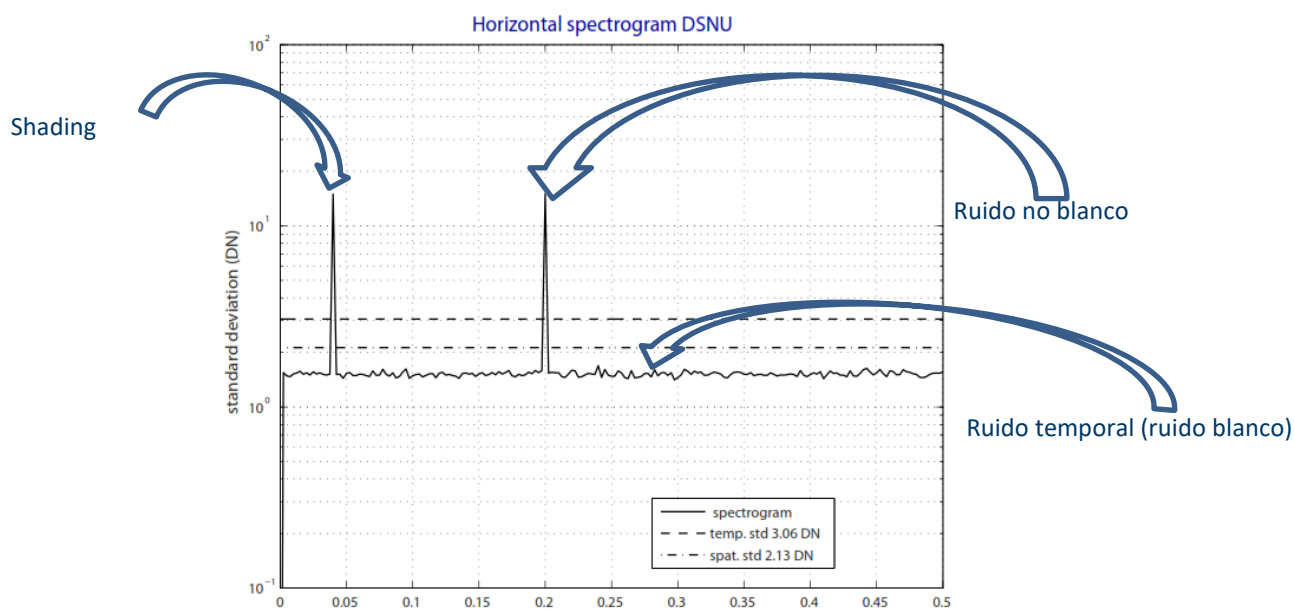


Figura 5-53 Fuentes de ruido DSNU

5.6 Otros artefactos

Aparte de los mencionados ruidos de patrón fijo y propios de la electrónica del sensor existen otros artefactos fácilmente identificables teniendo en cuenta las condiciones en las que se producen como podría ser el Blooming o el Lag, en esta sección explicaremos de forma general su base teórica.

	Dark	Illuminated	
		Below saturation	Above saturation
Fixed Pattern Noise (FPN)	Dark signal nonuniformity Pixel random Shading	Photo-response nonuniformity Pixel random Shading	
	Dark current nonuniformity (Pixel-wise FPN) (Row-wise FPN) (Column-wise FPN)		
	Defects		
Temporal Noise	Dark current shot noise	Photon shot noise	
	Read noise (Noise floor) Amplifier noise, etc. (Reset noise)		
			Smear, Blooming
	Image Lag		

Figura 5-54 Conjunto de ruidos y artefactos en un CIS

5.6.1 Blooming

Tras una exposición en un píxel a una señal lumínica muy fuerte en el fotodiodo se llena rápidamente y la carga desborda provocando que los píxeles vecinos saturan igualmente, esto es el efecto blooming y se ilustra en la siguiente figura.

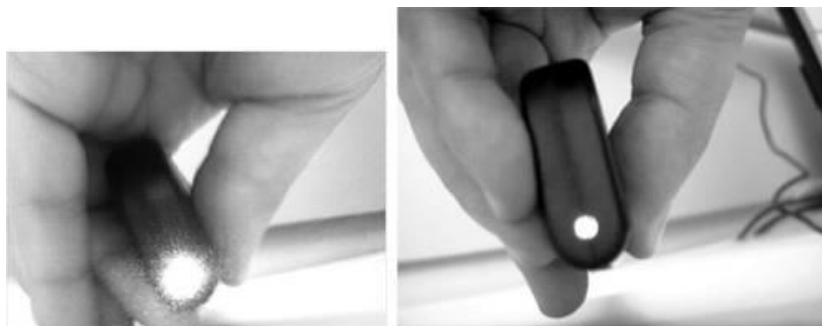


Figura 5-55 Imágenes con y sin efecto Blooming

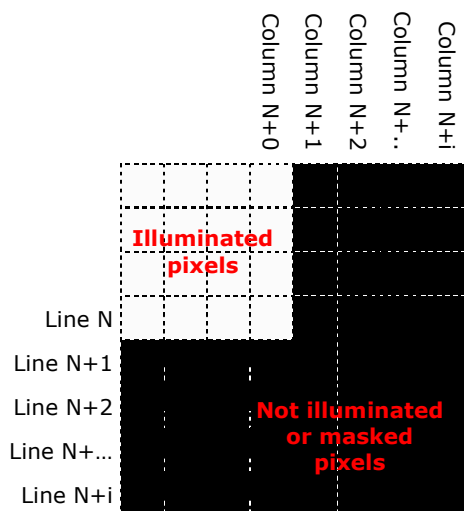


Figura 5-56 Medida de Blooming

El blooming corresponde a las cargas recibidas por un píxel cuando su vecino satura. La medida requiere utilizar píxeles saturados junto a píxeles no saturados. Es posible utilizar para ello líneas llamadas “dummies” en el área sensora. En condiciones de sobreexposición, la zona útil del área sensora estará saturada mientras que los píxeles tapados deberían ser oscuros.

5.6.2 Image Lag

El lag o image lag es un efecto de memoria del píxel provocado por una ineficiente transferencia de carga desde el PPD a la FD de manera que la carga residual se queda en el siguiente frame a capturar. En un píxel 4T o 5T una transferencia completa de carga es crucial sobre la performance del píxel en términos de ruido y lag. El umbral de potencial bajo la puerta de la TRF debe ser ajustado minuciosamente para establecer una barrera de potencial suficiente entre el fotodiodo y la FD para poder alcanzar una transferencia completa de carga al aplicar un valor lógico alto en la puerta de este transistor. La posición relativa y la geometría del PPD combinado con la forma y tamaño de la TRF afectan directamente en performances del píxel como puede ser el ruido, corriente oscura, o lag. Existe un compromiso entre una buena capacidad de llenado, baja corriente oscura y un significativo lag para cualquier diseñador de píxel. A continuación se van a introducir diferentes causas de lag en un píxel.

➤ Barrera de potencial entre el PPD y la TG.

El PPD puede ser modelado como una red RC por lo que hay cierto retardo desde que la carga sale completamente a través del diodo o atraviesa la barrera de potencial. Si el pulso que se aplica a la puerta de la TRF (TG) es menor que este retardo resulta en una carga residual en el fotodiodo lo que se traduce en lag en el sensor.

La transferencia de carga del PPD puede corresponder matemáticamente como la corriente inversa de un diodo, la cual requiere de cierto tiempo para superar la barrera y este tiempo de vaciado se traduce

en lag.

➤ Efecto de atrapado en la TG.

Es un fenómeno común en los MOSFETs que cuando los portadores están siendo atrapados en la TG, estas capturas pueden capturar a su vez portadores libres y liberarlos en cualquier momento lo que resultaría en lag de nuevo.

➤ Gran nivel de señal.

Un nivel alto de señal puede resultar como lag ya que algunos electrones pueden retroceder al PPD si el potencial aplicado a la TG no es suficientemente grande para la gran cantidad de carga acumulada.



Figura 5-57 Efecto Lag

6 SIMULACIONES Y RESULTADOS

Este capítulo 6 contiene la introducción al software empleado y los resultados en dos casos concretos de dos sensores con características diferentes.

6.1 Lenguaje y entorno

Como se discutió al inicio de la memoria, uno de los principales objetivos de este proyecto era presentar el lenguaje de Python como una herramienta en común para poder llevar a cabo la caracterización de CIS en detrimento de software de carácter comercial como Matlab.



Figura 6-1 Entorno Python de trabajo

Python es un lenguaje de programación multiparadigma. Esto significa que más que forzar a los programadores a adoptar un estilo particular de programación, permite varios estilos: programación orientada a objetos, programación imperativa y programación funcional. Otros paradigmas están soportados mediante el uso de extensiones. Esto hace de Python la herramienta ideal para integrarla en cualquier entorno de trabajo, sector o industria.

Hasta ahora la caracterización se realizaba a través de Matlab por lo que a la hora de migrar a Python es necesario un entorno de programación ligero y versátil y que encima esté preparado para la programación científica, Spyder es el IDE perfecto, no sólo por su interfaz similar a Matlab, sino por una serie de características que destacamos a continuación:

- Consolas Python y iPython integradas.
- Explorador de variables e inspector de objetos.
- Gran cantidad de paquetes de computación y análisis de datos integrados como numpy, scipy, matplotlib, etc.

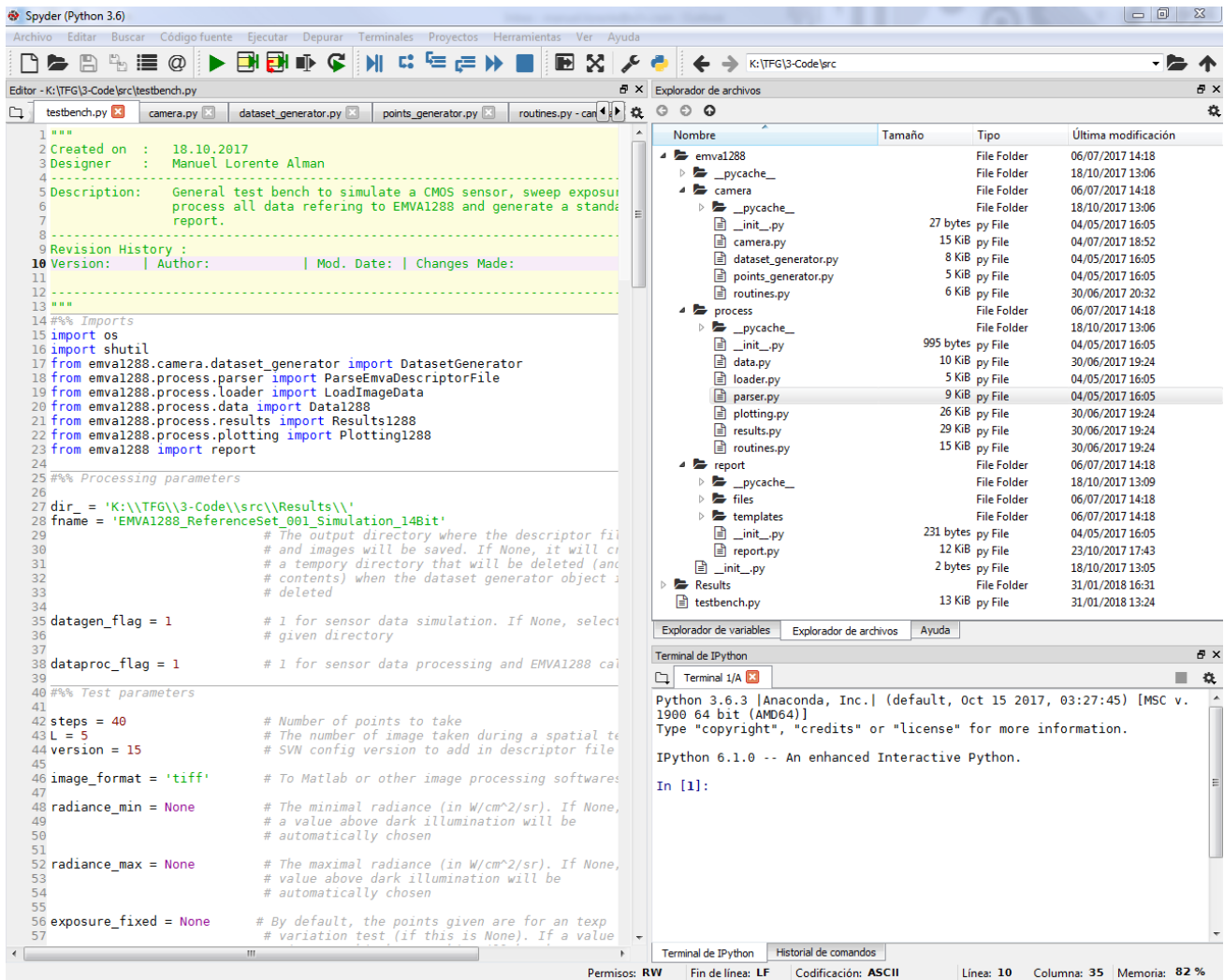


Figura 6-2 IDE Spyder

6.2 Librería en Python

Se ha creado una librería escrita en Python que con la siguiente taxonomía definida a continuación y detallada en el capítulo de anexos:

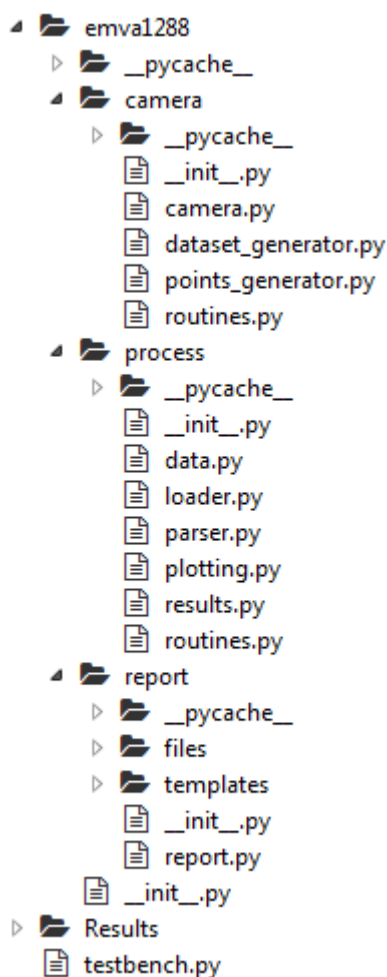


Figura 6-3 Librería creada para caracterización y simulación

Tabla 6-1 Descripción funciones y scripts creados

<i>camera.camera.py</i>	Contiene las clases y rutinas necesarias para simular el comportamiento de un CIS a partir de unos parámetros típicos propios del sensor.
<i>camera.dataset_generator.py</i>	Permite crear un fichero descriptor y sus correspondientes imágenes siguiendo los parámetros generados por <i>camera.py</i> .
<i>camera.points_generator.py</i>	Contiene las clases y rutinas precisas para poder crear vectores de puntos para parámetros como la exposición o la irradiancia.
<i>camera.routines.py</i>	Rutinas para calcular QE, irradiancia y demás parámetros útiles haciendo uso de las ecuaciones matemáticas descritas en la memoria.
<i>process.data.py</i>	Clase que permite leer del fichero descriptor generado, cargar las imágenes indicadas y crea un objeto de datos para ser procesado según la EMVA1288.
<i>process.loader.py</i>	Clase que obtiene la información relevante de una imagen para el cálculo de performances.
<i>process.parser.py</i>	Clase que traduce la información del fichero descriptor a información útil para la carga de imágenes.
<i>process.plotting.py</i>	Clase que a partir de los resultados del procesamiento dibuja las gráficas

	necesarias para el reporte final.
<i>process.results.py</i>	Clase que a partir de los datos procesados realiza todos los cálculos siguiendo el estándar EMVA1288 como se ha descrito a lo largo del proyecto.
<i>process.routines.py</i>	Funciones útiles a la hora de calcular ajustes lineales, formas de las imágenes, FFT, etc.
<i>report.report.py</i>	Conjunto de funciones que a partir de los resultados y las gráficas generadas crea un reporte automático en LaTeX.
<i>testbench.py</i>	Script de pruebas utilizado para testar todas las funciones y realización de las simulaciones propuestas.

6.3 Gráficas y resultados

Se van a generar dos sets de datos para procesarlos posteriormente y mostrar los conceptos más importantes en un sistema simulado de manera fiel al real. Por lo que nos encontraremos dos casos:

- Sensor A: sensor matricial de 8bits de resolución y una resolución de 4320x2048 píxeles.
- Sensor B: sensor matricial de 12 bits de resolución y una resolución de 640x480 píxeles.

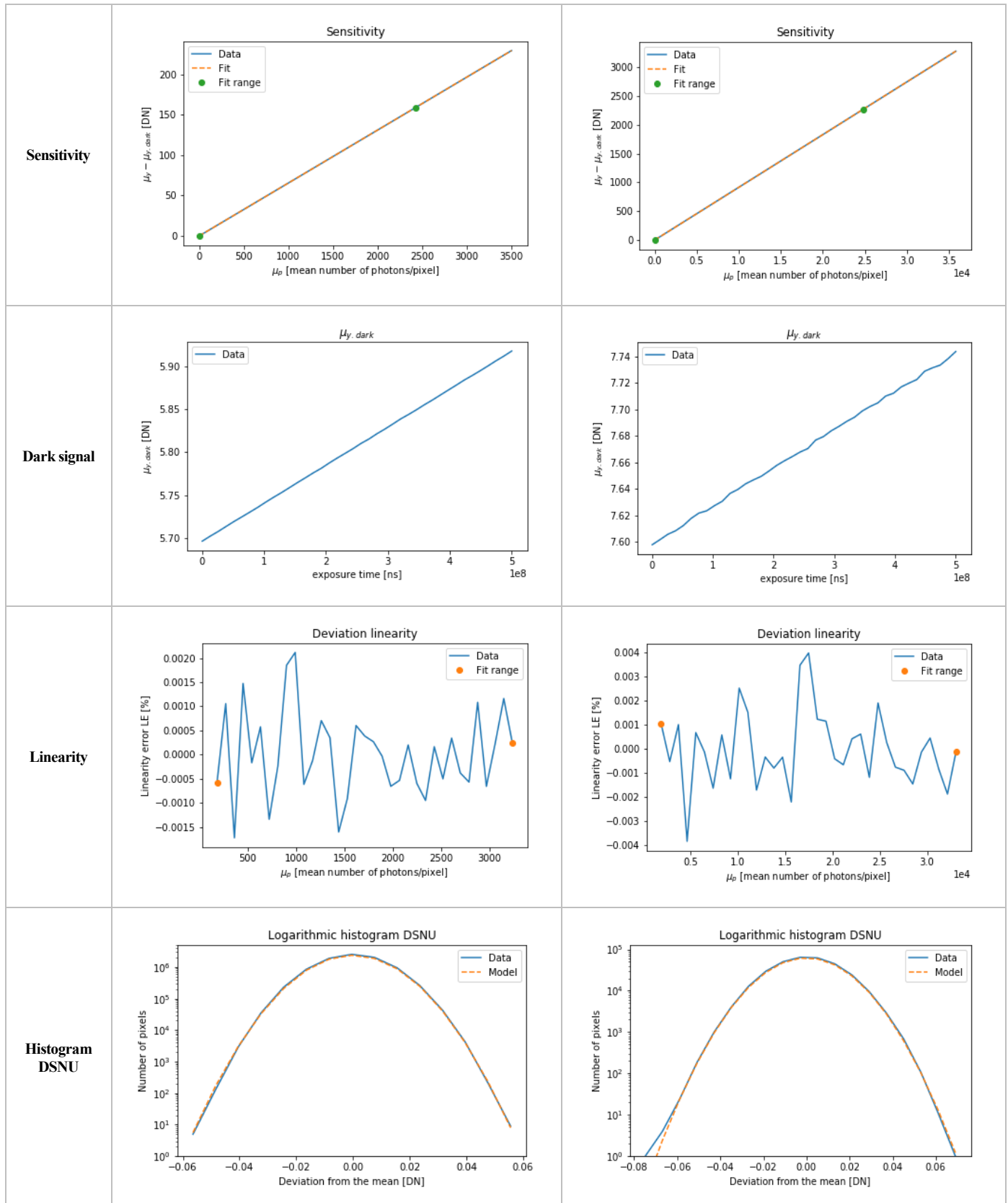
Tabla 6-2 Sensores simulados para la comparativa

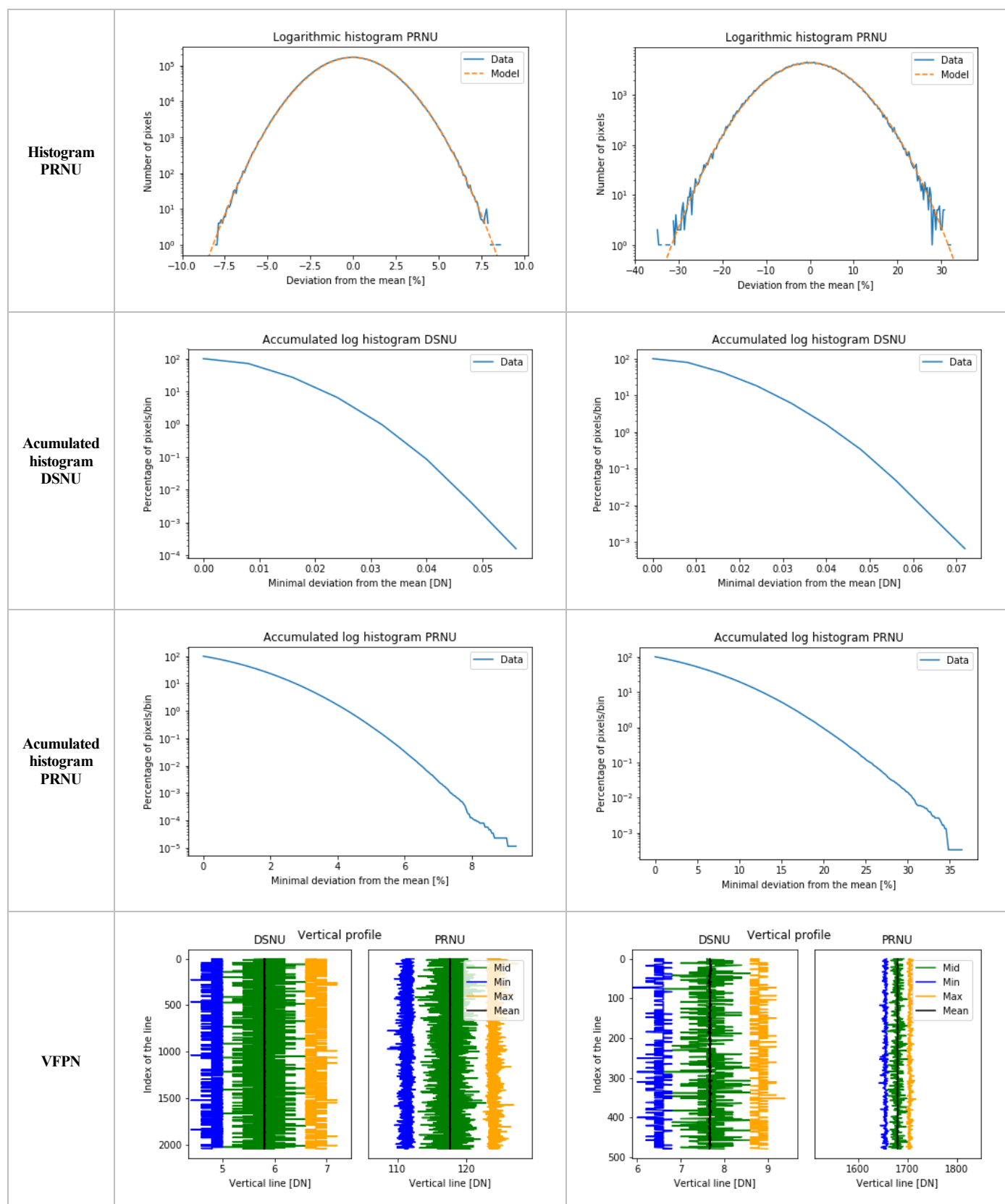
	Sensor A	Sensor B
Sensor type	Area	Area
Resolution [pixels]	4320x2048	640x480
Data output [bits]	8	12
Pixel size [μm]	5x5	5x5
Frame-rate	35fps	120fps
Interface type	Camera-Link	GigE
Light source	Led	Led

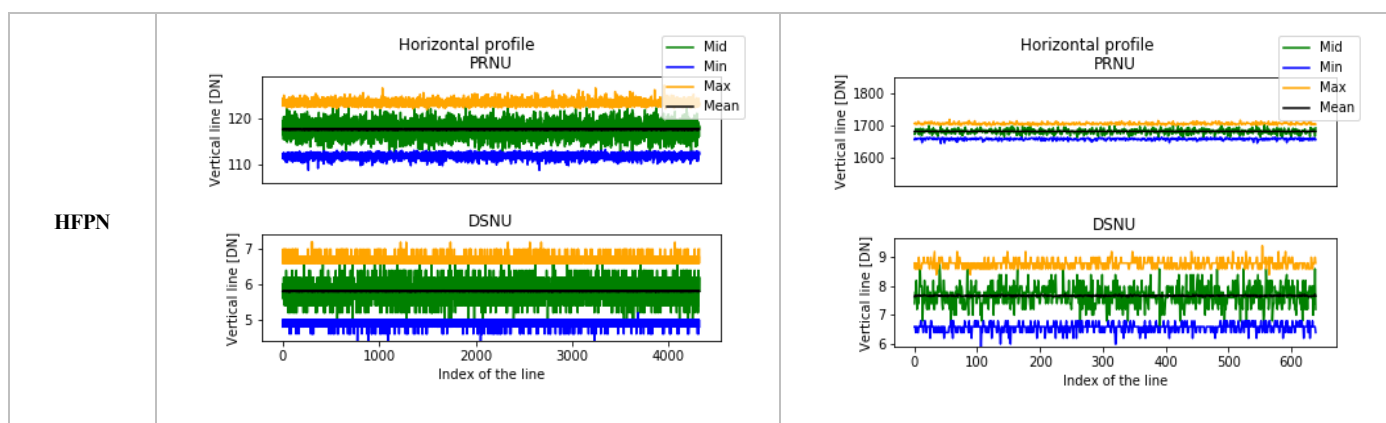
El procesamiento de las imágenes generadas en ambos casos queda resumido en la siguiente tabla:

Tabla 6-3 Comparativa performance electro-óptica para los dos casos en estudio

	Sensor A	Sensor B
QE [%]	56.05	49.86
CG [DN/e ⁻]	0.117	0.183
$\sigma_{y, \text{dark}}$ [e ⁻]	4.598	4.182
SNR _{max} [dB]	32.93	42.52
$\mu_{e, \text{min}}$ [e ⁻]	5.721	4.968
$\mu_{e, \text{sat}}$ [e ⁻]	1964	17867
DR [dB]	50.7	71.1
DSNU [%]	< 0.1	< 0.1
PRNU [%]	< 0.1	< 0.1
Linearity error [%]	0.002	0.004
Dark current [e ⁻ /s]	3.78	1.573
PTC		
SNR		







6.4 Generación automática de reporte

Para generar un datasheet automáticamente en PDF se ha hecho uso a partir de datos y plantillas creadas en LaTeX de un software portable como es MiKTeX en su versión portable (<https://miktex.org/download>) que permite convertir del formato LaTeX a otros formatos entre ellos PDF.

Ambos reportes generados como datos de simulación están adjuntos en la sección de anexos.

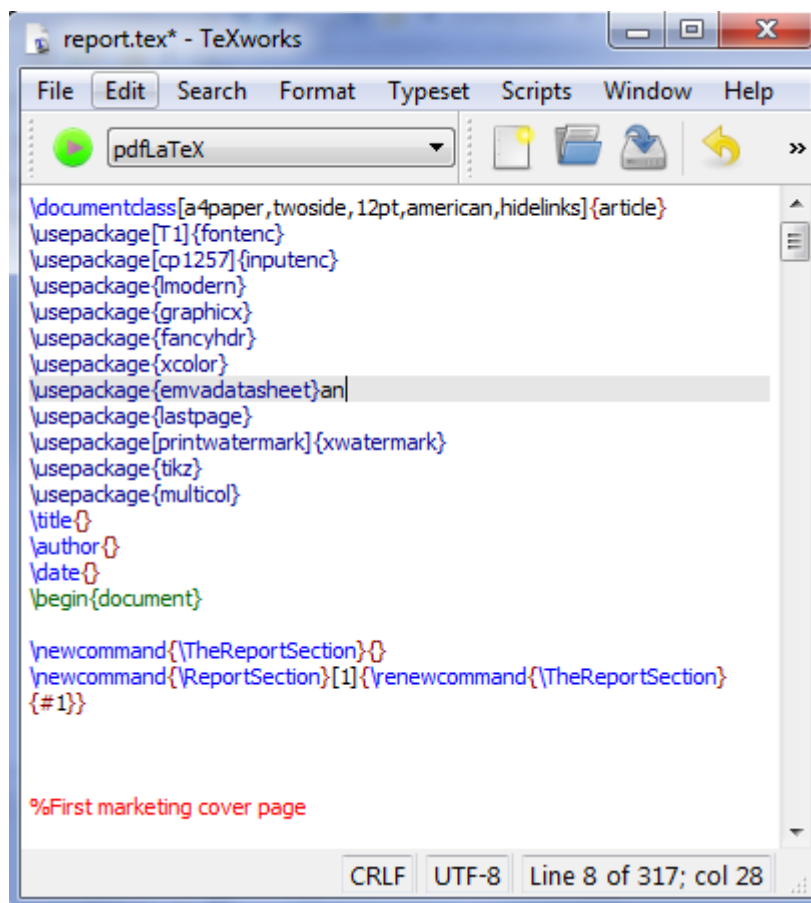


Figura 6-4 Ejemplo uso MikTex

7 CONCLUSIONES

Este penúltimo capítulo incluye las conclusiones del trabajo y líneas maestras a desarrollar en un futuro en este campo de semiconductores en general y sensores de imagen CMOS en particular.

7.1 Conclusiones

- Principalmente uno de los factores más importantes a la hora de caracterizar un dispositivo opto-electrónico es tener una fuente de luz estable y precisa la cual emita luz con una intensidad constante.
- Para medir las características principales de un sensor de imagen y obtener información sobre la linealidad, valor de ganancia de conversión, rango dinámico, etc. Idealmente es independiente barrer el tiempo de exposición manteniendo la luz constante que barrer la luz con una fuente de luz lineal manteniendo el tiempo de exposición constante. A este producto de luz y tiempo de exposición le llamaremos exposición, la cual es constante en todos los CIS.
- Para el cálculo de la ganancia de conversión el método de la varianza cuadrática respecto a la media de un barrido de exposición es el método más preciso.
- Existen diferentes métodos para calcular el MTF de los cuales el método de borde inclinado es el más preciso y rápido. Es tedioso calcular la MTF ya que implica un montón de cálculo matemático. El primer paso es obtener los datos sobremuestreados para la función de extensión de borde. Un detalle interesante sobre el procesamiento de datos es que para un píxel de tamaño grande conviene aplicar polyfit lineal en datos muestreados debido a que para píxeles pequeños es una posibilidad de que una muestra sea resultado de ruido, pero no así los datos reales. De manera similar, para los datos de función de dispersión de borde, existe una necesidad de interpolación.
- Para poder sacar una conclusión generalizada de las mediciones de ruido; es importante saber qué tipo de ruido se está midiendo. Cuando se trata de ruido en oscuridad, que es el ruido inherente del sensor de imagen sin iluminación, debe caracterizarse en completa oscuridad y se debe medir en un sistema de temperatura estable. Del mismo modo, para la medición del ruido con luz o PRNU, el tiempo de integración para adquirir una imagen debe ser el menor posible para eliminar la influencia del DSNU.
- El lag no es muy significativo en los sensores de imagen CMOS, pero con aumento de la demanda de alta velocidad y alta capacidad de carga puede ser un problema en el futuro por una ineficiencia en la limpieza del píxel para alcanzar esta velocidad.

7.2 Líneas futuras de trabajo

Como líneas futuras de trabajo a este proyecto se proponen varias alternativas:

- Creación de una GUI (Graphic User Interface) con el fin de mejorar la experiencia del usuario que utilice estas librerías.
- Adición de otras medidas propias de CIS orientados a aplicaciones que no sean del ámbito industrial como podría ser la caracterización de la medida de distancia en sensores de Time Of Flight o medidas especiales de ruido para sensores de muy bajo nivel de ruido.
- Para la realización de este trabajo de manera secundaria tenía el objetivo de separar la adquisición de imágenes y el control del sensor del procesamiento de las mismas por lo que se podría estudiar cómo implementar la adquisición de imágenes a través de Python de interfaces como GigE, CameraLink o USB.

ANEXO: CÓDIGO PYTHON

Se incluyen en este anexo sólo las clases principales para simular el comportamiento de un CIS y su posterior medida de performance óptica acorde a la EMVA1288, obviándose algunas de librerías propias menos relevantes pues únicamente sirven para generar un reporte en formato LaTeX, algo perfectamente prescindible en esta memoria.

testbench.py

```

"""
Name      :   testbench.py
Created on :   18.10.2017
Designer  :   Manuel Lorente Alman
-----
Description:   General test bench to simulate a CMOS sensor, sweep exposure,
               process all data refering to EMVA1288 and generate a standard
               report.
-----
"""

#%% Imports
import os
import shutil
from emval288.camera.dataset_generator import DatasetGenerator
from emval288.process.parser import ParseEmvaDescriptorFile
from emval288.process.loader import LoadImageData
from emval288.process.data import Data1288
from emval288.process.results import Results1288
from emval288.process.plotting import Plotting1288
from emval288 import report

#%% Processing parameters
dir_ = 'K:\\TFG\\3-Code\\src\\Results\\'
fname = 'EMVA1288_Simulation_12Bit'

                                # The output directory where the descriptor file
                                # and images will be saved. If None, it will create
                                # a tempory directory that will be deleted (and its
                                # contents) when the dataset generator object is
                                # deleted

datagen_flag = 1                # 1 for sensor data simulation. If None, select the
                                # given directory

dataproc_flag = 1               # 1 for sensor data processing and EMVA1288 calcs

```

```

#%% Test parameters
steps = 40                # Number of points to take
L = 5                    # The number of image taken during a spatial test.
version = 15             # SVN config version to add in descriptor file
image_format = 'tiff'    # To Matlab or other image processing softwares
radiance_min = None      # The minimal radiance (in W/cm^2/sr). If None,
                        # a value above dark illumination will be
                        # automatically chosen
radiance_max = None      # The maximal radiance (in W/cm^2/sr). If None, a
                        # value above dark illumination will be
                        # automatically chosen
exposure_fixed = None    # By default, the points given are for an texp
                        # variation test (if this is None). If a value is
                        # given to this kwarg, this will be the camera's
                        # exposure time (in ns) at which the operation
                        # points will be set for an illumination variation
                        # test.

#%% Sensor parameters
f_number = 8             # F-number of the light source/camera setup
pixel_area = 25          # The area of one pixel (in  $\mu\text{m}^2$ )
bit_depth = 12           # Bit depth of the image [8, 10, 12, 14]
width = 640              # The number of columns in the image
height = 480             # The number of rows in the image
temp = 45,               # Sensor temperature in degrees Celsius
temp_ref = 25            # Reference temperature (at which the dark current
                        # is equal to the reference dark current)
temp_doubling = 3        # The doubling temperature (at which the dark
                        # current is two times the reference dark
                        # current)
wavelength = 550         # illumination wavelength in nm
qe = None                # Quantum efficiency for the given wavelength
                        # (between 0 and 1). If None, a simulated
                        # quantum efficiency is choosen
exposure = 1000000       # Exposure time in ns
exposure_min = 100000    # Minimum exposure time in ns
exposure_max = 500000000 # Maximum exposure time in ns
K = 0.21                 # Overall system gain (in DN/e^-)
K_min = 0.05             # The overall minimal system gain (in DN/e^-)
K_max = 17.              # The overall maximal system gain (in DN/e^-)

```

```

K_steps = 255          # The number of available intermediate overall
                        # system gains between K_min and K_max
blackoffset = 3        # The dark signal offset for each pixel (in DN)
blackoffset_min = 0    # The min dark signal offset for each pixel (in DN)
blackoffset_max = None # The max dark signal offset for each pixel (in DN)
blackoffset_steps = 255 # The number of available blackoffsets between the
                        # minimal and maximal blackoffsets.
dark_current_ref = 5    # The reference dark current used for computing the
                        # total dark current
dark_signal_0 = 20.     # Mean number of electrons generated by the
                        # electronics (offset)
sigma2_dark_0 = 15.     # Variance of electrons generated by the
                        # electronics
u_esat = 45000.         # Full well capacity
radiance_factor = 1.    # The factor or an array with the same size of the
                        # image for modifie the radiance. By default 1 for
                        # no modification

dsnu = 5                # DSNU image in DN, array with the same shape of
                        # the image that is added to every image
prnu = 0.8              # PRNU image in percentages (1 = 100%), array with
                        # the same shape of the image. Every image is
                        # multiplied by it

%% Report parameters
# Description of the setup
s_standard_version = 3.1 # The EMVA1288 standard version number used
s_ls = 'Led'             # The light source type (e.g. integrating sphere)
s_ls_nonuniform = '0.5%' # The light source introducing non uniformity
# Basic information
b_vendor = 'Electronic \
            Engineering \
            Department' # The vendor name that manufactures theabc
b_model = 'CMOS Sensor' # The model of the tested camera
b_data_type = 'Single'  # The label given to the data used for the test
b_sensor_type = 'Area'  # The type of the tested sensor within the camera
b_resolution = '640x480' # The camera's resolution
b_pixel_size = 5        # The sensor's pixel size
b_shutter_type = 'Global' # The shutter type of the sensor
b_max_readout = '120fps' # The camera's maximal readout rate
b_intf = 'GigE'         # The camera's interface type

```

```

b_qe_plot = None          # The sensor's quantum efficiency plots
# Marketing information
m_logo = None             # The path to the logo icon
m_watermark = None        # A text that will be printed on every page of the
                           # report in the background in transparent red.
m_missingplot = None      # The path to a missing plot icon
m_cover_page = None       # The path to a custom cover page for the report
# Operation point
op_name = 'Config case B'
op_id = 1
op_summary = False
#%% Algorithm
if datagen_flag :
    if dir_ and fname:
        outdir = os.path.join(dir_, fname)
        if os.path.exists(outdir):
            shutil.rmtree(outdir,1)
    else:
        outdir = None
dataset_generator = DatasetGenerator(steps = steps,
                                     L = L,
                                     version = version,
                                     image_format = image_format,
                                     outdir = outdir,
                                     radiance_min = radiance_min,
                                     radiance_max = radiance_max,
                                     exposure_fixed = exposure_fixed,
                                     f_number = f_number,
                                     pixel_area = pixel_area,
                                     bit_depth = bit_depth,
                                     width = width,
                                     height = height,
                                     temperature = temp_ref,
                                     temperature_doubling = temp_doubling,
                                     wavelength = wavelength,
                                     qe = qe,
                                     exposure = exposure,
                                     exposure_min = exposure_min,
                                     exposure_max = exposure_max,
                                     K = K,

```



```

        K_min = K_min,
        K_max = K_max,
        K_steps = K_steps,
        blackoffset = blackoffset,
        blackoffset_min = blackoffset_min,
        blackoffset_max = blackoffset_max,
        blackoffset_steps =
blackoffset_steps,

        dark_current_ref = dark_current_ref,
        dark_signal_0 = dark_signal_0,
        sigma2_dark_0 = sigma2_dark_0,
        u_esat = u_esat,
        radiance_factor = radiance_factor,
        dsnu = dsnu,
        prnu = prnu
    )

if dataproc_flag :
    if datagen_flag :
        imp = dataset_generator.descriptor_path
    else :
        imp = os.path.join(os.path.join(dir_, fname), 'EMVA1288descriptor.txt')
    # Parse the descriptor file
    parser = ParseEmvaDescriptorFile(imp)
    # Load images
    imgs = LoadImageData(parser.images)
    # Process the collected data
    data = Data1288(imgs.data)
    res = Results1288(data.data, pixel_area = pixel_area)
    res.print_results()
    plot = Plotting1288(res)
    plot.plot()
    # Initialize the report with the marketing data
    # Provide a non existent name for the output directory
    myreport = report.Report1288(os.path.join(os.path.join(dir_, fname), \
                                                'Report'), \
                                marketing={ 'logo': m_logo,
                                              'watermark': m_watermark,
                                              'missingplot': m_missingplot,
                                              'cover_page': m_cover_page
                                              },

```

```

        setup={'Light source' : s_ls,
               'Light source non uniformity' : \
               s_ls_nonuniform,
               'Standard version' : s_standard_version
               },
        basic= {'vendor': b_vendor,
                'model': b_model,
                'data_type': b_data_type,
                'sensor_type': b_sensor_type,
                'resolution': b_resolution,
                'pixel_size': b_pixel_size,
                'bit_depth': bit_depth,
                'shutter_type': b_shutter_type,
                'maximum_readout_rate': b_max_readout,
                'interface_type': b_interf,
                'qe_plot': b_qe_plot
                })

op = {'name': op_name,
      'id': op_id,
      'summary_only': op_summary,
      'camera_settings': {'Number of images' : steps,
                           'Number of frames' : L,
                           'Configuration SVN version' : version,
                           'Image format' : image_format,
                           'Radiance minimum' : radiance_min,
                           'Radiance maximum' : radiance_max,
                           'Exposure fixed' : exposure_fixed
                           }
      }

# Add the operation point to the report
# we can add as many operation points as we want
# we pass the emval288.Data1288 object to extract automatically all
# the results and graphics
myreport.add(op, data.data)
# Generate the latex files
myreport.latex()

```

camera.py

```

"""
Name      :   camera.py
Created on :   18.10.2017
Designer  :   Manuel Lorente Alman

-----
Description:   Camera simulator. It creates images according to the given
                parameters.

Parameters
-----
f_number : float, optional
            The emval288 f_number for the camera.
pixel_area : float, optional
            The area of one pixel (in um ^ 2)
bit_depth : int, optiona
            The number of bits allowed for one pixel value.
width : int, optional
            The number of columns in the the image.
height : int, optional
            The number of rows in the image.
temperature : float, optional
            The camera's sensor temperature in degrees Celsius.
temperature_ref : float, optional
            The reference temperature (at which the dark current
            is equal to the reference dark current).
temperature_doubling: float, optional
            The doubling temperature (at which the dark
            current is two times the reference dark
            current).
wavelength : float, optional
            The light wavelength hitting the sensor (in nm).
qe : float, optional
            Quantum efficiency (between 0 and 1). If None, a simulated
            quantum efficiency is choosen with the
            :func:`~emval288.camera.routines.qe` function.
exposure : float, optional
            The camera's exposure time in ns.
exposure_min : float, optional
            The camera's minimal exposure time in ns.
exposure_max : float, optional

```

```

        The camera's maximal exposure time in ns.
K : float, optional
    The overall system gain (in DN/e^-).
K_min : float, optional
    The overall minimal system gain (in DN/e^-).
K_max : float, optional
    The overall maximal system gain (in DN/e^-).
K_steps : int, optional
    The number of available intermediate overall system gains
    between K_min and K_max.
blackoffset : float, optional
    The dark signal offset for each pixel (in DN).
blackoffset_min: float, optional
    The minimal dark signal offset for each pixel (in DN)
blackoffset_max : float, optional
    The maximal dark signal offset for each pixel
    (in DN).
blackoffset_steps : int, optional
    The number of available blackoffsets between the
    minimal and maximal blackoffsets.
dark_current_ref : float, optional
    The reference dark current used for computing the
    total dark current.
dark_signal_0 : float, optional
    Mean number of electrons generated by the electronics (offset)
sigma2_dark_0 : float, optional
    Variance of electrons generated by the electronics
u_esat : float, optional
    Full well capacity
radiance_factor : float or array, optional
    The factor or an array with the same size of the image
    for modifie the radiance. By default 1 for no
    modification, but if you want a bayer filter use
    :func:`~emval288.camera.routines.get_bayer_filter`
    function.
dsnu : np.array, optional
    DSNU image in DN, array with the same shape of the image
    that is added to every image
prnu : np.array, optional
    PRNU image in percentages (1 = 100%), array with the same shape

```

```

of the image. Every image is multiplied by it
-----
"""
#%% Imports
import numpy as np
from emval288.camera import routines

#%% Main class
class Camera(object):
    def __init__(self,
        f_number=8, # F-number of the light source/camera setup
        pixel_area=25, # um^2
        bit_depth=8, # Bit depth of the image [8, 10, 12, 14]
        width=640,
        height=480,
        temperature=22, # Sensor temperature in ^oC
        temperature_ref=30, # Reference temperature
        temperature_doubling=8, # Doubling temperature
        wavelength=525, # illumination wavelength
        qe=None, # Quantum efficiency for the given wavelength
        exposure=1000000, # Exposure time in ns
        exposure_min=500000, # Minimum exposure time in ns
        exposure_max=500000000, # Maximum exposure time in ns
        K=0.1, # Overall system gain
        K_min=0.1,
        K_max=17.,
        K_steps=255,
        blackoffset=0,
        blackoffset_min=0,
        blackoffset_max=None,
        blackoffset_steps=255,
        dark_current_ref=30,
        dark_signal_0=10.,
        sigma2_dark_0=10.,
        u_esat=15000.,
        radiance_factor=1.,
        dsnu=None,
        prnu=None
    ):
        """Camera simulator init method.

```

```

"""
self._pixel_area = pixel_area
self._bit_depth = bit_depth
self._img_max = 2 ** int(bit_depth) - 1
if bit_depth <= 8:
    self._img_type = np.uint8
elif bit_depth <= 16:
    self._img_type = np.uint16
else:
    self._img_type = np.uint64
self._width = width
self._height = height
self._shape = (self.height, self.width)
self._temperature_ref = temperature_ref
self._temperature_doubling = temperature_doubling
self._qe = qe
# When no specific qe is provided we simulate one
if qe is None:
    self._qe = routines.qe(wavelength)
self._dark_current_ref = dark_current_ref
self._dark_signal_0 = dark_signal_0
self._sigma2_dark_0 = sigma2_dark_0
self._exposure = exposure
self._exposure_min = exposure_min
self._exposure_max = exposure_max
self._u_esat = u_esat
self._Ks = np.linspace(K_min, K_max, num=K_steps)
self._K = None
self.K = K
# A good gestimate for maximum blackoffset is 1/16th of the full range
if not blackoffset_max:
    blackoffset_max = self._img_max // 16
self._blackoffsets = np.linspace(blackoffset_min, blackoffset_max,
                                  num=blackoffset_steps)

self._blackoffset = None
self.blackoffset = blackoffset
self._radiance_factor = radiance_factor
self._dsnu = dsnu
self._prnu = prnu
if dsnu is None:

```

```

        self._dsnu = np.zeros(self._shape)

    if prnu is None:
        self._prnu = np.ones(self._shape)
    self.environment = {'temperature': temperature,
                       'wavelength': wavelength,
                       'f_number': f_number}

    @property
    def bit_depth(self):
        """The number of bits allowed for a gray value for one pixel."""
        return self._bit_depth

    @property
    def pixel_area(self):
        """The area of one pixel (in um ^ 2)."""
        return self._pixel_area

    @property
    def img_max(self):
        """The maximal value for one pixel (in DN)."""
        return self._img_max

    @property
    def width(self):
        """The number of columns"""
        return self._width

    @property
    def height(self):
        """The number of rows"""
        return self._height

    @property
    def exposure(self):
        """The camera's exposure time (in ns)."""
        return self._exposure

    @exposure.setter
    def exposure(self, value):
        self._exposure = value

```

```

@property
def exposure_min(self):
    """The camera's minimal exposure time (in ns)."""
    return self._exposure_min

@property
def exposure_max(self):
    """The camera's maximal exposure time (in ns)."""
    return self._exposure_max

@property
def K(self):
    """The overall system gain (in DN/e^-).
    :Setter: The setter uses the
        :func:`~emval288.camera.routines.nearest_value` function
        to set the system gain to the nearest value given to the
        setter.
        This is because not all system gains are possible but rather a
        linear sample between the minimal and maximal value.
    """
    return self._K

@K.setter
def K(self, value):
    self._K = routines.nearest_value(value, self.__Ks)

@property
def Ks(self):
    """The array of all the available system gains (in DN/e^-)."""
    return self.__Ks

@property
def blackoffset(self):
    """The system dark signal offset (in DN).

    :Setter: The setter uses the
        :func:`~emval288.camera.routines.nearest_value` function
        to set the black signal offset to the nearest value given
        to the setter. This is because not all black offsets are
    """

```



```

        possible but rather a linear sample between the minimal
        and maximal value.

    """
    return self._blackoffset

@blackoffset.setter
def blackoffset(self, value):
    self._blackoffset = routines.nearest_value(value,
                                                self.__blackoffsets)

@property
def blackoffsets(self):
    """The array of all blackoffsets (in DN)."""
    return self.__blackoffsets

def grab(self, radiance, temperature=None, wavelength=None, f_number=None):
    """
    Create an image based on the mean and standard deviation from the
    EMVA1288 parameters.
    The image is generated using a normal distribution for each pixel.
    Parameters
    -----
    radiance : float
        The sensor's illumination in W/cm^2/sr. This is the only
        mandatory argument because it is frequently changed during
        an test.

    temperature : float, optional
        The camera's temperature in degrees Celsius.
        If None, the environment's
        temperature will be taken.

    wavelength : float, optional
        The illumination wavelength in nanometers.
        If None, the environment's wavelength will be taken.

    f_number : float, optional
        The optical setup f_number.
        If None, the environment's f_number will be taken.

    """
    clipping_point = int(self.img_max)
    #####
    # Thermally induced electrons image

```

```

u_d = self._u_therm(temperature=temperature)
# Noise centred on the number of electrons thermally generated
img_e = np.random.poisson(u_d, size=self._shape)
#####
# Light induced electrons image
u_e = self._u_e(radiance, wavelength=wavelength, f_number=f_number)
# Noise centred on the number of electrons from the Light
img_e += np.random.poisson(u_e, size=self._shape)
#####
# Electronics induced electrons image and Dark Signal non uniformity
variance = np.sqrt(self._sigma2_dark_0)
dark_signal = self._dsnu + np.random.normal(loc=self._dark_signal_0,
                                             scale=variance,
                                             size=self._shape)

img_e = img_e + dark_signal
#####
# Clip of the Full well electrons capacity
np.clip(img_e, 0, self._u_esat, img_e)
# Analog to Digital gain
img = self.K * img_e
# Quantization noise image
img_q = np.random.uniform(-0.5, 0.5, self._shape)
img += img_q
# Offset on the dark_signal
img += self.blackoffset
np rint(img, img)
np.clip(img, 0, clipping_point, img)
return img.astype(self._img_type)

def _u_e(self, radiance, wavelength=None, f_number=None):
    """
    Light induced electrons image for the exposure time.
    """
    # Alteration of the Quantum Efficiency by the
    # photon response non uniformity variation (prnu).
    qe = self._qe * self._prnu
    # Mean number of electrons per pixel during exposure time.
    photons_exposure = self.get_photons(radiance, wavelength=wavelength,
                                         f_number=f_number)
    # Influence of the radiance fator on the number of photons

```

```

        photons = self._radiance_factor * photons_exposure
        # Electrons generation
        u_e = qe * photons
        return u_e

def _u_therm(self, temperature=None):
    """
    Mean number of electrons due to temperature.
    """
    u_d = self._u_i(temperature=temperature) * self.exposure / (10 ** 9)
    return u_d

def _u_i(self, temperature=None):
    """
    Dark current (in DN/s).
    """
    if temperature is None:
        temperature = self.environment['temperature']
    u_i = 1. * self._dark_current_ref * 2 ** (
        (temperature - self._temperature_ref) / self._temperature_doubling)
    return u_i

def get_radiance_for(self, mean=None, exposure=None):
    """Radiance to achieve saturation.
    Calls the :func:`~emval288.camera.routines.get_radiance` function
    to get the radiance for saturation.
    Parameters
    -----
    mean : float, optional
        The saturation value of the camera. If None, this value
        is set to the :attr:`~img_max` attribute.
    exposure : float, optional
        The camera's exposure time at which the radiance
        for saturation value will be computed. If None, the
        exposure time taken will be the camera's actual exposure
        time.
    Returns
    -----
    float :
        The radiance at which, for the given saturation value and

```

```

        the given exposure time, the camera saturates.
    """
    if not mean:
        mean = self.img_max
    if not exposure:
        exposure = self.exposure
    ud = self._u_therm()
    ue = (mean / self.K) - ud
    up = ue / self._qe
    radiance = routines.get_radiance(exposure,
                                     self.environment['wavelength'],
                                     up,
                                     self.pixel_area,
                                     self.environment['f_number'])

    return radiance

def get_photons(self, radiance, exposure=None,
                wavelength=None, f_number=None):
    """Computes the number of photons received by one pixel.
    Uses the :func:`~emval288.camera.routines.get_photons` function
    to compute this number.
    Parameters
    -----
    radiance : float
        The radiance exposed to the camera (in Wsr-1cm-2).
    exposure : float, optional
        The pixel's exposure time in ns.
    Returns
    -----
    float :
        The number of photons received by one pixel.
    """
    if exposure is None:
        exposure = self.exposure
    if f_number is None:
        f_number = self.environment['f_number']
    if wavelength is None:
        wavelength = self.environment['wavelength']
    return routines.get_photons(exposure,
                                wavelength,

```

```

        radiance,
        self.pixel_area,
        f_number)

```

dataset_generator.py

```

"""
Name      :   dataset_generator.py
Created on :   18.10.2017
Designer   :   Manuel Lorente Alman

-----
Description:   This module contains a generator that generates a descriptor
                file and the corresponding images using the implemented camera.
                The generator uses a
                :class:`~emval288.camera.points_generator.PointsGenerator`
                object to create the operation points. It then grabs the images
                for these points using a
                :class:`~emval288.camera.camera.Camera` simulator object.
                The camera is intialized according to the
                given kwargs. Then, after getting the test points,
                it :meth:`~makes <run_test>` the images
                with it by changing its
                exposure time, or the radiation and :meth:`~saves <save_images>`
                the images and the descriptor file.

Parameters
-----
L : int, optional
    The number of image taken during a spatial test point.
version : str, optional
    Data version to add in descriptor file.
image_format : str, optional
    The image's format when they are saved.
outdir : str, optional
    The output directory where the descriptor file and the images
    will be saved. If None, it will create a tempory directory
    that will be deleted (and its contents) when the dataset
    generator object is deleted.
radiance_min : float, optional
    Same as in
    :class:`~emval288.camera.points_generator.PointsGenerator`
radiance_max : float, optional

```

```

        Same as in
        :class:`~emval288.camera.points_generator.PointsGenerator`
    exposure_fixed : float, optional
        Same as in
        :class:`~emval288.camera.points_generator.PointsGenerator`
    kwargs : All other kwargs are passed to the camera.

-----
"""

""" Imports
from emval288.camera.camera import Camera as Cam
from emval288.camera.points_generator import PointsGenerator
from collections import OrderedDict
import numpy as np
import tempfile
import os
from PIL import Image

""" Main methods
def _get_emva_gain(cam):
    """Find the gain to satisfy EMVA1288 requirements"""
    gini = cam.K
    # Find gain with a minum temporal noise of 0.5DN
    g = cam.Ks[0]
    for gain in cam.Ks:
        cam.K = gain
        g = gain
        img1 = cam.grab(0).astype(np.int64)
        img2 = cam.grab(0).astype(np.int64)
        if (img1 - img2).std() > 0.5:
            break
    cam.K = gini
    return g

def _get_emva_blackoffset(cam):
    """Find the blackoffset to satisfy EMVA1288 requirements"""
    bini = cam.blackoffset
    # Find black offset with a maximum of 0.5% of values at Zero
    bo = cam.blackoffsets[0]
    pixels = cam.width * cam.height
    for i in cam.blackoffsets:

```

```

        cam.blackoffset = i
        img = cam.grab(0)
        bo = i
        if np.count_nonzero(img) > pixels * .995:
            break
    cam.blackoffset = bini
    return bo

```

```

class DatasetGenerator:

```

```

    """Dataset generator.
    Creates a descriptor file and the corresponding linked images for a
    a exposure variant test example according to the emval288 standart.
    The images are created using the implemented camera in the emva module.
    """

    def __init__(self,
                  steps=100,
                  L=50,
                  version='3.0',
                  image_format='png', # best memory consumption
                  outdir=None, # directory where to save the dataset
                  radiance_min=None,
                  radiance_max=None,
                  exposure_fixed=None,
                  **kwargs
                  ):

        self._steps = steps # number of points to take
        self.cam = Cam(**kwargs)
        # set the camera parameters for the test
        self.cam.exposure = self.cam.exposure_min
        # If no blackoffset/gain are specified find them according to standard
        if 'blackoffset' not in kwargs:
            self.cam.blackoffset = _get_emva_blackoffset(self.cam)
        if 'K' not in kwargs:
            self.cam.K = _get_emva_gain(self.cam)
        # create test points
        points = PointsGenerator(self.cam,
                                radiance_min=radiance_min,
                                radiance_max=radiance_max,
                                exposure_fixed=exposure_fixed,
                                steps=self._steps)

```

```

self._points = points.points
self._L = L # number of images to take for a spatial test
self._version = version # data version
# store image format
self._image_format = image_format
# images will be saved one at a time during the generation into outdir
self.outdir = outdir
# create temporary directory to store the dataset
if outdir is None:
    self.tempdir = tempfile.TemporaryDirectory()
    self.outdir = self.tempdir.name
# create dir where images will be saved
os.makedirs(os.path.join(self.outdir, 'images'))
# run test
self._descriptor_path = self.run_test()

@property
def points(self):
    """The test points suite."""
    return self._points

@property
def descriptor_path(self):
    """The absolute path to the descriptor file."""
    return self._descriptor_path

def _is_point_spatial_test(self, i):
    """Check if a point index should be a spatial test.

    Spatial points are done at midpoint of bright and dark series.
    """
    v = self._steps // 2
    if i in (v, self._steps + v):
        return True
    return False

def _get_descriptor_line(self, exposure, radiance):
    """Create the line introducing a test point images in descriptor."""
    if radiance == 0.0:
        # dark image

```



```

        return "d %.1f" % exposure

    # bright image
    # round photons count to three decimals
    return "b %.1f %.3f" % (exposure,
                             round(self.cam.get_photons(radiance), 3))

def _get_image_names(self, number, L):
    """Create an image filename."""
    names = []
    for l in range(L):
        names.append("img_%04d.%s" % (number, self._image_format))
        number += 1
    return names, number

def _get_imgs(self, radiance, L):
    """Create a list of image from the given radiances.
    """
    # computes an array of dict whose keys are the name of the file
    # and the data of the image to save
    imgs = []
    for l in range(L):
        imgs.append(self.cam.grab(radiance))
    return imgs

def run_test(self):
    """Run the test points, save the images and generate descriptor."""
    descriptor_text = OrderedDict()
    image_number = 0
    # descriptor file path
    path = os.path.join(self.outdir, "EMVA1288descriptor.txt")
    # open descriptor file to write images in it
    with open(path, "w") as f:
        # write version
        f.write("v %s\n" % self._version)
        # write camera's properties
        f.write("n %i %i %i\n" % (self.cam.bit_depth,
                                self.cam.width,
                                self.cam.height))

        for kind in ('temporal', 'spatial'):
            # number of image to take

```

```

        L = 2
        if kind == 'spatial':
            L = self._L
        for texp, radiances in self.points[kind].items():
            # set camera
            self.cam.exposure = texp
            # Grab all images for these radiances
            for radiance in radiances:
                # Get descriptor line introducing the images
                f.write("%s\n"
                        % self._get_descriptor_line(texp, radiance))
            for l in range(L):
                # grab
                img = self.cam.grab(radiance)
                # write the name in descriptor
                name = "image%i.%s" % (image_number,
                                       self._image_format)
                f.write("i images\\%s\n" % name)
                image_number += 1
                # save image
                self.save_image(img, name)

    # return descriptor path
    return path

def save_image(self, img, name):
    """Save the image.
    """
    # save the images contained in d
    dtype = np.uint32
    mode = 'I'
    if self.cam.bit_depth <= 8:
        # 8 bit images have special format for PIL
        dtype = np.uint8
        mode = 'L'
    im = Image.fromarray(img.astype(dtype), mode)
    path = os.path.join(self.outdir, 'images', name)
    # (filename already contains image format)
    im.save(path)
    # erase image from dict to reduce memory consumption

```

routines.py

```

"""
Name      :   routines.py
Created on :   18.10.2017
Designer  :   Manuel Lorente Alman

-----

Description:   Useful functions for optical performances calcs.
-----

"""

#%% Imports
import numpy as np

#%% Classes
def qe(wavelength):
    """Simulate quantum efficiency for a specific wavelengths.

    Parameters
    -----
    wavelength : float
                  The wavelength to compute the quantum efficiency for.

    Returns
    -----
    float :
            The simulated quantum efficiency.
    """
    # For the time being we just simulate a simple gaussian
    s = 0.5
    u = 0.
    min_ = 350
    max_ = 800
    w = -1 + (wavelength) / (max_ - min_)
    qe = -.1 + np.exp(-(w - u) ** 2) /
            (2 * s ** 2)) / (np.sqrt(np.pi * 2 * s ** 2))

    if qe < 0:
        return 0
    return qe

def nearest_value(value, array):
    """Returns the nearest value in vals.

    Parameters

```

```

-----
value : float
    The value we want to get as near as possible.
array : array_like
    The array containing the available values to get near the value.
Returns
-----
The nearest element of `array` to `value`.
"""
# http://stackoverflow.com/questions/2566412/find-nearest-value-in-numpy-
array
idx = (np.abs(array - value)).argmin()
return array[idx]

def get_irradiance(radiance, f):
    """Get The irradiance, in w/cm^2.

    Parameters
    -----
    radiance : float
        The radiance (in W/sr/cm^2) to compute the irradiance from.
    f : float
        The f number of the setup.
    Returns
    -----
    float :
        The irradiance in W/cm^2
    """
    j = np.pi * radiance / (1 + ((2 * f) ** 2))
    return j

def get_photons(exposure, wavelength, radiance, pixel_area, f_number):
    """Get the number of photons hitting one pixel.

    Parameters
    -----
    exposure : float
        The pixel exposure time (in ns) to the light.
    wavelength : float
        The light wavelength hitting the pixel (in nm).
    radiance : float

```

```

        The radiance hitting the sensor (in W/sr/cm^2).
    pixel_area : float
        The pixel area in um ^ 2
    f_number : float
        The f number of the setup.

    Returns
    -----
    float :
        The number of photons that hit the pixel.
    """
    h = 6.63e-34
    c = 3.00e8
    w = wavelength * 1e-9
    t = exposure * 1e-9
    a = pixel_area * 1e-12
    j = get_irradiance(radiance, f_number)
    return j * a * t * w / (h * c)

def get_radiance(exposure, wavelength, photons, pixel_area, f_number):
    """From the number of photons, get the radiance hitting a pixel.

    Parameters
    -----
    exposure : float
        The pixel exposure time to the light (in ns).
    wavelength : float
        The photons' wavelength (in nm).
    photons : float
        The number of photons that hit the pixel.
    pixel_area : float
        The pixel area in um^2.
    f_number : float
        The f number of the setup.

    Returns
    -----
    float :
        The radiance that hit the pixel and gave the number of photons.
    """
    h = 6.63e-34
    c = 3.00e8
    w = wavelength * 1e-9

```

```

t = exposure * 1e-9
a = pixel_area * 1e-12
# p = j * a * t * w / (h * c)
# j = np.pi * radiance / (1 + ((2 * f) ** 2))
j = photons * h * c / (a * t * w)
r = j * (1 + ((2 * f_number) ** 2)) / np.pi
return r

```

```

def get_tile(arr, height, width):
    """From an array with a pattern, repeat the array to fill an 2D array
        with the given dimention.

    Parameters
    -----
    arr : array
        The pattern wanted to be replecate in a bigger shape.
    height : int
        The height of the array wanted in return.
    width : int
        The width of the array wanted in return.

    Returns
    -----
    array :
        The pattern given(arr) replecate in the size given.
    """
    # To reduce the execution time, we will reduce the width an height of
    # arguments given in the np.tile function. We take the width and height
    # wanted, dived by size of the array dimention. By careful, for an 2D
    # array, the shape[1] is the width and the shape[0] is the height, but
    # for an 1D array the height is 1 so the shape[0] is the width.
    # The size need to be a int, so we use np.floor to arroud the division
    # and add one to be sure than the size is not to short. To be sure to
    # return the array with the good shape we use [:height, :width].
    # if the array given is an 1D array.
    if len(arr.shape) == 1:
        # the height suppose to be 1
        h_r = height
        # only the width change
        w_r = int(np.floor(width / arr.shape[0]) + 1)
    # if the array given is a 2D array.

```

```

if len(arr.shape) == 2:
    # number of repetition needed
    h_r = int(np.floor(height / arr.shape[0]) + 1)
    w_r = int(np.floor(width / arr.shape[1]) + 1)
    # the case if the array given is an multidimentinal array.
    # else:
    # TODO: make a error message for to many dimention.
    # create a array with the dimension given and the array given
    tile = np.tile(arr, (h_r, w_r))[:height, :width]
    # if we want an 1D array so with a shape of (width,) and not (1, width)
    if height == 1:
        # tile = array([[0,0,0,...]]) so we only want the first line.
        return tile[0]
    return tile

def get_bayer_filter(t00, t01, t10, t11, width, height):
    """From different values of transmtion and the size, get a bayer filter.
    +-----+-----+-----+-----+-----+-----+
    |           Pattern           |
    +=====+=====+=====+=====+=====+=====+
    |   This case   |   Example   | Suggested values |
    +-----+-----+-----+-----+-----+-----+
    |   t00   |   t01   |   G1   |   R   |   1   |   0.15   |
    +-----+-----+-----+-----+-----+-----+
    |   t10   |   t11   |   B   |   G2  |  0.02 |   1   |
    +-----+-----+-----+-----+-----+-----+
    Where *G1* and *G2* are transmtion values for a green filter, *R* for a
    red filter and *B* for a blue filter. The suggested values are values
    for a standard bayer filter like the example.
    Parameters
    -----
    t00 : float
            The transmtion, in pourcentage, of the first pixel.
    t01 : float
            The transmtion, in pourcentage, of the second pixel.
    t10 : float
            The transmtion, in pourcentage, of the third pixel.
    t11 : float
            The transmtion, in pourcentage, of the fourth pixel.
    width : int

```

```

        The number of columns in the the image.
height : int
        The number of rows in the image.

Returns
-----
array :
    The array with the bayer filter of the size gived.
"""
pattern_rep = np.array([[t00, t01], [t10, t11]])
b_filter = get_tile(pattern_rep, height, width)
return b_filter

```

data.py

```

"""
Name      :    data.py
Created on :    18.10.2017
Designer  :    Manuel Lorente Alman

-----

Description:    Transform image data into data
                This class takes a dictionary with image data
                (product of :class:`emval288.process.loader.LoadImageData`)
                and transforms it into data that can be used for the EMVA1288
                computations.
                It sets the logging configuration and creates a
                :class:`python:logging.Logger` object using
                :func:`python:logging.getLogger` and sets the loglevel.
                Creates the data dictionaries with the call of
                :meth:`_get_temporal` for temporal data and :meth:`_get_spatial`
                for spatial data.

Parameters
-----
data : The data descriptor file.
loglevel : int, optional
        The loglevel for the Logger object.

-----

"""
""" Imports
import numpy as np

```



```

import logging

#%% Main class
class Data1288(object):
    """Take data from parsed images (descriptor file)
    and expose it as temporal and spatial dicts
    This dicts have the appropriate form for the processing
    """
    def __init__(self,
                  data,
                  pixels=None,
                  loglevel=logging.INFO):
        """Data processing object init method.
        """
        logging.basicConfig()
        self.log = logging.getLogger('Data')
        self.log.setLevel(loglevel)
        self.pixels = pixels or data['width'] * data['height']
        self.data = {}
        self.data['temporal'] = self._get_temporal(data['temporal'])
        self.data['spatial'] = self._get_spatial(data['spatial'])

    def _get_temporal(self, data):
        """Fill the temporal dict, with the stuff that we need.
        Compute the averages and variances from the sums (sum and pvar)
        If there is only one exposure time,
        the arrays in the returned dict will all have the same length as the
        photon count array. For this case, the exposure times and the dark
        value data array elements will all be the same.
        Parameters
        -----
        data : The data dictionary containing the temporal data sets.
        Returns
        -----
        dict : A dict containing all temporal test data.
               The keys are the following:
               *'texp'*: the array of the
               exposure times used for the test,
               *'u_p'*: the array of
               photon count in a pixel for each exposure time and photon count,

```

```

        *'u_y'*: the
        array of the mean digital value for each exposure
        time and photon count,
        *'s2_y'*:
        the array of the digital value variance for each exposure
        time and photon count,
        *'u_ydark'*: the array of the mean digital dark value
        for each exposure time
        and *'s2_ydark'*: the
        array of the digital dark value variance for each exposure time.
    Raises
    -----
    ValueError
        If there is no one 0.0 photon count entry for each exposure time
        and at least one bright point
    """
    temporal = {}
    # List of exposure times
    exposures = np.asarray(sorted(data.keys()))
    # texp is now an array of exposure times
    temporal['texp'] = exposures
    u_p = []
    u_y = []
    s2_y = []
    u_ydark = []
    s2_ydark = []
    for t in exposures:
        # photons is a list of photon counts
        # images for each exposure time
        photons = sorted(data[t].keys())
        if 0.0 not in photons:
            raise ValueError('Every exposure point must have a 0.0 photon')
        if len(photons) < 2:
            raise ValueError('There must be at least one bright photon')
        # get data for dark image
        d = self._get_temporal_data(data[t][0.0])
        u_ydark.append(d['mean'])
        s2_ydark.append(d['var'])
        for p in photons[1:]:
            # For each photon count, get the data

```

```

        u_p.append(p)
        d = self._get_temporal_data(data[t][p])
        u_y.append(d['mean'])
        s2_y.append(d['var'])

# Append all data to temporal dict
temporal['u_p'] = np.asarray(u_p)
temporal['u_y'] = np.asarray(u_y)
temporal['s2_y'] = np.asarray(s2_y)
temporal['u_ydark'] = np.asarray(u_ydark)
temporal['s2_ydark'] = np.asarray(s2_ydark)

# In case we have only one exposure, we need arrays with the
# same length as the up
# we just repeat the same value over and over
if len(exposures) == 1:
    l = len(temporal['u_p'])
    v = temporal['texp'][0]
    temporal['texp'] = np.asarray([v for _i in range(l)])
    v = temporal['u_ydark'][0]
    temporal['u_ydark'] = np.asarray([v for _i in range(l)])
    v = temporal['s2_ydark'][0]
    temporal['s2_ydark'] = np.asarray([v for _i in range(l)])
return temporal

def _get_temporal_data(self, d):
    """Convert temporal image data to mean and variance
    The mean is the sum of the pixels of the two images divided by
    (2 * self.pixels)
    The variance is the pseudo variance(integer), divided by
    (4 * self.pixels)
    Parameters
    -----
    d : dict
        The data dictionary that contains the sum and pvar of the
        pixels of a sum of two consecutive images with
        the same photon count
        and exposure time.
    Returns
    -----
    dict : A data dictionary with the following keys:
        *'mean': the mean as described above and

```

```

        *'var'*: the variance as described above.

    """
    mean_ = d['sum'] / (2.0 * self.pixels)
    var_ = d['pvar'] / (4.0 * self.pixels)
    return {'mean': mean_, 'var': var_}

def _get_spatial(self, data):
    """Fill the spatial dictionary.
    The images (sum and pvar) are preserved,
    they are needed for processing.
    Parameters
    -----
    data : The data dictionary to take data from.
    Returns
    -----
    dict : A dict containing all spatial test data.
           The keys are the following:
           - *'texp'*: the array of exposure times for spatial tests,
           - *'u_p'*: the array of photon count average for
             each exposure times.
           - *'sum'*: the array of the image sum for each
             photon count and exposure time,
           - *'pvar'*: the array of the pvar image for each
             photon count and exposure time,
           - *'L'*: the number of image taken to make the sum and
             pvar images for each photon count and each exposure time,
           - *'avg'*: the average computed from the sum image for
             each photon count and each exposure time,
           - *'var'*: the varianve computed from the pvar image for each
             photon count and each exposure time,
           - *'sum_dark'*: the sum image in the dark for
             each exposure time,
           - *'pvar_dark'*: the pvar image in the dark
             for each exposure time,
           - *'L_dark'*: the number of image taken in the dark to
             compose the sum and pvar images for each exposure time,
           - *'avg_dark'*: the average computed from the dark
             sum image for each exposure time,
           - *'var_dark'*: the variance computed from the dark
             pvar image for each exposure time.
    """

```

```

    Raises
    -----
    ValueError
        If the there is no exactly one dark and one bright point
    """
    if len(data) != 1:
        raise ValueError('Spatial data must contain only one exposure'
                          ' time')

    # the spatial exposure is the only exposure
    exposure = list(data.keys())[0]
    spatial = {}
    spatial['texp'] = exposure
    photons = sorted(data[exposure].keys())
    if 0.0 not in photons:
        raise ValueError('there must be a 0.0 photon count for spatial')
    if len(photons) != 2:
        raise ValueError('There must be one bright and one dark')
    # get dark spatial data
    d = self._get_spatial_data(data[exposure][0.0], postfix='_dark')
    spatial.update(d)
    # Photon count for bright spatial
    bphoton = photons[1]
    d = self._get_spatial_data(data[exposure][bphoton])
    spatial['u_p'] = bphoton
    spatial.update(d)
    return spatial

def _get_spatial_data(self, d, postfix=''):
    """Add the mean and variance to the spatial image data.
    The mean is the sum of the images divided by L,
    which is the number of images for the spatial test.

    The variance is the pseudovariance divided by
    (L^2 * (L-1)).
    Parameters
    -----
    d : dict
        The data dictionary containing the sum and pvar of the images.
    postfix: str, optional
        String to add in the resulting dictionary keys

```

```

Returns
-----
dict : A data dictionary processed from the input. The keys (+ postfix)
are:
    - 'sum': the sum image preserved from input,
    - 'L': the number of image summed,
    - 'avg_var': variance of the the average image
    - 'avg_mean': mean of the the average image
    - 'var_mean': the variance of the variance image
"""
# This cast is just in case the original images are uint
sum_ = d['sum'].astype(np.int64)
pvar_ = d['pvar'].astype(np.int64)
L = d['L']
avg_ = sum_ / (1.0 * L)
var_ = pvar_ / (1.0 * np.square(L) * (L - 1))
return {'sum' + postfix: sum_,
        'var_mean' + postfix: var_.mean(),
        'L' + postfix: L,
        # ddof = 1 (delta degrees of freedom) accounts for the minus 1
        # in the divisor for the calculation of variance
        'avg_var' + postfix: np.var(avg_, ddof=1),
        'avg_mean' + postfix: avg_.mean()
       }

```

results.py

```

"""
Name      :   results.py
Created on :   18.10.2017
Designer  :   Manuel Lorente Alman
-----
Description:   Compute EMVA1288 values from data
               This class takes the data from data.Data1288 and compute the actual
               EMVA1288 values.

               This class uses a :class:`python:logging.Logger` object to display
               informations for users.

Parameters

```

```

-----
data : dict
    The data dictionary to compute the results from.
pixel_area : float, optional
    The area of one pixel in um^2.
index_u_ysat : int, optional
    The index of the u_y array at which we consider
    that the camera saturates. This is used if forcing
    the saturation point is necessary.
loglevel : int, optional
    The level for the :class:`python:logging.Logger` object.
-----
"""
""" Imports
from __future__ import print_function
import logging
import numpy as np
from emval288.process import routines
from scipy.ndimage import convolve

""" Main class
class Results1288(object):
    """Class used to process data and to generate pdf report using LaTeX.
    When properties from this class are computed, their docstring are also
    parsed using the :func:`~emval288.process.routines.cls_1288_info`
    function to retrieve more data informations (like units or full name).
    """
    #####
    # Docstrings with .. emval288:: flags are used when computing the      #
    # properties to add relevant information to data for further processing #
    # like plotting or creating a report                                     #
    #####
    def __init__(self,
                  data,
                  pixel_area=None,
                  index_u_ysat=None,
                  loglevel=logging.DEBUG):
        """Results computation init method.
        """
        logging.basicConfig()

```

```

self.log = logging.getLogger('Results')
self.log.setLevel(loglevel)
self.temporal = data['temporal']
self.spatial = data['spatial']
self.pixel_area = pixel_area
self._s2q = 1.0 / 12.0
self._index_start = 0
self._index_sensitivity_min = 0
self._histogram_Qmax = 256 # Maximum number of bins in histograms
# Convolution kernel for high pass filter in defect pixel
self._histogram_high_pass_box = (-1) * np.ones((5, 5), dtype=np.int64)
self._histogram_high_pass_box[2, 2] = 24
# Sometimes we need to force the saturation point
# in those cases pass the index in the initialization of Results1288
self._index_u_ysat = index_u_ysat

@property
def s2q(self):
    """Quantification noise."""
    return self._s2q

@property
def index_start(self):
    """The array's starting index.
    .. emval1288::
        :Section: info
        :Short: Start array index
    """
    return self._index_start

@property
def index_u_ysat(self):
    """Index of saturation.
    .. emval1288::
        :Section: sensitivity
        :Short: Saturation index
    """
    if self._index_u_ysat:
        return self._index_u_ysat
    max_ = 0

```



```

max_i = 0
# we have to loop backwards because sometimes we have some
# noise pics that really bother the computation
s2_y = self.temporal['s2_y']
for i in range(len(s2_y) - 1, -1, -1):
    # Check that is not a local max
    if (s2_y[i] >= max_) or \
        (s2_y[abs(i - 1)] >= max_):
        max_ = s2_y[i]
        max_i = i
    elif (s2_y[i] < max_) and \
        (s2_y[abs(i - 1)] < max_):
        break
self._index_u_ysat = max_i
return self._index_u_ysat

@property
def index_sensitivity_max(self):
    """Index for linear fits in sensitivity part of the standard
    (70% of saturation).
    .. emval288::
        :Section: sensitivity
        :Short: Sensitivity fit maximum index
    """
    Y = self.temporal['u_y'] - self.temporal['u_ydark']
    m = 0.7 * (Y[self.index_u_ysat])
    return max(np.argwhere(Y <= m)) [0]

@property
def index_sensitivity_min(self):
    """Sensitivity minimum index.
    Index for linear fits in sensitivity part of the standard
    (70% of saturation)
    .. emval288::
        :Section: sensitivity
        :Short: Sensitivity fit minimum index
    """
    return self._index_sensitivity_min

```

```

@property
def R(self):
    """Responsivity.
    Slope of the (u_y - u_y_dark) Vs u_p. Fit with offset = 0
    Uses the :func:`~emval288.process.routines.LinearB0` function
    to make the fit.
    .. emval288::
        :Section: sensitivity
        :Short: Responsivity
        :Symbol: R
        :Unit: DN/p
    """
    Y = self.temporal['u_y'] - self.temporal['u_ydark']
    X = self.temporal['u_p']
    val, _error = routines.LinearB0(X[self.index_sensitivity_min:
                                     self.index_sensitivity_max + 1],
                                    Y[self.index_sensitivity_min:
                                     self.index_sensitivity_max + 1])

    return val[0]

@property
def K(self):
    """Overall system gain.
    Slope of (s2_y - s2_y_dark) Vs (u_y - u_y_dark). Fit with
    offset = 0. Uses the :func:`~emval288.process.routines.LinearB0`
    to make the fit.
    .. emval288::
        :Section: sensitivity
        :Short: System gain
        :Symbol: K
        :Unit: $DN/e^-$
        :LatexName: K
    """
    X = self.temporal['u_y'] - self.temporal['u_ydark']
    Y = self.temporal['s2_y'] - self.temporal['s2_ydark']
    val, _error = routines.LinearB0(X[self.index_sensitivity_min:
                                     self.index_sensitivity_max + 1],
                                    Y[self.index_sensitivity_min:
                                     self.index_sensitivity_max + 1])

    return val[0]

```

```

def inverse_K(self):
    """Inverse of overall system gain.
    .. emval288::
        :Section: sensitivity
        :Short: Inverse of overall system gain
        :Symbol: 1/K
        :Unit:  $\text{e}^{-}/\text{DN}$ 
        :LatexName: InvK
    """
    return 1. / self.K

@property
def QE(self):
    """Quantum efficiency.
    It is retrieved as the ratio of the responsivity to the overall
    system gain.
    .. emval288::
        :Section: sensitivity
        :Short: Quantum efficiency
        :Symbol:  $\eta$ 
        :Unit: \%
        :LatexName: QE
    """
    return 100.0 * self.R / self.K

@property
def sigma_y_dark(self):
    """Temporal Dark Noise.
    Uses :func:`~emval288.process.routines.LinearB` to make the fit.
    .. emval288::
        :Section: sensitivity
        :Short: Temporal Dark Noise
        :Symbol:  $\sigma_{y.\text{dark}}$ 
        :Unit: DN
        :LatexName: SigmaYDark
    """
    if len(np.unique(self.temporal['texp'])) <= 2:
        s2_ydark = self.temporal['s2_ydark'][0]
    else:

```

```

        fit, _error = routines.LinearB(self.temporal['texp'],
                                       self.temporal['s2_ydark'])

        s2_ydark = fit[1]

        # Lower limit for the temporal dark noise
        # The temporal dark noise in this range is dominated by the
        # quantization noise
        if s2_ydark < 0.24:
            s2_ydark = 0.24
        return np.sqrt(s2_ydark)

@property
def sigma_d(self):
    """Temporal Dark Noise.
    .. emval288::
        :Section: sensitivity
        :Short: Temporal Dark Noise
        :Symbol:  $\sigma_d$ 
        :Unit:  $\text{e}^{-}$ 
        :LatexName: SigmaDark
    """
    return np.sqrt((self.sigma_y_dark ** 2) - self._s2q) / self.K

@property
def u_p_min(self):
    """Absolute sensitivity threshold.
    .. emval288::
        :Section: sensitivity
        :Short: Absolute sensitivity threshold
        :Symbol:  $\mu_{p.min}$ 
        :Unit:  $\text{p}$ 
        :LatexName: UPMIn
    """
    return (100.0 / self.QE) * ((self.sigma_y_dark / self.K) + 0.5)

@property
def u_p_min_area(self):
    """Sensitivity threshold per pixel area.
    Returns None if pixel area is not defined or 0.
    .. emval288::
        :Section: sensitivity

```

```

        :Short: Sensitivity threshold
        :Symbol:  $\mu_{p.min.area}$ 
        :Unit:  $\text{p}/\mu\text{m}^2$ 
        :LatexName: UPMIn
    """
    if not self.pixel_area:
        return None
    return self.u_p_min / self.pixel_area

@property
def u_e_min(self):
    """Sensitivity threshold.
    .. emval288::
        :Section: sensitivity
        :Short: Sensitivity threshold
        :Symbol:  $\mu_{e.min}$ 
        :Unit:  $\text{e}^-$ 
        :LatexName: UEMin
    """
    return self.QE * self.u_p_min / 100.0

@property
def u_e_min_area(self):
    """Sensitivity threshold per pixel area.
    Returns None if the pixel area is not defined or 0.
    .. emval288::
        :Section: sensitivity
        :Short: Sensitivity threshold
        :Symbol:  $\mu_{e.min.area}$ 
        :Unit:  $\text{e}^-/\mu\text{m}^2$ 
        :LatexName: UEMin
    """
    if not self.pixel_area:
        return None
    return self.u_e_min / self.pixel_area

@property
def u_p_sat(self):
    """Saturation Capacity.
    .. emval288::

```

```

        :Section: sensitivity
        :Short: Saturation Capacity
        :Symbol:  $\mu_{p.sat}$ 
        :Unit:  $p$ 
        :LatexName: UPSat
    """
    return self.temporal['u_p'][self.index_u_ysat]

@property
def u_p_sat_area(self):
    """Saturation Capacity per pixel area.
    Returns None if pixel area is not defined or 0.
    .. emval288::
        :Section: sensitivity
        :Short: Saturation Capacity
        :Symbol:  $\mu_{p.sat.area}$ 
        :Unit:  $p/\mu m^2$ 
        :LatexName: UPSat
    """
    if not self.pixel_area:
        return None
    return self.u_p_sat / self.pixel_area

@property
def u_e_sat(self):
    """Saturation Capacity.
    Number of electrons at saturation.
    .. emval288::
        :Section: sensitivity
        :Short: Saturation Capacity
        :Symbol:  $\mu_{e.sat}$ 
        :Unit:  $e^-$ 
        :LatexName: UESat
    """
    return self.QE * self.u_p_sat / 100.0

@property
def u_e_sat_area(self):
    """Saturation Capacity per pixel area.
    Returns None if pixel area is not defined or 0.

```

```

    .. emval288::
        :Section: sensitivity
        :Short: Saturation Capacity
        :Symbol:  $\mu_{e.sat.area}$ 
        :Unit:  $\text{e}^{-}/\mu\text{m}^2$ 
        :LatexName: UESat
    """
    if not self.pixel_area:
        return None
    return self.u_e_sat / self.pixel_area

@property
def SNR_max(self):
    """Maximum Signal-to-Noise Ratio.
    .. emval288::
        :Section: sensitivity
        :Short: Signal-to-Noise Ratio
        :Symbol:  $\text{SNR}_{\{max\}}$ 
        :LatexName: SNRMax
    """
    return np.sqrt(self.u_e_sat)

def SNR_max_dB(self):
    """Maximum Signal to Noise Ratio in Db.
    .. emval288::
        :Section: sensitivity
        :Short: Maximum Signal to Noise Ratio in Db
        :Symbol:  $\text{SNR}_{\{max.db\}}$ 
        :Unit: dB
        :LatexName: SNRMaxDB
    """
    return 20. * np.log10(self.SNR_max)

def SNR_max_bit(self):
    """Maximum Signal to Noise Ratio in Bits.
    .. emval288::
        :Section: sensitivity
        :Short: Maximum Signal to Noise Ratio in Bits
        :Symbol:  $\text{SNR}_{\{max.bit\}}$ 
        :Unit: bit

```

```

        :LatexName: SNRMaxBit
    """

    return np.log2(self.SNR_max)

def inverse_SNR_max(self):
    """Inverse Maximum Signal to Noise Ratio.
    .. emval288::
        :Section: sensitivity
        :Short: Inverse Maximum Signal to Noise Ratio
        :Symbol:  $SNR_{max}^{-1}$ 
        :Unit: \%
        :LatexName: InvSNRMax
    """
    return 100. / self.SNR_max

@property
def DR(self):
    """Dynamic Range.
    Defined as the saturation capacity divided by the absolute sensitivity
    threshold. The greater this number is, the greater the operational
    range of a camera (between the dark noise level and the saturation
    level).
    .. emval288::
        :Section: sensitivity
        :Short: Dynamic Range
        :Symbol: DR
        :LatexName: DR
    """
    return self.u_p_sat / self.u_p_min

def DR_dB(self):
    """Dynamic Range in deciBels.
    It is defined as  $20 * \log_{10} ( \text{Dynamic Range} )$ .
    .. emval288::
        :Section: sensitivity
        :Short: Dynamic Range
        :Symbol:  $DR_{dB}$ 
        :Unit: dB
        :LatexName: DRDB
    """

```



```

"""

return 20. * np.log10(self.DR)

def DR_bit(self):
    """Dynamic Range in bits.
    It is defined as log_2 ( Dynamic Range ).
    .. emval288::
        :Section: sensitivity
        :Short: Dynamic Range
        :Symbol: $DR_{bit}$
        :Unit: bit
        :LatexName: DRBit
    """
    return np.log2(self.DR)

@property
def index_linearity_min(self):
    """Linearity fit minimum index.
    Minimum index for linear fit (5% of saturation).
    .. emval288::
        :Section: linearity
        :Short: Linearity fit minimum index
    """
    Y = self.temporal['u_y'] - self.temporal['u_ydark']
    vmin = 0.05 * (Y[self.index_u_ysat])
    return min(np.argwhere(Y >= vmin))[0]

@property
def index_linearity_max(self):
    """Linearity fit maximum index.
    Maximum index for linear fit (95% of saturation).

    .. emval288::
        :Section: linearity
        :Short: Linearity fit maximum index
    """
    Y = self.temporal['u_y'] - self.temporal['u_ydark']
    vmax = 0.95 * (Y[self.index_u_ysat])
    return max(np.argwhere(Y <= vmax))[0]

```

```

def linearity(self):
    """Returns a dictionary containing linearity information.
    It fits the mean digital signal in function of the mean photon count
    (Linear fit) using the EMVA1288 standard for linear fit.
    Returns
    -----
    dict : Linearity dictionary.
        The keys are:
        - *'fit_slope'* : The slope of the linear fit.
        - *'fit_offset'* : The offset of the fit.
        - *'relative_deviation'* : The relative deviation of the real
            data from the fit (in %) for the whole array.
        - *'linearity_error_min'* : The minimal value of the
            relative deviation.
        - *'linearity_error_max'* : The maximal value of the
            relative deviation.
    """
    Y = self.temporal['u_y'] - self.temporal['u_ydark']
    X = self.temporal['u_p']
    # The maximum index has to be included, this is the reason for the +1
    imax = self.index_linearity_max + 1
    imin = self.index_linearity_min

    #####
    # Following the emva1288 standart for the computation of the fit #
    #####
    X_ = X[imin: imax]
    Y_ = Y[imin: imax]
    xy = np.sum(X_ / Y_)
    xy2 = np.sum(X_ / (Y_ ** 2))
    x2y2 = np.sum((X_ / Y_) ** 2)
    _y = np.sum(1. / Y_)
    _y2 = np.sum(1. / (Y_ ** 2))
    b = ((xy * xy2) - (x2y2 * _y)) / ((xy2 ** 2) - (x2y2 * _y2))
    a = (xy - (b * xy2)) / x2y2
    # The equivalent numpy polyfit is
    # a, b = np.polynomial.polynomial.polyfit(X_, Y_, 1, w=1 / Y_)
    dev = 100. * (Y - (a * X + b)) / (a * X + b)
    lin = {}
    lin['fit_slope'] = a

```

```

lin['fit_offset'] = b
lin['relative_deviation'] = dev
lin['linearity_error_min'] = np.min(dev[imin: imax])
lin['linearity_error_max'] = np.max(dev[imin: imax])
return lin

@property
def LE_min(self):
    """ Min Linearity error.
    .. emval288::
        :Section: linearity
        :Short: Min Linearity error
        :Symbol: $LE_{min}$
        :Unit: \%
        :LatexName: LEMin
    """
    return self.linearity()['linearity_error_min']

@property
def LE_max(self):
    """Max Linearity error.
    .. emval288::
        :Section: linearity
        :Short: Max Linearity error
        :Symbol: $LE_{max}$
        :Unit: \%
        :LatexName: LEMax
    """
    return self.linearity()['linearity_error_max']

@property
def u_I_var_DN(self):
    """Dark Current from variance.
    The dark current from variance is the square root of the slope of the
    dark signal variance in function of the exposure time.
    Returns NaN if u_I_var is imaginary (if the fit slope is negative).
    .. emval288::
        :Section: dark_current
        :Short: Dark Current from variance
        :Symbol: $\mu_{I.var.DN}$

```

```

        :Unit: $DN/s$
        :LatexName: UIVar
    """
    if len(np.unique(self.temporal['texp'])) <= 2:
        return np.nan
    fit, _error = routines.LinearB(self.temporal['texp'],
                                   self.temporal['s2_ydark'])

    if fit[0] < 0:
        return np.nan

    # Multiply by 10^9 because exposure times are in nanoseconds
    return fit[0] * (10 ** 9)

@property
def u_I_var(self):
    """Dark Current from variance.

    The dark current from variance is the square root of the slope of the
    dark signal variance in function of the exposure times divided
    by the overall system gain.

    Returns NaN if u_I_var is imaginary (if the fit slope is negative).

    .. emval288::

        :Section: dark_current
        :Short: Dark Current from variance
        :Symbol:  $\mu_{I.var}$ 
        :Unit:  $\text{e}^-/\text{s}$ 
        :LatexName: UIVar
    """
    ui = self.u_I_var_DN
    if ui is np.nan:
        return np.nan
    return ui / (self.K ** 2)

@property
def u_I_mean_DN(self):
    """Dark Current from mean.

    The dark current from mean is the slope of the dark signal mean in
    function of the exposure time.

    Returns NaN if the number of different exposure times is less than 3.

    .. emval288::

        :Section: dark_current

```

```

        :Short: Dark Current from mean
        :Symbol:  $\mu_{I.mean.DN}$ 
        :Unit: DN/s$
    """
    if len(np.unique(self.temporal['texp'])) <= 2:
        return np.nan
    fit, _error = routines.LinearB(self.temporal['texp'],
                                   self.temporal['u_ydark'])
    # Multiply by 10 ^ 9 because exposure time in nanoseconds
    return fit[0] * (10 ** 9)

@property
def u_I_mean(self):
    """Dark Current from mean.
    The dark current from mean is the slope of the dark signal mean in
    function of the exposure times divided by the overall system gain.
    Returns NaN if the number of different exposure times is less than 3.
    .. emval288::
        :Section: dark_current
        :Short: Dark Current from mean
        :Symbol:  $\mu_{I.mean}$ 
        :Unit:  $\text{e}^-/\text{s}$ 
    """
    ui = self.u_I_mean_DN
    if ui is np.nan:
        return np.nan
    return ui / self.K

@property
def sigma_2_y_stack(self):
    """Temporal variance stack.
    Mean value of the bright variance image.
    .. emval288::
        :Section: spatial
        :Short: Temporal variance stack
        :Symbol:  $\sigma^2_{y.stack}$ 
        :Unit: DN2
    """
    return self.spatial['var_mean']

```

```

@property
def sigma_2_y_stack_dark(self):
    """Temporal variance stack dark.
    Mean value of the dark variance image.
    .. emval288::
        :Section: spatial
        :Short: Temporal variance stack dark
        :Symbol:  $\sigma^2_{y.stack.dark}$ 
        :Unit: DN2
    """
    return self.spatial['var_mean_dark']

@property
def s_2_y_measured(self):
    """Spatial variance measure.
    Variance value of the bright variance image.
    .. emval288::
        :Section: spatial
        :Short: Spatial variance measure
        :Symbol:  $s^2_{y.measured}$ 
        :Unit: DN2
    """
    # ddof = 1 (delta degrees of freedom) accounts for the minus 1 in the
    # divisor for the calculation of variance
    return self.spatial['avg_var']

@property
def s_2_y_measured_dark(self):
    """Spatial variance measured dark.
    Variance value of the dark variance image.

    .. emval288::
        :Section: spatial
        :Short: Spatial variance measured dark
        :Symbol:  $s^2_{y.measured.dark}$ 
        :Unit: DN2
    """
    return self.spatial['avg_var_dark']

@property

```

```

def s_2_y(self):
    """Spatial variance from image.
    .. emval288::
        :Section: spatial
        :Short: Spatial variance from image
        :Symbol:  $s^2_y$ 
        :Unit: DN2
    """
    return self.s_2_y_measured - (self.sigma_2_y_stack /
                                   self.spatial['L'])

@property
def s_2_y_dark(self):
    """Spatial variance from image,
    .. emval288::
        :Section: spatial
        :Short: Spatial variance from image
        :Symbol:  $s^2_y$ 
        :Unit: DN2
    """
    return self.s_2_y_measured_dark - (self.sigma_2_y_stack_dark /
                                         self.spatial['L_dark'])

@property
def DSNU1288(self):
    """DSNU.
    Dark Signal NonUniformity (in e-) is defined as the deviation
    standard of the dark signal divided by the overall system gain.
    If the variance is negative, it will return NaN instead of an
    imaginary number.
    .. emval288::
        :Section: spatial
        :Short: DSNU
        :Symbol:  $DSNU_{1288}$ 
        :Unit: e-
        :LatexName: DSNU
    """
    if self.s_2_y_dark < 0.:
        return np.nan
    return np.sqrt(self.s_2_y_dark) / self.K

```

```

def DSNU1288_DN(self):
    """DSNU in DN.

    Defined as the DSNU in  $e^-$  multiplied by the overall system gain.
    Returns NaN if the dark signal variance is negative.

    Returns
    -----
    float : The DSNU in DN.
    .. emval1288::
        :Section: spatial
        :Short: DSNU in DN
        :Symbol: $DSNU_{1288.DN}$
        :Unit: DN
        :LatexName: DSNUDN
    """
    if self.s_2_y_dark < 0:
        return np.nan
    return np.sqrt(self.s_2_y_dark)

@property
def PRNU1288(self):
    """PRNU.

    Photo Response NonUniformity (in %) is defined as the square root of
    the difference between the spatial variance of a bright image (or from
    an average of bright images to remove temporal difformities) and the
    spatial variance of dark signal, divided by the difference between the
    mean of a bright image and the mean of a dark image.

    .. emval1288::
        :Section: spatial
        :Short: PRNU
        :Symbol: $PRNU_{1288}$
        :Unit: \%
        :LatexName: PRNU
    """
    return (np.sqrt(self.s_2_y - self.s_2_y_dark) * 100 /
            (self.spatial['avg_mean'] - self.spatial['avg_mean_dark']))

@property
def histogram_PRNU(self):
    """PRNU histogram.

```



```

    Uses the :func:`~emval288.process.routines.Histogram1288` function
    to make the histogram.
    .. emval288::
        :Section: defect_pixel
        :Short: PRNU histogram
    """
    # For prnu, perform the convolution
    y = self.spatial['sum'] - self.spatial['sum_dark']
    # Slicing at the end is to remove boundary effects.
    y = convolve(y, self._histogram_high_pass_box)[2:-2, 2:-2]
    h = routines.Histogram1288(y, self._histogram_Qmax)
    # Rescale the bins
    h['bins'] /= (self.spatial['L'] * 25.)

    return h

@property
def histogram_PRNU_accumulated(self):
    """Accumulated PRNU histogram.
    Uses the :func:`~emval288.process.routines.Histogram1288` function
    to make the histogram.
    .. emval288::
        :Section: defect_pixel
        :Short: accumulated PRNU histogram
    """
    # For prnu, perform the convolution
    y = self.spatial['sum'] - self.spatial['sum_dark']
    y = convolve(y, self._histogram_high_pass_box)[2:-2, 2:-2]
    # For the accumulated histogram subtract the mean
    y = np.abs(y - int(np.mean(y)))
    h = routines.Histogram1288(y, self._histogram_Qmax)
    # Rescale the bins
    h['bins'] /= (self.spatial['L'] * 25.)
    # Perform the cumulative summation
    h['values'] = np.cumsum(h['values'][:, :-1])[:, :-1]
    # we want it as percentage of pixels
    h['values'] = 100 * h['values'] / y.size
    return h

@property

```

```

def histogram_DSNU(self):
    """DSNU histogram.

    Uses the :func:`~emval288.process.routines.Histogram1288` function
    to make the histogram.

    .. emval288::
        :Section: defect_pixel
        :Short: DSNU histogram
    """
    # For dsnu, the image is just the dark image, upscaled to have
    # only integers
    y = self.spatial['sum_dark']
    h = routines.Histogram1288(y, self._histogram_Qmax)
    # Rescale the bins, this is due to upscaling the average image to have
    # only integers
    scale = self.spatial['L_dark'] * 25.
    h['bins'] /= scale
    # The image is not centered around zero, so we shift the bins
    h['bins'] -= (y.mean() / scale)
    return h

@property
def histogram_DSNU_accumulated(self):
    """Accumulated DSNU histogram.

    Uses the :func:`~emval288.process.routines.Histogram1288` function
    to make the histogram.

    .. emval288::
        :Section: defect_pixel
        :Short: accumulated DSNU histogram
    """
    # Dark image upscaled to have only integers
    y = self.spatial['sum_dark']
    # For the accumulated dsnu histogram, subtract the mean from the image
    y = np.abs(y - int(np.mean(y)))
    h = routines.Histogram1288(y, self._histogram_Qmax)
    # Rescale the bins
    h['bins'] /= (self.spatial['L_dark'] * 25.)
    # Perform the cumulative summation (the ::-1 means backwards)
    # because the cumsum function is performed contrary to what we need
    h['values'] = np.cumsum(h['values'][:-1])[:-1]

```

```

        # we want it as percentage of pixels
        h['values'] = 100 * h['values'] / y.size
    return h

def xml(self, filename=None): # pragma: no cover
    """Method that writes the results in xml format to a file.
    Parameters
    -----
    filename : str, optional
        The file to write the results. If None, the xml string
        won't be written but will be returned instead.
    Returns
    -----
    str : If the xml string is not written into a file, it is returned.
    """
    results = self.results_by_section
    if not filename:
        return routines.dict_to_xml(results)
    routines.dict_to_xml(results, filename=filename)

@property
def results(self):
    """Dictionnary with all the values and metadata for EMVA1288 values.
    It uses the :func:`~emva1288.process.routines.obj_to_dict` to compute
    all the results at once.
    """
    return routines.obj_to_dict(self)

@property
def results_by_section(self): # pragma: no cover
    """Results ordered by section."""
    return routines._sections_first(self.results)

def print_results(self): # pragma: no cover
    """Print results to the screen."""
    results = self.results_by_section

    for section, attributes in results.items():
        print('!' * 50)
        print(section)

```

```

        print('-' * 50)

        for attribute, info in attributes.items():
            if 'value' not in info:
                continue

            print('{:<50}{:<30}{:>10}'.format(info.get('short'),
                                              str(info.get('symbol')),
                                              str(info.get('value'))))

    print('*' * 50)
    print(' ')

```

plotting.py

```

"""
Name      :   plotting.py
Created on :   18.10.2017
Designer  :   Manuel Lorente Alman

-----
Description:   Plot the results

               This class takes a results.Results1288 object and produces all the
               plots needed to create a reference datasheet of the EMVA1288 test.

               The only mandatory attribute is the name, the rest are for use
               in the :meth:`setup_figure` method.

Parameters
-----
figure : The :class:`matplotlib:matplotlib.figure.Figure`
         object to plot.

-----
"""

#%% Imports
from __future__ import print_function
import numpy as np
from . import routines

#%% Main class
class Emva1288Plot(object):
    """Base class for emva plots."""

```

```

name = ""
"""The figure's name (used as title if title is none)."""
title = None
"""The figure's title."""
xlabel = None
"""The x axis label."""
ylabel = None
"""The y axis label."""
xscale = None
"""The x axis scale."""
yscale = None
"""The y axis scale."""

def __init__(self, figure):
    """Base class for emva plots init function.
    """
    self.figure = figure
    self.setup_figure()

def setup_figure(self):
    """Simple wrapper for one plot per figure
    Takes the name, xlabel, ylabel, xscale and yscale
    for one plot case.
    If more than one plot, just overwrite as you wish.
    """
    ax = self.figure.add_subplot(111)
    if self.title:
        ax.set_title(self.title)
    else:
        ax.set_title(self.name)
    if self.xlabel:
        ax.set_xlabel(self.xlabel)
    if self.ylabel:
        ax.set_ylabel(self.ylabel)
    if self.xscale:
        ax.set_xscale(self.xscale)
    if self.yscale:
        ax.set_yscale(self.yscale)
    self.ax = ax

```

```

def plot(self, test):
    """Method to show the figures.

    Parameters
    -----
    test : Do nothing for this method but can be
           used for the subclass method.

    Raises
    -----
    NotImplementedError
        If this method is not overridden.

    Notes
    -----
    Must be overridden in subclasses.
    """
    raise NotImplementedError

def set_legend(self, ax):
    """Shortcut to add legend.

    Parameters
    -----
    ax : The :class:`matplotlib.matplotlib.axes.Axes` object to which
          the legend will be added.
    """
    ax.legend(loc='best')
    legend = ax.get_legend()
    if legend is not None:
        if getattr(legend, 'draggable', False):
            legend.draggable(True)

def rearrange(self):
    """Opportunity to change axis or limits after all the tests have
    been plotted.

    Uses :meth:`matplotlib.matplotlib.figure.Figure.tight_layout` method.
    """
    self.figure.tight_layout()

def reduce_ticks(self, ax, axis, n=4):
    """Reduce the number of ticks in ax.axis

    Uses the :meth:`matplotlib.matplotlib.axes.Axes.locator_params` method.

    Parameters

```

```

-----
ax : The :class:`matplotlib.matplotlib.axes.Axes` object to which
    the number of ticks will be changed.
axis : str, {'x', 'y', 'both'}
        Axis on which to operate.
n : int, optional
    Number of bins between ticks to be left.
"""
ax.locator_params(axis=axis, nbins=n)

```

```
class PlotSensitivity(Emva1288Plot):
```

```

    name = 'Sensitivity'
    xlabel = '$\mu_p$ [mean number of photons/pixel]'
    ylabel = '$\mu_y - \mu_{y.dark}$ [DN]'

    def plot(self, test):
        ax = self.ax
        ax.plot(test.temporal['u_p'],
                test.temporal['u_y'] - test.temporal['u_ydark'],
                label='Data',
                gid='%d:data' % test.id)
        ax.plot(test.temporal['u_p'],
                test.R * test.temporal['u_p'], '--',
                label='Fit',
                gid='%d:fit' % test.id)

        xi = test.temporal['u_p'][test.index_sensitivity_min]
        xf = test.temporal['u_p'][test.index_sensitivity_max]
        yi = (test.temporal['u_y'] -
              test.temporal['u_ydark'])[test.index_sensitivity_min]
        yf = (test.temporal['u_y'] -
              test.temporal['u_ydark'])[test.index_sensitivity_max]
        ax.plot((xi, xf), (yi, yf),
                label='Fit range',
                linestyle='None',
                marker='o',
                gid='%d:marker' % test.id)
        self.set_legend(ax)

    def rearrange(self):
        self.ax.ticklabel_format(axis='x', style='sci', scilimits=(1, 4))

```

```

self.figure.tight_layout()

class PlotUyDark(Emval288Plot):
    '''Create  $\mu_{y.dark}$  plot'''
    name = 'Mean gray value in dark'
    title = ' $\mu_{y.dark}$ '
    xlabel = 'exposure time [ns]'
    ylabel = ' $\mu_{y.dark}$  [DN]'

    def plot(self, test):
        ax = self.ax
        if np.mean(test.temporal['texp']) == test.temporal['texp'][0]:
            ax.plot(test.temporal['texp'],
                    test.temporal['u_ydark'],
                    'o',
                    markersize=5,
                    label='Data',
                    gid='%d:data' % test.id)
        else:
            ax.plot(test.temporal['texp'],
                    test.temporal['u_ydark'],
                    label='Data',
                    gid='%d:data' % test.id)
        ax.ticklabel_format(axis='x', style='sci', scilimits=(1, 4))
        self.set_legend(ax)

class PlotPTC(Emval288Plot):
    '''Create Photon Transfer plot'''
    name = 'Photon Transfer'
    xlabel = ' $\mu_y - \mu_{y.dark}$  [DN]'
    ylabel = ' $\sigma_y^2 - \sigma_{y.dark}^2$  [DN2]'

    def plot(self, test):
        ax = self.ax
        X = test.temporal['u_y'] - test.temporal['u_ydark']
        Y = test.temporal['s2_y'] - test.temporal['s2_ydark']
        ax.plot(X, Y,
                label='Data',
                gid='%d:data' % test.id)
        ax.plot(X, test.K * X,

```



```

        linestyle='--',
        label='Fit',
        gid='%d:fit' % test.id)
    ax.plot((X[test.index_u_ysat], ), (Y[test.index_u_ysat], ),
            marker='s',
            linestyle='None',
            label='Saturation',
            gid='%d:marker' % test.id)
    ax.plot((X[test.index_sensitivity_min],
             X[test.index_sensitivity_max]),
            (Y[test.index_sensitivity_min],
             Y[test.index_sensitivity_max]),
            linestyle='None',
            marker='o',
            label='Fit range',
            gid='%d:marker' % test.id)
    self.set_legend(ax)

class PlotSNR(Emva1288Plot):
    '''Create SNR plot '''
    name = 'Signal to Noise Ratio'
    xlabel = '$\mu_{p}$ [mean number of photons/pixel]'
    ylabel = 'SNR'

    def setup_figure(self):
        super(PlotSNR, self).setup_figure()
        self.ax.loglog()
        max_ideal = []
        self.max_ideal = max_ideal

    def plot(self, test):
        ax = self.ax
        X = np.arange(test.u_p_min, test.u_p_sat,
                      (test.u_p_sat - test.u_p_min) / 100.0)

        # remove the zeros on the denominator, at saturation the temporal
        # noise is zero!
        nz = np.nonzero(test.temporal['s2_y'])
        ax.plot(test.temporal['u_p'][nz],
                (test.temporal['u_y'] - test.temporal['u_ydark'])[nz] /

```

```

        np.sqrt(test.temporal['s2_y'][nz]),
        marker='.',
        ls=' ',
        label='Data',
        gid='%d:data' % test.id)
    ax.plot(X,
            ((test.QE / 100) * X) /
            np.sqrt((test.sigma_d ** 2 +
                    (test.s2q / (test.K) ** 2) +
                    ((test.QE / 100) * X))),
            linestyle=':',
            label='Theoretical',
            gid='%d:fit' % test.id)
    ideal = np.sqrt(X)
    self.max_ideal.append(ideal[-1])
    ax.plot((X),
            ideal,
            linestyle='-.',
            label='Ideal',
            gid='%d:fit' % test.id)
    ax.axvline(test.u_p_min,
               label='$\mu_{p.min} = %.1f[p]$' % test.u_p_min,
               gid='%d:marker' % test.id)
    ax.axvline(test.u_p_sat,
               label='$\mu_{p.sat} = %.1f[p]$' % test.u_p_sat,
               gid='%d:marker' % test.id)
    ax.plot(X,
            ((test.QE / 100) * X) /
            np.sqrt((test.sigma_d ** 2 +
                    (test.s2q / (test.K) ** 2) +
                    ((test.QE / 100) * X) +
                    (test.DSNU1288 ** 2) +
                    (((test.PRNU1288 / 100) *
                     (test.QE / 100.) * X) ** 2))),
            linestyle='--',
            label='Total SNR',
            gid='%d:fit' % test.id)
    ax.set_ylim(1, max(self.max_ideal))
    self.set_legend(ax)

```

```

class PlotLinearity(Emval1288Plot):
    '''Create Linearity plot'''
    name = 'Linearity'
    xlabel = '$\mu_{p}$ [mean number of photons/pixel]'
    ylabel = '$\mu_y - \mu_{y.dark}$ [DN]'

    def plot(self, test):
        ax = self.ax
        X = test.temporal['u_p']
        Y = test.temporal['u_y'] - test.temporal['u_ydark']
        ax.plot(X, Y,
                label='Data',
                gid='%d:data' % test.id)
        ax.plot(X,
                test.linearity()['fit_slope'] *
                X + test.linearity()['fit_offset'],
                linestyle='--',
                label='Fit',
                gid='%d:fit' % test.id)
        ax.plot((X[test.index_linearity_min], X[test.index_linearity_max]),
                (Y[test.index_linearity_min], Y[test.index_linearity_max]),
                label='Fit range',
                linestyle='None',
                marker='o',
                gid='%d:marker' % test.id)
        self.set_legend(ax)
        self.ax.ticklabel_format(axis='x', style='sci', scilimits=(1, 4))

class PlotDeviationLinearity(Emval1288Plot):
    '''Create Deviation Linearity plot'''
    name = 'Deviation linearity'
    xlabel = '$\mu_{p}$ [mean number of photons/pixel]'
    ylabel = 'Linearity error LE [%]'

    def plot(self, test):
        ax = self.ax
        X = test.temporal['u_p'][test.index_linearity_min:
                                test.index_linearity_max]
        deviation = test.linearity()['relative_deviation']
        Y = deviation[test.index_linearity_min: test.index_linearity_max]

```

```

ax.plot(X, Y,
        label='Data',
        gid='%d:data' % test.id)
ax.plot((X[0], X[-1]),
        (Y[0], Y[-1]),
        label='Fit range',
        linestyle='None',
        marker='o',
        gid='%d:marker' % test.id)
self.set_legend(ax)
self.ax.ticklabel_format(axis='x', style='sci', scilimits=(1, 4))

```

```

class PlotHorizontalSpectrogramPRNU(Emval288Plot):
    '''Create Horizontal spectrogram PRNU plot'''
    name = 'Horizontal spectrogram PRNU'
    xlabel = 'cycles [periods/pixel]'
    ylabel = 'Standard deviation and\nrelative presence of each cycle [%]'
    yscale = 'log'

    def plot(self, test):
        ax = self.ax
        data = (test.spatial['sum'] - test.spatial['sum_dark'])
        data = data / test.spatial['L']
        data_mean = np.mean(data)
        spectrogram = routines.FFT1288(data) / data_mean
        ax.plot(routines.GetFrecs(spectrogram),
                (np.sqrt(spectrogram)),
                label='Data',
                gid='%d:data' % test.id)
        ax.axhline(test.PRNU1288,
                   label='$PRNU_{1288}$',
                   linestyle='--',
                   color='r',
                   gid='%d:marker' % test.id)
        ax.axhline(100 * np.sqrt(test.sigma_2_y_stack) / data_mean,
                   label='$\sigma^2_{y.stack}$',
                   linestyle='-.',
                   color='g',
                   gid='%d:marker' % test.id)
        self.set_legend(ax)

```

```

class PlotHorizontalSpectrogramDSNU(Emva1288Plot):
    '''Create Horizontal spectrogram DSNU plot'''
    name = 'Horizontal spectrogram DSNU'
    xlabel = 'cycles [periods/pixel]'
    ylabel = 'Standard deviation and\nrelative presence of each cycle [DN]'
    yscale = 'log'

    def plot(self, test):
        ax = self.ax

        spectrogram = routines.FFT1288(test.spatial['sum_dark'],
                                       n=test.spatial['L'])
        ax.plot(routines.GetFrecs(spectrogram),
               np.sqrt(spectrogram),
               label='Data',
               gid='%d:data' % test.id)
        ax.axhline(test.DSNU1288_DN(),
                  label='$DSNU_{1288.DN}$',
                  linestyle='--',
                  color='r',
                  gid='%d:marker' % test.id)
        ax.axhline(np.sqrt(test.sigma_2_y_stack_dark),
                  label='$\sigma^2_{y.stack.dark}$',
                  linestyle='-.',
                  color='g',
                  gid='%d:marker' % test.id)
        self.set_legend(ax)

class PlotVerticalSpectrogramPRNU(Emva1288Plot):
    '''Create Vertical spectrogram PRNU plot'''
    name = 'Vertical spectrogram PRNU'
    xlabel = 'cycles [periods/pixel]'
    ylabel = 'Standard deviation and\nrelative presence of each cycle [%]'
    yscale = 'log'

    def plot(self, test):
        ax = self.ax

        data = test.spatial['sum'] - test.spatial['sum_dark']
        data = data / test.spatial['L']

```

```

data_mean = np.mean(data)
spectrogram = routines.FFT1288(data, rotate=True) / data_mean
ax.plot((routines.GetFrecs(spectrogram)),
        (np.sqrt(spectrogram)),
        label='Data',
        gid='%d:data' % test.id)

ax.axhline(test.PRUNU1288,
           label='$PRNU_{1288}$',
           linestyle='--',
           color='r',
           gid='%d:marker' % test.id)
ax.axhline(100 * np.sqrt(test.sigma_2_y_stack) / data_mean,
           label='$\sigma^2_{y.stack}$',
           linestyle='-.',
           color='g',
           gid='%d:marker' % test.id)
self.set_legend(ax)

```

```

class PlotVerticalSpectrogramDSNU(Emval1288Plot):

```

```

    '''Create Vertical spectrogram DSNU plot'''

```

```

    name = 'Vertical spectrogram DSNU'

```

```

    xlabel = 'Cycles [periods/pixel]'

```

```

    ylabel = 'Standard deviation and\nrelative presence of each cycle [DN]'

```

```

    yscale = 'log'

```

```

    def plot(self, test):

```

```

        ax = self.ax

```

```

        spectrogram = routines.FFT1288(test.spatial['sum_dark'],
                                       rotate=True,
                                       n=test.spatial['L'])

```

```

        ax.plot(routines.GetFrecs(spectrogram),
                np.sqrt(spectrogram),
                label='Data',
                gid='%d:data' % test.id)

```

```

        ax.axhline(test.DSNU1288_DN(),
                   label='$DSNU_{1288.DN}$',
                   linestyle='--',
                   color='r',
                   gid='%d:marker' % test.id)

```

```

        ax.axhline(np.sqrt(test.sigma_2_y_stack_dark),
                    label='$\sigma^2_{y.stack.dark}$',
                    linestyle='-.',
                    color='g',
                    gid='%d:marker' % test.id)

        self.set_legend(ax)

class PlotLogarithmicHistogramDSNU(Emva1288Plot):
    '''Create Logarithmic histogram DSNU plot'''
    name = 'Logarithmic histogram DSNU'
    xlabel = 'Deviation from the mean [DN]'
    ylabel = 'Number of pixels'
    yscale = 'log'

    def plot(self, test):
        ax = self.ax

        hist = test.histogram_DSNU

        ax.plot(hist['bins'], hist['values'],
                gid='%d:data' % test.id,
                label='Data')
        ax.plot(hist['bins'], hist['model'],
                '--',
                gid='%d:fit' % test.id,
                label='Model')

        self.set_legend(ax)

        ax.axis(ymin=1.0, ymax=np.max(hist['values']) * 2)

class PlotLogarithmicHistogramPRNU(Emva1288Plot):
    '''Create Logarithmic histogram PRNU plot'''
    name = 'Logarithmic histogram PRNU'
    xlabel = 'Deviation from the mean [%]'
    ylabel = 'Number of pixels'
    yscale = 'log'

    def plot(self, test):
        ax = self.ax

```

```

hist = test.histogram_PRNU

ax.plot(hist['bins'], hist['values'],
        gid='%d:data' % test.id,
        label='Data')
ax.plot(hist['bins'], hist['model'], '--',
        gid='%d:fit' % test.id,
        label='Model')
self.set_legend(ax)
ax.axis(ymin=0.5, ymax=np.max(hist['values']) * 2)

```

```

class PlotAccumulatedLogHistogramDSNU(Emval288Plot):

```

```

    '''Create Accumulated log histogram DSNU plot'''

```

```

    name = 'Accumulated log histogram DSNU'

```

```

    xlabel = 'Minimal deviation from the mean [DN]'

```

```

    ylabel = 'Percentage of pixels/bin'

```

```

    yscale = 'log'

```

```

    def plot(self, test):

```

```

        ax = self.ax

```

```

        hist = test.histogram_DSNU_accumulated

```

```

        ax.plot(hist['bins'], hist['values'],

```

```

                gid='%d:data' % test.id,

```

```

                label='Data')

```

```

        self.set_legend(ax)

```

```

        self.figure.tight_layout()

```

```

class PlotAccumulatedLogHistogramPRNU(Emval288Plot):

```

```

    '''Create Accumulated log histogram PRNU plot'''

```

```

    name = 'Accumulated log histogram PRNU'

```

```

    xlabel = 'Minimal deviation from the mean [%]'

```

```

    ylabel = 'Percentage of pixels/bin'

```

```

    yscale = 'log'

```

```

    def plot(self, test):

```

```

        ax = self.ax

```



```

hist = test.histogram_PRNU_accumulated

ax.plot(hist['bins'], hist['values'],
        gid='%d:data' % test.id,
        label='Data')

self.set_legend(ax)

class ProfileBase(Emval288Plot):
    def __init__(self, *args, **kwargs):
        Emval288Plot.__init__(self, *args, **kwargs)
        # Dict to keep track of max and min values to adjust the
        # limit of the axis
        self.axis_limits = {'bright': {'max': [], 'min': [], 'length': []},
                             'dark': {'max': [], 'min': [], 'length': []}}

    def _get_extremes(self, mean_, min_, max_):
        min_min_i = np.argmax(mean_ - min_)
        min_min = min_[min_min_i]
        min_perc = np.abs(100. - (min_min * 100. / mean_[min_min_i]))
        min_label = 'Min ({:.1f} {:.1f}%)'.format(min_min,
                                                  min_perc)

        max_max_i = np.argmax(max_ - min_)
        max_max = max_[max_max_i]
        max_perc = np.abs(100. - (max_max * 100. / mean_[max_max_i]))
        max_label = 'Max ({:.1f} {:.1f}%)'.format(max_max,
                                                  max_perc)

        return {'min_deviation': min_min, 'min_precentage': min_perc,
                'min_label': min_label, 'max_label': max_label,
                'max_deviation': max_max, 'max_percentage': max_perc}

    def _get_image_profiles(self, image):
        """Get the images profiles. Supports the masked arrays.
        Masked arrays cannot be shown directly with continuous lines.
        To fix this, we only return the profiles excluding the masks
        constants.
        Also returns the x arrays corresponding the the profile.
        """
        img = image

```

```

    if self.vertical:
        img = np.transpose(image)

    profile = np.mean(img, axis=0)
    profile_min = np.min(img, axis=0)
    profile_max = np.max(img, axis=0)
    mid_i = np.shape(img)[0]
    profile_mid = img[mid_i // 2, :]
    # Verifications for profile_mid
    if isinstance(profile_mid, np.ma.core.MaskedArray):
        if profile_mid.mask.all():
            # If by chance mid is a column of masked constant,
            # take the next one
            profile_mid = img[mid_i // 2 + 1, :]
    # use _get_x_y to get the x and profile of unmasked values
    d = {'mean': self._get_x_y(profile),
        'min': self._get_x_y(profile_min),
        'max': self._get_x_y(profile_max),
        'mid': self._get_x_y(profile_mid),
        }
    return d

def _get_x_y(self, profile):
    x = np.arange(len(profile))
    if isinstance(profile, np.ma.MaskedArray):
        masks = profile.mask
        # Index lists of where there is masked constants
        index = [i for i in range(len(masks)) if not masks[i]]
        # chop out masked values
        x = x[index]
        profile = profile[index]
    return (x, profile)

def get_profiles(self, bright, dark):
    b_p = self._get_image_profiles(bright)
    # index 1 is the profile (0 is x-axis)
    b_mean = np.mean(b_p['mean'][1])
    self.axis_limits['bright']['min'].append(0.9 * b_mean)
    self.axis_limits['bright']['max'].append(1.1 * b_mean)

```

```

d_p = self._get_image_profiles(dark)
self.axis_limits['dark']['min'].append(0.9 * np.mean(d_p['min'][:1]))
self.axis_limits['dark']['max'].append(1.1 * np.mean(d_p['max'][:1]))
return {'bright': b_p, 'dark': d_p}

def plot(self, test):
    # index of profiles of what will be the x and y axis
    x = 0
    y = 1
    legend_loc = 'upper right'
    if self.vertical:
        x = 1
        y = 0
        legend_loc = (0.8, 0.65)
    ax = self.ax
    ax2 = self.ax2
    L = test.spatial['L']
    bimg = (test.spatial['sum'] - test.spatial['sum_dark']) / L
    dimg = test.spatial['sum_dark'] / L
    profiles = self.get_profiles(bimg, dimg)

    # to keep the lines number for legend
    bright_plots = []
    labels = []
    for typ, color in (('mid', 'green'),
                      ('min', 'blue'),
                      ('max', 'orange'),
                      ('mean', 'black')):
        # label has first letter capital
        label = typ.capitalize()
        labels.append(label)
        # bright plot
        line = ax.plot(profiles['bright'][typ][x],
                      profiles['bright'][typ][y],
                      label=label,
                      color=color,
                      gid='%d:marker' % test.id)[0]
        bright_plots.append(line)
        # dark plot
        ax2.plot(profiles['dark'][typ][x],

```

```

        profiles['dark'][typ][y],
        label=label,
        color=color,
        gid='%d:data' % test.id)

    # Place legend
    self.figure.legend(bright_plots,
                       labels,
                       loc=legend_loc)

class PlotHorizontalProfile(ProfileBase):
    '''Create Horizontal profile plot
    Profile is done with spatial images.
    '''
    name = 'Horizontal profile'
    vertical = False

    def setup_figure(self):
        self.ax = self.figure.add_subplot(211)
        self.ax2 = self.figure.add_subplot(212)
        self.figure.suptitle(self.name)
        self.ax.set_title('PRNU')
        self.ax.set_ylabel('Vertical line [DN]')
        self.ax2.set_title('DSNU')
        self.ax2.set_xlabel('Index of the line')
        self.ax2.set_ylabel('Vertical line [DN]')

    def rearrange(self):
        self.ax.set_xticks([])
        self.ax.axis(ymin=min(self.axis_limits['bright']['min']),
                     ymax=max(self.axis_limits['bright']['max']))
        self.ax2.axis(ymin=min(self.axis_limits['dark']['min']),
                      ymax=max(self.axis_limits['dark']['max']))
        self.reduce_ticks(self.ax2, 'y')
        self.reduce_ticks(self.ax, 'y')
        self.figure.tight_layout(pad=2)

class PlotVerticalProfile(ProfileBase):
    '''Create Vertical profile plot.
    Profile is done with spatial images.
    '''

```

```

name = 'Vertical profile'
vertical = True

def setup_figure(self):
    self.ax2 = self.figure.add_subplot(121)
    self.ax = self.figure.add_subplot(122)
    self.figure.suptitle(self.name)
    self.ax2.set_title('DSNU')
    self.ax2.set_xlabel('Vertical line [DN]')
    self.ax2.set_ylabel('Index of the line')
    self.ax.set_title('PRNU')
    self.ax.set_xlabel('Vertical line [DN]')

def rearrange(self):
    self.ax.set_yticks([])
    self.ax.axis(xmin=min(self.axis_limits['bright']['min']),
                 xmax=max(self.axis_limits['bright']['max']))
    self.ax2.axis(xmin=min(self.axis_limits['dark']['min']),
                  xmax=max(self.axis_limits['dark']['max']))
    self.ax2.invert_yaxis()
    self.ax.invert_yaxis()
    self.reduce_ticks(self.ax2, 'x')
    self.reduce_ticks(self.ax, 'x')
    self.figure.tight_layout()

EVMA1288plots = [PlotPTC,
                  PlotSNR,
                  PlotSensitivity,
                  PlotUyDark,
                  PlotLinearity,
                  PlotDeviationLinearity,
                  PlotHorizontalSpectrogramPRNU,
                  PlotHorizontalSpectrogramDSNU,
                  PlotVerticalSpectrogramPRNU,
                  PlotVerticalSpectrogramDSNU,
                  PlotLogarithmicHistogramDSNU,
                  PlotLogarithmicHistogramPRNU,
                  PlotAccumulatedLogHistogramDSNU,
                  PlotAccumulatedLogHistogramPRNU,
                  PlotHorizontalProfile,

```

PlotVerticalProfile]

```
class Plotting1288(object):
    """EMVA1288 plots
    Creates and shows all plots necessary to prepare a camera or sensor
    descriptive report compliant with EMVA Standard 1288 version 3.1.
    """
    def __init__(self, *tests):
        """
        Parameters
        -----
        tests: list
            List of tests to Plot
        """
        self.tests = []
        for test in tests:
            if not getattr(test, 'id', False):
                test.id = id(test)
        self.tests.append(test)

    def plot(self, *plots):
        """Plot EMVA1288 plots
        Parameters
        -----
        plots: list
            List of plots to plot
        """
        import matplotlib.pyplot as plt
        if not plots:
            plots = EVMA1288plots
        for i, plot_cls in enumerate(plots):
            figure = plt.figure(i)
            plot = plot_cls(figure)
            for test in self.tests:
                plot.plot(test)
            plot.rearrange()
            figure.canvas.set_window_title(plot.name)
        plt.show()
```

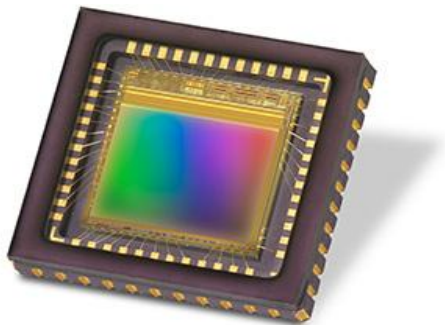
REFERENCIAS

- [1] Jacques Leconte, "Formation base capteurs e2v", *e2v internal*, 2007
- [2] Arnaud Camboly, "Intro au CMOS", *e2v internal*, 2009
- [3] Thierry Lizogat, "Response and sensitivity", *Atmel internal*, 2005
- [4] Jacques Leconte, "How to Choose between Global and Rolling Shutter", *Atmel internal*, 2006
- [5] Arnaud Camboy, "Image sensor validation capability", *e2v internal*, 2014
- [6] Rémi Boget, "Capteur d'image CMOS ONYX1.3M", *e2v internal & Grenoble INP*, 2016
- [7] Arnaud Camboly, "Characterization of CIS", *FillFactory internal*, 2004
- [8] Francis Pang, "CIS characterization Handbook", *e2v internal*, 2007
- [9] Cinta Ruiz, Gema Vallés, Manuel Lorente, "CIS Characterization - Optical Performances Methodology", *AnaFocus internal*, 2015
- [10] Rafael Dominguez, Fernando Medeiro, "Image sensor measurements based in EMVA Standard 1288", *AnaFocus internal*, 2015
- [11] Arnaud Camboly, "EMVA1288 VS AF methodology review", *AnaFocus internal*, 2016
- [12] Arnaud Camboly, "e2v Optical-Bench setup", *e2v internal*, 2010
- [13] Rafael Domínguez, Fernando Medeiro, "CMOS pixel architecture", *AnaFocus internal*, 2015
- [14] EMVA Standard 1288, "Standard for Characterization of Image Sensors and Cameras", Release 3.1, November 2012
- [15] Junichi Nakamura, "Image Sensors and Signal Processing for Digital Still Cameras", 2006
- [16] Federico Ariza, "Python library to create LaTeX files", <https://github.com/fariza/PyLaTeX>

EMVA 1288 Datasheet

Higher Technical School of Engineering of the University of Seville Final Degree Thesis - Manuel Lorente Alman This datasheet describes the specification according to the standard 1288 Standard for Characterization and Presentation of Specification Data for Image Sensors and Cameras of European Machine Vision Association (EMVA) (See www.standard1288.org).

Vendor	Electronic Engineering Department	Dark current compensation	
Model	CMOS Sensor	Interface type	Camera-Link
Data type	Single	Light source	Led
Sensor type	Area	Light source non uniformity	0.5
Resolution	4320x2048 pixels	Standard version	3.1
Pixel size	5 μm		
Maximum readout rate	35fps		



Summary sheet for Operation Point: **Config case A**

Camera setting

Number of images	40
Number of frames	5
Configuration SVN version	15
Image format	tiff

Radiance minimum

None

Radiance maximum

None

Exposure fixed

None

Performance

Quantum efficiency

η 56.05 %

System gain

K 0.117 DN/e⁻
1/K 8.566 e⁻/DN

Temporal dark noise

σ_d 4.598 e⁻
 $\sigma_{y.dark}$ 0.609 DN

Signal-to-Noise Ratio

SNR_{max} 44
32.93 dB
5.5 bit
 SNR_{max}^{-1} 2.256 %

Absolute sensitivity threshold

$\mu_{p.min}$ 10.206 p
 $\mu_{p.min.area}$ - p/ μm^2
 $\mu_{e.min}$ 5.721 e⁻
 $\mu_{e.min.area}$ - e⁻/ μm^2

Saturation Capacity

$\mu_{p.sat}$ 3504 p
 $\mu_{p.sat.area}$ - p/ μm^2
 $\mu_{e.sat}$ 1964 e⁻
 $\mu_{e.sat.area}$ - e⁻/ μm^2

Dynamic Range

DR 343
50.7 dB
8.4 bit

Spatial Nonuniformities

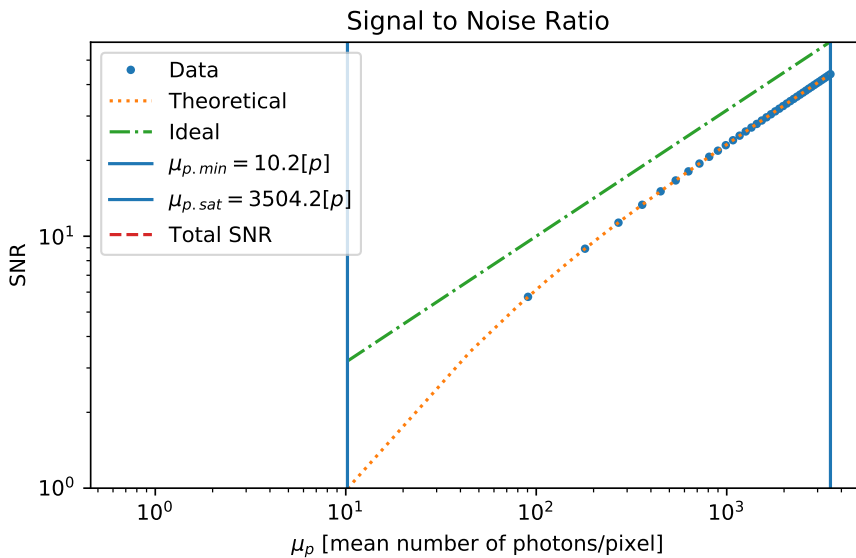
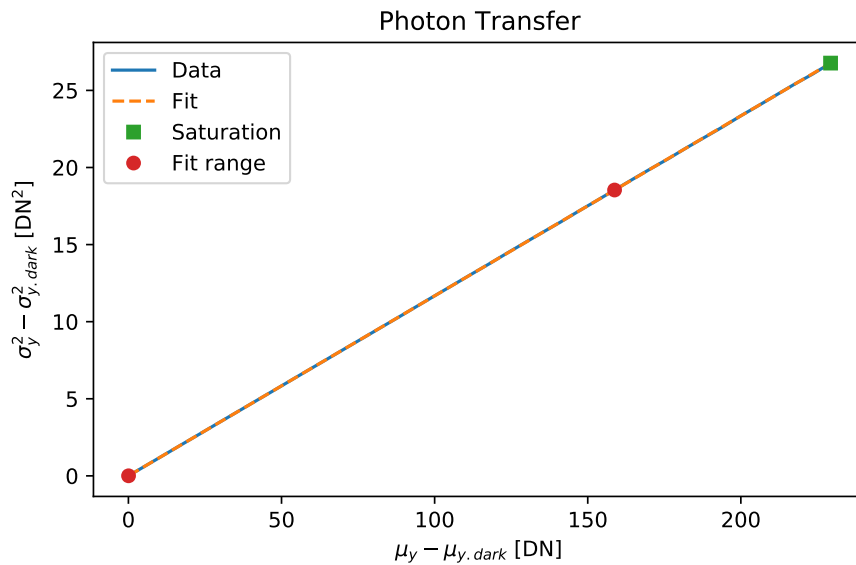
$DSNU_{1288}$ nan e⁻
nan DN
 $PRNU_{1288}$ 0.0 %

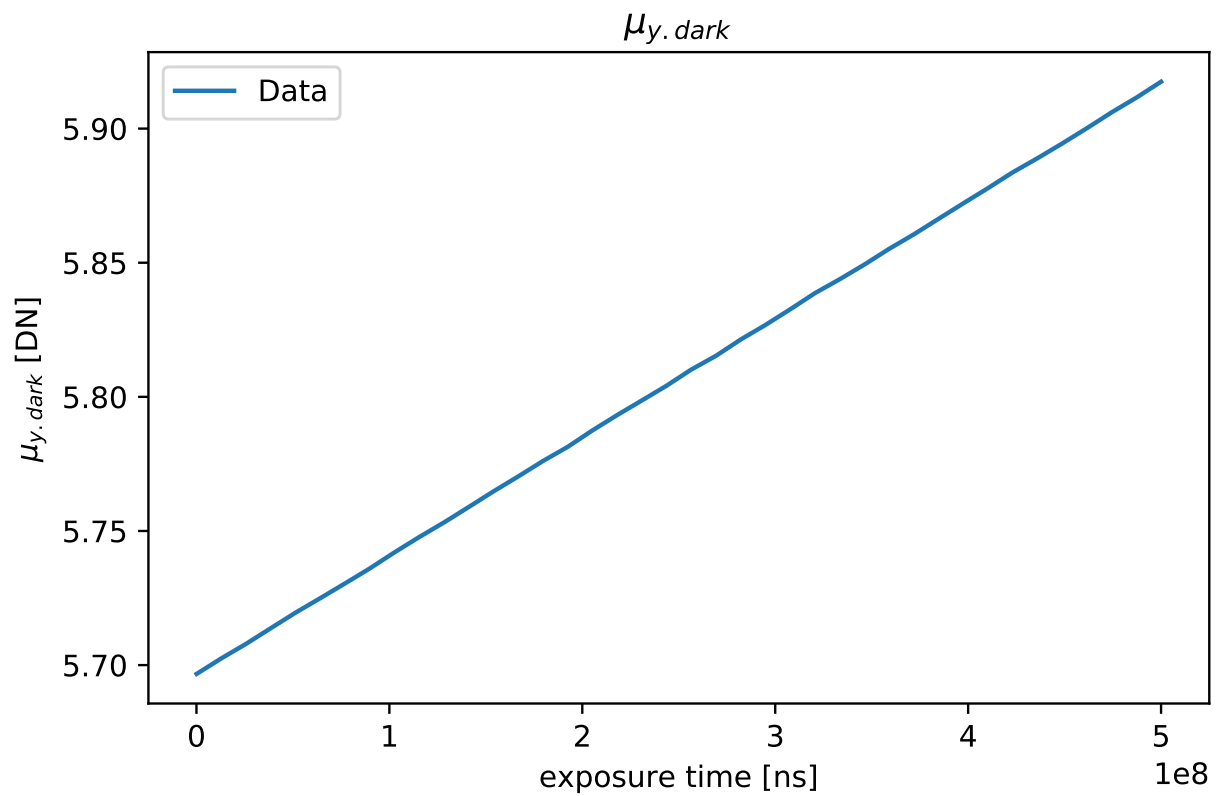
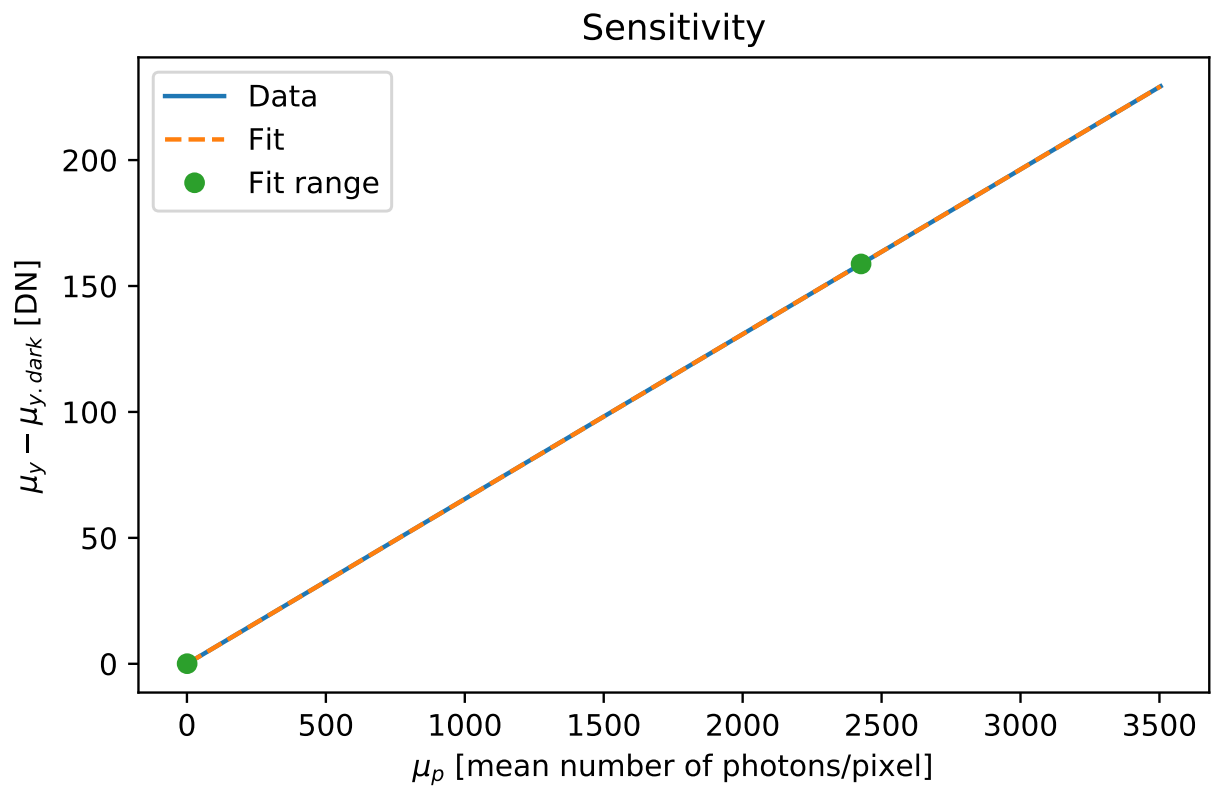
Linearity error

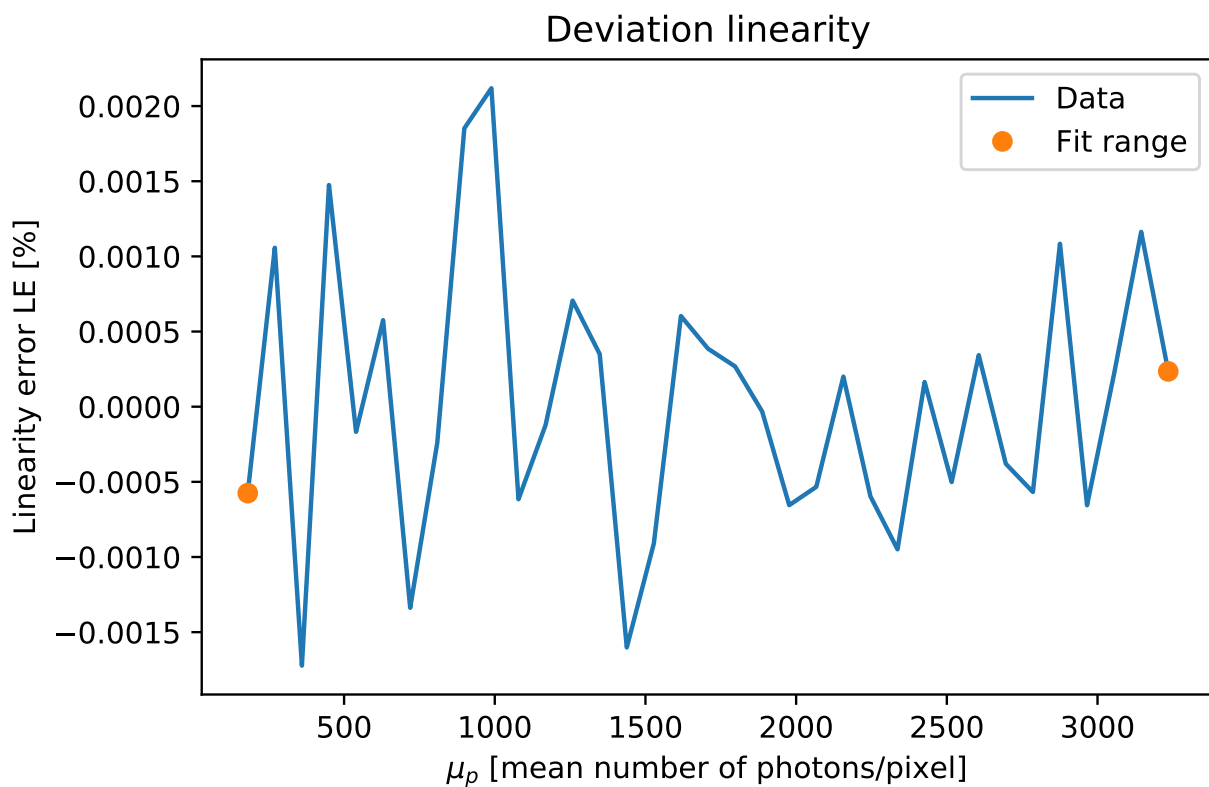
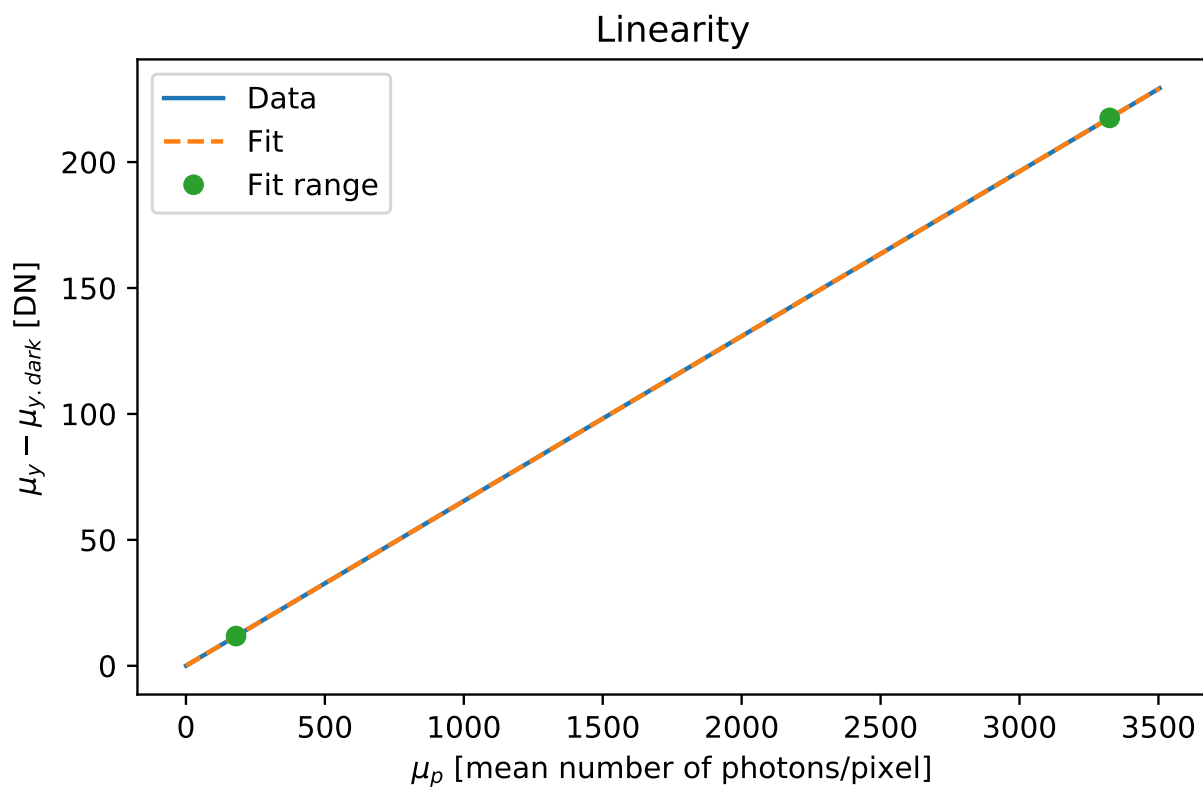
LE_{min} -0.002 %
 LE_{max} 0.002 %

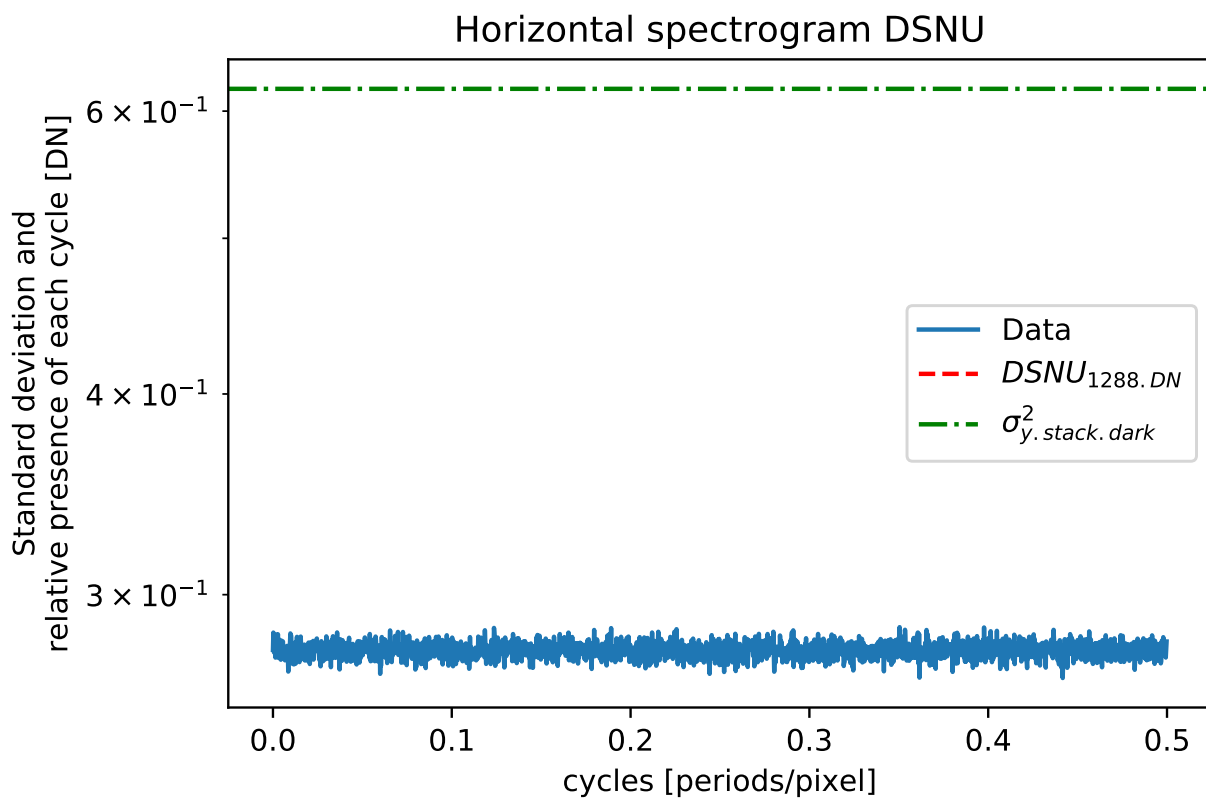
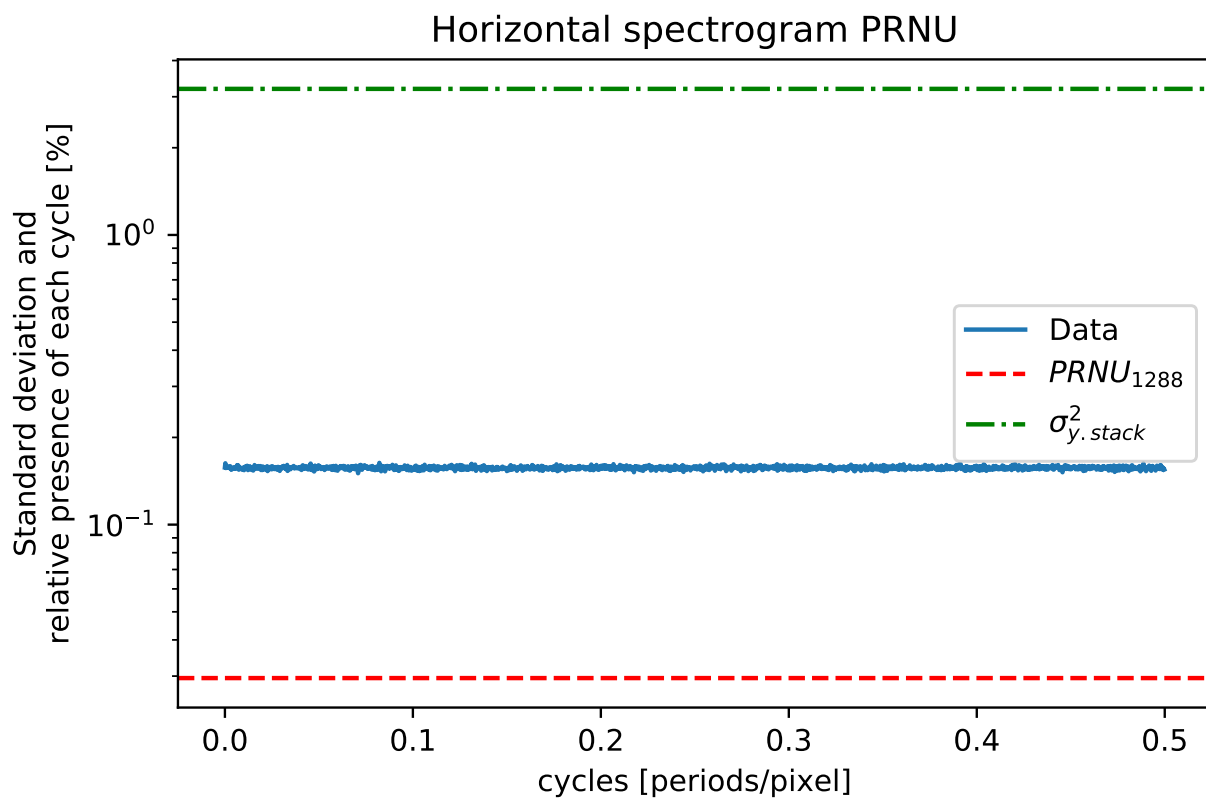
Dark current

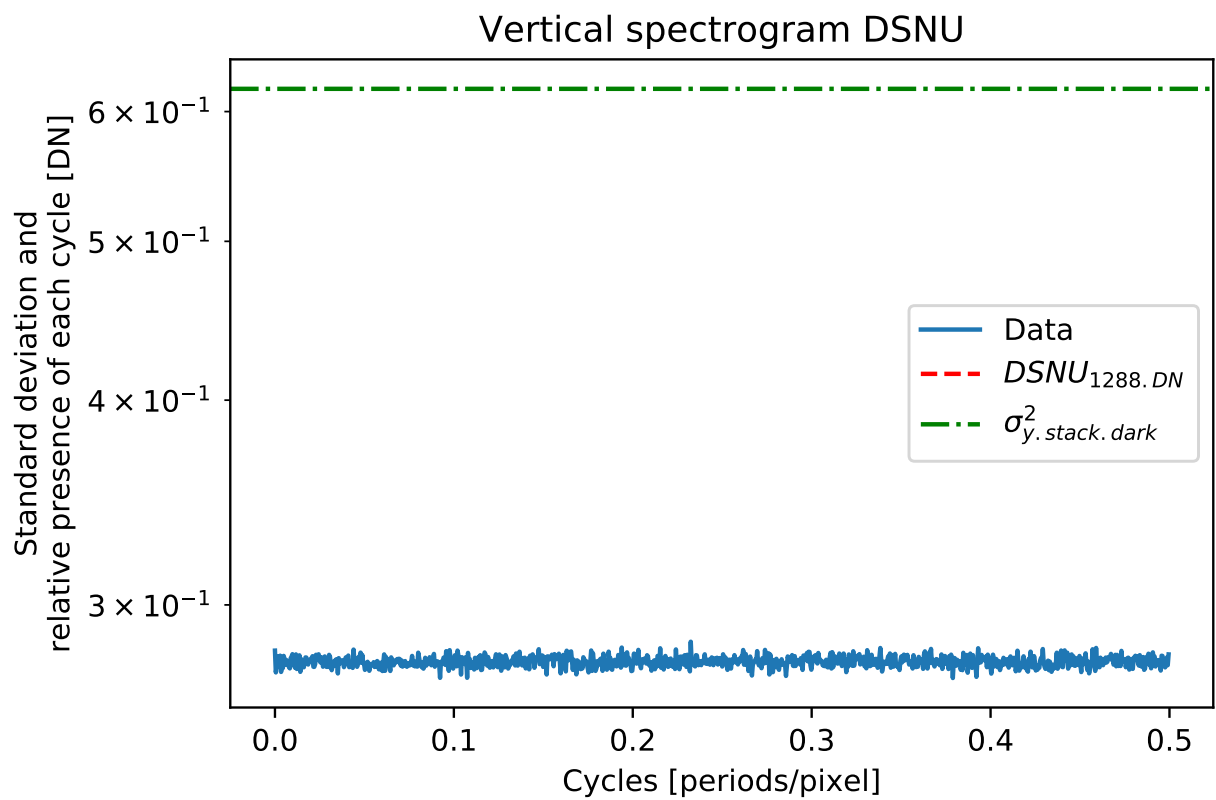
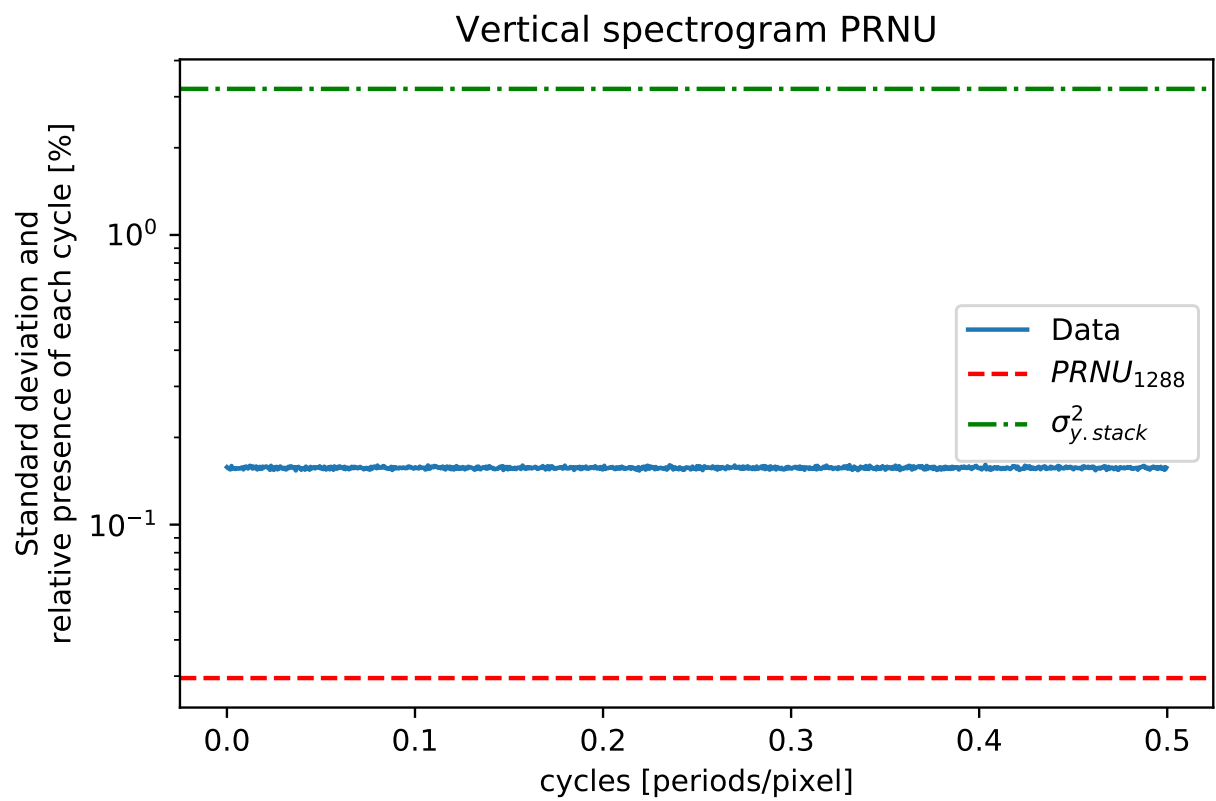
$\mu_{I.mean}$ 3.780 e⁻/s
0.441 DN/s
 $\mu_{I.var}$ 3.578 e⁻/s
0.049 DN/s

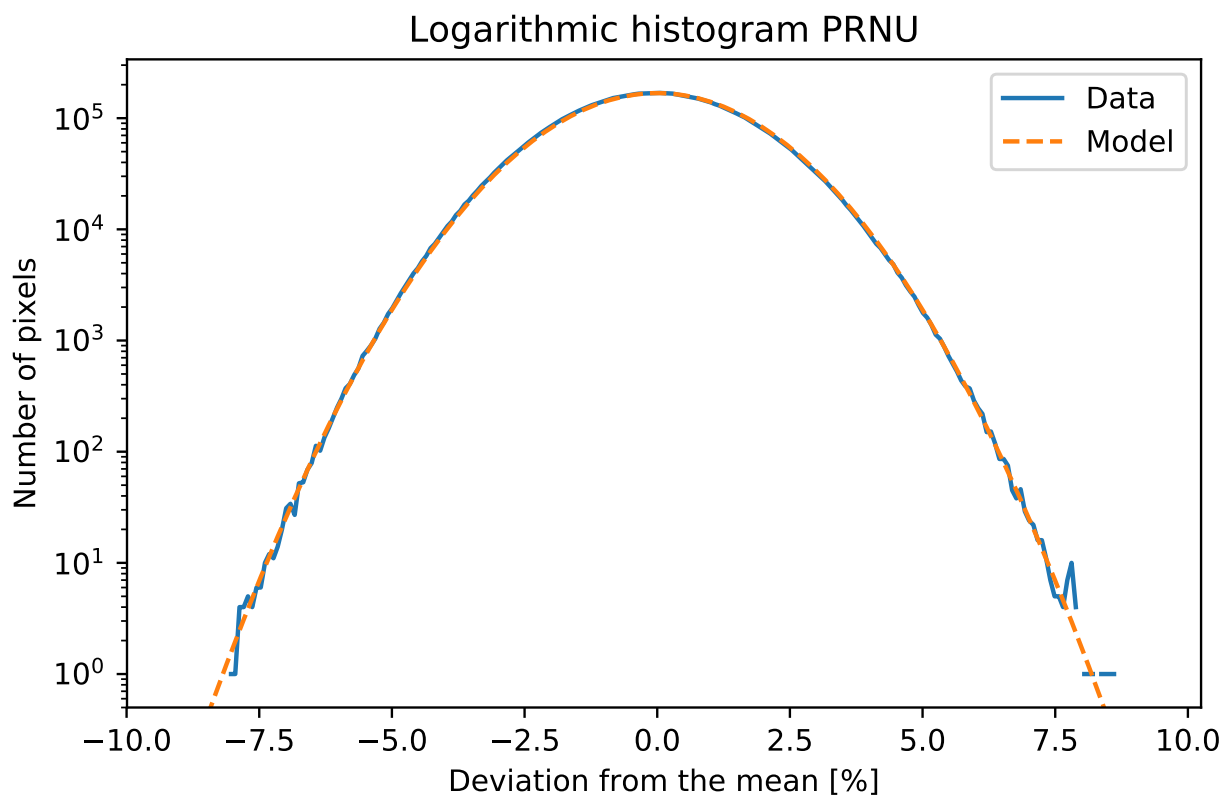
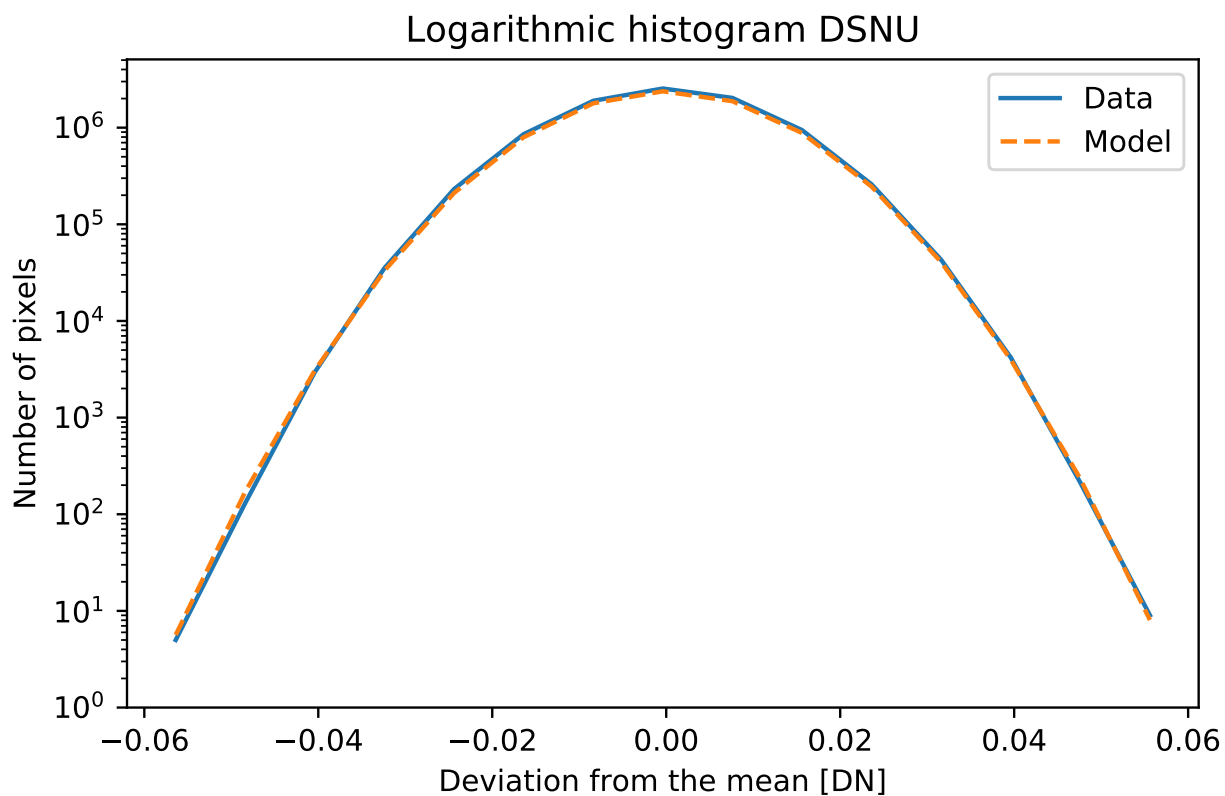


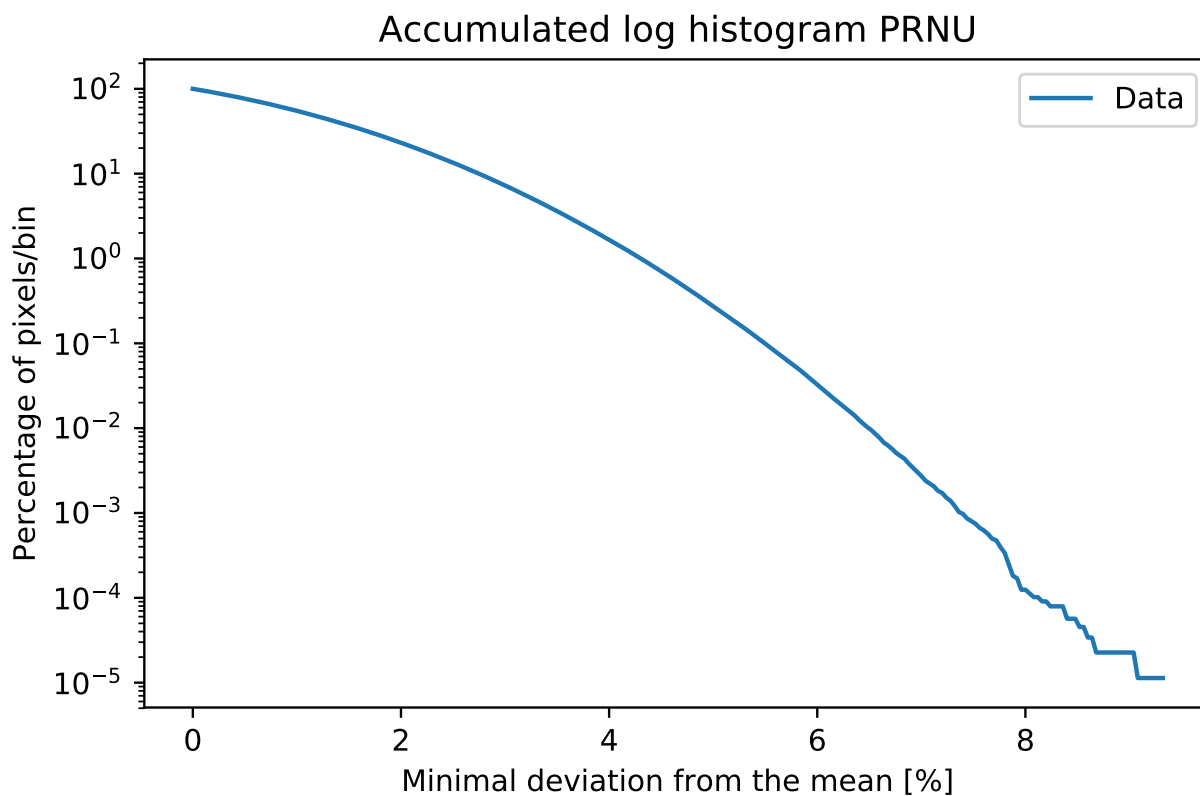
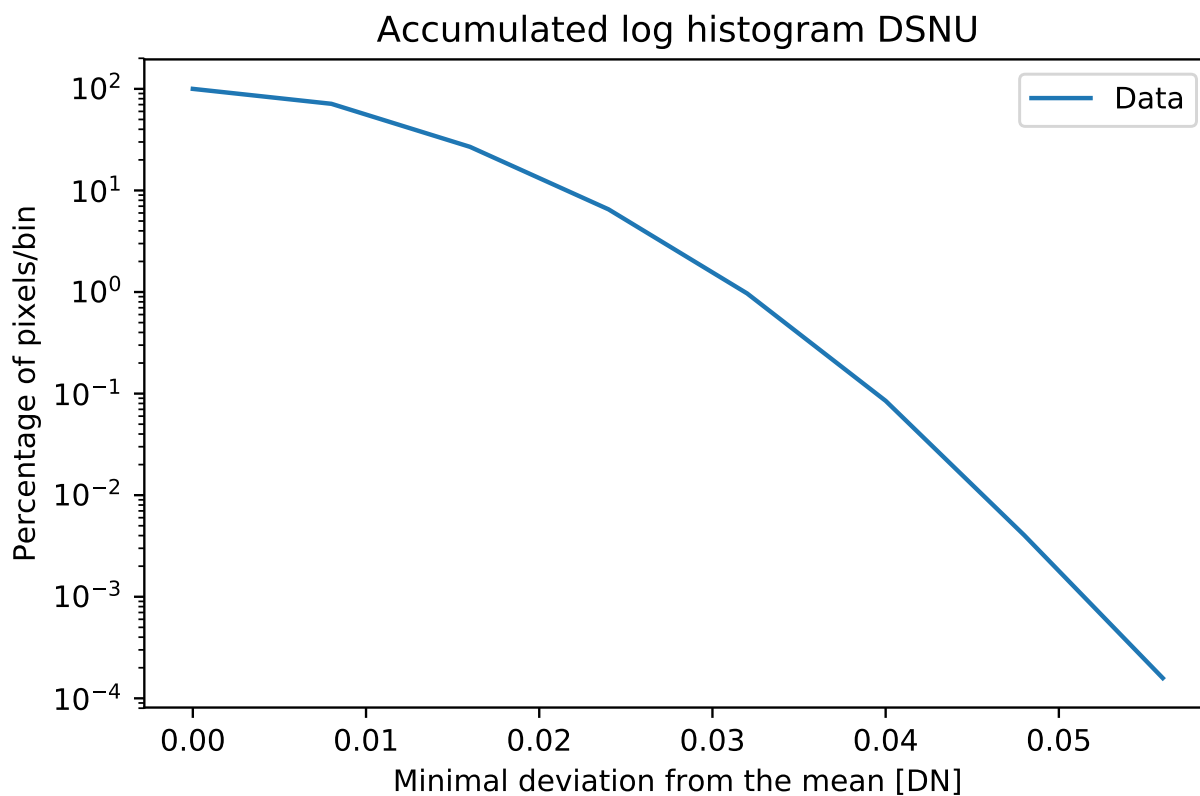


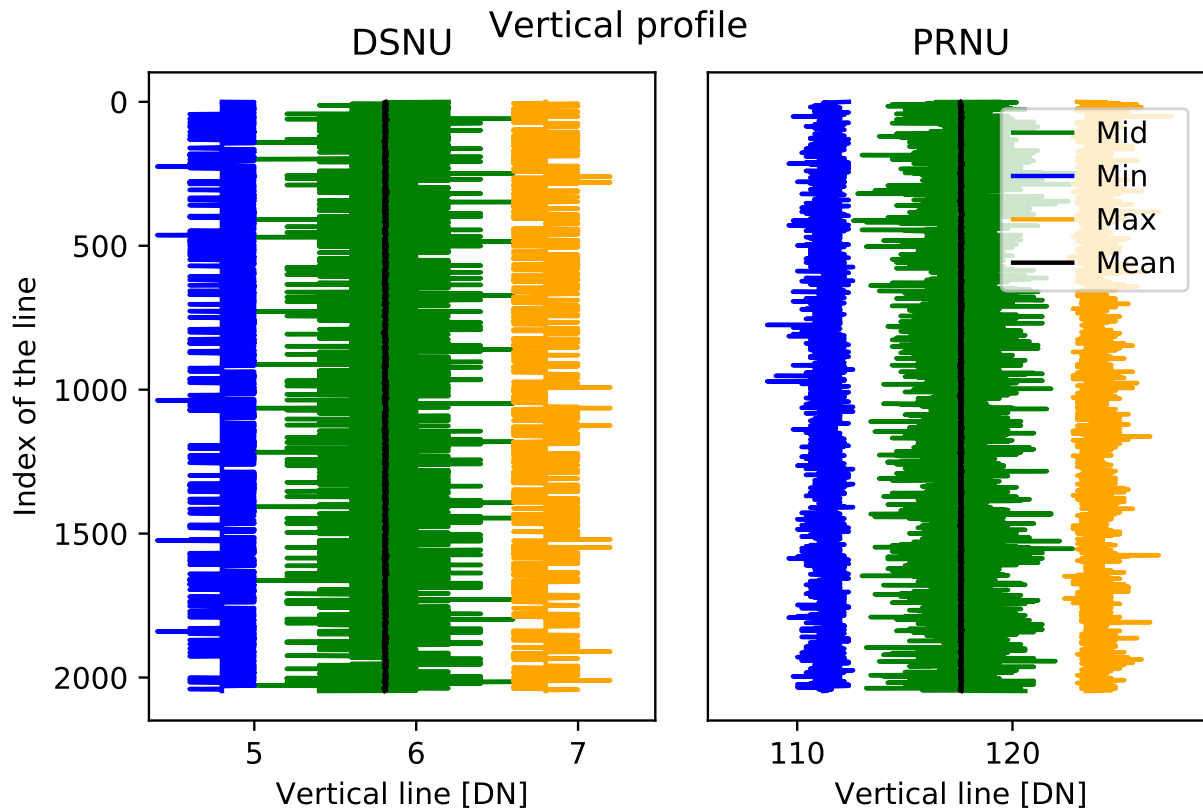
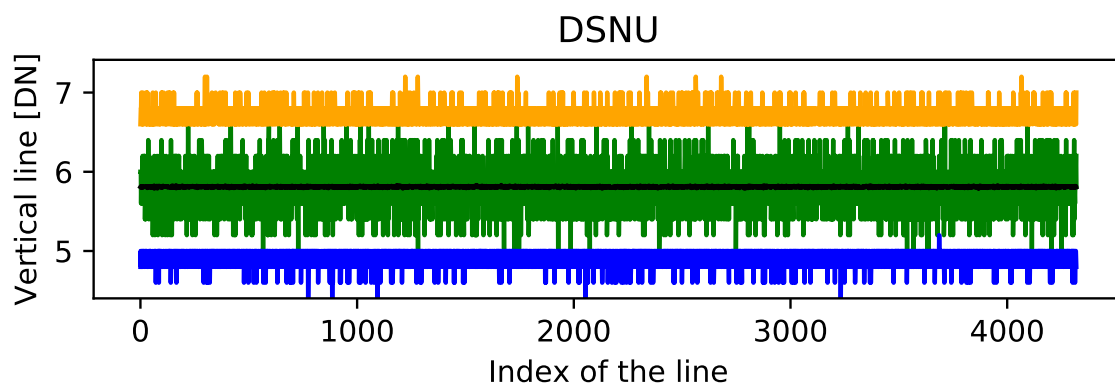
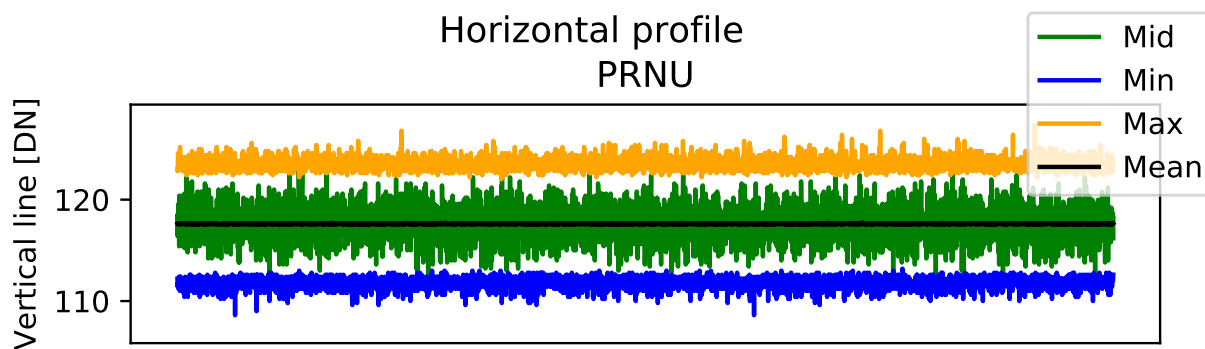








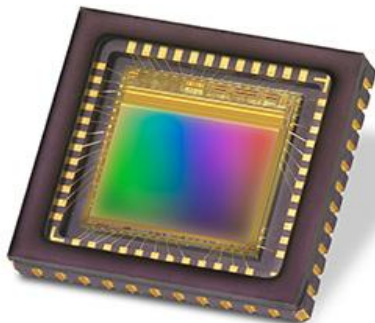




EMVA 1288 Datasheet

Higher Technical School of Engineering of the University of Seville Final Degree Thesis - Manuel Lorente Alman This datasheet describes the specification according to the standard 1288 Standard for Characterization and Presentation of Specification Data for Image Sensors and Cameras of European Machine Vision Association (EMVA) (See www.standard1288.org).

Vendor	Electronic Engineering Department	Dark current compensation	
Model	CMOS Sensor	Interface type	GigE
Data type	Single	Light source	Led
Sensor type	Area	Light source non uniformity	0.5
Resolution	640x480 pixels	Standard version	3.1
Pixel size	5 μm		
Maximum readout rate	120fps		



Summary sheet for Operation Point: **Config case B**

Camera setting

Number of images	40
Number of frames	5
Configuration SVN version	15
Image format	tiff

Radiance minimum

None

Radiance maximum

None

Exposure fixed

None

Performance

Quantum efficiency

η 49.86 %

System gain

K 0.183 DN/ e^-

1/K 5.454 e^- /DN

Temporal dark noise

σ_d 4.182 e^-

$\sigma_{y.dark}$ 0.819 DN

Signal-to-Noise Ratio

SNR_{max} 134

42.52 dB

7.1 bit

SNR_{max}^{-1} 0.748 %

Absolute sensitivity threshold

$\mu_{p.min}$ 9.965 p

$\mu_{p.min.area}$ - p/ μm^2

$\mu_{e.min}$ 4.968 e^-

$\mu_{e.min.area}$ - $e^-/\mu m^2$

Saturation Capacity

$\mu_{p.sat}$ 35835 p

$\mu_{p.sat.area}$ - p/ μm^2

$\mu_{e.sat}$ 17867 e^-

$\mu_{e.sat.area}$ - $e^-/\mu m^2$

Dynamic Range

DR 3596

71.1 dB

11.8 bit

Spatial Nonuniformities

$DSNU_{1288}$ 0.1 e^-

0.0 DN

$PRNU_{1288}$ nan %

Linearity error

LE_{min} -0.004 %

LE_{max} 0.004 %

Dark current

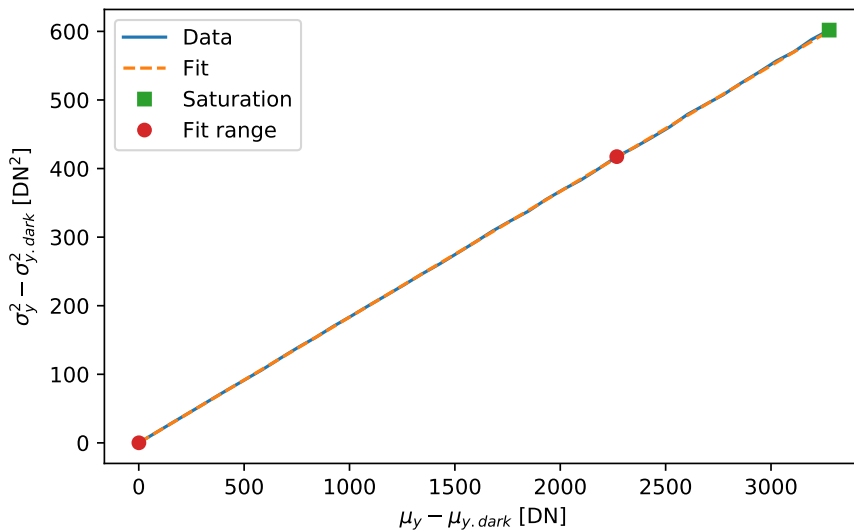
$\mu_{I.mean}$ 1.573 e^-/s

0.288 DN/s

$\mu_{I.var}$ 1.559 e^-/s

0.052 DN/s

Photon Transfer



Signal to Noise Ratio

